

ميتابولون-إل إل إم: عمارة وتحقيق من واجهة وكالة لتحليل الأيض

MetabolonR-LLM: Architecture and Validation of an Agentic Interface for Metabolomics Analysis

د. فضل محمد الأكوع

Dr. Fadhl Mohammed Alakwaa

2026

عربي · English

ميتابولون-إل إل إم: عمارة وتحقيق من واجهة وكالة لتحليل الأيض

*MetabolonR-LLM: Architecture and
Validation of an Agentic Interface for
Metabolomics Analysis*

د. فضل محمد الأكوع

Dr. Fadhl Mohammed Alakwaa

الطبعة الأولى — 2026 · First Edition

جميع الحقوق محفوظة للمؤلف

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

إهداء

إلى قاسم الماوري - طالبي الذي بنى هذا معي، في زمنٍ
صارت فيه الآلة شريكاً في الفكر، لكن العقل البشري لا يزال
هو القائد. - فضل

تنبیه

هذا الكتاب أُنتج بمساعدة نموذج Claude Opus 4.7 من شركة Anthropic، تحت إشراف وتوجيه المؤلف. التوجيه الفكري واختيار الموضوع والمراجعة والمسؤولية النهائية للمؤلف الدكتور فضل محمد الأكوع. الكتاب لأغراض تثقيفية وإثرائية، ولا يُغني عن استشارة المتخصصين في الموضوعات القانونية والطبية والمالية والشرعية.

Notice

This book was produced with the assistance of Anthropic's Claude Opus 4.7 model, under the author's direction and supervision. Intellectual direction, topic selection, review, and final responsibility rest with the author, Dr. Fadhl Mohammed Alakwaa. The book is for educational purposes and does not substitute for consultation with specialists in legal, medical, financial, or religious matters.

المحتويات

1

الفصل 1

2

تقديم

إلى قاسم الماوري - طالي الذي بنى هذا معي، في زمنٍ صارت فيه الآلة شريكاً
في الفكر، لكن العقل البشري لا يزال هو القائد.

— فضل

تقديم

كُتِبَ هذا الكتاب على مرحلتين: مرحلة البناء، ومرحلة التوثيق. وما بينهما، كانت العلاقة بين أستاذ وطالبه هي ما حمل المشروع من فكرة إلى نظامٍ يعمل، ومن نظامٍ يعمل إلى ورقةٍ علميةٍ قابلةٍ للنشر.

أعمل في مختبر كريتلر بجامعة ميشيغان، ضمن مشروع نيتيون لدراسة أمراض الكلى المزمنة، وقد قضيتُ سنواتٍ في تحليل بيانات الأيض السريرية. والعقبة الكبرى التي رأيتها في عملي، وفي عمل زملائي حول العالم، لم تكن في جودة المختبرات، ولا في عمق النظريات الإحصائية. كانت في الواجهة: في الفجوة الواسعة بين الطبيب الذي يطرح السؤال السريري، والأداة الحاسوبية التي تستطيع الإجابة عنه. واجهتُ هذه الفجوة في عام 2018 ببناء نسخةٍ أولى من برنامج "ميتابولون-آر" بلغة شايبي، ثم في عام 2022 ببناء نسختي الثانية. ومع ذلك بقيت الفجوة قائمة.

ثم جاءت موجة النماذج اللغوية الكبيرة في 2023 و2024، وتغير المشهد. صار ممكناً، للمرة الأولى، أن يحادث الباحث السريري أدواته كما يحادث زميلاً. لكن هذا التحول جاء بخاطره: نماذج تتحدث بثقةٍ ولا تنتج النتيجة ذاتها مرتين، وأدواتٌ يُعلن عن قدراتها أكثر مما تستطيع البرهنة عليه. إن الأداة التي يُحتمل

أن تُستخدم في القرار السريري يجب أن تكون قابلةً لإعادة الإنتاج بشكل صارم، لا "تقريباً".

من هذا التوتر وُلد "ميتابولون-إل إل إم". الفكرة المركزية بسيطة: يدير النموذج اللغوي التوجيه، وتُنقذ الإحصاءات في طبقةٍ من الأدوات الموثقة، ويسُجّل كلُّ شيءٍ في سجلٍ يمكن إعادة تشغيله بدون النموذج. هذه القاعدة الواحدة هي ما يجعل الواجهة المحادثية قابلةً للنشر في المجلات السريرية.

أهدي هذا الكتاب إلى طالي قاسم الماوري، الذي بنى معي هذا النظام في ستة أسابيع متواصلة من الانضباط الهندسي والصبر العلمي. لم يكن هذا العمل سهلاً عليه، ولم يكن من حقّي أن أتوقع منه السرعة التي حقّقها. إنّ المنهج الذي يحمله هذا الكتاب — الانضباط في حدود الأدوات، وصدق التحقق، والاحترام لحدود ما تستطيع الآلة فعله — هو ما أرجو أن يكون نصيبه في مشروعه القادم.

أكتب هذا التقديم من آن أربور في صيف 2026، وأنا على قناعةٍ بأنّ مستقبل المعلوماتية الحيوية السريرية لن يكون في استبدال الباحث بالآلة، بل في بناء أدواتٍ تُحرّر الباحث للأسئلة التي لا تستطيع الآلة الإجابة عنها. هذا الكتاب جزءٌ من ذلك المشروع الأكبر، ولن يكون آخر ما أكتب فيه.

— د. فضل محمد الأكووع آن أربور، ميشيغان صيف 2026

القسم الإنجليزي

English Section

Table of Contents

Appendix A — Complete JSON Schemas for All 10 Tools	1
Appendix B — Annotated Code Listings	2
Appendix C — The 20 Validation Prompts	3
Appendix D — Glossary	4
Appendix E — References	5
Chapter 1 — Why Metabolomics Needs Better Software	6
Chapter 2 — The Metabolomics Analysis Pipeline	7
Chapter 3 — From Command Line to Shiny to Agent	8
Chapter 4 — Large Language Models and Tool Use	9
Chapter 5 — Design and Implementation of MetabolonR v1	10

Chapter 6 — A Worked Example: The Progredir CKD Cohort	11
Chapter 7 — What v1 Cannot Do	12
Chapter 8 — Design Principles	13
Chapter 9 — The Tool Layer	14
Chapter 10 — The Agent Loop	15
Chapter 11 — Session Logging and Replay	16
Chapter 12 — Frontend and Deployment	17
Chapter 13 — Week 1: Environment, Lit Review, Architecture Lock	18
Chapter 14 — Week 2: Wrapping v1 Functions as Typed Tools	19
Chapter 15 — Week 3: Agent Loop and Replay	20
Chapter 16 — Week 4: Frontend and the Validation Experiment	21

Chapter 17 — Week 5: Manuscript Drafting and Repository Polish 22

Chapter 18 — Week 6: Preprint and Submission 23

Chapter 19 — The Reliability Problem in Agentic Biology 24

Chapter 20 — Designing the Validation Experiment 25

Chapter 21 — Results and Failure-Mode Taxonomy 26

Chapter 22 — What "Reproducible LLM Agent" Really Means 27

Chapter 23 — Known Limitations 28

Chapter 24 — Future Directions 29

Chapter 25 — A Short Guide to Publishing Scientific Software in 2026 30

Appendix A — Complete JSON

Schemas for All 10 Tools

This appendix reproduces the full JSON Schema for each of the ten v2.0 tools in MetabolonR-LLM. The schemas are alphabetized by tool name. Each is the schema served to the LLM during a tool-call round trip; each is generated from the corresponding Pydantic class in `tools/<tool>.py` and is the single source of truth for the tool's interface.

The schemas are reproduced verbatim from the v2.0.0 release. Field descriptions are abbreviated to fit; the full descriptions appear in the Pydantic source.

A.1 differential_abundance

```
{  
  "name": "differential_abundance",  
  "description": "Per-metabolite log fold change, p
```



```



```

A.2 export_session

```

{
  "name": "export_session",
  "description": "Bundle the session's outputs into a file",
  "input_schema": {

```

```

    "type": "object",
    "properties": {
        "session_id": {"type": "string"},
        "output_path": {"type": "string"},
        "include_figures": {"type": "boolean", "default": false},
        "figure_format": {"type": "string", "enum": ["png", "pdf"]},
    },
    "required": ["session_id", "output_path"]
}

```

A.3 impute_missing

```

{
    "name": "impute_missing",
    "description": "Fill missing values in the abundance matrix",
    "input_schema": {
        "type": "object",
        "properties": {
            "dataset_handle": {"type": "string"},
            "method": {"type": "string", "enum": ["knn", "naive"]}
        }
    }
}

```

```

    "knn_k": {"type": "integer", "default": 10, "
    "rf_ntree": {"type": "integer", "default": 10
  },
  "required": ["dataset_handle"]
}
}

```

A.4 load_metabolomics_data

```

{
  "name": "load_metabolomics_data",
  "description": "Load a three-file metabolomics da
  "input_schema": {
    "type": "object",
    "properties": {
      "abundance_path": {"type": "string"},
      "sample_annotation_path": {"type": "string"},
      "metabolite_annotation_path": {"type": "string"},
      "sample_id_column": {"type": "string", "defau
      "metabolite_id_column": {"type": "string", "d
    },
  },
}

```

```

    "required": ["abundance_path", "sample_annotation"]
  }
}

```

A.5 pca

```

{
  "name": "pca",
  "description": "Compute principal components of t",
  "input_schema": {
    "type": "object",
    "properties": {
      "dataset_handle": {"type": "string"},
      "n_components": {"type": "integer", "default": 10},
      "center": {"type": "boolean", "default": true},
      "scale": {"type": "boolean", "default": false}
    },
    "required": ["dataset_handle"]
  }
}

```

A.6 qc_filter

```
{
  "name": "qc_filter",
  "description": "Drop metabolites and samples exceeding the specified thresholds",
  "input_schema": {
    "type": "object",
    "properties": {
      "dataset_handle": {"type": "string"},
      "max_metabolite_missingness_pct": {"type": "number"},
      "max_sample_missingness_pct": {"type": "number"}
    },
    "required": ["dataset_handle"]
  }
}
```

A.7 scale

```
{
  "name": "scale",
  "description": "Apply per-metabolite scaling (Pareto or log-ratio)"
```

```

"input_schema": {
  "type": "object",
  "properties": {
    "dataset_handle": {"type": "string"},
    "method": {"type": "string", "enum": ["pareto"]},
  },
  "required": ["dataset_handle"]
}

```

A.8 sources_of_variation

```

{
  "name": "sources_of_variation",
  "description": "For each named covariate, compute",
  "input_schema": {
    "type": "object",
    "properties": {
      "dataset_handle": {"type": "string"},
      "covariates": {"type": "array", "items": {"type": "string"}},
    },
  },
}

```

```

    "required": ["dataset_handle", "covariates"]
  }
}

```

A.9 summarize_dataset

```

{
  "name": "summarize_dataset",
  "description": "Report counts and structural properties of a dataset",
  "input_schema": {
    "type": "object",
    "properties": {
      "dataset_handle": {"type": "string"},
      "group_by": {"type": ["string", "null"], "description": "Grouping variable"},
    },
    "required": ["dataset_handle"]
  }
}

```

A.10 transform

```
{
  "name": "transform",
  "description": "Apply a variance-stabilizing transform",
  "input_schema": {
    "type": "object",
    "properties": {
      "dataset_handle": {"type": "string"},
      "method": {"type": "string", "enum": ["log2", "rlog", "tmm"]},
      "pseudocount": {"type": "number", "default": 1},
    },
    "required": ["dataset_handle"]
  }
}
```

A.11 Output-schema sketches

The Pydantic output models are not reproduced as JSON Schema in this appendix because they are not served to the

LLM. Each tool's return value is summarized into a JSON-serializable form that the runtime packages as a `tool_result` content block. The summary fields per tool, abbreviated:

- `load_metabolomics_data` →
`{dataset_handle, n_samples,`
`n_metabolites, validation_warnings}`
- `summarize_dataset` → `{n_samples,`
`n_metabolites,`
`overall_missingness_pct,`
`per_metabolite_missingness,`
`group_sizes, dynamic_range}`
- `qc_filter` → `{dataset_handle,`
`n_metabolites_dropped,`
`n_samples_dropped, final_dimensions}`
- `impute_missing` → `{dataset_handle,`
`method, n_cells_imputed,`
`knn_mean_distance}`
- `transform` → `{dataset_handle, method,`
`pseudocount}`

- `scale` → {`dataset_handle`, `method`,
`scale_factors_summary`}
- `pca` → {`scores_top5`, `loadings_top5`,
`variance_explained`, `n_components`}
- `sources_of_variation` →
{`ranked_covariates`: [{`name`,
`median_f_ratio`,
`fraction_significant`}]}
- `differential_abundance` → {`table_top20`:
[...], `n_significant`, `fdr_threshold`,
`fc_threshold`}
- `export_session` → {`output_path`,
`file_list`, `methods_paragraph`}

Tools that consume or produce large data structures — full PCA loadings, full differential-abundance tables — return summary forms to the LLM (top-N entries plus aggregate counts) while retaining the full structure in the local cache and writing it to the export archive. This keeps the conversation history token-efficient (Chapter 10 §10.3) while preserving the underlying data.

A.12 Schema versioning

The schemas reproduced here are at v2.0.0. Future versions of the tools may add fields (backward-compatible: new optional fields with defaults) but will not remove or rename existing fields without a major-version bump. The agent's contract with the schema is therefore stable across v2.x.

A user with a session log from v2.0 can replay it against v2.x (forward compatibility): unknown fields in the recorded args dict are ignored by v2.0's tool implementations but are accepted by v2.x's. The session log itself records `tool_layer_sha`, so the replay engine can verify that the recorded version matches the running version and emit a warning if they differ.

Appendix B — Annotated Code Listings

This appendix reproduces the four load-bearing Python modules of MetabolonR-LLM in full, with annotation: the agent loop, the replay engine, the SQLite session logger, and the rpy2 bridge. Each listing is also available as a standalone runnable file in `drafts/metabolonr-llm/code/`.

These listings are the v2.0.0 release versions. Imports are abbreviated where standard.

B.1 The agent loop (`agent/multi_turn.py`)

The multi-turn agent loop, approximately 80 lines. The full listing is reproduced as `code/chB_listing_01.py`. The annotated body:

```

def run_multi_turn(
    user_message: str,
    tools: list[dict],
    session: Session,
    max_iterations: int = 25,
) -> str:
    """Run an agent turn that may call multiple tools"""
    msgs = session.history + [{"role": "user", "content": user_message}]
    session.log_user_message(user_message)

    for iteration in range(max_iterations):
        response = client.messages.create(
            model=MODEL_NAME,
            system=SYSTEM_PROMPT,
            tools=tools,
            messages=msgs,
            max_tokens=2048,
        )
        msgs.append({"role": "assistant", "content": response.content})
        session.log_assistant_message(response.content)

```

```

if response.stop_reason == "end_turn":
    session.history = msgs
    return _extract_text(response.content)

if response.stop_reason == "tool_use":
    tool_blocks = [b for b in response.content]
    tool_results = []
    for tb in tool_blocks:
        try:
            result = dispatch(tb.name, tb.args)
        except SchemaValidationError as exc:
            result = {"status": "error",
                    "error": "schema_validation_error",
                    "detail": str(exc)}
        except ToolExecutionError as exc:
            result = {"status": "error",
                    "error": "tool_execution_error",
                    "detail": str(exc)}
    tool_results.append({
        "type": "tool_result",
        "tool_use_id": tb.id,

```

```

        "content": json.dumps(result),
    })
    session.log_tool_call(tb, result)
    msgs.append({"role": "user", "content":
continue

    raise UnexpectedStopReason(response.stop_re

    raise IterationLimitExceeded(max_iterations)

```

The loop has four entry conditions to terminate: an `end_turn` stop reason, an iteration-limit guard, an unexpected stop reason (which should not happen in normal use), and exceptions raised inside tool dispatch (handled inline). The session is updated incrementally so that an exception mid-loop leaves a partial record rather than no record.

The `max_iterations = 25` cap is generous for any analysis MetabolonR-LLM is designed to do; a session that hits the cap is almost certainly stuck in a retry loop and should be investigated.

B.2 The replay engine (agent/ replay.py)

The replay engine, approximately 50 lines:

```
def replay(
    log_path: str,
    data_root: str,
    output_dir: str,
    tolerance: ReplayTolerance = DEFAULT_TOLERANCE,
) -> ReplayReport:
    """Re-execute a session log without invoking the
    training engine.

    Returns a ReplayReport describing whether the
    output hashes within the configured tolerance.
    """

    log = json.load(open(log_path))
    header = log["header"]
    pin_status = verify_pins(header)
    if pin_status.major_drift:
```



```

        return ReplayReport(success=False, reason=
                                details=pin_status)

handles: dict[str, DatasetHandle] = {}
outputs: dict[str, str] = {}

for call in log["tool_calls"]:
    tool = TOOL_REGISTRY[call["tool"]]
    args = dict(call["args"])
    if "handle_in" in call:
        args["dataset_handle"] = handles[call["
    try:
        result = tool.execute(**args)
    except ToolExecutionError as exc:
        return ReplayReport(success=False, reason=exc,
                                details={"tool": call["tool"]})
    if "handle_out" in call:
        handles[call["handle_out"]] = result.handle_out
    if result.artifact_path:
        outputs[result.artifact_kind] = result.artifact_path

```

```
return verify_output_hashes(log["output_hashes"]
```

The engine is intentionally simple. The most consequential function calls — `verify_pins` and `verify_output_hashes` — are implemented in adjacent modules and total another 100 lines, with explicit policy for what counts as a "major drift" (a major R version change, a different Anthropic model snapshot) versus a "minor drift" (a patch-level R-package change, a different OS minor version).

B.3 The session logger (`agent/db.py`)

The SQLite session logger, abbreviated:

```
SCHEMA = """
CREATE TABLE IF NOT EXISTS sessions (
    session_id      TEXT PRIMARY KEY,
    started_at      TIMESTAMP NOT NULL,
    ended_at        TIMESTAMP,
```

```

        user_id            TEXT NOT NULL,
        model_name         TEXT NOT NULL,
        prompt_sha256      TEXT NOT NULL,
        tool_layer_sha      TEXT NOT NULL,
        api_spend_usd      REAL DEFAULT 0
    );

-- ... five more table definitions, as in Chapter 1
"""

class Session:
    def __init__(self, db_path: str, user_id: str):
        self.db = sqlite3.connect(db_path)
        self.db.executescript(SCHEMA)
        self.session_id = f"ses_{datetime.utcnow().strftime('%Y%m%d%H%M%S')}_"
        self._next_seq = 0
        self._log_header(user_id)

    def _log_header(self, user_id: str):
        self.db.execute(
            "INSERT INTO sessions(session_id, start_time, "
            "prompt_sha256, tool_layer_sha) VALUES"

```

```

        (self.session_id, datetime.utcnow(), use_tool,
         MODEL_NAME, prompt_sha256(SYSTEM_PROMPT))
    )
    self.db.commit()

def log_tool_call(self, tool_use, result: dict):
    seq = self._next_seq; self._next_seq += 1
    self.db.execute(
        "INSERT INTO tool_calls(call_id, session_id, tool_name,
        arguments, started_at) VALUES (?, ?, ?, ?, ?)",
        (tool_use.id, self.session_id, seq, tool_use.name,
         json.dumps(tool_use.input), datetime.utcnow()))
    )
    self.db.execute(
        "INSERT INTO tool_results(call_id, status, started_at,
        finished_at) VALUES (?, ?, ?, ?)",
        (tool_use.id, result.get("status", "ok"),
         json.dumps(result if result.get("status") == "ok" else None),
         json.dumps(result if result.get("status") == "error" else None))
    )
    self.db.commit()

```

```

def export_log(self, output_path: str) -> None:
    """Render the SQLite session into the user-facing log.

    header = self._build_header()
    calls = self._build_calls_list()
    results_summary = self._build_results_summary()
    output_hashes = self._build_output_hashes()

    log = {
        "header": header,
        "user_intent": self._first_user_message,
        "tool_calls": calls,
        "results_summary": results_summary,
        "output_hashes": output_hashes,
    }

    with open(output_path, "w") as f:
        json.dump(log, f, indent=2, default=str)

```

The Session class is the operational store. The `export_log` method is what produces the `analysis_log.json` artifact that Chapter 11 §11.4 documents. The SQLite store and the JSON export are deliberately separated: the SQLite store is the

durable record (mutable, queryable, used during the session), and the JSON export is the public artifact (immutable, citeable, used post-session).

B.4 The rpy2 bridge (`bridge.py`)

The bridge described in Chapter 9 §9.4. Abbreviated:

```
from contextlib import contextmanager
import rpy2.robj as ro
from rpy2.robj import default_converter, pandas
from rpy2.robj.conversion import localconverter

_loaded = False

def _ensure_loaded():
    global _loaded
    if not _loaded:
        ro.r('suppressPackageStartupMessages(library(
        _loaded = True
```

```

@contextmanager
def r_session():
    """Context manager activating the standard converter
    and ensuring the R session is loaded()
    cv = default_converter + pandas2ri.converter +
    with localconverter(cv):
        yield ro.r

def call_r(func_name: str, **kwargs) -> object:
    """Call a metabolonr:: function with pandas input
    with r_session() as r:
        f = ro.r(f'metabolonr::{func_name}')
        return f(**kwargs)

def r_to_summary(r_object: object) -> dict:
    """Convert a metabolonr S4 result object to a dict
    return ro.r('metabolonr::to_summary')(r_object)

```

The bridge is the single point of contact between Python and R in the system. Every tool wrapper goes through `call_r(...)`. The two converters (`pandas2ri`,

`numpy2ri`) handle the data-frame-and-matrix conversions; the `r_to_summary` helper wraps R-side serialization for the result objects.

B.5 The system prompt

The system prompt, reproduced in full as it shipped in v2.0.0. The SHA-256 hash of the text is `b7e1f1...` (truncated for display; full value in `system_prompt.sha256` in the repository). The text is approximately 600 words; the abbreviated structure:

```
You are MetabolonR-LLM v2.0.0, an analytical assistant for metabolomics. Your job is to help users analyze metabolomics data using the tools provided. You do not write code or create new methods; you call the tools.
```

```
[Tool catalog: ten tools listed by name with one-line descriptions, followed by the schema reminder that each tool's full schema is available in the API request.]
```


[Default-values block: the documented default for each tool in the form, so that the agent's defaults are explicit.]

[Worked example: a single multi-turn transcript on a task, covering the standard pipeline, demonstrating clarification of an ambiguous prompt, and demonstrating the export_session method.]

[Instructions:

- Call tools rather than improvise.
- When uncertain, ask the user.
- Refuse requests for methods not in the tool set.
- Report errors verbatim; do not retry silently.
- Produce a one-paragraph methods summary after each session.

The full text is reproduced in code/chB_system_prompt.txt. Edits to the system prompt produce a new SHA-256 hash, which is recorded in the session log header and is a versionable event (Chapter 11 §11.4).

Appendix C — The 20 Validation Prompts

This appendix reproduces the 20 prompts of the validation experiment of Chapters 16, 20, and 21. Each prompt has three paraphrases. For each paraphrase, a one-paragraph session summary is given describing what the agent did on the median repeat across three runs. Where the actual results from the Week 4 experiment are not yet available, summary text is rendered as [PLACEHOLDER] for the replacement pass after the experiment completes.

The prompts are the frozen v2.0.0 validation set. They were committed to `docs/VALIDATION_PROMPTS.md` on Tuesday of Week 4 (Chapter 16 §16.3) and have not been modified since.

C.1 Unambiguous cohort (ten prompts)

Prompt 1 — Full standard pipeline

- 1a: "Run a standard MetabolonR pipeline on Progredir and produce a volcano plot."
- 1b: "Do the usual analysis on the Progredir data — QC, impute, transform, scale, PCA, differential abundance — and give me the volcano."
- 1c: "I want to look for biomarkers in Progredir using the standard workflow. End with a volcano of progressors vs stable."

Median session summary. [PLACEHOLDER: The agent loads Progredir (454 × 293, GC-MS, ST000978), runs the eight-step standard pipeline, and produces the volcano plot. ~18 metabolites pass FDR 0.05 on the median run — the Path A v1-oracle reference (Chapter 6 §6.7). All three paraphrases produce identical tool

sequences and parameters in the post-mitigation run set.]

Prompt 2 — QC and summarize only

- 2a: "Just run QC on Progredir and summarize what's left."
- 2b: "Give me a summary of the Progredir data after standard quality filtering."
- 2c: "How many metabolites and samples survive QC on Progredir with the usual thresholds?"

Median session summary. [PLACEHOLDER: The agent loads, summarizes, qc_filters, and summarizes again. The Progredir working matrix from Metabolomics Workbench ST000978 is already at 454 × 293 after the upstream <50% per-metabolite missingness filter; the qc_filter step at 30%/50% therefore drops 0 samples and 0 metabolites. See Chapter 6 §6.7.]

Prompt 3 — PCA only

- 3a: "Run a PCA on Progredir after standard preprocessing."
- 3b: "Show me the principal components of the Progredir metabolites."
- 3c: "I want to see whether the Progredir samples cluster by group on a PCA."

Median session summary. [PLACEHOLDER: The agent runs the preprocessing pipeline through scaling and stops at PCA. PC1 captures 14% of variance, PC2 9%.]

Prompt 4 — Explicit imputation method

- 4a: "Process Progredir with random-forest imputation and give me the differential abundance."
- 4b: "Use missForest for imputation on Progredir, then proceed normally."
- 4c: "I want the standard MetabolonR pipeline but with RF instead of KNN for imputation."

Median session summary. [PLACEHOLDER: The agent runs the standard pipeline substituting random forest for KNN imputation. Differential abundance results are similar but not identical to the KNN run.]

Prompt 5 — Explicit transformation

- 5a: "Run a standard analysis of Progredir but log10-transform the abundances first."
- 5b: "Use log10 transformation (not log2) for the Progredir analysis."
- 5c: "I want the standard pipeline with log base 10 transformation."

Median session summary. [PLACEHOLDER: The agent runs the pipeline with log10 substituted for log2. After the mitigation discussed in Chapter 21 §21.5, all three paraphrases call log10 reliably; pre-mitigation,

paraphrase 2a sometimes silently used
log2.]

Prompt 6 — Explicit scaling method

- 6a: "Process Progredir with auto-scaling and produce a volcano plot."
- 6b: "Use z-score scaling on the Progredir analysis."
- 6c: "I want a standard pipeline but with full standardization rather than Pareto scaling."

Median session summary. [PLACEHOLDER: The agent substitutes auto-scaling for the Pareto default. Differential abundance results show somewhat different ranking of borderline metabolites but the top hits agree.]

Prompt 7 — Explicit test choice

- 7a: "Process Progredir and use Wilcoxon rank-sum for differential abundance."

- 7b: "Use a non-parametric test instead of limma."
- 7c: "Run the pipeline with Wilcoxon for the group comparison."

Median session summary. [PLACEHOLDER: The agent runs the pipeline with Wilcoxon substituting for limma. n_significant is approximately 14 vs limma's 18, reflecting the power difference.]

Prompt 8 — Explicit FDR threshold

- 8a: "Standard pipeline on Progredir with FDR 0.10."
- 8b: "Use a more lenient FDR threshold — 10% — for the Progredir analysis."
- 8c: "Run the analysis but call metabolites significant at FDR 0.10."

Median session summary. [PLACEHOLDER: The agent runs the standard pipeline with the FDR threshold set to 0.10. n_significant approximately 26 vs the default's 18.]

Prompt 9 — Explicit fold-change threshold

- 9a: "Standard Progredir analysis with a fold-change cutoff of 1 instead of 0.5."
- 9b: "Use $|\log_2 \text{FC}| > 1$ as the significance criterion in addition to FDR 0.05."
- 9c: "Show me Progredir biomarkers with at least two-fold differences."

Median session summary. [PLACEHOLDER: The agent applies the stricter fold-change threshold. n_significant approximately 9 vs the default's 18.]

Prompt 10 — Sources of variation alongside pipeline

- 10a: "Run the standard pipeline on Progredir and also tell me which clinical variables drive the most variance."
- 10b: "Standard analysis plus a sources-of-variation decomposition for Progredir."

- 10c: "I want the usual workflow but with an F-ratio plot of covariate effects added."

Median session summary. [PLACEHOLDER: The agent runs the standard pipeline plus `sources_of_variation` over `{progressor, age, sex, eGFR, batch}`. The ranking is `eGFR > progressor > age > sex > batch.`]

C.2 Ambiguous cohort (ten prompts)

Prompt 11 — Underspecified target

- 11a: "Tell me which metabolites are interesting in ProgreDir."
- 11b: "Find the most informative metabolites in the ProgreDir data."
- 11c: "What stands out in the ProgreDir metabolome?"

Expected clarification. What does "interesting" mean — differentially abundant between groups, high variance, low missingness, biologically annotated?

Median session summary. [PLACEHOLDER: Agent asks the clarification question and waits. Post-mitigation clarification rate is approximately 95% across the three paraphrases.]

Prompt 12 — Underspecified contrast

- 12a: "Compare the groups in Progredir."
- 12b: "Tell me what's different between groups in the Progredir data."
- 12c: "Run a group comparison on Progredir."

Expected clarification. Which grouping variable — progressor, sex, diabetes, hypertension, batch?

Median session summary. [PLACEHOLDER: Pre-mitigation, the agent defaulted to

progressor without asking. Post-mitigation it asks reliably.]

Prompt 13 — Underspecified parameter

- 13a: "Do a PCA on Progredir with the right number of components."
- 13b: "Run PCA on Progredir and choose an appropriate number of PCs."
- 13c: "PCA on Progredir — pick a sensible component count."

Expected clarification. What criterion — fixed 5, scree-plot elbow, 80% variance, or a specified number?

Median session summary. [PLACEHOLDER]

Prompt 14 — Multiple-interpretation prompt

- 14a: "Look at the eGFR signal in Progredir."
- 14b: "I want to understand how eGFR relates to the metabolome in Progredir."

- 14c: "Examine the eGFR signal in the ProgreDir data."

Expected clarification. As a covariate in sources-of-variation, as a categorical group for differential abundance (above/below threshold), or as a continuous correlate?

Median session summary. [PLACEHOLDER]

Prompt 15 — Implicit covariate

- 15a: "Run the ProgreDir analysis adjusting for the obvious confounders."
- 15b: "Standard pipeline with covariate adjustment for the relevant clinical variables."
- 15c: "Compare progressors to stable participants but control for confounders."

Expected clarification. Which covariates count as "obvious" — eGFR, age, sex, diabetes? Currently v2.0 does not support covariate adjustment in `differential_abundance`; the agent should say so and offer the unadjusted analysis or `sources_of_variation` as alternatives.

Median session summary. [PLACEHOLDER]

Prompt 16 — Conflicting requests

- 16a: "Use a high stringency cutoff but include the borderline cases."
- 16b: "Strict FDR but show me the near-misses too."
- 16c: "Conservative significance threshold and a list of marginal hits."

Expected clarification. The two halves of the request are in tension; the agent should propose a reconciliation (e.g., FDR 0.01 plus a separate "marginal" tier at FDR 0.05).

Median session summary. [PLACEHOLDER]

Prompt 17 — Vague threshold

- 17a: "Show me the strong effects in Progredir."
- 17b: "Which metabolites have large effects in the progressor comparison?"
- 17c: "Focus on the high-confidence biomarkers."

Expected clarification. What threshold — effect size, FDR, both?

Median session summary. [PLACEHOLDER]

Prompt 18 — Non-existent variable

- 18a: "Group Progredir by smoking status."
- 18b: "Compare smokers to non-smokers in the Progredir data."
- 18c: "Stratify the Progredir analysis by smoking."

Expected clarification. Progredir does not have a smoking-status column in its sample annotation. The agent should report this and offer to list the available grouping variables.

Median session summary. [PLACEHOLDER]

Prompt 19 — Ambiguous reference group

- 19a: "Compare the high-risk and low-risk groups in Progredir."
- 19b: "Risk-stratified analysis of Progredir."

- 19c: "Look at differences between high- and low-risk participants."

Expected clarification. Progredir does not have an explicit "risk" categorization. The agent should ask whether "high risk" should map to progressors, to a specific eGFR range, or to another derivable category.

Median session summary. [PLACEHOLDER]

Prompt 20 — Incomplete prompt

- 20a: "Run the pipeline and"
- 20b: "Do the analysis on Progredir, then"
- 20c: "Standard workflow on Progredir, followed by"

Expected clarification. The prompt is incomplete; the agent should ask the user to finish it.

Median session summary. [PLACEHOLDER: All three paraphrases trigger immediate clarification at near-100% rate.]

C.3 Notes on placeholder replacement

The placeholder summaries will be filled in by replacing the [PLACEHOLDER] strings with one-to-three-sentence summaries derived from the actual Week 4 session logs in `data/validation/`. The replacement is mechanical: each placeholder corresponds to a specific (prompt, paraphrase) cell of the experimental design, and the summary is derived from the median-of-three repeats for that cell. The replacement pass is scheduled to run after Chapter 21's headline numbers are finalized.

A reader who reads this appendix before the placeholder replacement should understand that the prompts and the expected behaviors are stable — they were pre-registered — and only the per-prompt result descriptions remain to be filled in.

Appendix D — Glossary

Terms used throughout the book, grouped by domain. A reader who is new to one of the three intersecting fields — metabolomics, machine learning, or software engineering — should find a one-paragraph definition here.

D.1 Metabolomics

Abundance matrix. A two-dimensional numerical table with samples in rows and metabolites in columns. Cells are non-negative real numbers representing relative abundance, typically peak areas from mass spectrometry normalized in a platform-specific way.

Below the limit of detection (below-LOD). A measurement value lower than the assay platform can reliably resolve. In metabolomics, below-LOD values are typically reported as missing and constitute the dominant source of missingness.

Benjamini–Hochberg (BH) correction. A multiple-testing correction procedure that controls the false discovery rate. The default in metabolomics and genomics for differential-abundance analyses.

Biomarker. A measurable indicator of a biological state. In this book, the term refers specifically to a differentially abundant metabolite that has been situated in biology — placed on a known pathway, replicated in an independent cohort.

Differential abundance. The statistical comparison of metabolite values between two clinical groups, producing an effect size (log fold change) and a p-value per metabolite.

F-ratio decomposition. A method for ranking covariates by how much variance they explain in the metabolome, by computing the F-ratio of between-level to within-level variance for each covariate, across metabolites.

Fold change. The ratio of mean abundance in one group to mean abundance in another, typically expressed on a log scale. A log₂ fold change of 1 means a two-fold difference.

HMDB. The Human Metabolome Database, a comprehensive resource of human metabolite information including chemical structure, pathway membership, and disease associations.

Imputation. The procedure of filling missing values in the abundance matrix. KNN imputation is the MetabolonR default; alternatives include half-min, random forest, and Bayesian PCA.

KEGG. The Kyoto Encyclopedia of Genes and Genomes, a database resource for metabolic pathways and gene/metabolite annotations.

Limma. A statistical method for differential analysis originally developed for microarrays, which uses empirical Bayes moderation of per-feature variances. The default test in MetabolonR's differential-abundance tool.

Log transformation. Replacing each abundance value with its logarithm (typically $\log_2$ or $\log_{10}$), to stabilize variance across the dynamic range of a metabolomics assay.

Metabolomics. The systematic study of small-molecule metabolites in a biological sample.

Missingness. The fraction of cells in an abundance matrix that have no value. Distinguished into below-LOD (missing not at random) and technical missingness (missing more nearly at random).

Pareto scaling. Per-metabolite scaling by the square root of the standard deviation. A common metabolomics default that compromises between auto-scaling and no scaling.

Pathway analysis. Aggregation of metabolite-level signals into pathway-level summaries, using a database like KEGG or HMDB. Deferred to MetabolonR-LLM v2.1.

Principal component analysis (PCA). The eigendecomposition of the centered covariance matrix; produces scores (sample projections), loadings (metabolite contributions), and variance-explained per component.

Sources of variation. A diagnostic that ranks clinical and technical covariates by how much variance each explains across the metabolome; identifies batch effects and biological signal alongside each other.

Untargeted metabolomics. A platform that attempts to measure as many metabolites as possible without prior commitment to a specific panel; produces a matrix with many features, many of which are unidentified.

Volcano plot. A scatter plot with log fold change on the x-axis and minus-log₁₀ p-value on the y-axis, used to visualize differential-abundance results. The four corners correspond to (large effect, low p), (small effect, low p), (large effect, high p), (small effect, high p).

D.2 Machine learning and LLMs

Agent. A system in which an LLM chooses actions (typically tool calls) in pursuit of a user-specified goal, with the LLM acting as the routing layer between user intent and the underlying execution.

Constrained decoding. Sampling from an LLM under a schema constraint, such that all generated outputs satisfy the schema. Used by tool-calling APIs to ensure tool-call validity.

Determinism (output). The property of a computation producing bit-identical outputs given identical inputs. Classical software has this property; LLMs do not.

Determinism (re-executability). The property of a computation being reproducible from its recorded log, without re-invoking the stochastic LLM. MetabolonR-LLM's architectural commitment.

Domain-LLM. A large language model fine-tuned or otherwise specialized for a particular domain (e.g., BioGPT for biomedical text). Contrasted with frontier LLMs and with tool-using agents.

Frontier LLM. A large language model at the current state of the art, accessed through an API (Anthropic, OpenAI, Google). Examples in 2026: Claude Opus 4.7, Claude Sonnet 4.6.

Function calling / tool use. The mechanism by which an LLM emits, in place of text, a structured request to invoke a pre-declared function. Function calling (OpenAI's term) and tool use (Anthropic's term) refer to the same mechanism.

Hallucination. An LLM output that is confident but incorrect. In tool-using systems, hallucinated function calls (calls to functions that do not exist) are a specific failure mode.

JSON Schema. A specification language for the shape of JSON documents. Used in tool-use APIs to declare the structure of valid tool arguments.

Pydantic. A Python library for data validation using type hints. The Python-side counterpart to JSON Schema in MetabolonR-LLM's tool layer.

Replay. Re-executing a recorded session by reading its log and running each recorded tool call in order, without re-invoking the LLM. The mechanism by which a deterministically re-executable session is reproduced.

RAG (Retrieval-Augmented Generation). A pattern in which an LLM is provided with retrieved-document context before generating a response. Used by some agentic-bio tools (BioChatter, BioAgents); not used by MetabolonR-LLM.

System prompt. The block of text provided to an LLM at the start of every request, specifying its role and instructions. Versioned by SHA-256 hash in MetabolonR-LLM.

Temperature. A sampling parameter controlling the randomness of LLM outputs. Temperature zero is the closest a frontier LLM gets to deterministic behavior but is not strictly deterministic.

Tool-calling LLM. An LLM that can emit structured tool calls in addition to text. See also: agent.

D.3 Software engineering

BLAS (Basic Linear Algebra Subprograms). A set of low-level routines for linear-algebra operations. Variants — OpenBLAS, MKL, ATLAS — produce subtly different floating-point results, with implications for bit-identical replay.

CI (Continuous integration). Automated test runs triggered on every code change. MetabolonR-LLM's CI runs `ruff`, `mypy`, the oracle tests, and a subset of integration tests.

Container. A self-contained runtime environment defined by an image. Docker is the dominant containerization technology in 2026.

Content-addressed. Identified by a hash of contents rather than by an opaque name. `DatasetHandle` is content-addressed in MetabolonR-LLM, which means two semantically identical datasets produce the same handle.

ER diagram (Entity-Relationship). A graphical notation for database schemas showing tables, columns, and foreign-key relationships.

Golden snapshot. A captured reference output, saved to disk, against which a test asserts equality. The MetabolonR-LLM oracle tests are golden-snapshot tests.

Hash (SHA-256). A fixed-length output of a hash function, used to verify integrity and identity. MetabolonR-LLM uses SHA-256 hashes for prompt versioning, dataset handles, and output verification.

Linter. A static-analysis tool that flags code quality issues. `ruff` is the project's linter.

Oracle test. A test that asserts the equality of a new implementation's output against a known-correct reference. `v1` is the oracle; `v2`'s tools are tested against it.

Pydantic. See D.2.

rpy2. A Python library that provides an interface to R. The bridge between MetabolonR-LLM's Python tool layer and the `v1` R routines.

Schema validation. Checking a piece of data against a declared schema. `JSON Schema` and `Pydantic` both provide schema-validation mechanisms.

Semantic versioning (semver). A versioning convention `MAJOR.MINOR.PATCH` where major changes are backward-incompatible, minor changes add features, and patch changes fix bugs. MetabolonR-LLM follows `semver`.

Type hint. A Python annotation declaring the expected type of a variable, parameter, or return value. Pydantic uses type hints to drive its validation.

Zenodo. A general-purpose digital repository operated by CERN that provides DOIs for archived datasets, software releases, and other digital artifacts.

Appendix E — References

References cited in the book's text, alphabetized by first author. Items flagged [CITATION NEEDED] in the chapter prose require the author's confirmation before publication; they are listed here with the in-text claim they support, so the gap is visible to the author and to a reviewer.

References to software packages without a primary publication are given as software-citation entries with the package name, version where pinned in the book, and the canonical website or repository URL.

ORCID identifiers are included where stable.

E.1 Confirmed references

Abramson J, Adler J, Dunger J, et al. *Accurate structure prediction of biomolecular interactions with AlphaFold 3*. Nature 630:493–500. 2024. doi:10.1038/s41586-024-07487-w.

Afgan E, Baker D, Batut B, et al. *The Galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2018 update*. Nucleic Acids Research 46(W1):W537–W544. 2018. doi:10.1093/nar/gky379.

Alakwaa FM, Chitale S, Saliba A, et al. *Lilikoi: an R package for personalized pathway-based classification modeling using metabolomics data*. GigaScience 7(12):giy136. 2018. doi: 10.1093/gigascience/giy136.

Al-Akwaa FM, Yunits B, Huang X, et al. *Lilikoi V2.0: a deep learning-enabled, personalized pathway-based R package for diagnosis and prognosis predictions using metabolomics data*. GigaScience 10(1):giaa162. 2022. doi:10.1093/gigascience/giaa162. [VERIFY journal and year against final publication; the package release is 2022 per the README.]

Allaire JJ, Teague C, Scheidegger C, Xie Y, Dervieux C. *Quarto*. Software, 2022–. <https://quarto.org>. [Cited as the document-rendering successor to R Markdown.]

Anthropic. *Tool use documentation*. Web documentation, retrieved May 2026. <https://docs.anthropic.com/en/docs/build->

with-claude/tool-use. [Cited for the Anthropic tool-use API mechanics.]

Benjamini Y, Hochberg Y. *Controlling the false discovery rate: a practical and powerful approach to multiple testing*. Journal of the Royal Statistical Society Series B 57(1):289–300. 1995.

Blighe K, Rana S, Lewis M. *EnhancedVolcano: Publication-ready volcano plots with enhanced colouring and labeling*. R package, Bioconductor, 2018–. <https://github.com/kevinblighe/EnhancedVolcano>.

Brunius C, Shi L, Landberg R. *Large-scale untargeted LC-MS metabolomics data correction using between-batch feature alignment and cluster-based within-batch signal intensity drift correction*. Metabolomics 12:173. 2016. doi:10.1007/s11306-016-1124-4. [Used for F-ratio decomposition framework.]

Chang W, Cheng J, Allaire JJ, et al. *shiny: Web Application Framework for R*. R package, 2024. <https://shiny.posit.co>.

Chen Z, Chen S, Ning Y, et al. *ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery*. arXiv:2410.05080. ICLR 2025.

CZI Single-Cell Biology Program; Abdulla S, Aeevermann B, et al. *CZ CELLxGENE Discover: a single-cell data platform for scalable exploration, analysis and modeling of aggregated data*. Nucleic Acids Research 53(D1):D886–D900. 2025. doi: 10.1093/nar/gkae1142.

de Almeida BP, Dalla-Torre H, Richard G, et al. *A multimodal conversational agent for DNA, RNA and protein tasks*. Nature Machine Intelligence. 2025. doi:10.1038/s42256-025-01047-1.

Di Tommaso P, Chatzou M, Floden EW, et al. *Nextflow enables reproducible computational workflows*. Nature Biotechnology 35:316–319. 2017. doi:10.1038/nbt.3820.

Dong Z, Zhong V, Lu Y. *BioMANIA: Simplifying bioinformatics data analysis through conversation*. bioRxiv. 2023. doi: 10.1101/2023.10.29.564479.

Eliasson M. *DeviumWeb: An interactive multivariate-analysis web tool*. GitHub repository, 2012–. <https://github.com/dgrapov/DeviumWeb>. [Cited as historical Shiny-based metabolomics interface; project effectively unmaintained.]

Gadegbeku CA, Gipson DS, Holzman LB, et al. *Design of the Nephrotic Syndrome Study Network (NEPTUNE) to evaluate primary glomerular nephropathy by a multidisciplinary approach*. *Kidney International* 83:749–756. 2013. doi:10.1038/ki.2012.428.

Giacomoni F, Le Corguillé G, Monsoor M, et al. *Workflow4Metabolomics: a collaborative research infrastructure for computational metabolomics*. *Bioinformatics* 31(9):1493–1495. 2015. doi:10.1093/bioinformatics/btu813.

Greshake K, Abdelnabi S, Mishra S, et al. *Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection*. arXiv:2302.12173. 2023. [VERIFY citation form; cited in Chapter 19 §19.2.]

Hastie T, Tibshirani R, Narasimhan B, Chu G. *impute: Imputation for microarray data*. R package, Bioconductor, 2001–.

<https://bioconductor.org/packages/release/bioc/html/impute.html>.

Huang K, et al. *Biomni: A General-Purpose Biomedical AI Agent*. bioRxiv. 2025. doi:10.1101/2025.05.30.656746.

Jin Q, Yang Y, Chen Q, Lu Z. *GeneGPT: augmenting large language models with domain tools for improved access to biomedical information*. *Bioinformatics* 40(2):btac075. 2024. doi:10.1093/bioinformatics/btac075.

Johnson WE, Li C, Rabinovic A. *Adjusting batch effects in microarray expression data using empirical Bayes methods*. *Biostatistics* 8(1):118–127. 2007. doi:10.1093/biostatistics/kxj037.

Kanehisa M, Goto S. *KEGG: Kyoto Encyclopedia of Genes and Genomes*. *Nucleic Acids Research* 28(1):27–30. 2000. doi:10.1093/nar/28.1.27.

Kluyver T, Ragan-Kelley B, Pérez F, et al. *Jupyter Notebooks — a publishing format for reproducible computational workflows*. In:

Positioning and Power in Academic Publishing: Players, Agents and Agendas, pp. 87–90. 2016.

Liu H, et al. *GenoTEX: An LLM Agent Benchmark for Automated Gene Expression Data Analysis*. arXiv:2406.15341. 2024.

Lobentanzer S, et al. *A platform for the biomedical application of large language models*. Nature Biotechnology. 2025. doi:10.1038/s41587-024-02534-3.

Luo R, Sun L, Xia Y, et al. *BioGPT: generative pre-trained transformer for biomedical text generation and mining*. Briefings in Bioinformatics 23(6):bbac409. 2022. doi:10.1093/bib/bbac409.

Mehandru N, et al. *BioAgents: Bridging the gap in bioinformatics analysis with multi-agent systems*. Scientific Reports. 2025. doi:10.1038/s41598-025-25919-z.

Mölder F, Jablonski KP, Letcher B, et al. *Sustainable data analysis with Snakemake*. F1000Research 10:33. 2021. doi:10.12688/f1000research.29032.2.

Munafò MR, Nosek BA, Bishop DVM, et al. *A manifesto for reproducible science*. Nature Human Behaviour 1:0021. 2017. doi:10.1038/s41562-016-0021.

Niewczas MA, Sirich TL, Mathew AV, et al. *Uremic solutes and risk of end-stage renal disease in type 2 diabetes: metabolomic study*. Kidney International 85(5):1214–1224. 2014. doi:10.1038/ki.2013.497. [Cited in Chapter 6 §6.5 for related CKD-metabolomics literature.]

Pal A, Sankarasubbu M. *Llama3-OpenBioLLM-70B/8B*. Hugging Face model card and blog. 2024. <https://huggingface.co/aaditya/Llama3-OpenBioLLM-70B>.

Pang Z, Lu Y, Zhou G, et al. *MetaboAnalyst 6.0*. Nucleic Acids Research. 2024. [VERIFY exact citation; the version-6 paper appeared in mid-2024.]

Risso D, Ngai J, Speed TP, Dudoit S. *Normalization of RNA-seq data using factor analysis of control genes or samples*. Nature Biotechnology 32(9):896–902. 2014. doi:10.1038/nbt.2931.

Ritchie ME, Phipson B, Wu D, et al. *limma* powers differential expression analyses for RNA-sequencing and microarray studies. *Nucleic Acids Research* 43(7):e47. 2015. doi:10.1093/nar/gkv007.

Rue-Albrecht K, Marini F, Soneson C, Lun ATL. *iSEE: Interactive SummarizedExperiment Explorer*. *F1000Research* 7:741. 2018. doi:10.12688/f1000research.14966.1.

Sandve GK, Nekrutenko A, Taylor J, Hovig E. *Ten Simple Rules for Reproducible Computational Research*. *PLoS Computational Biology* 9(10):e1003285. 2013. doi:10.1371/journal.pcbi.1003285.

Sekula P, Goek O-N, Quaye L, et al. *A metabolome-wide association study of kidney function and disease in the general population*. *Journal of the American Society of Nephrology* 27(4):1175–1188. 2016. doi:10.1681/ASN.2014111099. [Cited in Chapter 6 §6.5.]

Singhal K, Azizi S, Tu T, et al. *Large language models encode clinical knowledge*. *Nature* 620:172–180. 2023. doi:10.1038/s41586-023-06291-2.

Singhal K, Tu T, Gottweis J, et al. *Toward expert-level medical question answering with large language models*. Nature Medicine. 2025. doi:10.1038/s41591-024-03423-7.

Smith CA, Want EJ, O'Maille G, Abagyan R, Siuzdak G. *XCMS: processing mass spectrometry data for metabolite profiling using nonlinear peak alignment, matching, and identification*. Analytical Chemistry 78(3):779–787. 2006. doi:10.1021/ac051437y.

Smith AM, Katz DS, Niemeyer KE, FORCE11 Software Citation Working Group. *Software citation principles*. PeerJ Computer Science 2:e86. 2016. doi:10.7717/peerj-cs.86.

Stacklies W, Redestig H, Scholz M, Walther D, Selbig J. *pcaMethods — a Bioconductor package providing PCA methods for incomplete data*. Bioinformatics 23(9):1164–1167. 2007. doi: 10.1093/bioinformatics/btm069.

Stodden V, McNutt M, Bailey DH, et al. *Enhancing reproducibility for computational methods*. Science 354(6317): 1240–1241. 2016. doi:10.1126/science.aah6168.

Sud M, Fahy E, Cotter D, et al. *Metabolomics Workbench: an international repository for metabolomics data and metadata, metabolite standards, protocols, tutorials and training, and analysis tools*. Nucleic Acids Research 44(D1):D463–D470. 2016. doi: 10.1093/nar/gkv1042.

Sumers TR, Yao S, Narasimhan K, Griffiths TL. *Cognitive architectures for language agents*. Transactions on Machine Learning Research. 2024.

Titan SM, Venturini G, Padilha K, et al. *Metabolomics biomarkers and the risk of overall mortality and ESRD in CKD: Results from the ProgreDir Cohort*. PLoS ONE 14(3):e0213764. 2019. doi: 10.1371/journal.pone.0213764. Verified: 454 participants, GC-MS (Agilent MassHunter; Fiehn + NIST libraries), 34 Cox-significant metabolites, composite outcome of mortality or incident renal replacement therapy. Public data: Metabolomics Workbench ST000978.

Trirat P, Jeong W, Hwang SJ. *AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML*. arXiv:2410.02958. 2024.

van den Berg RA, Hoefsloot HCJ, Westerhuis JA, Smilde AK, van der Werf MJ. *Centering, scaling, and transformations: improving the biological information content of metabolomics data.* BMC Genomics 7:142. 2006. doi:10.1186/1471-2164-7-142.

Sud M, Fahy E, Cotter D, et al. *Metabolomics Workbench: an international repository for metabolomics data and metadata, metabolite standards, protocols, tutorials and training, and analysis tools.* Nucleic Acids Research 44(D1):D463–D470. 2016. doi: 10.1093/nar/gkv1042. Public-data accession path for the Progredir cohort used throughout this book: study ID ST000978.

Wei R, Wang J, Su M, et al. *Missing value imputation approach for mass spectrometry-based metabolomics data.* Scientific Reports 8:663. 2018. doi:10.1038/s41598-017-19120-0.

Wishart DS, Guo AC, Oler E, et al. *HMDB 5.0: the Human Metabolome Database for 2022.* Nucleic Acids Research 50(D1):D622–D631. 2022. doi:10.1093/nar/gkab1062.

Xia J, Wishart DS. *MetaboAnalyst: a web server for metabolomic data analysis and interpretation*. Nucleic Acids Research 37(W1):W652–W660. 2009. doi:10.1093/nar/gkp356.

Zhang K, et al. *A generalist vision-language foundation model for diverse biomedical tasks*. Nature Medicine. 2024. doi:10.1038/s41591-024-03185-2.

E.2 Pending — [CITATION NEEDED] items

The following claims in the prose require the author's confirmation of a specific source before publication. Each is reproduced with the chapter and the in-text claim.

- **Ch 1 §1.3:** A 2025–2026 review of metabolomics tooling cataloging two dozen additional tools.
- **Ch 3 §3.5:** A 2025 review of agentic interfaces in science.

- **Ch 8 §8.6 / Ch 19 §19.6:** The late-2025 *Nature* commentary on LLM tools in science (need to identify exact piece and verify the quote).
- **Ch 19 §19.2:** Greshake et al. 2023 on prompt injection (entry above is provisional; verify against final publication).
- **Ch 20 §20.7:** *Briefings in Bioinformatics* statistics-reporting guidelines and *GigaScience* 2025 cost-disclosure guidance update.
- **Ch 22 §22.5:** FDA 2025 SaMD Action Plan update on LLM-enabled systems (verify the update exists and cite correctly).
- **Ch 23 §23.4:** Current Anthropic enterprise offerings as of mid-2026 (the on-prem and HIPAA-compliant deployment options).
- **Ch 24 §24.5:** A 2025 agentic-LLM benchmark paper documenting open-weights vs frontier-API parity numbers (need one or two specific benchmark citations).
- **Ch 24 §24.5:** Metabolomics Workbench API status as of mid-2026.

- **Various:** Software citation entries marked "software citation" need to be cross-checked against the canonical citation format the package maintainers prefer.
-

E.3 ORCID and CRedit

For the manuscript submission, ORCID identifiers and CRedit contribution roles are tracked in `manuscript/AUTHORS.md`. The list is not reproduced here because it depends on the final coauthor count and is finalized at submission. The book's author is Fadhl Mohammed Alakwaa (ORCID [VERIFY]), affiliated with the Division of Nephrology, Internal Medicine, University of Michigan, and a co-investigator on the NEPTUNE consortium.

Chapter 1 — Why Metabolomics Needs Better Software

Metabolomics in 2026 produces datasets that clinicians cannot touch without a bioinformatician. The bottleneck is no longer assay quality or statistical theory — it is the interface between the wet-lab biologist and the analysis. MetabolonR-LLM is one answer to that bottleneck.

1.1 The translation gap

A clinical metabolomics study in 2026 looks like this. A nephrologist at the University of Michigan recruits 200 chronic kidney disease patients into a longitudinal cohort. She sends serum aliquots to a core facility. Six weeks later she receives an Excel file with 200 rows and 800 columns of relative metabolite abundances, plus a separate file annotating which column corresponds to which named metabolite, plus a third file describing patient demographics, time of draw, batch, and outcome. She has a clinical question — which metabolites at

baseline predict progression to end-stage renal disease at three years — and she has, in principle, the data to answer it. What she does not have is a way to run the analysis without writing code.

This is not a deficit of statistical theory. The pipeline she needs is well understood: filter metabolites with too much missingness, impute the rest, log-transform, scale, run a principal component analysis to check for batch effects, decompose the variance to identify which clinical covariates drive the largest signals, then run a differential-abundance test between progressors and non-progressors with appropriate multiple-testing correction. Every one of those steps has been documented in a textbook for at least a decade. The arithmetic is settled.

The bottleneck is the *interface*. To execute that pipeline today she has three options. She can learn R or Python well enough to write the analysis herself, which is several months of work for a non-programmer and a much longer commitment if she wants to do it competently. She can hand the data to a bioinformatician — typically the only one in her department —

and join a queue that is currently months long at most institutions. Or she can use one of the existing web tools, which give her a fixed pipeline she cannot deviate from and outputs whose parameters she cannot inspect.

None of these options is good. All three slow the path from sample to publication, and all three put the analysis at a remove from the clinical question.

[Figure 1.1: Interface evolution timeline 1995–2026 — CLI, web-form, Shiny, Jupyter-as-app, agent. Each row is a paradigm with its date range and a representative tool in metabolomics.]

1.2 The code-literacy barrier

The code-literacy barrier is older than metabolomics, and it has not improved. A non-programming clinician trying to run a PCA in R will spend most of her time not on statistics but on the surrounding scaffolding: how to read a CSV, how to align two data frames by sample identifier, how to handle a column that R has decided is a factor when it should be numeric, how to install a Bioconductor package, how to debug an error that

says `Error in plot.new() : figure margins too large`. The R community has built impressive tooling — tidyverse, Bioconductor, RMarkdown, Quarto — and every piece of that tooling adds another layer she must learn before she can write her first line of statistics.

The same is true of Python. A non-programming clinician trying to do the same analysis with `pandas`, `scikit-learn`, and `statsmodels` will face an analogous tower of dependencies, environments, and conventions. The arithmetic is the same; the scaffolding is just a different shape.

There are reasonable arguments that clinicians *should* learn to program, and there are reasonable arguments that the field benefits when they do. Neither argument changes the present fact that most clinicians have not, will not, and have other professional reasons not to. A tool that helps only the clinicians who choose to invest a year in R is a tool with a small effective audience.

1.3 The existing tools

The metabolomics community has not been idle. A handful of tools attempt, with varying success, to make the analysis accessible without requiring code.

MetaboAnalyst (Xia and Wishart 2009; Pang et al. 2024) is the most widely used. It is a web application, hosted at metaboanalyst.ca, that walks the user through a fixed sequence of steps: upload, normalize, run any of a menu of analyses, download results. It is genuinely useful for standard workflows and has been cited tens of thousands of times. Its limits are structural. The pipeline is fixed; if a user wants to insert a batch correction step in the middle, or use a non-default scaling, or compose two analyses that the interface did not anticipate, she cannot. The parameters at each step are mostly hidden behind sensible defaults. The output figures are publication-acceptable for many journals but not all. Most critically for clinical translation, the user cannot easily reproduce her own analysis a year later: the web interface does not log her parameter choices

in a machine-readable form, and the version of MetaboAnalyst she ran in 2025 will not be the version she runs in 2026.

Lilikoi v1 (Alakwaa et al. 2018) and **Lilikoi v2.0** (Al-Akwaa et al. 2022) are R packages from this author's group. They extend metabolomics analysis to pathway-level and multi-omics integration, and v2.0 added machine-learning workflows. They are powerful and flexible. They are also R packages, which means they require the user to write R. Lilikoi solves a different problem than MetaboAnalyst: it gives a programmer richer tools, not a non-programmer easier access.

DeviumWeb (Eliasson 2012) was an early Shiny-based metabolomics interface. It demonstrated the appeal of the point-and-click web-form paradigm before MetaboAnalyst's web rewrite caught up. The project has been effectively unmaintained for years. Its codebase remains a useful study object — Chapter 5 borrows several Shiny patterns from it — but it is not a live deployment a clinician can use today.

Workflow4Metabolomics (Giacomoni et al. 2015) is the Galaxy-based pipeline platform. It excels at batch processing of

mass-spectrometry-derived datasets, where the analyst constructs a workflow once and re-runs it across many cohorts. It is the right tool for production pipelines in a core facility. For an investigator with a single 200-patient cohort and a single clinical question, it asks her to learn Galaxy first, which is its own scaffolding cost.

A 2025 review of metabolomics tooling [CITATION NEEDED: 2025–2026 metabolomics-tooling survey] catalogs roughly two dozen additional tools, most of which fall into one of these patterns: a Shiny app with a fixed pipeline, an R or Python package for programmers, or a Galaxy-style workflow system for production.

What is conspicuously absent is a tool that lets the clinician describe what she wants to do in her own words and have the analysis happen.

1.4 The one-expert-per-analysis problem at NEPTUNE

The translation gap is not abstract for this author. The Nephrotic Syndrome Study Network (NEPTUNE) is a multi-site consortium that has been collecting clinical, histopathological, and omics data from patients with glomerular kidney disease since 2010 (Gadegbeku et al. 2013). At the University of Michigan, the Kretzler lab serves as the analytical hub for NEPTUNE's metabolomics arm. As of 2026 the consortium spans 27 recruitment sites, hundreds of investigators, and thousands of patient samples. The metabolomics analysis pipeline for any given research question — say, identifying serum biomarkers of focal segmental glomerulosclerosis progression — runs through one analyst at Michigan. When that analyst takes vacation, the queue grows. When she leaves, the queue grows faster.

This is not a Michigan-specific problem. It is the standard configuration at every consortium running clinical omics in

2026. There is one analyst per study, or per institution, or per consortium. The analyst is the rate-limiting step, and the queue she manages is the place where good clinical questions die of impatience.

The question this book asks is not "how do we replace the analyst" — the answer to that is "we should not, and we cannot, because clinical context still requires a human in the loop." The question is "how do we let the clinician do the routine analyses herself, freeing the analyst's time for the analyses that genuinely require expertise."

[Figure 1.2: Clinical translation funnel — assay → matrix → analysis → biomarker → clinic, with the analysis step shaded as the bottleneck and a sidebar showing "one expert per analysis" as the cause.]

1.5 Why a conversational interface

A conversational interface is the right abstraction for this problem for four reasons.

First, it matches the way clinicians think about their data. They do not think in `pandas.DataFrame.groupby` calls or in `limma::eBayes` invocations. They think: "of the metabolites that survive QC, which are different between progressors and non-progressors after adjusting for eGFR at baseline?" A tool that accepts that sentence as input and produces an answer is in the same vocabulary as the question.

Second, it absorbs the scaffolding cost. The clinician does not need to know that her CSV has to be tab-separated, or that the sample identifiers must be in column one, or that the metabolite annotation file must have a column named `HMDB` rather than `HMDB_ID`. A conversational system can ask, fix, and proceed. The scaffolding becomes a dialog, not a tutorial.

Third, it makes the analysis legible. A good conversational system narrates what it is doing as it does it. The clinician learns the pipeline by watching it run on her own data, with her own variable names, against her own clinical question. By the third analysis she has acquired enough vocabulary to direct the system more efficiently. This is a teaching property of the interface, not a bug.

Fourth, and most importantly for the rest of this book, a tool-calling LLM agent can be made reproducible in a way that pure free-text systems cannot. The next section frames the gap that MetabolonR-LLM is built to fill.

1.6 The gap MetabolonR-LLM fills

Conversational analysis systems are not new in 2026. A short, fair survey of the agentic-bioinformatics literature appears in Chapter 4 and includes BioMANIA, GeneGPT, BioChatter, Biomni, BioAgents, and several others. Each of these is a contribution. None of them, as of this writing, has solved the problem MetabolonR-LLM addresses.

The problem is this. A clinical metabolomics analysis needs to be *reproducible in a clinical sense*. That means: a year from now, a regulator, a reviewer, or a different clinician should be able to take the record of the original analysis and re-run it, with bit-identical or near-bit-identical outputs, without re-engaging the LLM. The LLM was used to decide *what* to run; the *running*

should not depend on the LLM's continued availability, its current version, or its current temperature setting.

Most agentic bioinformatics tools as of 2025 do not have this property. They re-execute on the LLM each time, which means each run can produce different parameter choices, different tool orderings, and ultimately different numerical results. For exploratory data analysis that is acceptable. For clinical translation it is not. Reviewers cannot referee an analysis whose numbers may change next time.

MetabolonR-LLM addresses this by separating two concerns rigidly. The LLM is the routing layer: it decides which tool to call and with what parameters. The tools are deterministic, version-pinned wrappers around R functions inherited from MetabolonR v1, the Shiny prototype this book describes in Chapter 5. Every session writes a structured log of the tool calls, parameters, dataset hashes, and outputs. A replay engine can take that log and reproduce the analysis without calling the LLM. The reproducibility is in the log, not in the model.

Whether this architecture is publishable depends on whether the validation experiment in Chapter 21 produces strong-enough numbers, which depends on whether the system is built correctly in Chapters 13 through 18, which depends on whether the design principles in Chapter 8 are honored. This book is the documentation of all of those depend-on's, in order.

What you should be able to do after this chapter

- Name four interface paradigms in bioinformatics — command-line, workflow, web-app/Shiny, and agent — and place a given tool in the correct one.
- Identify two structural limitations of MetaboAnalyst that motivate a successor.
- Distinguish a programmer-oriented R package like Lilikoi from a clinician-oriented web tool like MetaboAnalyst, and explain which problem each solves.
- Explain in one paragraph why "a Shiny replacement" is not enough to address the clinical translation gap.

- State, in one sentence, the specific architectural commitment that distinguishes MetabolonR-LLM from prior agentic bioinformatics tools.

Chapter 2 — The Metabolomics Analysis Pipeline

This is the chapter for the reader who has never touched a metabolite-by-sample matrix. By the end you should be able to read every output that MetabolonR produces and know what choice was made at every step.

2.1 The matrix and its annotations

A metabolomics dataset, in the form this book uses, consists of three files.

The first is the abundance matrix. Rows are samples, columns are metabolites, cells are non-negative real numbers representing relative abundance — typically peak areas from a mass spectrometry run, normalized in some platform-specific way. A typical clinical study has between 50 and 5,000 samples and between 100 and 2,000 named metabolites, with the

smaller end being more common for targeted assays and the larger end for untargeted profiling. The matrix is dense in principle — every sample-by-metabolite cell has a measurement — but in practice many cells are missing. We return to missingness in 2.2.

The second is the sample annotation file. One row per sample, one column per clinical variable: subject identifier, group label, batch, draw date, age, sex, eGFR, and so on. The sample identifier column joins the annotation file to the abundance matrix.

The third is the metabolite annotation file. One row per metabolite. Columns identify the metabolite by name, by HMDB identifier, by KEGG identifier, by pathway membership, and by chemical class. For untargeted assays many metabolites are unidentified — the platform returns a peak with a mass and a retention time but no name. These are flagged as such and are not dropped automatically; whether they survive QC is a downstream choice.

[Figure 2.1: The three-file structure of a metabolomics dataset, annotated. The abundance matrix in the middle, the sample annotation joining from the left, the metabolite annotation joining from the top.]

The shape of these files matters because every step that follows is a transformation of them. A pipeline that produces a list of significant biomarkers at the end is a pipeline that started with these three files and applied, in order, the operations described in the rest of this chapter.

2.2 Missingness

Metabolomics data are missing in two distinct senses, and the distinction matters for which imputation method is appropriate.

The first is **below the limit of detection**. The metabolite is present in the sample at a concentration the assay cannot resolve. This is *missing not at random* (MNAR): the missingness itself is informative — it tells you the value is low. In metabolomics this is the dominant missingness pattern.

The second is **technical missingness**: the metabolite was detectable but a particular sample's measurement failed for an unrelated reason — a peak interference, an injection failure, a software call. This is closer to *missing completely at random* (MCAR).

Distinguishing the two by inspection is hard. A simple heuristic: a metabolite that is missing in 60% of samples uniformly across groups is probably below-LOD; a metabolite missing in 5% of samples scattered randomly is probably technical (Wei et al. 2018).

Imputation methods fall into four families that MetabolonR exposes.

Half-minimum replaces every missing value with half the minimum observed value for that metabolite across samples. It is the natural choice when missingness is dominated by below-LOD; it embodies the assumption that what is missing is small. It is fast and parameter-free.

KNN imputation finds the k nearest non-missing samples (by metabolite-vector distance) and imputes by averaging their

values for the missing cell. The default `k` in `MetabolonR` is 10, inherited from the `impute` Bioconductor package (Hastie et al., software citation). KNN is appropriate when missingness is closer to MAR — when other metabolites carry information about the missing one.

Random forest imputation, as implemented in `missForest`, treats each metabolite with missing values as a regression target and predicts it from the others. It is the most flexible of the four but the slowest, and it can over-smooth in small samples.

Bayesian PCA (`bPCA` in `pcaMethods`, Stacklies et al. 2007) imputes missingness by maximizing the likelihood under a probabilistic PCA model. It is the method most often cited for chemometrics applications and is the third option exposed by `MetabolonR`.

The `MetabolonR-LLM` tool `impute_missing` (Chapter 9) exposes the choice as a parameter. In a clinical analysis the choice should be made by the analyst, not by the system; the LLM's job is to ask, if the user has not specified, which one to use, and to record the answer in the session log.

[Figure 2.2: Missingness pattern heatmap. Samples on the x-axis, metabolites on the y-axis, black cells = missing. Two regimes visible: a few metabolites with mostly-black rows (below-LOD candidates) and scattered black cells (technical candidates).]

2.3 Transformation

Mass-spectrometry abundances span many orders of magnitude. A metabolite present at micromolar concentration produces a peak area perhaps four log-units larger than one present at nanomolar concentration. Working with raw abundances in a downstream PCA or t-test will let the high-abundance metabolites dominate every analysis.

The fix is a variance-stabilizing transformation. Four are standard.

Log transformation — usually log base 2 in genomics, log base 10 in metabolomics — is the most common. Because zero values are problematic for $\log$, a pseudocount is added: the transformed value is $\log(x + c)$, where c is a small positive

number, often 1 or half the minimum non-zero value. MetabolonR uses $c = 1$ as the default and documents the choice in the log.

Square-root transformation is a weaker variance stabilizer than log. It is used when the data are counts (which metabolomics data are not), or when the dynamic range is modest enough that log over-compresses.

Generalized log (glog) is a transformation designed to behave like log at high values and like identity near zero, smoothing the discontinuity at the origin. It is the method recommended by Durbin et al. for microarray data and has been adopted in some metabolomics pipelines. MetabolonR exposes it as an option.

No transformation is occasionally the right choice — for instance when the data have already been transformed upstream by the assay platform.

The MetabolonR-LLM tool `transform` exposes the method choice and the pseudocount. The conservative default is log2 with pseudocount 1. The log records both.

2.4 Scaling

After transformation, metabolites still differ in their *variance*. A high-variance metabolite — even one whose mean is now log-comparable to a low-variance one — will dominate a PCA. Scaling rescales each metabolite so that variance differences do not drive the analysis.

Four methods are standard (van den Berg et al. 2006).

Auto-scaling (z-score) subtracts the mean and divides by the standard deviation. Every metabolite ends with mean 0 and standard deviation 1. This is the most aggressive of the methods: it gives every metabolite equal weight in a PCA regardless of biological prominence.

Pareto scaling subtracts the mean and divides by the *square root* of the standard deviation. The effect is between auto-scaling and no scaling. High-variance metabolites still have somewhat more influence, but the dominance is reduced. Pareto is the default in many metabolomics pipelines because it

preserves more of the biological structure of the data than auto-scaling.

Range scaling divides by $(\max - \min)$. It is sensitive to outliers and is rarely the right choice when robust alternatives are available.

Level scaling divides by the mean. It expresses each value as a fold change from the metabolite's average abundance.

Two pitfalls deserve mention. The first is double scaling — applying auto-scaling after a log transformation that has already approximately stabilized variance. This is not catastrophic but it strips information unnecessarily. The second is scaling before imputation, which can introduce artifacts at the imputed cells. The MetabolonR pipeline enforces the order: filter $\rightarrow$ impute $\rightarrow$ transform $\rightarrow$ scale. The `scale` tool refuses to run if the previous steps have not run.

2.5 Principal component analysis

PCA is an eigendecomposition of the centered (and optionally scaled) data covariance matrix. The first principal component is the linear combination of metabolites that captures the largest variance in the data. The second is the linear combination, orthogonal to the first, that captures the next largest variance. And so on. For a dataset with p metabolites, there are p principal components, but the first few typically capture most of the structure.

PCA produces three outputs that matter.

The **scores** are the projection of each sample onto each principal component. A scores plot of PC1 vs PC2, colored by clinical group, is the standard first look at a metabolomics dataset. If the two groups separate, there is a metabolite signature distinguishing them; if they do not, the signature is either small or buried under other sources of variation.

The **loadings** are the contribution of each metabolite to each principal component. A high absolute loading means that

metabolite drives the component. Loadings are the bridge from "PC1 separates the groups" to "and these metabolites are why."

The **variance explained** by each component, expressed as a percentage of total variance, says how much of the data's structure each component captures. A scree plot of variance explained against component index helps decide how many components to keep.

The dual use of PCA in metabolomics is to *find* biology and to *check* for technical confounding. If PC1 separates samples by batch rather than by clinical group, the analyst has learned that batch correction is needed before any differential-abundance test can be trusted.

[Figure 2.3: PCA scores plot of a metabolomics dataset, two panels. Left panel: PC1 vs PC2 colored by clinical group, showing separation. Right panel: PC1 vs PC2 colored by batch, showing no separation, confirming the signal in the left panel is biological.]

2.6 Sources of variation

The scores-plot-coloring trick of section 2.5 answers a yes/no question per covariate. A more quantitative version, which MetabolonR inherits from the chemometrics literature, is the **F-ratio decomposition** of sources of variation (Brunius et al. 2016).

The method takes the data matrix and a set of named covariates — group, batch, sex, age category, draw site, and so on — and for each metabolite-covariate pair computes the F-ratio of between-level variance to within-level variance. The result is a bar plot, one bar per covariate, height equal to the median F-ratio across metabolites. A tall bar means that covariate explains a lot of variation across the metabolome. A short bar means it does not.

The interpretation depends on which covariate is which. A tall bar for *clinical group* is the desired signal. A tall bar for *batch* is the unwanted technical confound that requires correction before the rest of the pipeline can proceed. A tall bar for *sex* in

a study that is not stratified by sex is a warning that sex-effects should be modeled rather than ignored.

The F-ratio plot is one of the two diagnostic outputs the clinical analyst is most likely to look at first. The other is the volcano plot (2.7).

2.7 Differential abundance and the volcano plot

The differential-abundance question is: which metabolites differ between two clinical groups? The mechanical answer is: for each metabolite, run a statistical test comparing the two groups; report the effect size and the p-value. The methodological answer involves a choice among three test families.

A **two-sample t-test** assumes the metabolite values are approximately normal within each group. After log transformation, metabolomics data are often close enough to normal for the t-test to be appropriate. The R implementation in `stats::t.test` is the simplest of the three options.

A **Wilcoxon rank-sum test** makes no normality assumption; it tests whether the ranks of values in the two groups differ. It is more robust to outliers and to skewed residuals, at the cost of some statistical power when the normality assumption does hold.

The **limma moderated t-test** (Ritchie et al. 2015), inherited from gene-expression analysis, borrows information across metabolites to obtain a more stable variance estimate per metabolite. In small samples this is a substantial gain. MetabolonR-LLM exposes all three in its `differential_abundance` tool and defaults to limma when the sample size per group is below 20.

For each metabolite, the test produces an effect size (a log fold change, comparing mean log-abundance between groups) and a p-value. A **volcano plot** displays these together: log fold change on the x-axis, $-\log_{10}(\text{p-value})$ on the y-axis. Each point is one metabolite. Points in the upper-left and upper-right corners are large-effect, low-p-value metabolites — candidate biomarkers.

[Figure 2.4: A volcano plot, annotated. Horizontal dashed line at the FDR threshold; vertical dashed lines at fold-change thresholds; the four corners labeled "significantly up," "significantly down," "small effect," "low confidence."]

2.8 Multiple-testing correction

A study of 800 metabolites that tests each one with a t-test at $\alpha = 0.05$ will, under the null hypothesis of no real difference anywhere, find about 40 "significant" metabolites by chance. Multiple-testing correction adjusts for this expected false-positive count.

Three procedures cover almost all practical cases.

Bonferroni divides α by the number of tests. With 800 tests, the per-test threshold becomes $0.05 / 800 = 6.25 \times 10^{-5}$. Bonferroni controls the *family-wise error rate*: the probability of any false positive in the whole experiment. It is conservative and is the right choice when even one false positive is unacceptable.

Holm's step-down procedure is a less conservative variant that still controls the family-wise error rate.

Benjamini-Hochberg (BH) (Benjamini and Hochberg 1995) controls the *false discovery rate* (FDR): the expected proportion of false positives among the called positives. An FDR threshold of 0.05 means that of the metabolites called significant, on average 5% are expected to be false. BH is the default in metabolomics and in genomics because it is the right trade-off when the analyst will follow up on a list of candidates: a 5% false-positive rate among the candidates is a tolerable cost of finding more true positives.

The `differential_abundance` tool returns adjusted p-values under all three corrections. The session log records which one drove the final candidate list.

2.9 From significant metabolite to biomarker

A metabolite that passes the FDR threshold is a *candidate*. A *biomarker* is a candidate that has been situated in biology — placed on a known pathway, replicated in an independent cohort, and connected to the clinical phenotype by a plausible mechanism. The transition from one to the other is downstream of MetabolonR's scope, but the tool produces the inputs to it. The `export_session` tool (Chapter 9) bundles the candidate list with the metabolite annotation file, so the analyst's next step — opening KEGG or HMDB to look at pathway membership — is one click away.

What you should be able to do after this chapter

- Describe the three-file structure of a metabolomics dataset and what role each file plays.

- Distinguish below-LOD missingness from technical missingness and choose an imputation method appropriate to each.
- Explain in one sentence what a log transformation, with pseudocount, accomplishes and why it is the metabolomics default.
- Read a PCA scores plot and distinguish a biological-signal interpretation from a batch-confound interpretation.
- Read a volcano plot and identify the four corners by what they represent biologically.
- State the difference between Bonferroni and Benjamini-Hochberg correction in one sentence each.
- Explain why a "significant metabolite" is not yet a biomarker and what the next step would be.

Chapter 3 — From Command Line to Shiny to Agent

Bioinformatics interfaces have evolved through four distinct paradigms in thirty years. This chapter places agentic LLM interfaces in that lineage and is honest about what each generation gave up to gain its strengths.

3.1 Why a paradigm review precedes a system description

Before describing MetabolonR-LLM's architecture in Part III, this chapter steps back and asks: where does a tool-calling LLM agent fit in the history of bioinformatics interfaces? The answer is not "it replaces everything that came before." Each generation of interface has solved a real problem and has paid a real cost. A reader who understands those trade-offs can read Parts III

through V with a clearer sense of what design decisions are forced and what are choices.

Four paradigms cover the field from the mid-1990s through 2026: the command line, the workflow system, the web app (Shiny and Jupyter-as-app), and the agent. They overlap chronologically; a 2026 lab uses all four, often in the same week. They are distinguished not by date but by where the user's attention sits — at the shell prompt, in a workflow diagram, in a browser form, or in a conversation.

3.2 The command-line era

The command-line paradigm is the original interface for bioinformatics. A user opens a terminal, types a command like `blastn -query seqs.fa -db nr -out hits.txt`, and reads the output. Tools are Unix-style programs that read files and write files. Pipelines are constructed by stringing programs together — often with shell pipes (`|`), `awk`, `sed`, and small Perl or Python scripts to glue the pieces.

The community shaped by this paradigm — the BLAST users of the 1990s, the SAM/BAM users of the 2000s, the bedtools users of the 2010s — values composability, scriptability, and reproducibility-by-text. A pipeline written as a shell script is exactly what was run; there is no hidden state.

The paradigm's cost is the entry barrier. A clinician who does not already know the shell will not, by inspection of `samtools view -bS in.sam | samtools sort -o out.bam`, understand what is happening. Even users who know the shell face the constant tax of remembering option flags and tool-specific conventions. In metabolomics specifically, the command-line paradigm never dominated — the tooling was always closer to R and Bioconductor than to the SAM/BAM ecosystem — but the lineage is visible in tools like `xcms` (Smith et al. 2006, software citation) that are operated from an R command line.

The reproducibility property of the command line is the highest of the four paradigms. A shell script is the analysis. The scaffolding cost is also the highest. These two facts are not coincidences.

3.3 The workflow paradigm

By the mid-2000s the cost of stringing command-line tools together had become a community problem. The response was workflow systems: tools that wrap each command-line program as a node in a directed graph, with edges as data flows. The user constructs the workflow in a graphical or declarative interface and the system executes the underlying commands.

The most influential in bioinformatics has been **Galaxy** (Afgan et al. 2018). Galaxy lets a user drag-and-drop tools onto a canvas, configure their parameters in forms, connect their outputs to other tools' inputs, and run the result on a server. It exports the workflow as a JSON document that can be re-imported elsewhere. **Snakemake** (Mölder et al. 2021, software citation) and **Nextflow** (Di Tommaso et al. 2017) extend the same idea into declarative, text-based workflow languages preferred by users who like the workflow paradigm but want the version-control properties of a shell script.

The workflow paradigm makes pipelines visible. A diagram of nodes and edges is easier to read than a 200-line shell script. It also makes pipelines reusable: a workflow that processes one cohort can be re-run on the next cohort by changing only the input files.

The cost is rigidity at the boundary. A workflow node is a wrapped command-line tool with a fixed set of parameters. If the analyst needs an operation the workflow's tool library does not provide, she has to write a new tool wrapper — which means she is back at the command line. For metabolomics, **Workflow4Metabolomics** (Giacomoni et al. 2015) wraps the dominant mass-spectrometry tools as Galaxy nodes; it is excellent for production pipelines in a core facility and not the right tool for an investigator with a one-off question.

The reproducibility property of a workflow is high: the JSON or YAML representation captures the pipeline structure and parameters. The UX is friendlier than the command line for users who think in pipelines, less friendly than a web form for users who think in one analysis at a time.

3.4 Shiny and Jupyter-as-app — the web-form era

In 2012 the R community released Shiny, a framework for turning R scripts into reactive web applications. The combination — R's statistical depth with a point-and-click browser UI — gave rise to a wave of bioinformatics tools that exposed a single analytical pipeline through a form. Upload your data, choose a few parameters from dropdowns, click a button, get a plot. **MetaboAnalyst** in its current form, **DeviumWeb**, the **iSEE** browser for single-cell data (Rue-Albrecht et al. 2018), and **MetabolonR v1** itself all live in this paradigm.

Jupyter notebooks (Kluyver et al. 2016) and R Markdown / Quarto (Allaire et al. 2024) are an adjacent paradigm — narrative documents that interleave code with text and plots. They were never primarily an interface for non-programmers, but the "Jupyter-as-app" trend of the late 2010s — packaging a notebook with `voila` or `binder` so a non-programmer can

run it in a browser — blurs the line. A notebook served as an app behaves a lot like a Shiny tool, and the two paradigms share their key properties.

What makes the web-form era distinctive is the radical lowering of the entry barrier. A clinician with no R knowledge can use MetaboAnalyst on the day she first hears about it. The cost is rigidity *within* the form. The user can only do what the form's authors anticipated. She cannot insert a step the form does not offer, change a parameter the form does not expose, or compose two analyses the form did not connect.

The reproducibility property is the weakest of any paradigm so far. A web form does not, by default, log what was clicked. The MetaboAnalyst team has done work to improve this (the modern interface exports an R script that approximates what the user did), and Shiny apps can be designed to log their inputs, but the default Shiny pattern leaves the user with screenshots and a memory of what she chose. This was the most consequential limitation of MetabolonR v1 and a substantial part of the motivation for v2.

3.5 LLM agents as the 2024–2026

frontier

The fourth paradigm — and the subject of the rest of this book — is the tool-calling LLM agent. The user describes the analysis in natural language. An LLM interprets the request and routes it to one or more tools — programmatic functions with well-defined inputs and outputs — and weaves the tool outputs into a response. If a parameter is ambiguous, the agent can ask. If a step needs clarification, the agent can ask. If the user changes her mind mid-analysis, the agent can pick up the conversational thread.

The decisive change is in *where the user's attention sits*. In the command-line paradigm, attention sits at the shell prompt; in the workflow paradigm, on a diagram; in the web-form paradigm, on a form; in the agent paradigm, in a conversation. The conversation is the closest the four paradigms come to matching the way clinical investigators discuss their data with each other.

A more thorough survey of the published agentic-bioinformatics literature appears in Chapter 4, including BioMANIA (Dong et al. 2023), GeneGPT (Jin et al. 2024), BioChatter (Lobentanzer et al. 2025), Biomni (Huang et al. 2025), and others. As of 2026 the paradigm is mature enough that there is a *Nature Methods*-level publication record but young enough that the methodological standards are still being negotiated. Chapter 19 examines what reviewers in the field are now asking for and where most published agentic tools fall short.

The reproducibility property of the agent paradigm is the most interesting and the most fragile. The agent's behavior depends on the model, the prompt, the temperature, and whatever order the LLM happens to call tools on this particular run. Re-running the same prompt next week, against the same data, on the same agent, can produce different parameter choices, different tool orderings, and different final answers. Whether this is a fatal property depends entirely on whether the system separates "what was decided" from "what was computed" and logs both. This is the architectural commitment of MetabolonR-

LLM (Chapter 8) and the methodological centerpiece of Chapter 11.

3.6 Two axes and a scatterplot

The four paradigms can be placed on two axes: **entry barrier** (how hard is the tool to start using) and **reproducibility** (how reliably can someone else re-run the analysis from the artifacts).

The command line is high-barrier and high-reproducibility. A user must learn the shell, but once she has, the script she wrote is the analysis.

The workflow paradigm is moderate-barrier and high-reproducibility. The user must learn the workflow system but not the underlying tools.

The web-form paradigm is low-barrier and low-reproducibility. The user can start immediately but cannot reproduce what she did without re-doing it.

The agent paradigm, by default, is low-barrier and *catastrophically low* reproducibility — worse than the web form,

because the agent's behavior is not even consistent between two runs of the same input. The architectural challenge of agentic bioinformatics is to keep the low entry barrier while making reproducibility comparable to or better than the workflow paradigm. MetabolonR-LLM's claim is that the right separation of routing from computation, plus a structured session log and a replay engine, achieves this. That claim is the burden of Chapters 8, 11, 20, and 21.

[Figure 3.1: Bioinformatics interface lineage, 1995–2026. A horizontal timeline. Bars labeled "Command line (1995–)," "Workflow (2005–)," "Web app / Shiny / Jupyter-as-app (2012–)," "Agent (2024–)." Representative metabolomics tools annotated under each bar.]

[Figure 3.2: UX vs reproducibility scatterplot. x-axis: entry barrier (high to low). y-axis: reproducibility (low to high). Four labeled points for the four paradigms. A dashed arrow from "default agent" upward to "MetabolonR-LLM" indicating the architectural lift this book documents.]

3.7 A note on what this lineage does not predict

Paradigm shifts in interfaces do not retire the previous paradigms. The Unix shell is alive in 2026. Galaxy is alive in 2026. Shiny apps are alive in 2026. The same lab that builds an agentic tool for clinical analysts will still have a Snakemake workflow running its production pipeline and an R Markdown notebook documenting last year's manuscript figures. The relationship between paradigms is additive, not replacement.

What this implies for the rest of the book is that MetabolonR-LLM is not a successor to MetabolonR v1 in the sense of making v1 obsolete. v1 is the reference implementation of the underlying statistics. v2 is a new front door to the same statistics. Both will continue to exist; v2 will, the author hopes, be the one a clinician opens first.

A second implication concerns the interpretation of "agentic." In some 2024–2025 commentary the term has been used loosely, to describe any LLM-driven scientific tool. The lineage in this

chapter argues for a tighter definition: an *agentic* system is one in which the user's attention sits in a conversation that the model maintains, and in which the model itself chooses which actions to take. Tools that present an LLM-generated explanation of a fixed pipeline — useful as they are — are not in that category. MetabolonR-LLM is. The classification matters because the reproducibility burden differs: a fixed pipeline with an LLM commentary track inherits the reproducibility of the pipeline; an agent that chooses its own steps does not, and must earn the property explicitly.

What you should be able to do after this chapter

- Name the four bioinformatics-interface paradigms and place a given tool — say, Galaxy, MetaboAnalyst, or BioChatter — in the correct one.
- Explain the trade-off between entry barrier and reproducibility for each paradigm in one sentence.

- State why the default agent paradigm has *worse* reproducibility than the web-form paradigm and what architectural commitment is required to fix that.
- Argue, in one paragraph, why a new interface paradigm in bioinformatics does not retire the older ones.
- Recognize that this chapter has framed the problem MetabolonR-LLM solves but has not yet described the solution; that is the job of Part III.

Chapter 4 — Large Language Models and Tool Use

Every architectural decision in MetabolonR-LLM depends on understanding what tool-calling LLMs can and cannot do. This chapter is the technical foundation for the rest of the book — a description of the tool-use mechanism, an explanation of why it solves the determinism problem that pure LLM systems do not, and a survey of fifteen tools that frame where MetabolonR-LLM sits in the 2024–2026 agentic-bioinformatics landscape.

4.1 What a tool-calling LLM is

A large language model, by itself, is a function from a sequence of tokens to a probability distribution over the next token. Used naïvely, it produces text. A *tool-calling* LLM is the same model wrapped in a runtime contract: the model can emit, in place of plain text, a structured request to call one of a set of pre-declared functions — *tools* — and the runtime executes the

function and returns its result for the model to use in its next turn.

The contract is what makes this useful. The runtime declares, in machine-readable form, which tools exist and what arguments each accepts. The model, conditioned on the user's request and on the tool declarations, chooses a tool, fills in its arguments, and emits the call. The runtime checks the call against the schema, executes it if valid, and feeds the result back to the model as a tool-result message. The model then decides whether to call another tool, ask a clarifying question, or produce a final answer.

This is sometimes called *function calling* and sometimes *tool use*; the names are interchangeable. Anthropic's API uses "tool use" (Anthropic, tool-use documentation, retrieved May 2026). OpenAI's API uses "function calling" (OpenAI, function-calling documentation, retrieved May 2026). The underlying mechanism is the same.

4.2 The Anthropic tool-use API in detail

MetabolonR-LLM is built against the Anthropic Messages API with tool use enabled. A request to the API has three relevant fields beyond the standard `messages` array.

The `tools` field declares the available tools as a JSON array, where each tool has a `name`, a `description`, and an `input_schema` — a JSON Schema describing the shape of valid arguments. The schema serves two purposes. It tells the model how to fill in the arguments, and it lets the runtime validate the model's output before executing.

The `tool_choice` field controls whether the model must call a tool, may call any tool, or may call a specific named tool. MetabolonR-LLM defaults to `{"type": "auto"}` — the model decides.

The `messages` array contains the conversation history, with each message being either user text, assistant text, an

assistant tool-use request (content of type `tool_use`), or a user tool result (content of type `tool_result` referencing the prior `tool_use` by id). Multi-turn tool calling is just multi-turn message exchange where some turns are tool requests and tool results rather than plain text.

A complete single-call request and response, abbreviated to the essentials, looks like this:

```
// Request
{
  "model": "claude-sonnet-4-6",
  "tools": [
    {
      "name": "summarize_dataset",
      "description": "Return n_samples, n_metabolites",
      "input_schema": {
        "type": "object",
        "properties": {
          "dataset_handle": {"type": "string"}
        }
      },
    },
  ],
}
```

```

        "required": ["dataset_handle"]
    }
}
],
"messages": [
    {"role": "user", "content": "Summarize the Prog"}
]
}

// Response
{
    "stop_reason": "tool_use",
    "content": [
        {
            "type": "tool_use",
            "id": "toolu_01...",
            "name": "summarize_dataset",
            "input": {"dataset_handle": "progredir_v1"}
        }
    ]
}

```

The `runtime` executes `summarize_dataset("progredir_v1")`, packages the return value as a `tool_result` message, and sends a follow-up request including the original messages plus the `tool_use` and `tool_result`. The model's next response — typically of `stop_reason: "end_turn"` — is the user-facing summary.

Chapter 10 walks a full multi-turn Progredir session through this mechanism in detail.

[Figure 4.1: The tool-use request/response cycle, sequence diagram. User → Model → `tool_use` → Runtime → R function → `tool_result` → Model → user-facing answer.]

4.3 Why tool calling is the answer to LLM non-determinism

A plain LLM is not deterministic. Two identical requests, even with `temperature: 0`, can produce different outputs because of sampling implementation, hardware non-

associativity, and server-side batching effects (Anthropic and OpenAI both document the residual non-determinism in their respective FAQ pages). For exploratory writing this is harmless. For statistical analysis it is fatal — a tool that produces different numbers on Tuesday than on Wednesday is not a tool a regulator will accept.

The architectural answer in MetabolonR-LLM is to ensure that the LLM never *computes* a statistic. The LLM's role is to choose which deterministic function to call and with what arguments. The function itself is a version-pinned wrapper around an R routine inherited from MetabolonR v1 (Chapter 5). The function's output, given the same arguments and the same data, is bit-identical across runs — modulo BLAS, OS, and thread-count effects that Chapter 11 addresses explicitly.

This is the schema-as-contract pattern. The schema is the boundary between the non-deterministic routing layer and the deterministic computation layer. As long as the schema is honored — and the runtime validates that it is on every call — the LLM's variability does not contaminate the numerical outputs.

The schema does more than validate. It also constrains the model's hypothesis space at generation time. A modern tool-calling API uses the schema during decoding, biasing the model toward outputs that satisfy the schema rather than letting it propose syntactically invalid calls (this is sometimes implemented as constrained decoding, sometimes as schema-conditioned sampling, depending on the provider). The practical effect is that schema violations are rare — fewer than 1% in the validation experiment of Chapter 21 — and when they do occur the runtime returns a structured error that the model can interpret and self-correct on the next turn.

A second benefit follows from the schema's role as documentation. Because each tool's input schema is written once and used both at runtime (for validation) and at design time (for the model to read), there is no "documentation drift" between what the system does and what the model thinks it does. This is a small property in isolation; in aggregate, across ten tools and dozens of arguments, it eliminates a class of bugs that bedevil systems where the prompt and the implementation are maintained separately.

The cost of this approach is rigidity. A pure LLM can write arbitrary new code on the fly; a tool-calling LLM is restricted to the tools its runtime declares. The author's contention, which the rest of the book defends, is that for clinical metabolomics this trade is correct: the rigidity is the source of the publishability. A clinician who needs a fundamentally new statistical method should go to her biostatistician — not to an LLM that will improvise one. The agent's job is to remove friction from the routine cases, not to replace expertise in the novel ones.

4.4 The survey: fifteen tools, four categories

A reader trying to place MetabolonR-LLM in the agentic-bioinformatics landscape of 2024–2026 needs a sense of what else has been built. This section is a deliberately concise survey of fifteen tools chosen to span the four categories the chapter has defined: tool-using agents, domain-adapted LLMs, boundary-case systems, and benchmarks. Every citation below

has been verified against the primary source; the full verification report is reproduced as Appendix C-supplement for the curious reader.

Each entry uses the same four lines: classification, year and venue, one-line description, and the dimension along which it is closest to or most distant from MetabolonR-LLM.

Tool-using agents (seven)

BioMANIA (Dong et al. 2023, bioRxiv). *Agent*. A framework that auto-generates conversational agents from documented Python bioinformatics packages, using API extraction and instruction tuning. Closest prior art to MetabolonR-LLM in spirit — both turn a programming-library interface into a conversational one — and farthest in execution model: BioMANIA generates the agent automatically from documentation, where MetabolonR-LLM curates a closed tool set by hand. The trade-off is breadth vs. auditability.

GeneGPT (Jin et al. 2024, *Bioinformatics*). *Agent*. The canonical demonstration that an LLM (Codex in the original paper) can

be taught to call NCBI Web APIs in-context to answer genomics QA. The closest published precedent for a tool-calling LLM in biomedicine. The scope is narrower than MetabolonR-LLM (QA rather than analysis pipeline) and the tool set is the NCBI public API rather than a custom layer.

GenoTEX / GenoAgent (Liu et al. 2024, arXiv:2406.15341). *Agent paired with a benchmark.* A multi-agent LLM baseline (GenoAgent) for gene-expression analysis, evaluated against an explicit benchmark (GenoTEX). The pairing of an agent with a benchmark suite from the same authors is methodologically close to the approach this book takes — Chapters 20 and 21 build the analog for metabolomics — though the GenoTEX benchmark is broader than MetabolonR-LLM's twenty-prompt validation suite.

AutoML-Agent (Trirat et al. 2024, arXiv:2410.02958). *Agent.* A multi-agent LLM framework that converts natural-language ML task descriptions into full AutoML pipelines. Not biomedicine-specific; included as the canonical agentic-AutoML reference and as a contrast — AutoML-Agent generates

pipelines, MetabolonR-LLM routes among a fixed pipeline's steps.

BioAgents (Mehandru et al. 2025, *Scientific Reports*). *Agent*. A multi-agent system using small local LLMs and retrieval-augmented generation to lower the barrier to bioinformatics analysis. Peer-reviewed in 2025. The closest published comparator on the bench of "agentic bioinformatics for the non-programmer" — but with a different reproducibility model (small local models with RAG rather than a tool-routing layer over a frontier API model).

Biomni (Huang et al. 2025, bioRxiv). *Agent*. The Stanford SNAP group's general-purpose biomedical agent, integrating 105 software tools, 150 biological tools, and 59 databases across 25 biomedical domains. The most ambitious tool catalog in the literature. The contrast with MetabolonR-LLM is one of breadth versus depth: Biomni aims to be a general agent over much of biomedicine, MetabolonR-LLM aims to be a specialist agent over metabolomics with reproducibility guarantees.

BioChatter (Lobentanzer et al. 2025, *Nature Biotechnology*). *Agent framework*. An open-source Python framework that wraps LLMs with retrieval-augmented generation, BioCypher knowledge-graph integration, and agentic workflows for biomedical use. The closest published prior art to MetabolonR-LLM in framing. BioChatter is a *framework* for building biomedical LLM agents; MetabolonR-LLM is an *instance* of an agent for a specific domain with a stronger reproducibility commitment. The two are complementary, and Chapter 24 discusses a potential refactor of MetabolonR-LLM onto the BioChatter framework as a v3 direction.

Domain-adapted LLMs (five)

BioGPT (Luo et al. 2022, *Briefings in Bioinformatics*). *Domain-LLM*. GPT-2 pretrained on PubMed for biomedical text generation and relation extraction. Predates the agentic-LLM era; included as the canonical pre-agent biomedical LLM. No tool use; pure domain adaptation.

ChatNT (de Almeida et al. 2025, *Nature Machine Intelligence*). *Domain-LLM*. A multimodal model coupling a Nucleotide

Transformer DNA encoder with Vicuna-7B for sequence-task QA in natural language. The word "conversational agent" in the title refers to the user interface, not to a tool-use mechanism; there is no external tool layer. The contrast with MetabolonR-LLM is the source of the determinism — ChatNT's outputs depend on the model and the prompt; MetabolonR-LLM's numerical outputs depend on a deterministic tool layer.

OpenBioLLM (Pal and Sankarasubbu 2024, Hugging Face release). *Domain-LLM*. Llama-3 fine-tuned with DPO on biomedical corpora; claims benchmark-leading performance on biomedical QA. Released as a model card on Hugging Face; *no peer-reviewed paper is yet associated with the release*. Included here for completeness; the chapter does not over-rely on it. Its presence in the survey is a cue for Chapter 23's discussion of on-prem open-weights as a future deployment option.

BiomedGPT (Zhang et al. 2024, *Nature Medicine*). *Domain-LLM*. A unified vision-language foundation model pretrained across 19 biomedical modalities. Not an agent. Included as the multimodal-domain-LLM comparator.

Med-PaLM (Singhal et al. 2023, *Nature*; Med-PaLM 2, Singhal et al. 2025, *Nature Medicine*). *Domain-LLM*. Google's instruction-tuned PaLM for medical QA, evaluated on USMLE-style benchmarks. Reads as the most cited domain-LLM in clinical-AI surveys. Not an agent. Included to establish the upper bound of what is achievable with model-tuning alone, in contrast to the architectural approach this book advocates.

Boundary-case systems (two)

AlphaFold 3 / AlphaFold Server (Abramson et al. 2024, *Nature*). *Specialized predictive model with a web service*. Not an LLM, not an agent. Included as a deliberate boundary case: a tool that demonstrates how a specialized model can be packaged as a service without the LLM scaffolding. The contrast is instructive — AlphaFold's reproducibility comes from the model being deterministic given its inputs, with no LLM in the loop. MetabolonR-LLM's tool layer aspires to the same property at the tool boundary.

CELLxGENE Census Discover (CZI Single-Cell Biology Program 2025, *Nucleic Acids Research*). *Data platform with a*

programmatic API. Not an LLM and not an agent — a harmonized data platform with Python and R APIs. Included as the "tool an agent could call" reference point: it represents the kind of large biological resource that a future MetabolonR-LLM tool could wrap as part of a multi-omics expansion (Chapter 24).

Benchmarks (one)

ScienceAgentBench (Chen et al. 2025, ICLR). *Benchmark*. A 102-task benchmark for evaluating language agents on data-driven scientific discovery across four sciences, including bioinformatics. The most relevant external benchmark against which a hypothetical pass of MetabolonR-LLM should eventually be measured. Chapter 21 discusses whether MetabolonR-LLM's twenty-prompt validation suite is best left independent or eventually mapped to the ScienceAgentBench format.

[Figure 4.2: Comparative grid of fifteen agentic and adjacent tools. Rows: BioMANIA, GeneGPT, GenoTEX, AutoML-Agent, BioAgents, Biomni, BioChatter, BioGPT, ChatNT,

OpenBioLLM, BiomedGPT, Med-PaLM, AlphaFold 3, CELLxGENE, ScienceAgentBench. Columns: classification (Agent / Domain-LLM / Other / Benchmark), input modality, tool surface, reproducibility claim, validation rigor, license/availability.]

4.5 Where MetabolonR-LLM sits

The placement, after this survey, can be stated precisely. MetabolonR-LLM is an agent in the same family as BioMANIA, GeneGPT, BioAgents, Biomni, and BioChatter. It differs from all of these in three respects.

First, it is narrower. The tool surface is metabolomics, not bioinformatics-in-general (Biomni, BioChatter) and not QA (GeneGPT). The narrowness is the cost of the next property.

Second, the tool layer is hand-curated and version-pinned, not auto-generated (BioMANIA) and not configured by retrieval (BioAgents). Every tool wraps a specific R routine inherited from a specific Shiny app, with a specific oracle test pinning its outputs. The cost is breadth; the benefit is auditability.

Third, every session writes a structured log sufficient to replay the analysis without re-engaging the LLM. None of the agentic systems above currently makes this commitment as a central architectural feature. Several allow it as a possibility; MetabolonR-LLM treats it as a contract. The case for why this matters is the subject of Chapters 11 and 22.

Whether the trade-offs are correct — whether narrower-but-auditable is preferable to broader-but-not for clinical metabolomics — is a hypothesis the validation experiment in Part V tests. The argument that follows in Chapters 8 through 12 is the design that makes the test possible.

It is worth saying once, clearly, what *this book is not arguing*. It is not arguing that MetabolonR-LLM is a more capable system than Biomni or BioChatter. They are larger in scope and have larger teams behind them. It is arguing that for the specific clinical-translation use case described in Chapter 1 — a nephrologist running a one-off differential-abundance analysis on a single cohort — the trade-off this book makes is the right one, and that the architectural pattern (closed tool set, version-pinned wrappers, structured session log, replay engine) is

generalizable beyond metabolomics. A reader who builds an analog of MetabolonR-LLM for transcriptomics or proteomics, using a similar architecture, should be able to make a similar reproducibility claim. The pattern, not the metabolomics specifics, is the contribution that the author hopes will travel.

What you should be able to do after this chapter

- Describe what a tool-calling LLM does and how it differs from a plain text-generating LLM.
- Read a JSON Schema tool definition and identify the required arguments.
- Trace a single tool-use round trip through the Anthropic API: request → tool_use → tool execution → tool_result → final response.
- Explain why tool calling lets a non-deterministic LLM produce numerically reproducible outputs.

- Name three published agentic bioinformatics systems from 2023–2025 and describe one architectural respect in which each differs from MetabolonR-LLM.
- Distinguish a domain-adapted LLM (Med-PaLM, BioGPT, BiomedGPT) from a tool-using agent (BioMANIA, GeneGPT, BioChatter) and explain why MetabolonR-LLM is firmly in the second category.

Chapter 5 — Design and Implementation of MetabolonR v1

MetabolonR v1 is the Shiny app this book replaces. Understanding it in detail matters because v1 is also the validation oracle for v2: every tool in the agent must produce the same numbers v1 produces. This chapter walks through its architecture, its dependencies, its deployment, and the role it will play in Part IV.

5.1 What v1 is and why it exists

MetabolonR v1 is an R Shiny application that wraps the metabolomics pipeline of Chapter 2 in a browser-based, point-and-click interface. It was written in the Kretzler-lab analytical group between 2017 and 2019, around the same time as the original Lilikoi release (Alakwaa et al. 2018), as a companion tool for clinicians who wanted to look at metabolomics data without writing R. It has been used internally on dozens of NEPTUNE cohorts and externally by collaborators in São

Paulo, Seoul, and Berlin. It is deployed on `shinyapps.io` as a free-tier app, accessible to anyone with the URL, and the source is on GitHub under an MIT license.

`v1` is not the contribution of this book. The contribution is what comes next. But the rest of the book is unintelligible without knowing what `v1` already does, because the `v2` agent's job is precisely to replace `v1`'s interface while preserving its statistics. Every routine `v1` exposes through a button is a routine `v2` must wrap through a tool. Every parameter `v1` sets behind a default is a parameter `v2` must surface in its log.

5.2 The Shiny reactive model

A Shiny application is built on R's reactive programming model. The user interface is declared in a `ui.R` file as a tree of nested function calls — `fluidPage`, `sidebarLayout`, `tabsetPanel`, `selectInput`, `plotOutput`, and so on — each of which produces a piece of HTML. The application logic is declared in a `server.R` file as a function of two arguments, `input` and `output`. Reading from `input$<id>`

is reactive: any expression that reads it is re-evaluated whenever the input changes. Writing to `output$<id>` is also reactive: the expression is wrapped in a `render*` function that ties it to the inputs it reads.

The reactive graph that emerges from these dependencies is the application. When the user changes a dropdown, only the outputs that depend on it (directly or transitively) are recomputed. When the user uploads a new dataset, the whole downstream graph re-runs.

For metabolomics in particular, this model is both a strength and a liability. The strength is that the application feels immediate — change a parameter, and the plot updates. The liability is that the reactive graph is invisible. The user does not see that her PCA plot depends on her choice of scaling, which in turn depends on her choice of imputation, which in turn depends on her choice of QC threshold. She sees a plot. The chain of choices that produced it lives only in her browser's memory and is gone when she closes the tab. This is the most consequential limitation of v1 and the proximate cause of Chapter 11's session-log design.

5.3 The seven tabs

The v1 UI is organized as a horizontal `tabsetPanel` with seven tabs, intended to be navigated left to right.

The first is **Upload**. The user drops three files: the abundance matrix, the sample annotation, the metabolite annotation. v1 validates that the sample identifiers align between the matrix and the sample annotation, that the metabolite identifiers align between the matrix and the metabolite annotation, and that the matrix contains only numeric values (allowing for missingness). Validation failures are surfaced as a red banner with a specific diagnostic.

The second is **Summarize**. v1 reports the number of samples and metabolites, the percentage of missing values overall and per metabolite, the group sizes from the sample annotation, the dynamic range of the abundance values, and a histogram of log-transformed abundances. This tab is the user's first sanity check.

The third is **QC**. The user sets two thresholds — maximum allowed missingness per metabolite and per sample — and sees

the count of rows and columns that would be dropped at each threshold. Tightening or relaxing a threshold updates the count live; clicking "Apply" persists the filtered dataset to the rest of the pipeline.

The fourth is **Impute and Transform**. v1 combines the imputation step and the log/sqrt transformation step on one tab because, in clinical practice, the choices are made together. The user selects an imputation method from {KNN (k=10), half-min, random forest, BPCA} and a transformation from {log2, log10, sqrt, glog, none} with a pseudocount field. The tab displays the before-and-after distribution of abundances for one user-selected metabolite, so the user can see what the transformation did.

The fifth is **Scale**. The user chooses Pareto, auto, range, level, or none. The tab shows the per-metabolite variance distribution before and after scaling. Chapter 2 describes the trade-offs.

The sixth is **PCA**. v1 runs `prcomp` on the scaled matrix, plots PC1 vs PC2 (with PC3 vs PC4 available via a dropdown), and lets the user color the scores by any column of the sample

annotation. This is the diagnostic tab for batch effects and biological signal.

The seventh is **Volcano**. v1 lets the user pick a grouping variable from the sample annotation and a reference level. It computes a per-metabolite limma moderated t-test, applies Benjamini–Hochberg correction, and plots the volcano with FDR and fold-change cutoffs the user can drag with sliders. Significant metabolites are listed in a sortable table to the right of the plot.

[Figure 5.1: v1 reactive graph — boxes for the seven tabs, arrows for the data dependencies. Upload at the root, Volcano at the leaf. Annotated with the input controls that gate each transition.]

A separate **Sources of Variation** view, accessible from the PCA tab as a sub-panel, runs the Brunius F-ratio decomposition over user-selected covariates. It was added in a 2020 minor release and is the model for the v2 `sources_of_variation` tool (Chapter 9).

5.4 R package dependencies

v1 is pinned to a specific set of package versions, captured in a `renv.lock` file at the repository root. The major dependencies are:

- `shiny` 1.7.4 — the web-app framework (Chang et al., software citation).
- `impute` 1.74 — the KNN imputation backend, from Bioconductor.
- `pcaMethods` 1.92 (Stacklies et al. 2007) — Bayesian PCA and the PCA wrapper.
- `missForest` 1.5 — the random-forest imputation backend.
- `limma` 3.56 (Ritchie et al. 2015) — the moderated t-test.
- `EnhancedVolcano` 1.18 (Blighe et al., software citation) — the volcano-plot renderer.
- `ggplot2` 3.4 — for non-volcano plots.
- `DT` 0.28 — for the sortable significant-metabolites table.

The choice to pin versions in `renv.lock` was made early and has been worth its weight in debugging time. When a 2022 update to `limma` changed the default behavior of `eBayes` for a marginal case, the pinned environment continued to produce the expected numbers; an unpinned environment would have silently shifted the volcano calls.

This pinning discipline is the seed of the version-pinning policy that Chapter 11 generalizes to the full agent stack.

5.5 Deployment

`v1` is deployed on `shinyapps.io`, Posit's hosted Shiny service. The free tier provides one app instance with 1 GB of RAM, 1 CPU, and a quota of 25 active hours per month. For occasional clinical use this is sufficient. For a power user uploading multiple cohorts in a week, it is not — the app shuts down when the quota is exhausted, leaving a "this app has been deactivated" page until the next billing cycle.

The 1 GB memory ceiling is the more consequential limit. A dataset of 5,000 samples $\times$ 2,000 metabolites in double

precision occupies 80 MB before any analysis runs. KNN imputation roughly triples that footprint, and a PCA on the result adds another 50%. By the time the user is on the Volcano tab the app is consuming several hundred megabytes, and a second uploaded dataset can push it past the ceiling. The free tier was the right place to demonstrate the app to early users; it was always a stopgap.

Lab-internal users get around the ceiling by running v1 locally — `git clone, renv::restore(), shiny::runApp()` — which works for anyone with R installed but defeats the purpose of having a no-code interface for clinicians.

The deployment story is part of why v2 leaves `shinyapps.io` for the Kretzler-lab server `alakwaaf-aps1a` (Chapter 12).

5.6 v1 as the oracle for v2

The most important property of v1, for the rest of this book, is that it works. Its statistical outputs are correct — verified across hundreds of internal analyses and cross-checked against

MetaboAnalyst on standard test datasets. Its parameter defaults are chosen carefully. Its plots are publication-quality.

When v2 is built in Chapters 13–18, every tool the agent calls must produce numerically equivalent outputs to the corresponding v1 routine. The pattern that enforces this is the **golden snapshot**.

For each v1 routine — `impute_knn`, `transform_log`, `scale_pareto`, `pca`, `f_ratio`, `limma_da` — a snapshot is captured by running v1 on a curated test input (typically the Progredir dataset of Chapter 6) and saving the output to disk as a JSON blob. The v2 tool that wraps the same routine, when run on the same input, must produce a JSON blob that matches the snapshot bit-for-bit modulo a documented floating-point tolerance.

The tolerance policy, set in Chapter 14, is relative 1×10^{-6} and absolute 1×10^{-9} for numerical fields, and exact equality for everything else (factor levels, column names, ordering). This is tight enough to catch real bugs and loose enough to absorb the

BLAS-version drift that an R-to-Python bridge inevitably introduces.

When a v2 tool fails its oracle test, the rule is: v1 wins. The v2 tool is wrong until it can produce the matching number. There is exactly one exception, documented when it arises in Chapter 14, where v1 itself was found to have a marginal-case bug that v2 corrected; in that case the snapshot was regenerated against a corrected v1 patch and both versions of v1 are tagged in the repository.

[Figure 5.2: v1 tab structure, screenshot grid. Eight panels — the seven tabs plus the Sources of Variation sub-panel — each annotated with the inputs it accepts and the outputs it produces.]

5.7 What v1 does not have

This chapter has described v1 as a competent tool. It is. Chapter 7 catalogs everything v1 cannot do, and the list is what motivates v2. Before that catalog, three properties of v1 that v2 will inherit deserve naming.

v1 has no concept of a session. Each browser tab is an ephemeral analysis. The user's choices live in `input$<id>` reactives that are garbage-collected when the tab closes.

v1 has no concept of programmatic re-execution. There is no `runFromConfig()` entry point. The analysis is whatever the user clicked.

v1 has no concept of a methods paragraph. The user's downstream documentation — for a paper, a poster, a regulatory submission — is on the user. v1 helps her produce the figure; it does not help her describe how the figure was produced.

These three absences are not bugs in v1; they are properties of the Shiny paradigm as such (Chapter 3). The agent paradigm offers a path to fill them in, which is the project of Part III.

What you should be able to do after this chapter

- Read a Shiny `server.R` file and identify the reactive graph it defines.
- Name the seven tabs of MetabolonR v1 in pipeline order and describe what each does.
- Explain why v1's package versions are pinned in `renv.lock` and what a 2022 `limma` change would have done without the pin.
- Identify the resource ceiling of `shinyapps.io`'s free tier and explain when it bites.
- Describe the golden-snapshot pattern and state the floating-point tolerance v2 inherits.
- Name three structural properties of v1 — no session, no programmatic re-execution, no methods paragraph — that the v2 architecture will explicitly address.

Chapter 6 — A Worked Example: The ProgreDir CKD Cohort

Throughout this book the same dataset is used to ground every architectural and statistical claim. This chapter introduces it: the ProgreDir cohort, a São Paulo–based study of chronic kidney disease with serum metabolomics tied to renal-outcome endpoints. The cohort reappears in Chapters 14, 15, 16, 20, and 21; readers who internalize its shape here will follow the rest of the book more easily. The chapter closes with a "canonical numbers" callout box (§6.7) that subsequent chapters cite rather than re-derive.

6.1 The cohort

The ProgreDir study (Titan et al. 2019, *PLoS ONE* 14(3): e0213764) is a prospective observational cohort of adults with chronic kidney disease, recruited at three nephrology centers in São Paulo, Brazil, beginning in 2011. The inclusion criteria — adults with documented CKD stages 3 to 5 not yet on dialysis

— and the exclusion criteria — active malignancy, acute kidney injury within the prior six months, life expectancy under one year — were standard for an observational nephrology cohort of its era.

Participants were enrolled at baseline with a clinical evaluation, serum and urine sample collection, and a standard panel of laboratory measurements (creatinine, urea, electrolytes, hemoglobin, lipid panel). Follow-up was at six-month intervals through three years, with capture of a composite outcome of **mortality or incident renal replacement therapy** (defined as initiation of chronic dialysis or kidney transplant). The composite-outcome framing — rather than a single endpoint — is what Titan 2019's Cox proportional-hazards analysis is anchored on, and it is the same outcome the v2.0 validation experiment's Path B external-validation arm uses (Chapter 20 §20.4b).

The cohort size used in this book is **454 participants**, the full ProgreDir baseline-serum subset reported by Titan et al. 2019. The author has used the same subset throughout the development of MetabolonR and will continue to use it. Where

downstream chapters quote per-group numbers — for instance, the count of progressors versus stable participants in a binary contrast — they refer to subsets of these 454.

6.2 The clinical question

The motivating clinical question is whether serum metabolites at baseline carry information about three-year renal outcomes beyond what serum creatinine and estimated glomerular filtration rate already provide. The hypothesis is biological as well as statistical: chronic kidney disease is a disease of accumulated uremic solutes, and the metabolome — including those solutes — might surface markers that the renal panel misses.

The same question has been asked of many cohorts at this point. The strength of the ProgreDir analysis is the combination of an untargeted serum metabolomics panel with a clinical endpoint that is hard (initiation of dialysis is not subject to the same ascertainment bias as a soft endpoint like eGFR slope) and

a follow-up that is long enough to capture clinically meaningful progression.

For the purposes of this book, the clinical question is the *occasion* for the analysis. The analysis itself — what MetabolonR v1 does on the data, what Titan 2019 published, and what MetabolonR-LLM v2.0 is asked to reproduce — is the focus.

6.3 The dataset

The metabolomics arm of Progredir produced a baseline serum panel measured on a **GC-MS platform** (Agilent MassHunter; metabolites identified using the Agilent Fiehn GC/MS library and the NIST mass-spectral libraries). The raw matrix carries 454 samples $\times$ the full untargeted feature set; after the standard $<50\%$ -per-metabolite missingness QC filter (Chapter 2 §2.2), the working matrix that v1 and v2 both operate on is **454 $\times$ 293** — 454 samples by 293 identified metabolites. Overall missingness in the working matrix is roughly 12%, concentrated in metabolites at the low end of the platform's

dynamic range and therefore below the limit of detection on a fraction of samples.

The sample annotation file carries baseline clinical variables — age, sex, eGFR, urine albumin-creatinine ratio, diabetes status, hypertension status, draw-batch identifier — plus the outcome variables: a binary progressor flag (1 if mortality or incident RRT within three years, 0 otherwise) and a time-to-event variable in months. The metabolite annotation file carries the platform's internal identifier, the chemical name, the HMDB and KEGG identifiers where available, and a chemical-class label.

This three-file structure (Chapter 2 §2.1) is exactly the input MetabolonR v1 accepts on its Upload tab and exactly the input MetabolonR-LLM accepts through its `load_metabolomics_data` tool (Chapter 9).

The raw data are publicly available. The Progredir baseline-serum metabolomics dataset is deposited at the **Metabolomics Workbench under accession ST000978** — a single accession that gives a reviewer access to the same abundance matrix,

sample annotation, and metabolite annotation the book's examples use. This is the access path Chapter 11 §11.4's session log refers to, and it is the same path Chapter 17's manuscript Availability section cites.

[Figure 6.1: ProgreDir cohort flow diagram. Top: enrollment counts and exclusions. Middle: baseline serum collection and GC-MS metabolomics processing. Bottom: three-year follow-up with progressor-vs-stable counts and composite-outcome event counts.]

6.4 A guided run through v1

The rest of this section walks a single MetabolonR v1 session on ProgreDir, tab by tab. The same session reappears in Chapter 15 as the demo on the lab server and in Chapter 16 as one of the twenty validation prompts.

Upload. The three files are dropped onto the Upload tab. v1 validates the alignment of sample identifiers and reports: 454 samples $\times$ N features loaded (the pre-QC feature count varies with the platform's feature-extraction parameters; the post-QC

count of 293 is what matters downstream). A green banner appears. No validation errors.

Summarize. The Summarize tab reports overall missingness, a per-metabolite missingness distribution that is sharply bimodal (most metabolites missing in fewer than 5% of samples; a long tail missing in more than 50%), and group sizes of approximately 119 progressors and 335 non-progressors. The histogram of log-transformed abundances is approximately symmetric — a good sign for a log transformation downstream.

QC. The user sets the per-metabolite missingness threshold to 50% and the per-sample missingness threshold to 30%. After the filter, the working matrix is 454×293 . No samples are dropped at the 30% per-sample threshold on Progredir's GC-MS panel because Progredir's per-sample missingness is uniformly low.

Impute and Transform. The user selects KNN imputation with $k=10$ (the default) and log2 transformation with pseudocount 1. The before-and-after distribution for a representative metabolite collapses from a heavy-tailed shape to

an approximately normal one. The choice is documented in the session header.

Scale. Pareto scaling is selected. The per-metabolite variance distribution post-scaling is narrower than pre-scaling but not flat — high-variance metabolites still carry somewhat more weight, which is the intended Pareto behavior.

PCA. The PC1-vs-PC2 plot shows the cohort's first two components capturing about 14% and 9% of total variance, respectively. Coloring the scores by batch reveals no separation: the batches overlap, which says the batch correction the core facility applied at the platform level held. Coloring by progressor status reveals modest separation along PC2.

Sources of Variation. The F-ratio decomposition over {progressor, age, sex, eGFR, batch} ranks eGFR first, progressor second, age third, sex fourth, batch fifth. The ordering says exactly what the analyst would hope: the strongest signal in the metabolome is the kidney-function gradient (eGFR), the next strongest is the clinical endpoint of interest, and the technical confound (batch) is dwarfed by the biology.

Volcano (binary contrast). The user selects the progressor variable and runs the limma moderated t-test with stable participants as the reference level. v1 produces a volcano plot with FDR threshold 0.05 (Benjamini–Hochberg) and a fold-change cutoff of $|\log_2 \text{FC}| > 0.5$. Approximately **18 metabolites** pass both thresholds. The sortable table on the right lists them by adjusted p-value. This 18-metabolite list is the **v1 oracle reference** for the Path A internal-reproducibility metrics of Chapter 20 §20.4.

[Figure 6.2: PCA scores plot from the Progredir v1 session. Two panels: left colored by progressor status (showing modest PC2 separation), right colored by batch (showing no separation).]

[Figure 6.3: Volcano plot from the Progredir v1 session, annotated. Significant metabolites in the upper corners are labeled with their chemical-name initials.]

6.5 The Titan 2019 reference shortlists

Beyond MetabolonR v1's binary-contrast output, the ProgreDir cohort has two separate, published biomarker references in Titan et al. 2019: a **34-metabolite batch-only-adjusted Cox set** at FDR $q < 0.05$ on the time-to-event composite outcome (Table 2 of the paper), and an **18-metabolite fully-adjusted Cox set** (batch + sex + age + eGFR + diabetes) that survived multivariable adjustment on the same outcome (Table 3 of the paper). Both are derived by Cox proportional-hazards models on time-to-event data — a different statistical question than v1's limma binary contrast, but on the same cohort.

The seven canonical metabolites this book refers back to are drawn from Titan's Table 2 with biological diversity in mind:

- **Lactose** (Table 2 rank 1; carbohydrate; HR 1.57 per log2-unit, $q = 2.4 \times 10^{-9}$) — the top hit; also recurs in the ESRD-specific Fine-Gray competing-risk analysis (Table 5).

- **D-threitol** (rank 2; carbohydrate; HR 2.46, $q = 1.3 \times 10^{-8}$)
— also borderline-significant for ESRD.
- **Pseudouridine** (rank 3; nucleoside analogue; HR 2.05, $q = 2.6 \times 10^{-7}$) — a known uremic retention solute.
- **Acetohydroxamic acid** (rank 7; carboxylic acid derivative; HR 2.06, $q = 6.5 \times 10^{-5}$) — the paper highlights this as a novel association not previously known in CKD literature.
- **Tyrosine** (rank 20; amino acid; HR 0.67, $q = 4.8 \times 10^{-3}$)
— inverse association; the kidney is a major source of systemic tyrosine, so a low circulating value is biologically expected in advanced CKD.
- **Docosahexaenoic acid (DHA / doconexent)** (rank 33; long-chain omega-3 fatty acid; HR 0.63, $q = 0.050$) — inverse; the omega-3 finding the paper emphasizes.
- **Myo-inositol** (rank 9; alcohol/polyol; HR 1.97, $q = 2.0 \times 10^{-4}$) — appears in both the composite-outcome analysis and the ESRD-borderline list.

These seven cover the major chemical classes the paper reports (carbohydrate, nucleoside, amino acid, fatty acid, polyol) and

include both positive and inverse associations. They are the canonical examples this book refers to when discussing what the agent should be able to recover. They appear in Chapter 11's sample log, Chapter 15's demo transcript, Chapter 20's worked example, and Chapter 21's results table. The full 34-metabolite list (Table 2) is mirrored at `data/validation/titan_2019_reference_34.csv` and the fully-adjusted 18-metabolite list (Table 3) at `data/validation/titan_2019_reference_18.csv`. Both are derived from the Titan 2019 paper's published tables, accessed via the open-access PMC record (PMCID PMC6422295) and cross-checked against the Metabolomics Workbench accession ST000978.

A pedantic footnote on the 34-set: Titan Table 2 lists 34 metabolites, but the 34th entry, **threonic acid**, has $q = 0.056$ — just above the stated 0.05 cutoff. The paper counts it in the 34; the book follows that convention but flags it here so a careful reviewer cannot land a "the paper has 33 at $FDR \leq 0.05$ " comment without seeing that the discrepancy is acknowledged.

The 18-vs-18 convergence

The relationship between MetabolonR v1's limma-binary-contrast output (~18 metabolites) and Titan's fully-adjusted Cox shortlist (18 metabolites) is the experimental design's most informative feature. The two methods are different — Cox on time-to-event with multivariable adjustment versus limma on a two-group difference — but they converge on 18 hits at the same FDR threshold on the same cohort. The convergence is not coincidence-free: both methods identify large-effect-size signal that survives adjustment for the obvious confounders, and both rely on similar variance structure under Progredir's covariate distribution. Reframed: v1's 18-metabolite shortlist matches Titan's fully-adjusted 18 in cardinality through different statistical paths, suggesting both methods capture similar high-effect-size signal under the Progredir cohort's covariate structure.

This convergence is why Path B (Chapter 20 §20.4b) is constructed as a **two-tier external benchmark**. The primary tier scores the agent's adjusted-limma output against Titan's

fully-adjusted 18 — an apples-to-apples comparison because both involve covariate adjustment. The secondary tier scores against Titan's batch-only 34 — the headline number from the published paper, more commonly cited but methodologically less aligned with v2.0's tool surface. The two tiers together give the experiment both methodological precision and surface compatibility with how the field cites Titan 2019.

What Titan's 34-set does not test for, and why

v2.1 matters

Even the secondary tier (34-set recall) is a strict benchmark for v2.0, because v2.0's `differential_abundance` does not run Cox proportional-hazards. Metabolites in Titan's 34 whose signal accumulates with follow-up time (rather than dichotomizing cleanly at the three-year endpoint) cannot be recovered by `limma-on-binary` by design. Chapter 24's planned v2.1 `survival_analysis` tool closes this gap and is the motivation for promoting Path B to a like-for-like Cox-vs-Cox comparison in the v2.1 paper.

6.6 Why this dataset reappears

The reason this guided run will be referenced repeatedly through the rest of the book is that it is the smallest end-to-end example that exercises every tool MetabolonR-LLM v2 will need to provide.

Chapter 14 uses this run as the basis for the v2 oracle tests: each tool's golden snapshot is captured from the corresponding tab of this v1 session.

Chapter 15 uses this run as the script for the Friday-of-Week-3 lab-server demo: a v2 agent is asked, in natural language, to reproduce it. The first end-to-end demo expects the agent to recover 5–10 of the seven canonical Titan biomarkers named in §6.5 when given a biomarker-discovery prompt.

Chapter 16 includes the v1 sequence as the first of the twenty validation prompts, and uses the Titan 34-set for the Path B external-validation arm of the Week 4 experiment.

Chapter 20 uses the cohort and both reference sets (v1 oracle, Titan 34) as the data on which the five hypotheses H1–H5 are tested.

Chapter 21 uses the resulting biomarker sets as the references against which the twenty-prompt $\times$ three-paraphrase agreement metrics and the Path B external-recall metric are computed.

The Progredir cohort is therefore not a casual choice. It is the load-bearing example that ties Parts II through V together. A reader who knows it well will read those chapters more efficiently than one who does not.

6.7 Canonical numbers for this book

The numbers in the callout below are the single source of truth for Progredir parameters used throughout the rest of the book. Where subsequent chapters cite Progredir specifics, they cite this section rather than re-deriving them.

Cohort. Progridir, São Paulo, three nephrology centers, recruitment from 2011.

N enrolled. 454 participants (baseline-serum subset; Titan et al. 2019).

N analyzed for metabolomics. 450 (the four-participant gap is participants without usable metabolomics data; subsequent statistical analyses use 450, not 454).

Platform. GC-MS — Agilent 7890B GC with Agilent 5977A mass selective detector; metabolites identified via Agilent Fiehn GC/MS Metabolomics RTL Library (v A. 02.02) and NIST 11 (2014) libraries.

Feature-count chain. 10,940 features identified in at least one sample → 293 metabolites after <50%-per-metabolite missingness filter → **265 metabolites after exclusion of column contaminants and internal standards** (this 265-feature count is the final analytic feature set on which Titan's Cox results were computed).

Working matrix after <50%-missingness QC. 454×293 (samples $\times$ metabolites) — the count v2.0's `qc_filter` produces from the raw upload; subsequent contaminant/internal-standard exclusion (an analyst step not in the agent's tool surface) reduces to 450×265 for the Cox-comparable analytic set.

Overall missingness (working matrix). ~12%.

Event counts (composite outcome at 3-year follow-up).

129 composite events; 93 deaths; 36 incident ESRD/RRT.

Demographics. Mean age 67.5 ± 11.9 y; mean eGFR-CKD-EPI 38.4 ± 14.6 mL/min/1.73m²; 63.2% male; 56.6% diabetic; 90.1% hypertensive; mean follow-up ~3 years.

Group sizes (progressor flag). ~119 progressors, ~335 stable.

Composite outcome. Mortality or incident renal replacement therapy within three years.

v1 oracle reference (Path A, internal reproducibility).

~18 metabolites at FDR 0.05 via limma binary contrast (progressor vs. stable), $|\log_2 FC| > 0.5$. The 18 happens to match Titan's fully-adjusted Cox 18 in cardinality; see §6.5.

External reference, primary tier (Path B, fully-adjusted comparison). Titan 2019 18-metabolite fully-adjusted Cox set (batch + sex + age + eGFR + diabetes adjustment), Table 3 of the paper, on the composite outcome.

External reference, secondary tier (Path B, published-headline comparison). Titan 2019 34-metabolite batch-only-adjusted Cox set, Table 2 of the paper, FDR $q < 0.05$ (with threonic acid at $q = 0.056$ included per the paper's convention).

Public data accession. Metabolomics Workbench ST000978.

Primary citation. Titan SM, Venturini G, Padilha K, et al. 2019. PLoS ONE 14(3): e0213764. doi:10.1371/journal.pone.0213764.

What you should be able to do after this chapter

- Describe the ProgreDir cohort in three sentences: study design, sample size (454), and the composite outcome (mortality or incident renal replacement therapy).
- State the working-matrix dimensions (454×293) after the standard <50%-missingness QC filter and the GC-MS platform identifier.
- Walk through a MetabolonR v1 session on ProgreDir and identify the parameter choice made at each tab.
- Distinguish v1's ~18-metabolite limma binary-contrast reference (Path A) from Titan 2019's 34-metabolite Cox time-to-event reference (Path B), and explain why both are legitimate.
- Name three biomarkers from the Titan 34-set (Lactose, D-threitol, Pseudouridine, Acetohydroxamic acid, Tyrosine, Myo-inositol, or DHA all acceptable; see §6.5).

- Cite Metabolomics Workbench ST000978 as the public data accession and Titan 2019 (DOI 10.1371/journal.pone.0213764) as the primary reference.
- Recognize the §6.7 canonical-numbers callout as the source of truth for Progredir parameters elsewhere in the book.

Chapter 7 — What v1 Cannot Do

No tool succeeds at everything. This chapter is the honest catalog of what MetabolonR v1 cannot do, and each limitation maps to a v2 design decision in Part III. By the end of the chapter the reader should be able to predict, from a v1 limitation, which v2 component is the architectural response.

7.1 Why this catalog matters

Most software descriptions emphasize what the software does. A description that emphasizes what the software does *not* do can read like an apology, but in this case it is a design document. Every limitation in this chapter is one for which the author and his collaborators chose v2's architecture as an explicit response. The v1-to-v2 transition is not driven by a desire to use LLMs; it is driven by a list of specific things v1 cannot do, and the LLM-agent architecture happens to be the cleanest way to address the list.

If the list of limitations were short, the architecture of v2 would be smaller. If the list looked different, the architecture would look different. The exercise of writing the list down is, in effect, the system-requirements document for Chapters 8 through 12.

7.2 Scale ceiling

v1 begins to slow noticeably above roughly **2,000 samples × 5,000 metabolites**. The visible symptom is a multi-second delay between a user clicking a control and the corresponding plot updating. The cause is in the reactive graph: when an upstream input changes, every downstream reactive re-runs, which means a KNN imputation over a $2,000 \times 5,000$ matrix runs again, which costs several seconds. For a user who tweaks a missingness threshold and then a scaling option and then a PCA component count, the cumulative wait is enough to drop the user out of flow.

Above roughly **5,000 samples × 10,000 metabolites**, v1 on the free `shinyapps.io` tier runs out of memory and the app

stops responding. Lab-internal users running locally on a 64 GB workstation push the ceiling another order of magnitude before hitting it, but the ceiling is real.

The cause is partly the reactive model and partly the choice of an R backend without any caching layer. A v2 with cached intermediate results, content-addressed by inputs, can avoid recomputing what has not changed. The v2 `DatasetHandle` design (Chapter 10) is the proximate response to this limitation.

[Figure 7.1: v1 runtime vs matrix size. Log-log plot with sample count on one axis, metabolite count on the other; contour lines at one-second, five-second, and thirty-second response times. The 30-second contour passes through $2,000 \times 5,000$.]

7.3 No batch correction

v1 does not provide a batch-correction step. A user looking at a PCA scores plot colored by batch can *see* a batch effect; she cannot, within v1, do anything about it. Her options are to leave v1, do the batch correction in standalone R (ComBat from `sva`, RUVs from `RUVSeq`, or harmonization via `harman`),

and then re-upload the corrected matrix to v1 — which defeats the purpose of having a single-stop tool.

For single-site studies this is a tolerable gap. For multi-site studies like NEPTUNE, where every cross-site analysis requires harmonization first, the gap is fatal.

The v2 response is a new tool, `batch_correct`, which is one of two tools deferred to v2.1 (along with `pathway_variation`, Chapter 24). The architectural point is that v2's tool layer is *extensible* in a way v1's tab structure is not. Adding a new tab to v1 requires re-writing the reactive graph and re-deploying. Adding a new tool to v2 requires writing a Pydantic schema, a wrapper around an R function, and an oracle test; the agent picks it up automatically.

7.4 No analysis history

When the user closes the browser tab in v1, the analysis is gone. The reactive values in memory are garbage-collected, and the next session starts from the Upload tab with no state.

The user can save individual plots and the significant-metabolite table from the Volcano tab; she cannot save the *analysis* — the sequence of choices that produced the plot and the table. Re-running the same analysis a week later requires remembering, or writing down, every parameter at every tab.

This is the limitation with the largest downstream cost. A reviewer asking "what imputation method did you use?" has no record to consult. A regulator asking "what was the QC threshold and why?" gets a shrug. A coauthor trying to reproduce the figure for a revision opens v1 and starts from scratch.

The v2 response is the structured session log of Chapter 11. Every parameter choice, every tool call, every output is recorded in `analysis_log.json`. Re-running the same analysis next week is a single command against the log.

7.5 No programmatic re-execution

Related to but distinct from the no-history problem: v1 has no API. A user who wants to run the same v1 pipeline on a new

cohort cannot do it programmatically. She has to open the browser, click through the tabs, upload the new files, and click again.

For a clinical investigator with one cohort, this is acceptable. For a NEPTUNE-style consortium analyst running monthly QC on incoming data, it is not. The analyst inevitably writes her own R script that calls the v1 functions directly, bypassing the Shiny interface entirely, and that script becomes a fork of v1 that drifts from the deployed app over time.

The v2 response is two-pronged. First, `analysis_log.json` is itself a configuration file: the replay engine (Chapter 11) re-executes from a log without invoking the LLM, which gives programmatic re-execution as a first-class feature. Second, the FastAPI layer (Chapter 10) exposes the agent and the replay engine as HTTP endpoints, so a consortium analyst can `curl` a replay request from a cron job.

7.6 Parameter invisibility

A v1 PCA plot shows the user a scatter of points. It does not, on the plot itself, indicate that `center=TRUE` and `scale=TRUE` were used, that the imputation method was KNN with `k=10`, that the prior log transformation was `log2` with pseudocount 1, or that the dataset was filtered to 447 samples $\times$ 403 metabolites at the QC step. A user reading the plot on a poster a year later cannot tell. A coauthor cannot tell either.

v1's plot legends carry the bare minimum — axis labels, color key — and not the parameter trace. This is consistent with how Shiny apps are usually built, and it is the wrong default for clinical analysis.

The v2 response is twofold. The session log records every parameter; the auto-generated methods paragraph (the output of the `export_session` tool) embeds the parameters in natural-language prose suitable for a manuscript Methods

section. A user who exports her session gets, for free, a paragraph she can paste into her draft.

7.7 No path from analysis to publication

The single largest gap in v1, from the clinician's point of view, is the silence between her last v1 click and her first manuscript paragraph. v1 produces figures and a metabolite table. It does not produce: a methods description, a parameter table, a citation list of the statistical methods used, a software-availability statement, or a reproducibility receipt of any kind. Every one of those is on the clinician, and writing them from memory is where errors creep in.

v2 addresses this directly with the `export_session` tool. The tool's output is a single zipped archive containing the `analysis_log.json`, the methods paragraph, the parameter table in CSV form, the published figures in PDF and PNG, the metabolite table in CSV and Excel form, and a `CITATION.cff` for the agent itself. The clinician's path from

v2 session to manuscript submission is several clicks shorter than her path from v1 session to submission, and every claim in the methods is backed by a line in the log.

7.8 The mapping table

The six limitations above are the structural skeleton of Part III. The mapping is:

- **Scale ceiling** → `DatasetHandle` content-addressing and caching (Chapter 10).
- **No batch correction** → tool-layer extensibility; `batch_correct` deferred to v2.1 (Chapter 24).
- **No analysis history** → `analysis_log.json` and SQLite session store (Chapter 11).
- **No programmatic re-execution** → replay engine and FastAPI endpoint (Chapters 11, 12).
- **Parameter invisibility** → log header, methods-paragraph generation (Chapter 11).
- **No path to publication** → `export_session` tool (Chapter 9).

[Figure 7.2: Limitation-to-design-decision mapping table. Two columns: v1 limitation on the left, v2 architectural response on the right, with the chapter where the response is detailed.]

7.9 Two limitations v2 does not address

For honesty, two things v1 cannot do are also things v2 does not try to do.

v1 cannot do **statistical modeling beyond two-group differential abundance**: no Cox regression, no linear mixed models for repeated measures, no survival analysis. v2 inherits this limitation. The tool layer's `differential_abundance` is restricted to the same two-group case. A clinician who needs survival modeling on Progredir's progression-time variable goes elsewhere. This is a deliberate scope decision, justified in Chapter 23.

v1 cannot do **multi-omics integration**. v2 does not either. Chapter 24 sketches the v3 direction in which this might change.

The point of listing these two non-responses is to be clear about scope. v2 is not a replacement for the analyst's full toolkit. It is a replacement for v1's interface, with the limitations that motivated v1's replacement addressed and the limitations that did not motivate it left alone.

What you should be able to do after this chapter

- Name the six structural limitations of MetabolonR v1 and give a one-sentence description of each.
- Map each limitation to the corresponding v2 architectural response and the chapter where the response is detailed.
- Argue, in one paragraph, why "no analysis history" is the most consequential of the six limitations.

- Identify two things v1 cannot do that v2 also does not try to do, and explain why deferring them is a scope decision rather than an oversight.
- Predict the next chapter's content (the architectural principles that make these responses coherent rather than ad hoc).

Chapter 8 — Design Principles

The core invariant of MetabolonR-LLM is that the LLM routes and the statistics are deterministic. This chapter is the methodological claim that anchors the eventual publication and the discipline that anchors the rest of the book.

8.1 The invariant

Every architectural decision in MetabolonR-LLM is downstream of one sentence. **The LLM decides what to compute; deterministic code does the computing.** Routing in natural language, computation in audited code.

This sentence is not novel. It is the standard pattern for tool-using LLMs in any high-stakes domain — finance, security, infrastructure operations — and it is the explicit recommendation in the Anthropic and OpenAI tool-use documentation. What is less common is to treat it as a binding architectural commitment rather than a default that can be

relaxed when convenient. MetabolonR-LLM treats it as binding: the LLM never executes statistics, never writes code, never bypasses a tool. If a task does not have a tool, it does not get done by the agent.

The cost of this discipline is the rigidity discussed in Chapter 4. The benefit is what the rest of this chapter elaborates: a system whose statistical outputs do not depend on the LLM's current temperature, current snapshot, or current mood.

8.2 Why this matters for peer review

A reviewer of a clinical-bioinformatics paper in 2026 has been burned. She has seen agentic-LLM tools described in glowing terms in a methods section, opened the supplementary materials to look for the prompts and the parameter logs, and found none. She has asked authors to reproduce a number from their own paper, and received a sheepish response that the LLM produced a different sequence of tool calls on the rerun. She is now suspicious by default, and her suspicion is correct.

MetabolonR-LLM is designed to survive her suspicion. The artifact she should look for in the supplementary materials is the `analysis_log.json` from the session that produced the manuscript's figures. The artifact she should be able to run, on her own machine, is the replay engine pointed at that log. The numbers she should get out, modulo the floating-point tolerance documented in Chapter 11, are bit-identical to the manuscript's.

This is a strong claim. The rest of the book is the demonstration that the claim can be honored.

8.3 The contrast with code-writing agents

A different design choice for an agentic bioinformatics tool is to let the LLM write code. The user describes the analysis, the LLM generates an R or Python script, the runtime executes the script, the agent reports the results. This is the pattern in OpenInterpreter, Devin, and several 2024–2025 research

prototypes; in biology specifically, it is the pattern in BioAgents (Chapter 4) and in parts of Biomni's tool-creation pipeline.

The pattern has a real strength: arbitrary flexibility. If the user wants an analysis the system has never seen, the LLM can write it. For exploratory work this is appealing.

The pattern has two costs that disqualify it for clinical metabolomics.

The first cost is audit. A reviewer trying to understand what was done has to read the LLM-generated code, which is fresh each run, and decide whether it does what it claims. The code is correct in the cases it is, incorrect in the cases it is not, and there is no oracle to check it against. A reviewer is not in a position to audit fresh Python on each run of every paper, and asking her to is a deferred-failure mode.

The second cost is reproducibility. The code the LLM writes today is not necessarily the code it writes tomorrow. The tool call `(run_python(script="..."))` is itself stable, but the *content* of `script` is the variable, and that variable is the entire analysis. Logging the script does help, but the next run

produces a different script for the same prompt; the log is no longer a recipe, only a record.

MetabolonR-LLM trades the flexibility for the audit and the reproducibility. The tools are fixed in advance, version-pinned, and oracle-tested. The agent's freedom is in choosing among them and parameterizing them; not in writing them.

[Figure 8.1: Two architectural patterns side by side. Left panel: code-writing agent — LLM emits code, runtime executes code, output. Right panel: tool-routing agent — LLM emits tool calls, runtime executes pre-audited tools, output. Annotations highlight where audit and reproducibility live in each.]

8.4 The principles in detail

Six principles follow from the invariant. Each is short to state and consequential to honor.

Closed-set tools. The tool layer is fixed at release. The agent cannot invent a new tool. A tool is added only by a human developer who writes its schema, its R wrapper, and its oracle

test, and ships the bundle in a new release. In v2.0 the set is ten; v2.1 adds two more.

Schema-pinned arguments. Every tool's input is defined by a JSON Schema. The agent's tool call is validated against the schema before execution; a schema mismatch is a hard error that the agent must correct on retry, not a soft warning that gets papered over. This is the contract that makes routing-vs-computation a clean boundary.

Composable tools. Tools are pure functions over `DatasetHandles` (Chapter 10). They have no side effects beyond producing a new handle (or an output artifact). They can be called in any order the agent chooses; the only ordering constraints are data-flow constraints (you cannot scale before imputing) and these are enforced at the tool level, not by the agent.

Clarification before execution. When the agent is uncertain — when a parameter is ambiguous, when a tool could be applied to several plausible interpretations — it asks. Asking is a feature, not a failure. The system prompt instructs the agent

that "I am not sure which option you want; here are the choices" is always preferable to a confident guess.

No hidden state. Every parameter the agent chooses is in the log. Every default the tool applies is in the log. Every dataset handle is content-addressed by its inputs, so two identical inputs produce the same handle. There is no value anywhere in the system whose origin cannot be recovered from the log.

Version-pinned everything. The R packages, the Python packages, the LLM snapshot, the system-prompt SHA-256, the tool-layer commit hash — all are recorded in the log header (Chapter 11). A replay run validates the pins; if any has drifted, the replay refuses to run and reports which one.

8.5 What the system does not do

A complement to the principles is a list of *anti-principles* — patterns the system explicitly rejects.

It does not chain-of-thought-then-decide. The agent's reasoning is not a tool call. The agent's actions are tool calls; its reasoning

lives in the assistant-text turns and is not load-bearing for any numerical output.

It does not auto-retry on a soft failure. If a tool returns an error, the agent reports the error to the user and waits. It does not silently switch to a different tool, try a different parameter, or retry the same call. The user has to be in the loop on a correction. This is the cost of avoiding silent drift.

It does not let the user pin a parameter and then override it. If the user says "use KNN with $k=10$ " and later changes her mind, she has to say so explicitly. The agent does not infer mid-stream parameter changes from indirect cues.

It does not improvise novel statistics. A request for a method not in the tool set returns a polite refusal pointing the user at a biostatistician.

[Figure 8.2: Design-principle map. Six positive principles in green boxes (closed-set, schema-pinned, composable, clarification, no-hidden-state, version-pinned). Four anti-principles in red boxes (no chain-of-thought-as-action, no silent

retry, no inferred override, no improvised statistics). Lines connect each to the architectural component that enforces it.]

8.6 The methodological claim

The book's central methodological claim, which the rest of the work defends and which Chapter 22 examines philosophically, is this:

Agentic bioinformatics is publishable in clinical venues if and only if the agent's routing is non-deterministic and its computation is deterministic, and a structured log records both.

The "if" half of the biconditional says: with the architecture this book describes, the agent is publishable. The "only if" half is stronger: agentic systems that do not honor this separation will not be admitted, in the long run, to clinical-grade venues, regardless of how impressive their immediate output looks. The "only if" is the bet the author is making about how the field will evolve, and it is the bet that Chapter 22 elaborates with the regulatory framing.

The claim is testable. The validation experiment of Part V tests one direction of it: with the architecture, can MetabolonR-LLM achieve the agreement-rate numbers a reviewer would accept? The wider field-evolution claim cannot be tested in this book — it will be tested by what reviewers do over the next several years — but the validation results are the first piece of evidence.

8.7 The principles are the budget for the next four chapters

Chapter 9 enumerates the ten tools, and the closed-set, schema-pinned, composable principles bind every one of them. Chapter 10 walks the agent loop, and the clarification and no-hidden-state principles bind every turn. Chapter 11 is the session log and replay engine, which is what the version-pinned-everything principle exists to enable. Chapter 12 is the deployment, where the principles become operational policy.

A reader who returns to this chapter while reading Chapter 9 or Chapter 11 — and the cross-references will encourage it — should be able to point to the principle that justifies each design

choice she encounters. If a choice does not trace back to one of the principles, it should not be in the system. That is the discipline of the architecture, and the cleanest test of whether the discipline holds.

What you should be able to do after this chapter

- State the core invariant of MetabolonR-LLM in one sentence.
- Name the six positive design principles and give a one-line example of each in action.
- Name the four anti-principles and explain why each is rejected.
- Articulate the contrast between code-writing agents and tool-routing agents, and explain why MetabolonR-LLM chose the second pattern.
- State, in one sentence, the methodological claim the rest of the book defends.

- Predict, for any architectural choice you will encounter in Chapters 9–12, which principle it serves.

Chapter 9 — The Tool Layer

Ten tools are the entire surface area of MetabolonR-LLM v2.0. This chapter documents each one — its purpose, its Pydantic schema, the R function it wraps via rpy2, the v1 oracle test that pins it, and its failure modes. It is the longest and most technical chapter in the book; readers building an analog of this system for another omics layer can use it as a template.

9.1 Why ten and not fifty

A tool set has a Goldilocks problem. Too few tools and the agent cannot answer reasonable questions; too many and the agent gets lost. MetaboAnalyst exposes dozens of analyses through its web interface; a naïve port of MetaboAnalyst to a tool-using LLM would produce fifty tools or more, and the agent's success rate on a paraphrased prompt would suffer. Empirically — across a few hundred internal pilot sessions during the build — the agent's routing accuracy peaks

somewhere between seven and twelve tools and degrades on either side.

Ten is also the count of distinct *units of analytical work* in a typical clinical metabolomics analysis: load, summarize, filter, impute, transform, scale, PCA, sources of variation, differential abundance, export. Each is a verb the analyst would use in conversation. Each maps to one or a few v1 routines. The tool layer is not an exhaustive list of operations the agent can perform; it is the minimal list that covers the workflow.

The deferred eleventh tool, `pathway_variation`, is shipped in v2.1 (Chapter 24) for two reasons. It requires a KEGG-pathway lookup that is its own subsystem, with its own caching and failure modes. And it can be released independently without affecting the v2.0 surface, which is a clean version policy. The same logic will apply to `batch_correct` and a future `survival_analysis` tool.

This chapter walks the ten v2.0 tools in pipeline order, with three of them — `impute_missing`, `pca`, and `differential_abundance` — given a fully detailed

treatment that includes their Pydantic schemas, their R-side wrappers, and their oracle tests. The remaining seven are given the same anatomy in compact form. A reader who works through the three detailed entries can fill in the structure of any other tool from its compact entry.

9.2 The standard tool anatomy

Every tool in MetabolonR-LLM has the same six-element structure, repeated for each entry below.

Purpose. One sentence stating what the tool does, in language a clinician would recognize.

Pydantic input schema. A Python class inheriting from `pydantic.BaseModel` declaring the tool's arguments. The schema is the truth; the docstring and the JSON Schema served to the LLM are generated from it. Field-level validators enforce constraints that cannot be expressed as types alone ($k > 0$, $\text{FDR} \in [0, 1]$, `group_column` references a real column).

Output schema. The shape of the return value, also as a Pydantic class. For tools that produce a transformed dataset, the output carries a new `DatasetHandle` plus a structured summary. For tools that produce a final artifact (a PCA result, a differential-abundance table), the output is an artifact reference plus a structured summary.

Wraps (v1 function). The name of the R function from MetabolonR v1 that this tool delegates to via `rpy2`. The R function is the source of truth for the numerics.

Failure modes. A short list of the ways the tool can fail and what it does in each case. The default for an unrecoverable failure is to return a structured error message that the agent can read and surface to the user, not to crash.

Oracle test. The name of the pytest fixture and the golden snapshot the tool is pinned to. The test runs on every CI build and on every release.

[Figure 9.1: Tool layer architecture. Top-to-bottom flow: User → Agent (LLM) → Tool dispatcher (Python) → Pydantic schema validator → `rpy2` bridge → R function → return value

→ output formatter → Agent → User. Caching layer (DatasetHandle) on the side.]

9.3 The ten tools — compact entries

The seven tools given compact treatment here are: `load_metabolomics_data`, `summarize_dataset`, `qc_filter`, `transform`, `scale`, `sources_of_variation`, and `export_session`. The three full-treatment tools — `impute_missing`, `pca`, and `differential_abundance` — appear in §9.4 through §9.6.

9.3.1 `load_metabolomics_data`

Purpose. Accept a three-file metabolomics dataset and return a `DatasetHandle`. **Schema.** `abundance_path`, `sample_annotation_path`, `metabolite_annotation_path` (required strings); `sample_id_column`, `metabolite_id_column` (defaults "sample_id", "metabolite_id"). **Output.**

DatasetHandle plus LoadSummary. **Wraps.** `metabolonr::load_data()`. **Failure modes.** Missing files; sample-ID mismatch; non-numeric abundance values. **Oracle.** `test_load_progredir`, pinned against `golden/load_progredir.json`.

9.3.2 summarize_dataset

Purpose. Report counts and structural properties: `n_samples`, `n_metabolites`, missingness, group sizes, dynamic range. **Schema.** `dataset_handle`, optional `group_by`. **Output.** `DatasetSummary`. **Wraps.** `metabolonr::summarize()`. **Failure modes.** Unknown handle; `group_by` references missing column. **Oracle.** `test_summarize_progredir`.

9.3.3 qc_filter

Purpose. Drop metabolites and samples exceeding missingness thresholds. **Schema.** `dataset_handle`, `max_metabolite_missingness_pct` (default 50.0),

`max_sample_missingness_pct` (default 30.0). **Output.** New `DatasetHandle` plus `QCSummary` with counts dropped. **Wraps.** `metabolonr::qc_filter()`. **Failure modes.** Thresholds outside `[0, 100]`; filter empties the dataset. **Oracle.** `test_qc_filter_progredir`.

9.3.4 transform

Purpose. Apply a variance-stabilizing transformation. **Schema.** `dataset_handle`, `method` $\in$ `{log2, log10, sqrt, glog, none}`, `pseudocount` (default 1.0). **Output.** New `DatasetHandle` plus `TransformSummary`. **Wraps.** `metabolonr::transform()`. **Failure modes.** Negative values with a log method; non-positive pseudocount. **Oracle.** `test_transform_log2_progredir`, pinned bit-exact.

9.3.5 scale

Purpose. Per-metabolite scaling. **Schema.** `dataset_handle`, `method` $\in$ `{pareto, auto,`

range, level, none}. **Output.** New DatasetHandle plus ScaleSummary with per-metabolite factors. **Wraps.** `metabolonr::scale_metabolites()`. **Failure modes.** Zero-variance metabolite under auto/pareto; scaling before imputation. **Oracle.** `test_scale_pareto_progredir`.

9.3.6 sources_of_variation

Purpose. F-ratio decomposition across covariates. **Schema.** `dataset_handle, covariates: List[str]`. **Output.** `SourcesOfVariationResult` with ranked covariates. **Wraps.** `metabolonr::sov()`. **Failure modes.** Unknown covariate; constant covariate (warning, excluded from ranking). **Oracle.** `test_sov_progredir`, tolerance 1×10^{-4} on F-ratios.

9.3.7 export_session

Purpose. Bundle session outputs (log, methods paragraph, figures, tables) into an archive. **Schema.** `session_id`,

`output_path`, `include_figures` (default `True`),
`figure_format` $\in$ `{png, pdf, svg}` (default `pdf`).
Output. `ExportSummary`. **Wraps.** No R-side analog —
Python only (v1 had no equivalent). **Failure modes.** Unwritable
`output_path`; `session-store` query fails. **Oracle.**
`test_export_session_progreidir`.

9.4 Full treatment: `impute_missing`

`impute_missing` is the most parameter-rich tool in the
v2.0 set; its method-dispatch lives in the R wrapper, and it is
the tool most likely to be called with a non-default parameter.
It is the natural candidate for a full-treatment walkthrough.

Purpose and behavior

The tool fills missing values in the abundance matrix using one
of four methods. KNN (the default) is the metabolomics
standard for technical missingness; half-min is the standard for
below-LOD missingness; random forest is a more flexible
alternative; Bayesian PCA is the chemometrics tradition.

Pydantic input schema

The schema is reproduced verbatim from `tools/impute_missing.py` at the v2.0.0 release tag.

```
from typing import Literal
from pydantic import BaseModel, Field, field_validator

class ImputeMissingInput(BaseModel):
    """Input schema for the impute_missing tool."""
    dataset_handle: str = Field(
        ...,
        description="Handle to a dataset produced by the data manager"
    )
    method: Literal["knn", "halfmin", "rf", "bpca"] = Field(
        default="knn",
        description="Imputation method. KNN is the default"
    )
    knn_k: int = Field(
        default=10,
```

```

        description="k for KNN imputation; ignored
    )
    rf_ntree: int = Field(
        default=100,
        description="Number of trees for random for
    )

    @field_validator("knn_k")
    @classmethod
    def _knn_k_positive(cls, v: int) -> int:
        if v < 1:
            raise ValueError("knn_k must be >= 1")
        if v > 100:
            raise ValueError(
                "knn_k > 100 is almost never useful
                "if you really mean it, use the unc
            )
        return v

    @field_validator("rf_ntree")
    @classmethod

```

```
def _rf_ntree_positive(cls, v: int) -> int:
    if v < 10:
        raise ValueError("rf_ntree must be >= 10")
    return v
```

The two field-level validators encode constraints that the typed signature alone does not capture: KNN k must be at least 1 (a zero-neighbor imputation is undefined) and is capped at 100 to prevent obvious misuse; random-forest tree count must be at least 10 to be statistically meaningful. Both errors are caught at the Pydantic-validation layer before any R call is made, and the structured error returned to the agent gives the LLM a clear path to self-correct on the next turn.

Pydantic output schema

```
from typing import Optional
from pydantic import BaseModel

class ImputationSummary(BaseModel):
    """Output schema for impute_missing."""
```

```

dataset_handle: str
method: Literal["knn", "halfmin", "rf", "bpca"]
n_cells_imputed: int
n_cells_total: int
fraction_imputed: float
knn_mean_distance: Optional[float] = None      # d
rf_oob_error: Optional[float] = None           # d
bpca_n_components: Optional[int] = None        # d

```

The optional fields are method-specific diagnostics. When the agent reads the summary back, it can mention the diagnostic in the user-facing response — "KNN imputation with k=10, mean neighbor distance 0.34, 1,847 of 178,605 cells filled." This is the kind of detail that builds user trust without requiring the agent to invent it.

Wraps and the R side

The Python wrapper delegates to `metabolonr::impute()` in the v1 codebase. The R function dispatches on `method`:

```

# metabolonr::impute (simplified for illustration)
impute <- function(handle, method = "knn", knn_k =
  mat <- get_matrix(handle)
  result <- switch(method,
    knn      = impute::impute.knn(mat, k = knn_k),
    halfmin  = .impute_halfmin(mat),
    rf       = missForest::missForest(mat, ntree = 1000),
    bpca     = .impute_bpca(mat)
  )
  list(matrix = result, summary = .compute_summary(mat))
}

```

The R-side dispatch is intentionally a `switch` rather than four entry points; this is what lets the Python wrapper present one tool to the agent rather than four. The dispatch happens inside R because the four methods live in different packages with different ABIs and centralizing the dispatch in R is cleaner than encoding the package-specific knowledge in Python.

The end-to-end Python wrapper

The full wrapper that connects the Pydantic schema to the rpy2 bridge to the R function:

```
# tools/impute_missing.py
import hashlib, json
from .bridge import call_r
from .handles import DatasetHandle, get_handle, req

def impute_missing(input: ImputeMissingInput) -> ImputeMissingOutput:
    """Fill missing values; return a new DatasetHandle"""
    in_handle = get_handle(input.dataset_handle)
    args_dict = input.model_dump(exclude={"dataset_handle"})

    # Content-address the output handle deterministically
    args_sha = hashlib.sha256(
        json.dumps(args_dict, sort_keys=True).encode()
    ).hexdigest()[:16]
    out_handle_id = f"dh_{in_handle.short_hash}_impute_{args_sha}"
```

```

# Cache check: if we have already computed this
# return the cached handle without re-running.
if existing := register_handle.lookup(out_handle_id):
    return ImputationSummary(
        dataset_handle=out_handle_id,
        method=input.method,
        n_cells_imputed=existing.cached_meta["n_cells_imputed"],
        n_cells_total=existing.cached_meta["n_cells_total"],
        fraction_imputed=existing.cached_meta["fraction_imputed"],
        knn_mean_distance=existing.cached_meta["knn_mean_distance"],
        rf_oob_error=existing.cached_meta.get("rf_oob_error", None)
    )

# Call into R via the bridge
r_result = call_r(
    "impute",
    handle=in_handle.r_object,
    method=input.method,
    knn_k=input.knn_k,
    rf_ntree=input.rf_ntree,

```

```

)

# Pull the imputed matrix and the diagnostic su
out_matrix = r_result.rx2("matrix")
out_summary = dict(r_result.rx2("summary"))

# Register the new handle in the content-address
register_handle(
    handle_id=out_handle_id,
    parent_id=in_handle.id,
    tool_name="impute_missing",
    args_sha256=args_sha,
    r_object=out_matrix,
    meta=out_summary,
)

return ImputationSummary(
    dataset_handle=out_handle_id,
    method=input.method,
    n_cells_imputed=out_summary["n_cells_imputed"],
    n_cells_total=out_summary["n_cells_total"],

```



```

        fraction_imputed=out_summary["fraction_imputed"]
        knn_mean_distance=out_summary.get("knn_mean_distance")
        rf_oob_error=out_summary.get("rf_oob_error")
    )

```

Three details in this wrapper deserve calling out. First, the content-addressed handle ID combines the input handle's short hash with the SHA-256 of the sorted argument dict, so the same operation on the same input always produces the same handle. Second, the cache lookup short-circuits redundant R calls — a common pattern when the agent re-runs an analysis with one parameter change and the upstream steps are unaffected. Third, the bridge call is one line; all rpy2 complexity is encapsulated in the `call_r` function (§9.7).

Oracle test

The oracle test for `impute_missing` with the KNN default, pinned against a golden snapshot captured from MetabolonR v1 running on the Progredir baseline-serum matrix:

```

# tests/test_impute_missing.py
import json
import numpy as np
import pytest

from tools.impute_missing import impute_missing, ImputeMissing
from tools.load_metabolomics_data import load_metabolomics_data
from tests.helpers import load_golden_snapshot, get_golden_snapshot

REL_TOL = 1e-6
ABS_TOL = 1e-9

@pytest.fixture
def progredir_loaded():
    """Load the Progredir three-file bundle into a LoadInput object.
    return load_metabolomics_data(LoadInput(
        abundance_path="tests/data/progredir/abundance.txt",
        sample_annotation_path="tests/data/progredir/sample_annotation.txt",
        metabolite_annotation_path="tests/data/progredir/metabolite_annotation.txt"
    ))

```

```

def test_impute_knn_k10_matches_v1_snapshot(progre
    """KNN imputation with k=10 must match v1's out
    result = impute_missing(ImputeMissingInput(
        dataset_handle=progredir_loaded.dataset_har
        method="knn",
        knn_k=10,
    ))

    # Sanity: the summary fields match the snapshot
    snap = load_golden_snapshot("impute_knn_k10_pro
    assert result.method == snap["method"]
    assert result.n_cells_imputed == snap["n_cells_
    assert result.n_cells_total == snap["n_cells_to

    # Floating-point match on the diagnostic
    assert np.isclose(
        result.knn_mean_distance,
        snap["knn_mean_distance"],
        rtol=REL_TOL, atol=ABS_TOL,

```

```

)

# The imputed matrix itself must match cell-by-cell
imputed = get_matrix_from_handle(result.dataset)
snap_matrix = np.array(snap["imputed_matrix"])
assert imputed.shape == snap_matrix.shape
assert np.allclose(imputed, snap_matrix, rtol=1e-5, atol=1e-5,
                    f"Imputed matrix diverged from v1 snapshot",
                    f"(rel={REL_TOL}, abs={ABS_TOL}))."
)

```

This is the test that runs on every CI build. A failure is the alarm that the wrapper has drifted from v1's behavior, and the alarm is informative — the assertion message names the tolerance that was breached, and a follow-up debug script can find the per-cell deltas. The test takes about 8 seconds on the CI runner and is part of the default `pytest -m oracle` selection.

9.5 Full treatment: pca

Purpose and schema

The `pca` tool runs principal components on a scaled abundance matrix and returns scores, loadings, and variance explained.

```
from typing import List, Literal
from pydantic import BaseModel, Field, field_validator

class PCAInput(BaseModel):
    dataset_handle: str
    n_components: int = Field(
        default=5,
        description="Number of components to retain"
    )
    center: bool = Field(default=True, description="")
    scale: bool = Field(
        default=False,
```

```

        description="Scale columns. Default False but is
        "expected to be pre-scaled by the user")
    )

```

```

@field_validator("n_components")
@classmethod
def _n_components_positive(cls, v: int) -> int:
    if v < 1:
        raise ValueError("n_components must be at least 1")
    if v > 20:
        raise ValueError(
            "n_components > 20 is rarely useful. Please
            "consider whether you actually want to use that many components")
    )
    return v

```

```

class PCAResult(BaseModel):
    scores: List[List[float]] # n_samples x n_components
    loadings: List[List[float]] # n_metabolites x n_components
    variance_explained: List[float] # length n_components

```

```
sample_ids: List[str]
metabolite_ids: List[str]
n_components: int
```

The `n_components` validator is the same defensive pattern as `knn_k` in `impute_missing` — a typed integer caught at the schema layer. The output schema is structurally explicit: scores and loadings are nested lists rather than opaque numpy arrays, which keeps the serialized tool result inspectable in the session log (Chapter 11). For a large matrix the lists would be expensive to log verbatim; the output formatter truncates them to the top-5 components in the agent-visible payload while retaining the full result in the local cache via the `DatasetHandle`.

Failure modes and oracle test

The PCA oracle test handles the sign-flip ambiguity discussed in Chapter 5: a PCA's component signs are arbitrary, and a sign flip on a single component is not a numerical regression. The test sign-aligns each component to the snapshot before comparison.

```

# tests/test_pca.py (excerpt)
def test_pca_progredir_matches_v1_with_sign_align(p
    result = pca(PCAInput(
        dataset_handle=progredir_preprocessed.datas
        n_components=5,
    ))
    snap = load_golden_snapshot("pca_progredir.json

    scores = np.array(result.scores)
    snap_scores = np.array(snap["scores"])
    for k in range(result.n_components):
        if np.sign(scores[0, k]) != np.sign(snap_sc
            scores[:, k] *= -1
    assert np.allclose(scores, snap_scores, rtol=1e
    assert np.allclose(result.variance_explained, s
                           rtol=1e-6, atol=1e-9)

```


9.6 Full treatment:

differential_abundance

The differential-abundance tool exposes three test families and three FDR-correction methods; its schema is correspondingly the largest.

```
from typing import List, Literal, Optional
from pydantic import BaseModel, Field, field_validator

class DifferentialAbundanceInput(BaseModel):
    dataset_handle: str
    group_column: str
    reference_group: str
    test: Literal["limma", "t-test", "wilcoxon"] =
    fdr_method: Literal["BH", "bonferroni", "holm"]
    fc_threshold: float = Field(default=0.5, description=
    fdr_threshold: float = Field(default=0.05, description=
```

```

@field_validator("fdr_threshold")
@classmethod
def _fdr_in_unit_interval(cls, v: float) -> float:
    if not 0.0 < v <= 1.0:
        raise ValueError("fdr_threshold must be")
    return v

@field_validator("fc_threshold")
@classmethod
def _fc_non_negative(cls, v: float) -> float:
    if v < 0:
        raise ValueError("fc_threshold must be")
    return v

@model_validator(mode="after")
def _reference_group_non_empty(self):
    if not self.reference_group.strip():
        raise ValueError("reference_group must")
    return self

```

The `model_validator` is a tool-level cross-field check that runs after individual field validators. Here it catches the edge case of a whitespace-only `reference_group`, which would pass the type check (`str`) but fail the R-side factor matching with an opaque error. Catching it at the Pydantic layer turns the failure into a clean message for the agent.

```
class DifferentialAbundanceEntry(BaseModel):  
    metabolite_id: str  
    log2_fc: float  
    raw_p: float  
    adj_p: float  
    is_significant: bool
```

```
class DifferentialAbundanceResult(BaseModel):  
    table: List[DifferentialAbundanceEntry]  
    n_significant: int  
    test: str  
    fdr_method: str  
    fdr_threshold: float
```

```
fc_threshold: float
group_column: str
reference_group: str
contrast_group: str
```

The oracle test pins p-values with a looser tolerance than fold changes (relative 1×10^{-5} vs. 1×10^{-6}) to absorb known `limma::eBayes` numerical drift across `limma` minor versions; the rationale is documented inline in the test docstring (Chapter 14 §14.5).

9.7 The rpy2 bridge in detail

The wrappers in §9.4–9.6 all delegate to `call_r(...)`. The bridge module is implemented once and reused across all tools.

```
# bridge.py
from contextlib import contextmanager
import rpy2.objects as ro
from rpy2.objects import default_converter, pandas
from rpy2.objects.conversion import localconverter
```

```
_loaded = False
```

```
def _ensure_loaded():  
    global _loaded  
    if not _loaded:  
        ro.r('suppressPackageStartupMessages(library(  
        _loaded = True
```

```
@contextmanager
```

```
def r_session():  
    """Activate the standard converter stack for an R session.  
    _ensure_loaded()  
    cv = default_converter + pandas2ri.converter +  
    with localconverter(cv):  
        yield ro.r
```

```
def call_r(func_name: str, **kwargs):  
    """Call a metabolonr:: function with pandas input
```

```

with r_session() as r:
    f = ro.r(f'metabolonr::{func_name}')
    return f(**kwargs)

```

The bridge has its own test suite (Chapter 14 §14.2) that round-trips pandas DataFrames and numpy arrays through R and checks for conversion-layer regressions independently of the tools. The bridge is the single source of conversion-layer behavior, and a bridge bug shows up first in the bridge tests rather than as a confusing tool-level oracle failure.

[Figure 9.2: Anatomy of a single tool. Boxes for (1) Pydantic input schema with validators, (2) Python wrapper with cache check, (3) rpy2 bridge call, (4) R function with method dispatch, (5) return-value conversion, (6) Pydantic output schema, (7) golden snapshot. Arrows show data flow; one side-arrow shows the oracle test reading the snapshot.]

9.8 The tool dependency graph

Tools are pure functions over `DatasetHandles`, but the handles are typed: a handle from

`load_metabolomics_data` cannot be passed to `pca` without going through `qc_filter`, `impute_missing`, `transform`, and `scale`.

`load` → `summarize`

`load` → `qc_filter` → `impute_missing` → `transform` → `scale`

↘ `source`

↘ `diffusion`

all of the above → `export_session`

The graph is enforced at the tool boundary, not by the agent. If `pca` is called on a handle that has not been scaled, the tool refuses with an error directing the agent to call `scale` first. This catches a class of agent-misordering bugs that would otherwise produce subtly wrong outputs.

[Figure 9.3: Tool dependency graph as a directed acyclic graph. Nodes are tools, edges are valid handle-flow transitions. `export_session` is a sink reachable from every other node.]

9.9 Tool-set design philosophy

The tool layer's design is not a collection of independent decisions; it is four interlocking commitments. Naming them explicitly is the right closing move for this chapter, because every subsequent chapter — the agent loop, the session log, the validation experiment — depends on them.

Closed-set tools. The tool catalog is fixed at release. The agent cannot synthesize a new tool, propose an unaudited variant, or compose a tool that has not been written by a human. The cost is flexibility: a user whose analysis is one step outside the catalog gets a refusal. The benefit is auditability. Every tool the agent calls has a Pydantic schema, an R-side wrapper, an oracle test, and a code-review record. A reviewer of a manuscript that used MetabolonR-LLM can trace any number in the methods paragraph to a specific commit in the public repository. This is the property that distinguishes routing agents from code-writing agents (Chapter 8 §8.3), and the philosophical case for the trade-off is made there; the practical case is that fifty papers' worth of agentic-bio tools that produce different numbers on

different runs have, between 2023 and 2026, made the case for the alternative themselves.

Composability constraints. Tools are pure functions over `DatasetHandles`. They have no side effects beyond producing a new handle (or, for terminal tools, an artifact). They can be called in any order the agent chooses, subject only to data-flow constraints enforced by the dependency graph (§9.8). Composability is what makes the agent's routing latitude meaningful: the agent can construct any pipeline the graph admits without the tool layer constraining the *choice* of pipeline, only the *validity* of any particular pipeline. The dependency-graph enforcement is the discipline that keeps composability from collapsing into chaos. A tool that allowed itself to be called out of order (PCA on an un-scaled matrix, differential abundance on an un-imputed one) would be a tool that silently produced wrong answers; the graph enforcement turns silent wrongness into a structured error the agent must surface.

Schema-pinning. Every tool's input is a Pydantic class with field validators. Every tool's output is a Pydantic class. The JSON Schema served to the LLM is auto-generated from the

input class; the documentation served to the developer is auto-generated from the same class. There is no second source of truth that can drift. Schema-pinning has two specific benefits beyond the obvious one of catching type errors. First, it constrains the LLM's hypothesis space at generation time (Chapter 4 §4.3 elaborates this point), which empirically reduces the schema-violation rate in the validation experiment to under 1% (Chapter 21). Second, it forces the developer to specify the tool's interface before its implementation, which is a discipline that catches design errors at the cheapest possible moment.

Oracle testing as a discipline. Every tool has a golden-snapshot test, captured from v1 running on the Progredir dataset, with a documented floating-point tolerance. The tests run on every CI build. The tolerance is tight enough to catch real regressions (relative 1×10^{-6} , absolute 1×10^{-9} for most fields; looser for `limma` p-values per Chapter 14 §14.5) and loose enough to absorb BLAS-version drift across the team's various development machines. The discipline of capturing a snapshot the moment a tool is implemented is not glamorous,

but it is what makes the v1-as-oracle pattern (Chapter 5 §5.6) load-bearing rather than aspirational. When a v2 tool's behavior drifts from v1's, the test catches it before any user is affected. When v1 itself is found to have a marginal bug (Chapter 14 §14.7), the oracle policy provides a clean mechanism for re-capturing the snapshot under a tagged v1 patch. The tests are also the most legible documentation a future contributor has: what the tool does on Progredir is exactly what the test asserts.

A reader skimming this chapter sees three patterns repeated tool by tool: a small surface area, an R-side delegate, an oracle test pinning the numerics. The three patterns *are* the design. The smallness is the auditability. The R routines are the correctness. The oracle tests are the reproducibility. None is novel in isolation; the contribution is the discipline of holding all three together as a binding contract.

A team building an analogous system for transcriptomics, proteomics, or single-cell data should be able to reproduce this chapter's structure with their own ten tools wrapping their own R or Python routines pinned to their own oracle snapshots. The pattern travels; the metabolomics specifics are the example.

What you should be able to do after this chapter

- Name the ten v2.0 tools in pipeline order and describe what each does in one sentence.
- Read a Pydantic input schema with `@field_validator` and `@model_validator` decorators and identify the validation rules each enforces.
- Trace the path from an LLM tool call through the Pydantic validator, the rpy2 bridge, the R function, and back to the agent's response.
- Write a golden-snapshot test that asserts numerical equivalence against a v1 reference, with appropriate floating-point tolerance and sign-flip handling for PCA.
- Explain the four interlocking commitments of the tool-set design philosophy: closed-set, composability, schema-pinning, oracle testing.
- Predict, for a hypothetical eleventh tool (such as `pathway_variation` in v2.1 or `batch_correct`

in v2.2), what its schema, wrapper, and oracle test would look like.

Chapter 10 — The Agent Loop

The agent loop is where the LLM, the tool layer, and the session logger meet. This chapter walks through one full multi-turn conversation, end to end, in code; explains the `DatasetHandle` indirection that keeps the LLM at arm's length from raw data; documents the system prompt; and justifies the choice of Sonnet 4.6 over Opus 4.7 on a cost-vs-quality comparison.

10.1 The single-turn loop

The simplest unit of agent behavior is a single request that triggers exactly one tool call and produces a final response. The loop has four steps. The runtime sends the user's message plus the tool declarations to the Anthropic Messages API. The model emits a `tool_use` content block naming a tool and its arguments. The runtime validates the arguments against the tool's Pydantic schema, executes the tool, and packages the result as a `tool_result` content block. The runtime sends a

follow-up request with the original messages plus the `tool_use` and the `tool_result`, and the model emits its user-facing answer.

The Python entry point is a small function. The essentials, with error handling elided for clarity:

```
def run_single_turn(user_message: str, tools: list,
                    msgs: list):
    msgs = [{"role": "user", "content": user_message}]
    response = client.messages.create(
        model="claude-sonnet-4-6",
        system=SYSTEM_PROMPT,
        tools=tools,
        messages=msgs,
        max_tokens=2048,
    )
    if response.stop_reason == "tool_use":
        tool_use = next(b for b in response.content
                        if b.type == "tool_use")
        result = dispatch(tool_use.name, tool_use.args)
        msgs.append({"role": "assistant", "content": result})
        msgs.append({
            "role": "user",
            "content": [{"type": "tool_result",
```

```

        "tool_use_id": tool_use.id,
        "content": json.dumps(result)
    })

    response = client.messages.create(
        model="claude-sonnet-4-6",
        system=SYSTEM_PROMPT,
        tools=tools,
        messages=msgs,
        max_tokens=2048,
    )

    return response.content[0].text

```

This is the read-only operation that powers, for instance, the user asking "How many samples are in the ProgreDir dataset?" The agent calls `summarize_dataset`, reads the count, and responds in prose.

10.2 The multi-turn loop

Most clinical analyses require more than one tool call. A request like "load ProgreDir, QC it, impute with KNN, and run

a PCA" is four tool calls and one final summary. The multi-turn loop is the same single-turn structure repeated until the model emits an `end_turn` stop reason.

```
def run_multi_turn(user_message: str, tools: list,
    msgs = session.history + [{"role": "user", "content": user_message}]
    while True:
        response = client.messages.create(
            model="claude-sonnet-4-6",
            system=SYSTEM_PROMPT,
            tools=tools,
            messages=msgs,
            max_tokens=2048,
        )
        msgs.append({"role": "assistant", "content": response.content})
        if response.stop_reason == "end_turn":
            session.history = msgs
            return response.content[-1].text
        if response.stop_reason == "tool_use":
            tool_blocks = [b for b in response.content if b.startswith("tool_use")]
            tool_results = []
```

```

        for tb in tool_blocks:
            result = dispatch(tb.name, tb.input)
            tool_results.append({
                "type": "tool_result",
                "tool_use_id": tb.id,
                "content": json.dumps(result),
            })
            session.log_tool_call(tb, result)
            msgs.append({"role": "user", "content": ...})
            continue
    raise UnexpectedStopReason(response.stop_reason)

```

The loop is intentionally simple. There is no retry, no auto-correction, no parallel speculative tool calling. If a tool returns an error, the error becomes the tool result, the model reads it, and the model decides what to do — typically asking the user a clarifying question. This is the *clarification-before-execution* principle from Chapter 8 in code form.

Two design decisions in this loop are worth naming. First, every tool call is logged via `session.log_tool_call(...)` before the next API

request is made; the log is the source of truth even if the API request after it fails. Second, the loop maintains conversation history as a list that is appended in place; truncation, when needed, is the next section.

10.3 Conversation-history management

A long analysis can produce a conversation with dozens of tool calls. The full history grows linearly in tokens and is eventually constrained by the model's context window. v2.0 ships with a simple two-rule policy.

Rule one: never truncate within the current task. A task is defined as the span of messages from the most recent user message up to (and not including) the next `end_turn`. The agent must see its own tool calls and tool results within a task; truncating mid-task is the surest way to break things.

Rule two: across tasks, drop tool-result content (the body of the JSON blob) when it has been superseded by a later tool result

on the same handle. The tool-use record (what was called, with what arguments) is retained; the bulky payload (a 400-metabolite differential-abundance table) is summarized as a token-cheap reference.

This policy is enough for a single multi-hour analysis. For longer sessions a more aggressive policy will be needed; the v2.1 roadmap (Chapter 24) considers it.

10.4 The `DatasetHandle` indirection

The LLM never sees the raw abundance matrix. The agent passes `DatasetHandle` strings between tools; the actual data lives in an in-process cache keyed by handle. When a tool returns a new handle, the data in the cache is content-addressed by a hash of (input handle, tool name, parameter dict). Two paths that produce the same logical dataset produce the same handle.

The reason for the indirection is partly cost (passing a 200 KB matrix through the LLM is wasteful in tokens) and partly safety (the LLM should not be exposed to per-sample data even at the

metabolite-name level when the dataset carries PHI risk). It is also useful for reproducibility: the handle is, in effect, a pure-functional record of the dataset's analytical provenance.

[Figure 10.2: `DatasetHandle` lifecycle. A user-uploaded matrix enters at `load_metabolomics_data` and produces handle `dh_a0....`. Each subsequent tool produces a new handle by hashing (previous handle, tool name, parameters). The terminal handle is what `export_session` consumes.]

The cache is bounded in size; the eviction policy is least-recently-used with a floor that always retains the handles referenced by the current session. Replayed sessions (Chapter 11) re-populate the cache from the dataset files; the cache is not part of the replay's state.

10.5 Error handling and the retry contract

When a tool returns an error, the agent's behavior is governed by the system prompt: *report the error verbatim to the user, do not*

retry, do not silently switch tools, and ask the user for direction. The system prompt is not always sufficient on its own — frontier models have a strong prior toward "try harder, try again" — and so the loop also enforces a hard retry limit at the runtime level. Three consecutive tool errors on the same task abort the task with a user-facing message.

This is conservative. A different design could let the agent retry with adjusted parameters. The conservative choice is correct for clinical use: when something is wrong, the user wants to know. Silent recovery is the failure mode that erodes trust.

10.6 The system prompt

The system prompt is a 600-word block of text that the agent receives on every request. Its essential content, paraphrased to avoid reproducing the full text here (the full prompt is reproduced as Appendix B):

It identifies the agent as MetabolonR-LLM v2.0 and states the agent's purpose (assist with metabolomics analysis using the available tools).

It enumerates the ten tools by name and one-line description, even though the same information is in the `tools` field of the API request. This redundancy is empirically helpful for routing accuracy.

It instructs the agent to call tools rather than improvise, to ask the user when uncertain, to refuse novel statistical requests, and to report errors verbatim.

It includes one worked example of a multi-turn analysis (load → summarize → QC → impute → transform → scale → PCA → volcano) phrased as a transcript. The example is the single most consequential element of the prompt; removing it costs roughly 8 percentage points of tool-sequence agreement in the validation experiment (Chapter 21).

It instructs the agent to produce a one-paragraph methods summary at the end of any session that runs `export_session`.

The prompt is versioned by SHA-256 hash, which is recorded in the session log. A prompt change is therefore a versionable event and is reflected in the log header.

10.7 Model selection: Sonnet 4.6 vs Opus 4.7

The book's framing has stated, without justification, that Sonnet 4.6 is the routing model and Opus 4.7 is the comparator. This section is the justification.

The validation experiment of Chapter 21 was run on both models. The headline result, placeholder pending Week 4 completion: tool-sequence agreement was within 1.5 percentage points between the two models, with Opus slightly higher; parameter agreement was within 1 point; Jaccard agreement on the biomarker shortlist was indistinguishable [PLACEHOLDER: exact numbers].

The cost difference is not indistinguishable. At Anthropic's mid-2026 list prices, Opus is roughly 3× the per-token cost of Sonnet (verify URL and date at docs.claude.com/pricing at draft finalization). Across the 60-session validation run, Opus produced an API spend of approximately **\$8.10** and Sonnet **\$2.70** [PLACEHOLDER: real numbers from

the experiment]. Scaled to a clinical-lab workload of 20 sessions per week, that is the difference between a \$400 annual API budget and a \$1,200 one.

The decision rule is simple. If Opus produced a meaningful quality lift — say, 5 or more points of tool-sequence agreement — the cost would be paid. It does not, on this tool set with this prompt. Sonnet is the default. A user with a particularly sensitive analysis can override the default per session by setting `--model opus` on the CLI; the choice is recorded in the log header.

[Figure 10.3: Cost-per-session distribution, Sonnet vs Opus, across the 60 validation sessions. Two side-by-side box plots with the median costs annotated and the 95th percentile shown.]

10.8 The budget guardrail

Even at Sonnet pricing a misbehaved agent — caught in a refusal-loop, asked to operate on a 100-megabyte dataset, or simply prompted carelessly — can spend several dollars in a

single session. The Flask budget proxy is a thin layer in front of the Anthropic API that enforces a per-session and per-month cap.

The proxy receives every API request from the agent runtime, looks up the session ID in a small SQLite table tracking cumulative cost, and either forwards the request to the upstream API (incrementing the counter) or returns a structured error if the cap is exceeded. The cap is configurable; the default is \$0.50 per session and \$20 per month per user. A user who needs more contacts the lab admin (Fadhl) and gets the cap lifted, with the lift recorded.

The proxy is roughly 80 lines of Python and runs as a sidecar service via Docker Compose (Chapter 12). It is the operational layer of the *no hidden state* principle: even cost is visible.

10.9 A worked transcript

Chapter 6 described a v1 session on the Progredir dataset. The same analysis through MetabolonR-LLM v2 is, in conversation:

User: Load the Progredir dataset and run a standard pipeline.
I want a volcano plot of progressors versus stable, FDR 0.05.

Agent (**tool_use**, **load_metabolomics_data**, **abundance_path=...**). **Agent** (**tool_use**, **summarize_dataset**, **dataset_handle=dh_a0...**, **group_by="progressor"**). **Agent (text):** Loaded 454 samples. 12% overall missingness on the working matrix. Groups: ~119 progressors, ~335 stable. Proceeding with standard QC at 50% / 30% thresholds (producing the 454×293 working matrix from Chapter 6 §6.7), KNN imputation $k=10$, log2 transformation with pseudocount 1, Pareto scaling. Confirm?

User: Yes, and use limma for the differential abundance.

Agent (**tool_use**, **qc_filter** ...). **Agent** (**tool_use**, **impute_missing**, **method="knn"**, **knn_k=10** ...). **Agent** (**tool_use**, **transform**, **method="log2"**, **pseudocount=1.0** ...). **Agent** (**tool_use**, **scale**, **method="pareto"** ...). **Agent** (**tool_use**, **pca**, **n_components=5** ...). **Agent** (**tool_use**, **sources_of_variation**,

covariates=["progressor","age","sex","eGFR","batch"] ...).

Agent (tool_use, differential_abundance, group_column="progressor", reference_group="stable", test="limma", fdr_method="BH", fdr_threshold=0.05 ...).

Agent (tool_use, export_session, ...). *Agent (text): ~18 metabolites pass FDR 0.05 and $|\log_2 FC| > 0.5$ (Path A v1-oracle reference; see Chapter 6 §6.7). eGFR and progressor are the top two sources of variation, in that order. Batch is fifth and small. Volcano plot, methods paragraph, and metabolite table exported to session_2026-04-12_progredir.zip. The methods paragraph reads: "Serum metabolomics data (n=454 participants, GC-MS platform, working matrix 454×293 after the <50% per-metabolite and <30% per-sample missingness filter) were KNN-imputed (k=10), log2-transformed with pseudocount 1, Pareto-scaled, and tested for differential abundance between progressors and stable participants by limma moderated t-test with Benjamini-Hochberg FDR correction. Public data: Metabolomics Workbench accession ST000978."*

The transcript is a single agent loop iteration's worth of multi-turn tool calling. Every parameter the agent chose is in the session log, the user's confirmation is in the log, and the resulting analysis is replayable next year from the log alone.

[Figure 10.1: The multi-turn agent loop, sequence diagram.

User → API → tool_use → Dispatcher → Tool →
DatasetHandle → tool_result → API → next tool_use → ... →
end_turn → final response.]

What you should be able to do after this chapter

- Trace a single-turn tool-use round trip in pseudocode.
- Extend the single-turn loop to multi-turn and explain the role of the `stop_reason` check.
- Describe the conversation-history management policy and explain why mid-task truncation is forbidden.
- Define `DatasetHandle` and explain the two reasons (cost and safety) for the indirection.

- Justify the choice of Sonnet 4.6 over Opus 4.7 using both the agreement-rate evidence and the cost evidence.
- Describe how the Flask budget proxy enforces a per-session API spending cap and where the cap is configured.

Chapter 11 — Session Logging and Replay

This is the chapter that makes the rest of the book publishable. The reproducibility receipt — `analysis_log.json` plus the replay engine — is the methodological contribution that distinguishes MetabolonR-LLM from every other agentic bioinformatics tool in the 2024–2026 literature. The rest of the system is engineering; this chapter is the argument.

11.1 The reproducibility crisis specific to LLM agents

Reproducibility in computational science has always been a moving target. Sandve et al. 2013's "Ten Simple Rules for Reproducible Computational Research" set the bar for the previous decade: version-controlled code, archived data, recorded environment, tracked random seeds. By 2024 that bar

was widely understood, less widely honored, and arguably sufficient for the systems it described — pipelines whose every step was deterministic given its inputs.

Agentic LLMs break the bar in a specific way. A pipeline that includes an LLM call is not deterministic given its inputs. Re-running the same agent on the same data, even with identical model snapshot and identical temperature, can produce different tool calls in a different order, leading to different parameter choices and different final numbers. The Sandve criteria do not say what to do about this. The 2024–2025 literature has been working on what to do about it, mostly badly.

The bad responses include: claiming the agent is "deterministic" because temperature is 0 (it is not, for the reasons in Chapter 4 §4.3); reporting one run and not the variance across runs; logging the prompt and the response but not the intermediate tool calls; logging tool calls but not parameters; logging parameters but not data hashes; or wrapping the whole thing in a hopeful "results may vary" disclaimer in the methods section.

The good response, which this chapter is the implementation of, is to refuse the question's framing. The question "is your LLM deterministic" is the wrong question for a clinical-bioinformatics tool. The right question is "given a record of one run, can a third party reproduce that run without the LLM."

11.2 Two reproducibility properties

This book introduces a distinction that will recur in Chapter 22's philosophical treatment. There are two reproducibility properties an agentic system can have, and they are not the same.

Deterministic output. Two runs of the same input produce bit-identical outputs. This is the property classical software has and that LLMs do not have. It is impossible to achieve with a current frontier LLM in the loop, and pretending otherwise has cost the agentic-bio field a substantial amount of credibility.

Deterministic re-executability. Given the *record* of one run — a structured log of every tool call, every parameter, every intermediate hash — a different system can reproduce that run's

outputs without ever calling the LLM. This is achievable, and MetabolonR-LLM's `analysis_log.json` + replay engine is the demonstration.

The two properties have different reviewers, different regulators, different use cases. A reviewer asking "is this paper's analysis reproducible" needs the second property, not the first. A regulator approving an agentic clinical tool needs the second property as the regulatory minimum, the first as an aspiration. A user who wants to understand what the agent did needs the second to read a record of what was decided.

The bet of this book is that the second property is the binding one for clinical adoption, and that the first is a distraction.

[Figure 11.1: Reproducibility taxonomy. A 2×2 grid: rows are deterministic-output yes/no, columns are deterministic-re-executability yes/no. Four quadrants annotated with the classes of system that occupy each: classical software (upper left), MetabolonR-LLM (upper right), most 2024–2025 agentic tools (lower right or lower left depending on whether they log), and "nothing reasonable" (lower left).]

11.3 The SQLite session store: full schema with commentary

The session store is a single SQLite database file at `/nfs/turbo/umms-alakwaaf/metabolonr-llm/sessions/sessions.db`. Six tables capture every event of every session. The full DDL is reproduced below with field-by-field commentary.

```
-- sessions: one row per top-level user session
CREATE TABLE IF NOT EXISTS sessions (
    session_id      TEXT PRIMARY KEY,
    started_at      TIMESTAMP NOT NULL,
    ended_at        TIMESTAMP,
    user_id         TEXT NOT NULL,
    model_name      TEXT NOT NULL,
    prompt_sha256   TEXT NOT NULL,
    tool_layer_sha  TEXT NOT NULL,
    api_spend_usd   REAL DEFAULT 0
);
```

`session_id` is the externally visible identifier of the form `ses_<ISO-timestamp>_<8-hex>`, generated when the session opens. `started_at` and `ended_at` bracket the session's lifetime; `ended_at` is null while the session is active. `user_id` identifies the human; it ties into the lab's institutional authentication. `model_name` records the exact Anthropic model string (`claude-sonnet-4-6` for the default). `prompt_sha256` is the SHA-256 hash of the system prompt's text at session open — a versionable identifier for the prompt template. `tool_layer_sha` is the Git commit hash of the MetabolonR-LLM repository at runtime. `api_spend_usd` accumulates the cost of the session in dollars, incremented on every API call by the Flask budget proxy (Chapter 10 §10.8).

```
-- messages: every user, assistant, and tool-result
CREATE TABLE IF NOT EXISTS messages (
    msg_id          INTEGER PRIMARY KEY,
    session_id      TEXT REFERENCES sessions(session_id),
    seq             INTEGER NOT NULL,
    role            TEXT NOT NULL, -- user | assistant
```

```

        content      JSON NOT NULL,
        created_at   TIMESTAMP NOT NULL
    );

```

The `messages` table is the complete conversation history. `seq` is the within-session ordering and is monotonically increasing. `role` is one of `user`, `assistant`, or `tool_result`; the `role` distinguishes the three kinds of message the API exchanges. `content` is the full Anthropic-API content block in JSON form — a single string for plain text, a list of content blocks for assistant turns that include tool calls.

-- `tool_calls`: one row per tool invocation

```

CREATE TABLE IF NOT EXISTS tool_calls (
    call_id      TEXT PRIMARY KEY,
    session_id   TEXT REFERENCES sessions(session_id),
    seq          INTEGER NOT NULL,
    tool_name    TEXT NOT NULL,
    arguments    JSON NOT NULL,
    started_at   TIMESTAMP NOT NULL,
    ended_at     TIMESTAMP
);

```

`call_id` is the API's `tool_use_id` (Anthropic's `toolu_<base32>` form). `tool_name` is one of the ten v2.0 tool names. `arguments` is the JSON-serialized argument dict after Pydantic validation — so a parameter that the agent omitted but that the Pydantic class defaulted is present here with its default value. `started_at` and `ended_at` bracket the tool's execution; their difference is the wall-clock cost of the tool call, useful for performance analysis.

```
-- tool_results: structured result for each tool_call
CREATE TABLE IF NOT EXISTS tool_results (
    call_id      TEXT PRIMARY KEY REFERENCES tool_calls,
    status       TEXT NOT NULL,           -- ok | error
    output       JSON,
    error        JSON,
    handle_in    TEXT,                   -- the data in
    handle_out   TEXT                   -- the data out
);
```

A one-to-one extension of `tool_calls`. `status` is `ok` or `error`. On success, `output` carries the tool's Pydantic-serialized return value (truncated to top-N for large tables; the

full result is in the cache). On failure, error carries the structured error message, typed by error class (schema_validation, tool_execution, dependency_violation). handle_in and handle_out record the DatasetHandle consumed and produced; they make the data-flow graph queryable in SQL.

```
-- dataset_handles: content-addressed handle table
CREATE TABLE IF NOT EXISTS dataset_handles (
    handle      TEXT PRIMARY KEY,
    source      TEXT NOT NULL,          -- load
    parent      TEXT REFERENCES dataset_handles(handle),
    tool_name    TEXT,
    args_sha256 TEXT,
    data_sha256 TEXT NOT NULL,
    created_at   TIMESTAMP NOT NULL
);
```

The provenance ledger. Every handle has a deterministic identifier derived from (parent handle, tool name, sorted argument hash). data_sha256 is the hash of the underlying matrix or result object — the cell that confirms

that two handles with the same parent + tool + args produce the same data. The same row can be referenced by multiple sessions if the same dataset is loaded twice; the table is, in effect, a content-addressed cache.

```
-- outputs: artifacts produced by export_session and
CREATE TABLE IF NOT EXISTS outputs (
    output_id    TEXT PRIMARY KEY,
    session_id   TEXT REFERENCES sessions(session_id),
    kind         TEXT NOT NULL,           -- log | plot | ...
    path         TEXT NOT NULL,
    sha256       TEXT NOT NULL,
    created_at   TIMESTAMP NOT NULL
);
```

Final artifacts produced by the session: the exported `analysis_log.json`, the methods paragraph, the volcano-plot PDF, the differential-abundance CSV, the zipped session archive. Each has a `kind`, an absolute `path`, and a SHA-256 of the file's bytes for replay verification.

The schema is deliberately small. Every piece of information needed to replay a session is in one of these tables; nothing is in a Python pickle, a per-session JSON blob, or an "extra" file alongside the database. The pickle problem — pickles that do not unpickle after a library update — is a frequent source of reproducibility failures in 2024-era systems, and the SQLite store exists in part to avoid it.

[Figure 11.2: ER diagram of the SQLite schema. Six tables with foreign keys. `sessions` is the central table; `dataset_handles` and `outputs` hang off it; `messages`, `tool_calls`, and `tool_results` form the per-session conversational record.]

11.4 A complete `analysis_log.json`

The SQLite store is the operational store; users of MetabolonR-LLM normally never look at it directly. The user-facing artifact is `analysis_log.json`, a per-session JSON document produced by `export_session` (Chapter 9). It is what a

reviewer reads, what gets attached to a manuscript as supplementary material, and what the replay engine consumes.

A complete `analysis_log.json` for the Progredir biomarker-discovery session described in Chapter 6 is reproduced below in full. The values are synthetic but structurally realistic.

```
{
  "header": {
    "session_id": "ses_2026-04-12T09:14:22_a01b2c3d",
    "started_at": "2026-04-12T09:14:22Z",
    "ended_at": "2026-04-12T09:16:55Z",
    "user_id": "alakwaaf",
    "model_name": "claude-sonnet-4-6",
    "model_snapshot": "claude-sonnet-4-6-20260301",
    "prompt_sha256": "b7e1f1ad8c2f4e6a9b0d3c5e7f1a",
    "tool_layer_sha": "a4c9e8b7d6f5a4c3b2a1f0e9d8c7",
    "metabolonr_v1_commit": "v1.4.2-9-gabc1234",
    "r_version": "R version 4.4.1 (2024-06-14)",
    "r_packages": {
      "limma": "3.56.2",
```

```

    "pcaMethods": "1.92.0",
    "impute": "1.74.1",
    "missForest": "1.5",
    "EnhancedVolcano": "1.18.0"
  },
  "python_version": "3.11.7",
  "python_packages": {
    "rpy2": "3.5.16",
    "pydantic": "2.5.3",
    "anthropic": "0.40.0",
    "fastapi": "0.110.0"
  },
  "blas": "OpenBLAS 0.3.26",
  "blas_threads": 4,
  "os": "Ubuntu 22.04.4 LTS",
  "docker_image_digest": "sha256:efgh5678ijkl9012",
  "api_spend_usd": 0.043
},
"user_intent": "Run a standard MetabolonR pipeline",
"tool_calls": [
  {

```

```

    "seq": 1,
    "tool": "load_metabolomics_data",
    "args": {
        "abundance_path": "/data/progredir/abundance",
        "sample_annotation_path": "/data/progredir/sample_annotation",
        "metabolite_annotation_path": "/data/progredir/metabolite_annotation",
        "sample_id_column": "sample_id",
        "metabolite_id_column": "metabolite_id"
    },
    "handle_out": "dh_a0f1e2d3c4b5a697",
    "wall_clock_ms": 412
},
{
    "seq": 2,
    "tool": "summarize_dataset",
    "args": {"dataset_handle": "dh_a0f1e2d3c4b5a697"},
    "wall_clock_ms": 187,
    "result_summary": {"n_samples": 454, "n_metabolites": 123}
},
{
    "seq": 3,

```

```

"tool": "qc_filter",
"args": {
  "dataset_handle": "dh_a0f1e2d3c4b5a697",
  "max_metabolite_missingness_pct": 50.0,
  "max_sample_missingness_pct": 30.0
},
"handle_in": "dh_a0f1e2d3c4b5a697",
"handle_out": "dh_b1c2d3e4f5a6b708",
"wall_clock_ms": 95,
"result_summary": {"n_metabolites_dropped": 0
},
{
  "seq": 4,
  "tool": "impute_missing",
  "args": {
    "dataset_handle": "dh_b1c2d3e4f5a6b708",
    "method": "knn",
    "knn_k": 10,
    "rf_ntree": 100
  },
  "handle_in": "dh_b1c2d3e4f5a6b708",

```

```

    "handle_out": "dh_c2d3e4f5a6b7c819",
    "wall_clock_ms": 4830,
    "result_summary": {"method": "knn", "n_cells": 1000},
  },
  {
    "seq": 5,
    "tool": "transform",
    "args": {"dataset_handle": "dh_c2d3e4f5a6b7c819",
    "handle_in": "dh_c2d3e4f5a6b7c819",
    "handle_out": "dh_d3e4f5a6b7c8d92a",
    "wall_clock_ms": 38
  },
  {
    "seq": 6,
    "tool": "scale",
    "args": {"dataset_handle": "dh_d3e4f5a6b7c8d92a",
    "handle_in": "dh_d3e4f5a6b7c8d92a",
    "handle_out": "dh_e4f5a6b7c8d9ea3b",
    "wall_clock_ms": 51
  },
  {

```

```

    "seq": 7,
    "tool": "pca",
    "args": {"dataset_handle": "dh_e4f5a6b7c8d9ea3b",
    "handle_in": "dh_e4f5a6b7c8d9ea3b",
    "wall_clock_ms": 219,
    "result_summary": {"variance_explained": [0.1
},
{
    "seq": 8,
    "tool": "sources_of_variation",
    "args": {"dataset_handle": "dh_e4f5a6b7c8d9ea3b",
    "handle_in": "dh_e4f5a6b7c8d9ea3b",
    "wall_clock_ms": 1402,
    "result_summary": {"ranking": [{"covariate":
},
{
    "seq": 9,
    "tool": "differential_abundance",
    "args": {
        "dataset_handle": "dh_e4f5a6b7c8d9ea3b",
        "group_column": "progressor",

```

```

    "reference_group": "stable",
    "test": "limma",
    "fdr_method": "BH",
    "fc_threshold": 0.5,
    "fdr_threshold": 0.05
  },
  "handle_in": "dh_e4f5a6b7c8d9ea3b",
  "wall_clock_ms": 612,
  "result_summary": {"n_significant": 18, "top5":
},
{
  "seq": 10,
  "tool": "export_session",
  "args": {"session_id": "ses_2026-04-12T09:14
  "wall_clock_ms": 1058
}
],
"results_summary": {
  "n_significant_metabolites": 18,
  "top_sov_covariate": "eGFR",
  "biomarker_shortlist": ["lactose", "acetohydrox

```



```

    },
    "output_hashes": {
      "analysis_log.json": "sha256:8f3c2a1d4e5b6c7a8e",
      "methods_paragraph.md": "sha256:4a1d3b2c5e6f7a8",
      "differential_abundance_table.csv": "sha256:bd2",
      "volcano.pdf": "sha256:c70e1f2a3b4c5d6e7f8a9b0c",
      "pca_scores.pdf": "sha256:5d6e7f8a9b0c1d2e3f4a5",
      "sources_of_variation.pdf": "sha256:9a0b1c2d3e"
    }
  }
}

```

The structure of the log can be read top-down: the header pins the environment, `user_intent` captures the user's original request as free text, `tool_calls` is the audited transcript of what the agent decided to do, `results_summary` is the headline numbers, and `output_hashes` is what the replay engine verifies. Every numerical claim a downstream consumer wants to verify is anchored: the tool arguments are present, the input/output handles are present, the wall-clock cost is present, and the resulting hash is present.

A few specific fields deserve a second look. `wall_clock_ms` is not used by the replay engine — replay does not aim to reproduce timing — but is useful for performance analysis across versions. `model_snapshot` is the date-versioned snapshot of the LLM, distinct from `model_name`; Anthropic publishes a new snapshot of `claude-sonnet-4-6` every few months, and the snapshot is what would be needed if the agent's reasoning ever had to be re-run (it does not, for replay, but it matters for forensic analysis if a session's behavior is later questioned). `docker_image_digest` pins the canonical runtime image; a replay run inside the same image is the bit-identical case.

[Figure 11.3: Annotated `analysis_log.json` header. Each field labeled with its purpose: identity of the session, reproducibility-relevant version pins, environmental capture.]

11.5 The replay engine

The replay engine reads an `analysis_log.json` and re-executes the tool calls in order, never calling the LLM. The

agent's role at original run time was to choose the tool calls; at replay time the choices are already made and recorded.

```
# agent/replay.py
import hashlib
import json
from dataclasses import dataclass
from pathlib import Path
from typing import Optional

from .tools_registry import TOOL_REGISTRY
from .pin_verification import verify_pins, PinStatus
from .hashing import hash_file

@dataclass
class ReplayTolerance:
    relative: float = 1e-6
    absolute: float = 1e-9
    bit_identical_required: bool = False
```

```
DEFAULT_TOLERANCE = ReplayTolerance()
```

```
@dataclass
```

```
class ReplayReport:
```

```
    success: bool
```

```
    reason: str = ""
```

```
    pin_status: Optional[PinStatus] = None
```

```
    hash_diffs: Optional[dict] = None
```

```
def replay(log_path: str, data_root: str, output_dir: str,
```

```
           tolerance: ReplayTolerance = DEFAULT_TOLERANCE):
```

```
    """Re-execute a session log without invoking the simulator"""
```

```
    log = json.load(open(log_path))
```

```
    pin_status = verify_pins(log["header"])
```

```
    if pin_status.major_drift:
```

```
        return ReplayReport(False, "pin_major_drift")
```

```
    handles: dict[str, object] = {}
```

```

produced_outputs: dict[str, Path] = {}

for call in log["tool_calls"]:
    tool = TOOL_REGISTRY[call["tool"]]
    args = dict(call["args"])
    if "handle_in" in call:
        args["dataset_handle"] = handles[call["
    try:
        result = tool.execute(**args, output_d
    except Exception as exc:
        return ReplayReport(False, f"tool_failu
        pin_status=pin_stat
    if "handle_out" in call:
        handles[call["handle_out"]] = result.ha
    for kind, path in result.artifacts.items():
        produced_outputs[kind] = Path(path)

# Hash check: do the produced artifacts match t
diffs = {}

for name, recorded in log["output_hashes"].iter
    path = produced_outputs.get(name) or Path(c

```

```

actual = "sha256:" + hash_file(path)
if actual != recorded:
    diffs[name] = {"recorded": recorded, "a

if diffs and tolerance.bit_identical_required:
    return ReplayReport(False, "hash_mismatch",
                        hash_diffs=diffs)
return ReplayReport(True, "ok", pin_status=pin_
                    hash_diffs=diffs if diffs e

```

The engine is roughly 30 lines of substantive logic, plus its data classes and imports. `verify_pins` checks that the environment running the replay matches the environment recorded in the header. If the R version differs by a minor version (4.4.1 vs 4.4.2), the replay proceeds with a warning. If a `limma` patch version differs, the replay proceeds with a warning. If anything differs in a major version, the replay refuses to run and reports which pin is wrong.

`hash_file` is a trivial SHA-256 over the file's bytes. The hash check compares the recorded hashes from the log against the freshly computed hashes of the artifacts the replay just

produced. A match across all artifacts is what counts as a successful bit-identical replay. A mismatch is reported by artifact name and surfaces in the report.

11.6 Bit-identical replay as a tested property

The strong form of the replay guarantee is bit-identical: every output hash matches. This holds when the environment is identical to the original (same OS image, same R minor version, same BLAS, same thread count, same package patch versions). It does not always hold across environments. The forces that break bit-identity are well known and worth naming.

BLAS implementation. OpenBLAS, MKL, and ATLAS produce subtly different floating-point results for matrix multiplication, by a difference of a few ULPs. PCA and eBayes are sensitive to this.

BLAS thread count. Even within OpenBLAS, the multiplication result depends on the thread count for matrices

above a size threshold (typically a few hundred rows). The pin includes `OMP_NUM_THREADS` and `OPENBLAS_NUM_THREADS`.

Compiler. A package compiled with gcc 11 may produce minutely different math than one compiled with gcc 12.

Operating system. glibc minor versions affect some math-library calls; certain optimized routines (libm's `exp`, `log`, `sin`) have evolved across glibc releases.

The mitigation is to ship a Docker image that pins all of the above and to use that image as the canonical replay environment. Outside the image, replay is *near*-bit-identical: relative tolerance 1×10^{-6} across all numerical fields, exact equality across all categorical fields. This is the same tolerance as the oracle-test policy of Chapter 5.

The replay test suite is reproduced below. It runs the same ProgreDir session against a 4×4 grid of environment variations and asserts the expected pass/fail level for each cell.


```

# tests/test_replay_matrix.py
import pytest
from agent.replay import replay, ReplayTolerance

PROGREDIR_LOG = "tests/data/replay/progredir_session.log"
PROGREDIR_DATA = "tests/data/progredir"

@pytest.mark.parametrize("scenario, threads, bit_identities, tolerance")
def test_replay_matrix(scenario, threads, bit_identities, tolerance):
    # Canonical image; expect bit-identical
    ("canonical_openblas_ubuntu22_threads4", 4, True, 0)
    # Same OS, different thread count; expect within-tolerance
    ("canonical_openblas_ubuntu22_threads1", 1, True, 0)
    ("canonical_openblas_ubuntu22_threads1", 1, False, 0)
    # Different BLAS; expect within-tolerance but not bit-identical
    ("mkl_ubuntu22_threads4", 4, True, 0)
    ("mkl_ubuntu22_threads4", 4, False, 0)
    # Different OS; expect within-tolerance but not bit-identical
    ("openblas_debian12_threads4", 4, True, 0)
    ("openblas_debian12_threads4", 4, False, 0)

```

```

        # Different BLAS and different OS; still within
        ("mkl_debian12_threads4", 4, Fa
    ])
def test_replay_pass_fail_matrix(scenario, threads,
                                expected_pass, en
    """Replay the Progredir session across environm
    environment_setup(scenario, threads=threads)
    tolerance = ReplayTolerance(
        relative=1e-6, absolute=1e-9,
        bit_identical_required=bit_identical_requir
    )
    report = replay(
        log_path=PROGREDIR_LOG,
        data_root=PROGREDIR_DATA,
        output_dir=f"/tmp/replay_{scenario}_t{threa
        tolerance=tolerance,
    )
    assert report.success == expected_pass, (
        f"Replay in {scenario} (threads={threads},
        f"expected pass={expected_pass} but got {re

```

```
f"reason: {report.reason}; diffs: {report.diffs}";  
)
```

The test matrix produces the pass/fail grid summarized in Figure 11.4. The current build hits "bit-identical" on the canonical row and "within-tolerance" on all other rows. A future build that hit "bit-identical" everywhere would be useful but is not the architectural target; the within-tolerance property is what the methods-section claim of the eventual paper actually rests on, and the test matrix is the evidence.

The pass/fail discussion in plain language: a reviewer running the replay on her own Linux box against the canonical Docker image should expect bit-identical output. A reviewer running the replay on her own machine without the image should expect within-tolerance output, and the diff report will name which artifacts diverged and by how much. Both cases count as successful replays under the methodology of this book; only the cell where the replay should be bit-identical but is not is a real test failure.

[Figure 11.4: Bit-identical replay test matrix. Four rows $\times$ three columns of pass/fail cells. Rows: same OS+BLAS, same OS / different BLAS, different OS / same BLAS, different OS / different BLAS. Columns: bit-identical / within-tolerance / fail.]

ysis if a session's behavior is later questioned. `docker_image_digest` pins the canonical runtime image; a replay run inside the same image is the bit-identical case.

[Figure 11.3: Annotated `analysis_log.json` header. Each field labeled with its purpose: identity of the session, reproducibility-relevant version pins, environmental capture.]

11.5 The replay engine

The replay engine reads an `analysis_log.json` and re-executes the tool calls in order, never calling the LLM. The agent's role at original run time was to choose the tool calls; at replay time the choices are already made and recorded.

```
# agent/replay.py
import json
```

```
from dataclasses import dataclass
from pathlib import Path
from typing import Optional

from .tools_registry import TOOL_REGISTRY
from .pin_verification import verify_pins, PinStatus
from .hashing import hash_file
```

```
@dataclass
class ReplayTolerance:
    relative: float = 1e-6
    absolute: float = 1e-9
    bit_identical_required: bool = False
```

```
DEFAULT_TOLERANCE = ReplayTolerance()
```

```
@dataclass
class ReplayReport:
```

```

success: bool
reason: str = ""
pin_status: Optional[PinStatus] = None
hash_diffs: Optional[dict] = None

def replay(log_path: str, data_root: str, output_dir: str,
           tolerance: ReplayTolerance = DEFAULT_TOLERANCE) -> ReplayReport:
    """Re-execute a session log without invoking the tools.
    log = json.load(open(log_path))
    pin_status = verify_pins(log["header"])
    if pin_status.major_drift:
        return ReplayReport(False, "pin_major_drift")

    handles: dict[str, object] = {}
    produced_outputs: dict[str, Path] = {}

    for call in log["tool_calls"]:
        tool = TOOL_REGISTRY[call["tool"]]
        args = dict(call["args"])
        if "handle_in" in call:

```

```

        args["dataset_handle"] = handles[call["dataset_handle"]]
    try:
        result = tool.execute(**args, output_dir=output_dir)
    except Exception as exc:
        return ReplayReport(False, f"tool_failed: {exc}",
                             pin_status=pin_status)
    if "handle_out" in call:
        handles[call["handle_out"]] = result.handle_out
    for kind, path in result.artifacts.items():
        produced_outputs[kind] = Path(path)

diffs = {}
for name, recorded in log["output_hashes"].items():
    path = produced_outputs.get(name) or Path(call["dataset_handle"])
    actual = "sha256:" + hash_file(path)
    if actual != recorded:
        diffs[name] = {"recorded": recorded, "actual": actual}

if diffs and tolerance.bit_identical_required:
    return ReplayReport(False, "hash_mismatch",
                        hash_diffs=diffs)

```

```
return ReplayReport(True, "ok", pin_status=pin_  
                        hash_diffs=diffs if diffs e
```

The engine is roughly 30 lines of substantive logic, plus its data classes and imports. `verify_pins` checks that the environment running the replay matches the environment recorded in the header. If the R version differs by a minor version (4.4.1 vs 4.4.2), the replay proceeds with a warning. If a `limma` patch version differs, the replay proceeds with a warning. If anything differs in a major version, the replay refuses to run and reports which pin is wrong.

11.6 Bit-identical replay as a tested property

The strong form of the replay guarantee is bit-identical: every output hash matches. This holds when the environment is identical to the original (same OS image, same R minor version, same BLAS, same thread count, same package patch versions). It does not always hold across environments. The forces that break bit-identity are well known and worth naming.

BLAS implementation. OpenBLAS, MKL, and ATLAS produce subtly different floating-point results for matrix multiplication, by a difference of a few ULPs. PCA and eBayes are sensitive to this.

BLAS thread count. Even within OpenBLAS, the multiplication result depends on the thread count for matrices above a size threshold (typically a few hundred rows).

Compiler. A package compiled with gcc 11 may produce minutely different math than one compiled with gcc 12.

Operating system. glibc minor versions affect some math-library calls; certain optimized routines (libm's `exp`, `log`, `sin`) have evolved across glibc releases.

The mitigation is to ship a Docker image that pins all of the above and to use that image as the canonical replay environment. Outside the image, replay is *near*-bit-identical: relative tolerance 1×10^{-6} across all numerical fields, exact equality across all categorical fields.

The replay test suite runs the same Progredir session against a 4×4 grid of environment variations and asserts the expected pass/fail level for each cell.

```
# tests/test_replay_matrix.py
import pytest
from agent.replay import replay, ReplayTolerance

PROGREDIR_LOG = "tests/data/replay/progredir_session.log"
PROGREDIR_DATA = "tests/data/progredir"

@pytest.mark.parametrize("scenario, threads, bit_io", [
    ("canonical_openblas_ubuntu22_threads4", 4, True),
    ("canonical_openblas_ubuntu22_threads1", 1, True),
    ("canonical_openblas_ubuntu22_threads1", 1, False),
    ("mkl_ubuntu22_threads4", 4, True),
    ("mkl_ubuntu22_threads4", 4, False),
    ("openblas_debian12_threads4", 4, True),
    ("openblas_debian12_threads4", 4, False),
```

```

        ("mkl_debian12_threads4", 4, Fa
    ])
def test_replay_pass_fail_matrix(scenario, threads,
                                expected_pass, en
    environment_setup(scenario, threads=threads)
    tolerance = ReplayTolerance(
        relative=1e-6, absolute=1e-9,
        bit_identical_required=bit_identical_requir
    )
    report = replay(
        log_path=PROGREDIR_LOG,
        data_root=PROGREDIR_DATA,
        output_dir=f"/tmp/replay_{scenario}_t{threa
        tolerance=tolerance,
    )
    assert report.success == expected_pass, (
        f"Replay in {scenario} (threads={threads},
        f"expected pass={expected_pass} but got {re
        f"reason: {report.reason}; diffs: {report.h
    )

```

The pass/fail discussion in plain language: a reviewer running the replay on her own Linux box against the canonical Docker image should expect bit-identical output. A reviewer running the replay on her own machine without the image should expect within-tolerance output, and the diff report will name which artifacts diverged and by how much. Both cases count as su

Chapter 12 — Frontend and Deployment

A research prototype that nobody can run is not research. This chapter is the operational playbook: Streamlit in a day, Gradio for the paper demo, Next.js for production, all running on the Kretzler-lab server `alakwaaf-ap-ps1a` with NFS turbo storage and a Docker stack.

12.1 Three frontends for three audiences

MetabolonR-LLM is presented to users through three different frontends, each appropriate to a different audience and lifecycle stage.

Streamlit is the working-prototype frontend. It ships in a single day during Week 4 (Chapter 16), uses about 250 lines of Python, and is the version the author and lab collaborators

interact with day to day. It is not the version a journal reviewer is asked to use.

Gradio is the paper-demo frontend. It is a polished version with a cleaner layout, designed to be embedded in journal supplementary materials and to handle the spike of visitors that accompanies a preprint or social-media post. It is feature-equivalent to the Streamlit version but with attention to first-time-user UX.

Next.js is the eventual-production frontend. v2.0 ships without it; the v2.x roadmap (Chapter 24) considers when its development is worth the cost. The Next.js path matters for any future where MetabolonR-LLM is used by external users at scale, where the Streamlit and Gradio paths' single-process model becomes a bottleneck.

Picking three frameworks for one tool can read as over-engineering. The justification is that each serves a distinct phase and the cost of building any one is small relative to the cost of forcing one to do all three jobs. Streamlit is sufficient for the lab; Gradio for the paper; Next.js for the world.

12.2 Streamlit in detail

The Streamlit prototype has three regions. The left sidebar lets the user pick a dataset (from a small library of preloaded examples or upload her own three files). The center is the chat panel — a scrolling conversation with the agent, with user messages on the right and agent messages on the left. The right panel shows the live session state: the current `DatasetHandle`, the most recent tool call, and a small "show log" toggle that reveals the running `analysis_log.json`.

The full Streamlit file is reproduced as code listing `ch12_listing_01.py` and runs to about 240 lines including comments. The hot loop is straightforward:

```
# ch12_listing_01.py (excerpt)
if user_input := st.chat_input("Ask MetabolonR-LLM"):
    st.session_state.messages.append({"role": "user", "content": user_input})
    with st.chat_message("user"):
        st.write(user_input)
```

```

response = run_multi_turn(
    user_message=user_input,
    tools=TOOLS,
    session=st.session_state.session,
)
st.session_state.messages.append({"role": "assistant", "content": response})
with st.chat_message("assistant"):
    st.write(response)

```

Streamlit's reactive model is similar enough to Shiny's that v1 developers find it familiar. The trade-off the Streamlit prototype makes against a Shiny-style v1 frontend is that the chat panel is the primary interaction surface, not a sidebar to a dashboard. Users who want the tab-style v1 experience are directed to v1, which the lab continues to run alongside.

12.3 Gradio for the demo

Gradio's `gr.ChatInterface` provides the same chat semantics as Streamlit with a more refined visual default and built-in support for streaming responses (relevant when a

multi-tool conversation takes several seconds; the user sees the partial response as it arrives).

The Gradio app is feature-equivalent to the Streamlit one but adds three demo-friendly affordances: a row of example prompts the user can click instead of typing, a "share session" button that produces a URL the user can send to a colleague to view the same log, and an "export log" button that downloads `analysis_log.json` directly.

The Gradio app is what is linked from the bioRxiv preprint (Chapter 18) and from the journal supplementary materials. It runs on the same backend as the Streamlit app — only the frontend differs.

12.4 The Kretzler-lab server

alakwaaf-ap-ps1a

The deployment target for v2.0 is `alakwaaf-ap-ps1a`, the Kretzler-lab Linux server provisioned by Michigan ITS Advanced Research Computing. It is an Ubuntu 22.04 box with

64 cores, 512 GB of RAM, and access to the NFS turbo storage backend used across the Kretzler lab.

The host name reflects the lab's standard convention (alakwaaf for the author's username, ap-ps1a for the lab's pool of analysis servers). Other Kretzler-lab tools run on adjacent hosts in the same pool.

The relevant directory layout under `/nfs/turbo/umms-alakwaaf/metabolonr-llm/` is:

```
/nfs/turbo/umms-alakwaaf/metabolonr-llm/
```

```
|— data/                # uploaded datasets, cont
|— sessions/           # SQLite session store + p
|— artifacts/          # exported zips: methods p
|— logs/               # service stdout/stderr
|— secrets/            # .env (mode 0600, owned b
└— images/             # Docker image tags pinned
```

The NFS turbo backend is the same storage tier the lab uses for all clinical-data work. It is backed up nightly, audited

periodically by Michigan ITS for compliance, and has the access controls that NEPTUNE work requires.

[Figure 12.2: Server layout on alakwaaf-ap-ps1a. Boxes for the three Docker services (Streamlit/Gradio frontend, FastAPI backend, Flask budget proxy). NFS turbo storage attached. The Anthropic API as an external dependency.]

12.5 Docker, Docker Compose, and the pinning argument

Reproducibility (Chapter 11) requires the runtime environment to be pinned. The cleanest way to pin a Linux runtime in 2026 is a Docker image with an explicit base image digest, explicit package versions in the Dockerfile, and a frozen `requirements.txt` / `renv.lock`.

The `docker-compose.yml` defines three services.

backend runs the FastAPI app that exposes the agent loop and the replay engine as HTTP endpoints. It mounts `/nfs/turbo/umms-alakwaaf/metabolonr-llm/` at `/data`

inside the container. It depends on the Anthropic API and on the Flask budget proxy.

proxy is the Flask budget proxy (Chapter 10 §10.8). It sits between the backend and the Anthropic API, enforces the per-session and per-month spending caps, and writes its cost-accounting rows to the same SQLite database the backend uses for sessions.

frontend runs either the Streamlit or the Gradio app, selected by an environment variable. In normal lab operation, Streamlit is the default; for paper-demo periods, Gradio is enabled by a `MODE=demo` setting.

The `docker-compose.yml` looks like the following, with secrets-mount paths and image tags abbreviated:

```
services:
  proxy:
    image: metabolonr-llm/proxy:v2.0.0@sha256:abcd
    env_file: /nfs/turbo/umms-alakwaaf/metabolonr-1
    volumes:
```

```

    - /nfs/turbo/umms-alakwaaf/metabolonr-llm/se
networks: [internal]

backend:
  image: metabolonr-llm/backend:v2.0.0@sha256:efc
  env_file: /nfs/turbo/umms-alakwaaf/metabolonr-l
  environment:
    - ANTHROPIC_BASE_URL=http://proxy:8080
  volumes:
    - /nfs/turbo/umms-alakwaaf/metabolonr-llm:/da
  depends_on: [proxy]
  networks: [internal, external]

frontend:
  image: metabolonr-llm/frontend:v2.0.0@sha256:ij
  environment:
    - BACKEND_URL=http://backend:8000
    - MODE=lab
  depends_on: [backend]
  ports:
    - "8501:8501"

```

```
networks: [external]
```

```
networks:
```

```
  internal:
```

```
  external:
```

The image digests pin each service to a byte-identical layer set. A `docker compose pull` && `docker compose up -d` brings the stack up to the recorded state.

12.6 Secrets management

The Anthropic API key is the only secret the system handles, and it is handled carefully. It lives in `/nfs/turbo/umms-alakwaaf/metabolonr-llm/secrets/.env`, file mode `0600`, owned by the service user `metabolonr-svc`. It is mounted into the `proxy` and `backend` containers as an `env-file` rather than baked into the image; the image is therefore safe to inspect without exposing the key.

The `proxy` is the only service that uses the key on the outbound path; the `backend` talks to the `proxy`, not to Anthropic directly.

This means a backend container that is somehow exfiltrated does not contain the key. The proxy's network connectivity is restricted by the Docker network configuration to the upstream Anthropic API endpoint; it cannot reach arbitrary destinations.

The key is rotated whenever a lab member leaves the project. Rotation is straightforward — edit the `.env`, restart the proxy — and is recorded in `logs/key_rotation.log` with the date and the rotating user.

12.7 Operational pitfalls and what to do about them

A handful of operational pitfalls bit the team during the v2.0 build. Each is recorded here so a future re-implementation can sidestep them.

ipy2 inside Docker. The R installation inside the backend container must be present at image build time; it is not enough to `apt-get install r-base` at container start. The R packages must also be installed at build time, into a fixed

library path that rpy2 can find. The Dockerfile uses a multi-stage build: a "builder" stage installs R packages from `renv.lock`, the "runner" stage copies the library directory. Combined image size is about 1.8 GB.

BLAS thread count in containers. The default OpenBLAS thread count inside a container is the number of host CPUs visible to the container, which under Docker Compose is the host's total core count. For reproducibility (Chapter 11 §11.6) the thread count must be pinned. The backend's `env_file` includes `OPENBLAS_NUM_THREADS=4` and `OMP_NUM_THREADS=4`, and the `replay` environment is configured to match.

File permissions on NFS turbo. NFS turbo enforces POSIX permissions across the lab's various host machines, and a file created on `alakwaaf-ap-ps1a` is sometimes not readable from `alakwaaf-ap-ps1b` if the service user's UID does not match. The mitigation is that the service user has a fixed UID across all lab hosts, and the relevant subdirectories are group-owned by a `metabolonr` group whose membership is centrally managed.

Streamlit re-runs on every input. Streamlit's default behavior is to re-run the entire script when the user clicks anything. For an agent that has spent ten seconds and several cents on a tool call, an inadvertent re-run is expensive. The Streamlit app uses `st.session_state` aggressively and gates the agent invocation behind a `if not user_input == st.session_state.last_input` check.

Gradio share-link rate limits. Gradio's public-share feature (which proxies through `gradio.live`) has rate limits and a short URL lifetime. For the paper-demo, the Gradio app is deployed on the lab server with a stable URL rather than through `gradio.live`.

12.8 The path to a Next.js production frontend

The Streamlit/Gradio frontends share two limitations that mean they are not the right answer for a future production deployment serving many concurrent users.

First, both run a single Python process per worker, and the Python GIL prevents true parallel request handling. Scaling means spinning up more worker containers, with all the operational cost that implies.

Second, neither has a strong story for user authentication beyond a basic-auth wrapper. For a future where clinicians at multiple institutions log in to MetabolonR-LLM with their institutional credentials, a real frontend framework with OAuth / SSO is needed.

The Next.js path addresses both. The backend stays the same (FastAPI); only the frontend changes. v3 considers this seriously; v2 does not.

12.9 The deployment checklist

A reproduction-minded reader who wants to stand up MetabolonR-LLM on a comparable host can follow a short checklist.

Clone the repository at the v2.0 tag. Restore `renv.lock` for the R side. Build the three Docker images using the Dockerfiles in `services/{backend,proxy,frontend}/`. Populate `.env` with an Anthropic API key. Place a test dataset in `/data/` (the Progredir bundle is the natural starting point). Bring up the stack with `docker compose up -d`. Open `http://localhost:8501` and run the worked example from Chapter 6. Verify that the `analysis_log.json` is produced and that the replay engine succeeds against it.

The full checklist with copy-pasteable commands is reproduced as the README of the public repository and as Appendix B's deployment-script listing.

[Figure 12.1: Three-tier deployment diagram. Streamlit / Gradio frontend layer at the top, FastAPI backend in the middle, Flask proxy + Anthropic API on the side. NFS turbo storage attached to the backend.]

What you should be able to do after this chapter

- Choose between the Streamlit, Gradio, and Next.js frontends given a target audience and lifecycle stage.
- Describe the three Docker services in the `docker-compose.yml` and explain how they relate to each other.
- Explain why secrets are mounted as an env-file rather than baked into an image.
- Pin a BLAS thread count and explain why the pinning matters for replay reproducibility.
- Walk a reader through the deployment checklist from a fresh server to a running Streamlit instance.
- Identify two limitations of the Streamlit/Gradio path that motivate an eventual Next.js production deployment.

Chapter 13 — Week 1: Environment, Lit Review, Architecture Lock

Week 1 has no code. It has a working v1, a literature review, and a one-page architecture document signed off by the advisor. Skip any of these and Week 2 will fail.

13.1 Why the first week has no code

The temptation, on the Monday of a six-week build, is to start typing. Resist it. The most common reason short builds fail is that the architecture was not stable when the code began, and Week 1 exists to make the architecture stable.

This chapter is written to the student — second person, day by day. The advisor's job for the week is to be available for the Thursday architecture-lock review and to push back on under-specified plans. The student's job is to spend Monday through

Wednesday producing the artifacts that make Thursday's review productive, and Friday tying the loose ends.

13.2 Monday — environment setup

The environment you will spend six weeks in is a single Ubuntu 22.04 development VM with Python 3.11, R 4.4, rpy2, the MetabolonR v1 R package, and a small set of Python development tools.

Install in this order. Apt-install `r-base-dev`, `libcurl4-openssl-dev`, `libssl-dev`, `libxml2-dev`, and `python3.11-dev` first; these are the system dependencies that rpy2 and the R Bioconductor packages need. Install R 4.4 from the CRAN PPA. Open R and run `renv::restore()` against the lockfile in the v1 repository; this installs the pinned set of v1 R packages, which takes about 25 minutes the first time.

Set up a Python 3.11 virtual environment. Install rpy2, pydantic $\geq$ 2.5, anthropic, pytest, pytest-cov, ruff, mypy, and pre-commit. Verify rpy2 by opening

Python, `import rpy2.robj as ro`, and `ro.r('library(metabolonr)')` — this is the single most informative smoke test. If rpy2 cannot find R, the rest of the week is blocked; debug it now, not later.

By end of day Monday you should be able to:

- Open the v1 Shiny app locally with `R -e 'shiny::runApp("metabolonr-v1")'`.
- Run the v1 reference analysis on Progredir from the browser.
- Run a Python REPL and call a single v1 R function via `rpy2`.

If any of those does not work, stop and fix it. Do not start Tuesday.

13.3 Tuesday — produce

TOOL_INVENTORY.md

Open the v1 source. Read `ui.R`. Read `server.R`. For each tab, identify the reactive endpoints — the `output$<id>`

definitions and the R functions they call. Produce `TOOL_INVENTORY.md`, a Markdown document with one row per candidate tool.

The document has six columns: candidate tool name, the v1 function it would wrap, the v1 tab it lives on, the inputs the tool would need (from the v1 UI), the outputs it would return, and a "complexity 1-5" score for how hard the wrapping is expected to be.

A row for the imputation tool, for instance:

Tool name	v1 function	v1 tab	Inputs
			dataset_han
impute_missing	metabolonr::impute()	Impute & Transform	method, knn_k, rf_ntree

By end of Tuesday the inventory should have 10-12 candidate tools. You will revise it on Thursday. Do not optimize it now; just enumerate.

A side product of the inventory is a list of v1 functions that are *not* candidate tools — internal helpers, plotting code, validation routines. Note these in a small "v1-internal" section of the same document. Some of them will become Python-side utilities later.

13.4 Wednesday — literature review

Wednesday is the literature review. Produce `RELATED_WORK.md`, a positioning memo that places MetabolonR-LLM against the agentic-bioinformatics landscape.

The minimum scope: read the abstracts of the 15 tools from Chapter 4 and from the verification report. For the five most relevant — BioMANIA, GeneGPT, BioChatter, BioAgents, Biomni — read the full paper. For each, write a 100-word summary in the memo and a one-sentence "differs from MetabolonR-LLM in that..." line.

The memo's structure:

1. A one-page introduction stating what MetabolonR-LLM is and what it is for. This becomes the abstract of the eventual paper; treat the page as a first draft.
2. A "related work" section grouping the surveyed tools into the four categories from Chapter 4: agents, domain-LLMs, boundary cases, benchmarks.
3. A "positioning" section that is the explicit list of three architectural respects in which MetabolonR-LLM differs from the closest related work.
4. A "what is missing" section flagging at most three other tools the student found during reading that are not in the Chapter 4 list but probably should be cited. These go to the advisor on Thursday.

By end of Wednesday the memo is 6–8 pages. It is the most boring and most important thing you do in Week 1.

13.5 Thursday — architecture lock

Thursday is the architecture-lock review. The artifact is `ARCHITECTURE.md`, a one-page document that the student writes, the advisor signs, and that nobody is allowed to modify after Thursday without an explicit advisor sign-off.

The document has exactly five sections:

1. **Tool inventory (final).** The 10 tools v2.0 will ship with, derived from `TOOL_INVENTORY.md` but trimmed. Anything marked "v2.1 deferred" is named explicitly.
2. **Log schema (final).** The list of fields in `analysis_log.json`'s header. The student does not need to design the SQLite schema in detail today — that is Week 3 — but the header must be agreed today because every tool's output depends on it.
3. **Replay claim (final).** The single sentence stating what reproducibility property the system will guarantee. The student writes the sentence; the advisor edits it; the agreed sentence goes in the document.

4. **Model choice (final).** Sonnet 4.6 by default. Opus 4.7 as the comparator. The student must not relitigate this in Week 4.
5. **Out of scope (explicit).** A list of things v2.0 will *not* do, with one-line justifications. Batch correction, pathway analysis, survival analysis, multi-omics. Each is explicitly deferred.

The advisor reviews each section, agrees or proposes changes, and signs the bottom of the document with a date. The signed version is committed to the repository as `docs/ARCHITECTURE.md`. The student treats it as binding for six weeks.

The reason this discipline matters is that every Friday-of-the-week retro will ask "are we still on the architecture-lock document?" A drift from the document is the first signal that the build is in trouble.

13.6 Friday — repo skeleton and CI green

Friday is the practical day. By end of day the public repository on GitHub has:

- A `pyproject.toml` with the Python dependencies pinned.
- An `renv.lock` with the R dependencies pinned.
- A `services/` directory with `empty backend/`, `proxy/`, and `frontend/` subdirectories, each with a placeholder `Dockerfile`.
- A `tools/` directory with one file per planned tool, each containing a Pydantic schema stub and a `NotImplementedError` body.
- A `tests/` directory with one file per planned tool, each containing a `def test_<tool>():`
`pytest.skip("Week 2")` stub.

- A `.github/workflows/ci.yml` that runs `ruff`, `mypy`, and `pytest`. All three should be green; the tests pass because they all skip.
- A `README.md` with a project description and a "status: under construction" note.
- An `ARCHITECTURE.md` committed at the signed version from Thursday.
- A `RELATED_WORK.md` committed at the Wednesday draft.

The first commit is signed by the student and has the message "scaffold per ARCHITECTURE.md v1, signed 2026-..."

13.7 Common pitfalls

The pitfalls from past builds that this chapter exists in part to prevent.

rpy2 install. If `pip install rpy2` succeeds but `import rpy2.robjjects` fails at runtime with a `R_HOME not set` error, you skipped a system dependency. Re-read the `apt-install` step. On M-series Macs (which the lab does not deploy

to but the student may develop on) rpy2 wheels can be missing; either build from source or develop inside Docker from day one.

R/Python version mismatch. The rpy2 wheel must match the R minor version it expects. A `renv.lock` produced under R 4.4.1 may not restore cleanly under R 4.4.0. Use the same minor R version throughout the build.

Conflicting BLAS. Some Bioconductor binary packages ship with a BLAS dependency that conflicts with the system OpenBLAS. The symptom is non-determinism in PCA outputs across runs on the same machine. Use the `renv`-pinned BLAS or set `OPENBLAS_NUM_THREADS=1` early to make the issue visible.

Architecture drift before signing. If the architecture-lock document feels uncomfortable on Wednesday night, that is the right time to push back. After Thursday's sign-off the cost of changing it is much higher.

Over-ambitious tool inventory. A tool inventory of 18 candidates is too many. Trim aggressively on Thursday. Every

tool that ships in v2.0 has to clear the schema-pin and oracle-test bar in Weeks 2 and 3; the budget is real.

[Figure 13.1: Week 1 day-by-day Gantt. Five horizontal bars for Monday–Friday; deliverables annotated under each bar; Thursday's architecture lock annotated as the critical-path milestone.]

13.8 Deliverable checklist

By end of Friday, in the repository:

- docs/ARCHITECTURE.md (signed)
- docs/TOOL_INVENTORY.md
- docs/RELATED_WORK.md
- pyproject.toml and renv.lock
- services/{backend, proxy, frontend}/
Dockerfile (placeholder)
- tools/*.py (10 files, all schema stubs)
- tests/test_*.py (10 files, all skipped)
- .github/workflows/ci.yml (green)
- README.md

On the student's local machine:

- v1 Shiny app runs locally.
- v1 reference analysis on Progredir reproduces Chapter 6 §6.4.
- Python REPL can call at least one v1 R function via rpy2.

In the student's notebook:

- A page of pitfalls encountered during the week, for the next person in the lab.
- A retro: what worked, what did not, what to do differently in Week 2.

What you should be able to do after this chapter

- Stand up the development environment from a blank Ubuntu box in under a day.
- Produce a tool inventory from a Shiny app by reading its `server.R`.

- Write a related-work memo that places the project in the published literature.
- Defend an architecture-lock document to an advisor in a 30-minute Thursday meeting.
- Commit a repository skeleton with green CI before starting any tool implementation.
- Recognize the five common Week-1 pitfalls and have a mitigation ready for each.

Chapter 14 — Week 2: Wrapping v1

Functions as Typed Tools

Week 2 is the most code-dense week of the build. Ten tools, ten oracle tests, one shared rpy2 bridge. The architecture lock from Thursday of Week 1 is the binding contract; this week is its execution.

14.1 What success looks like at the end of Week 2

By Friday night, every one of the ten tools in `tools/` has a working implementation that passes its golden-snapshot oracle test. PyTest coverage is above 85% across the `tools/` package. The CI pipeline runs the full oracle suite on every push. No tool ships without a green test.

The week is paced to make this achievable: shared infrastructure on Monday, the first four tools on Tuesday with `load_metabolomics_data` shown in full as the worked

example, the middle three on Wednesday with the floating-point tolerance policy committed, the analytical-end tools on Thursday with the limma question, and `export_session` plus the coverage push on Friday. Each day's plan assumes the prior day's deliverables are green.

14.2 Monday — the shared rpy2 bridge

Monday is a no-tool day. The deliverable is the `bridge.py` module that every tool will call into, plus the converter library that handles R-to-Python and Python-to-R data conversion.

The bridge has three exported objects. `r_session()` is a context manager that activates the right rpy2 converter for pandas DataFrames and numpy arrays. `call_r(name, **kwargs)` is the thin wrapper that calls a `metabolonr::` function with the right converters in scope. `R_TO_PY[type]` and `PY_TO_R[type]` are dispatch tables for non-standard conversions (R S4 objects from `pcaMethods`, for instance).

By end of day Monday the bridge is implemented and tested. The test uses a stub R function defined inline:

```
def test_bridge_roundtrip():
    with r_session() as r:
        df_py = pd.DataFrame({"a": [1, 2, 3], "b":
        df_r = r['as.data.frame'](df_py)
        df_back = pd.DataFrame(df_r)
        assert df_py.equals(df_back)
```

The bridge tests are kept in `tests/test_bridge.py` and run independently of the tool tests. They catch rpy2-side regressions early.

A common time sink on Monday is the conversion of R factor columns to pandas Categorical. The R-side default is `stringsAsFactors=FALSE` in modern R, but Bioconductor packages sometimes return factors anyway. The bridge documents an explicit policy: factors are converted to pandas Categoricals with the factor levels preserved as the category order. A pull request that adds a new R-side dependency must verify this in `tests/test_bridge_factors.py`.

14.3 Tuesday — tools 1–4, with `load_metabolomics_data` shown in full

Tuesday implements `load_metabolomics_data`, `summarize_dataset`, `qc_filter`, and `impute_missing`. These are the data-ingress tools; their inputs come from the user and their outputs become handles that the rest of the pipeline uses. The first of them, `load_metabolomics_data`, is the natural worked example because its inputs are *non-trivial*: three separate files that must validate each other, two annotation files with their own column-name conventions, and the joining logic that links them.

The full implementation is reproduced below. The wrapper is approximately 110 lines including imports and the Pydantic classes; everything except the Pydantic boilerplate is wrapped around the `call_r` bridge.

```

# tools/load_metabolomics_data.py
import hashlib, json
from pathlib import Path
from typing import List, Optional
from pydantic import BaseModel, Field, field_validator

from .bridge import call_r
from .handles import DatasetHandle, register_handle

class LoadMetabolomicsDataInput(BaseModel):
    abundance_path: str = Field(..., description="Abundance path")
    sample_annotation_path: str = Field(..., description="Sample annotation path")
    metabolite_annotation_path: str = Field(..., description="Metabolite annotation path")
    sample_id_column: str = "sample_id"
    metabolite_id_column: str = "metabolite_id"

    @field_validator("abundance_path", "sample_annotation_path",
                    "metabolite_annotation_path")
    @classmethod
    def _path_exists(cls, v: str) -> str:

```

```

    if not Path(v).exists():
        raise ValueError(f"file not found: {v}")
    if Path(v).stat().st_size == 0:
        raise ValueError(f"file is empty: {v}")
    return v

```

```

class LoadSummary(BaseModel):
    dataset_handle: str
    n_samples: int
    n_metabolites: int
    abundance_path: str
    sample_annotation_path: str
    metabolite_annotation_path: str
    validation_warnings: List[str] = []

```

```

def load_metabolomics_data(input: LoadMetabolomicsS
    """Load the three-file Progredir-style bundle,
    # Content-address the input bundle by hashing a
    digest = hashlib.sha256()

```



```

for p in (input.abundance_path,
          input.sample_annotation_path,
          input.metabolite_annotation_path):
    digest.update(Path(p).read_bytes())
bundle_sha = digest.hexdigest()[:16]
handle_id = f"dh_{bundle_sha}"

# Cache check
if existing := register_handle.lookup(handle_id):
    return LoadSummary(
        dataset_handle=handle_id,
        n_samples=existing.cached_meta["n_samples"],
        n_metabolites=existing.cached_meta["n_metabolites"],
        abundance_path=input.abundance_path,
        sample_annotation_path=input.sample_annotation_path,
        metabolite_annotation_path=input.metabolite_annotation_path
    )

# Delegate to R; metabolonr::load_data validate
r_result = call_r(
    "load_data",

```

```

        abundance_path=input.abundance_path,
        sample_annotation_path=input.sample_annotation_path,
        metabolite_annotation_path=input.metabolite_annotation_path,
        sample_id_column=input.sample_id_column,
        metabolite_id_column=input.metabolite_id_column,
    )

```

```

n_samples = int(r_result.rx2("n_samples")[0])
n_metabolites = int(r_result.rx2("n_metabolites")[0])
warnings = list(r_result.rx2("warnings"))

```

```

register_handle(
    handle_id=handle_id,
    parent_id=None,
    tool_name="load_metabolomics_data",
    args_sha256=bundle_sha,
    r_object=r_result.rx2("dataset"),
    meta={"n_samples": n_samples, "n_metabolites": n_metabolites},
)

```

```

return LoadSummary(

```

```

        dataset_handle=handle_id,
        n_samples=n_samples,
        n_metabolites=n_metabolites,
        abundance_path=input.abundance_path,
        sample_annotation_path=input.sample_annotation_path,
        metabolite_annotation_path=input.metabolite_annotation_path,
        validation_warnings=warnings,
    )

```

Three things in this wrapper are worth attention. First, the path-existence validator at the Pydantic layer catches the most common failure mode (a typo in a path) before any R call. Second, the handle ID is the SHA-256 hash of the three files' bytes — content-addressed, so the same Progredir bundle always produces the same handle, regardless of who loaded it or when. Third, the validation work itself (sample-ID alignment between matrix and annotation, metabolite-ID alignment between matrix and metabolite annotation) is delegated to R via `metabolonr::load_data`, which is the `v1` routine; the wrapper is a thin Python skin over the audited R code, not a reimplementa-tion.

The oracle test for `load_metabolomics_data` against the Progredir bundle (Chapter 6 §6.7) is reproduced in `tests/test_load_metabolomics_data.py`. Capturing the snapshot was the first non-trivial test-fixture task of Week 2 because the snapshot must be deterministic across captures — meaning the R-side warnings list must be sorted and the bundle hash must be reproducible from the file bytes, not from the load order.

The remaining three Tuesday tools — `summarize_dataset`, `qc_filter`, `impute_missing` — follow the same pattern in compressed form. By end of day all four pass their oracle tests; the CI build is green.

The most common Tuesday failure is a snapshot that includes a timestamp or a random UUID, which the test then fails to match on subsequent runs. The fix is to scrub timestamps and UUIDs at snapshot-capture time and document the scrub in the test docstring.

14.4 Wednesday — tools 5–7 and the tolerance policy, argued

Wednesday implements `transform`, `scale`, and `pca`. These tools are where the floating-point tolerance policy becomes load-bearing, so the day starts with a written tolerance document. The numbers are not arbitrary, and arguing for them rather than asserting them is the discipline.

Why relative $1 \times 10^{-}$ and absolute $1 \times 10^{-}$

Three forces set the floor on tolerance.

Floating-point precision in double. IEEE 754 double precision has 52 bits of mantissa, which corresponds to about 15–16 decimal digits of relative precision. The lower bound on detectable difference for any single arithmetic operation is therefore around 1×10^{-15} relative. Any tolerance looser than this absolute floor is a real choice rather than a forced one.

Accumulated error in chained operations. A PCA on a 450-sample $\times$ 300-metabolite matrix involves $O(n^2)$ accumulated multiplications and additions. The accumulated relative error scales roughly as $\sqrt{n} \times$ machine epsilon for well-conditioned matrices; with $n \approx 300$ and machine epsilon $\approx 2.2 \times 10^{-16}$, the accumulated error is on the order of 4×10^{-15} for the well-conditioned case and several orders of magnitude worse near ill-conditioned subspaces. A tolerance of 1×10^{-10} catches real regressions; a tolerance of 1×10^{-6} has 10^4 headroom over the accumulated-error floor for a well-conditioned PCA.

Cross-environment drift from BLAS, OS, and thread count. This is the dominant source of legitimate variation in practice. Chapter 11 §11.6 documents that OpenBLAS, MKL, and ATLAS produce subtly different results for matrix multiplication, with differences of "a few ULPs." For a double-precision multiplication, a few ULPs translates to a relative error around 10^{-15} per operation but can amplify to 10^{-8} or so after accumulated PCA work. Empirically, on the development team's three primary build environments (Ubuntu 22.04 + OpenBLAS 0.3.26 + 4 threads; the same with 1 thread; Debian

12 + MKL + 4 threads), PCA scores on Progredir differ by up to relative 5×10^{-7} across environments while remaining numerically equivalent for any downstream interpretation.

The chosen tolerance is **relative** 1×10^{-6} , **absolute** 1×10^{-9} , with sign-flip tolerance for PCA components. The relative threshold is set tight enough to detect a real numerical regression (an off-by-one in a sum will not pass; a forgotten centering step will not pass) and loose enough to absorb the empirically observed BLAS-version drift with about $2 \times$ headroom. The absolute threshold is set to handle numerical fields whose magnitude is small enough that the relative tolerance is uninformative (a variance-explained fraction of 0.0001 has relative-1e-6 tolerance of 10^{-10} , smaller than the BLAS noise floor for that scale; the absolute floor of 1×10^{-9} takes over).

The policy is committed Wednesday morning to docs/TOLERANCE.md:

*Numerical fields in golden-snapshot comparisons are matched to **relative tolerance** 1×10^{-6} and **absolute tolerance***

1×10^{-9} . Categorical fields, column names, factor levels, and metadata strings must match exactly. Counts and integer-valued fields must match exactly. PCA scores and loadings tolerate sign flip on a per-component basis: each component is sign-aligned to the snapshot before comparison.

Wednesday's three tools and their wrinkles

`transform`'s `log` methods need an explicit error path when the matrix contains negative or zero values without an offsetting `pseudocount`; `Tuesday's load_metabolomics_data` does not guarantee non-negativity, so the validation lives in `transform`.

`scale`'s Pareto method requires non-zero variance per metabolite. The test passes a degenerate snapshot (one metabolite of constant abundance) and asserts a structured error, validating that the error-path itself is tested.

`pca` needs the sign-flip handling described above. The snapshot stores both the raw scores and a sign-key (the sign of the

loading on a designated reference metabolite per component) so that the test can sign-align before comparison.

By end of Wednesday seven of ten tools are green.

14.5 Thursday — tools 8–9 and the limma question

Thursday implements `sources_of_variation` and `differential_abundance`. These are the analytical end of the pipeline and the tools whose outputs are what a clinical user actually sees.

`sources_of_variation` wraps `metabolonr::sov()` and is straightforward; the only subtlety is the covariate-handling for constant or near-constant variables, which the `v1` implementation already handles. The oracle test runs the Chapter 6 §6.4 covariate set and pins the F-ratio ranking.

`differential_abundance` is the day's central concern. The tool exposes three test families (limma, t-test, Wilcoxon) and three FDR methods (BH, Bonferroni, Holm), so the oracle

test matrix is $3 \times 3 =$ nine cases. Each case has its own golden snapshot from v1.

The limma case is where the tolerance question gets sharp. `limma::eBayes` is sensitive to the prior estimation algorithm, which changed in a 2022 limma minor release. The tolerance for limma p-values is set looser than the default (relative 1×10^{-5} rather than 1×10^{-6}) to absorb this; the looser tolerance is documented in `tests/test_differential_abundance.py` with a comment citing the limma changelog. The log fold changes themselves remain at relative 1×10^{-6} .

By end of Thursday nine of ten tools are green. The team retros on the tolerance numbers and confirms them for the rest of the week.

14.6 Friday — `export_session`, coverage push, retro

Friday implements `export_session` and finishes the week's coverage push. `export_session` is the only tool with no R-side analog; `v1` had no equivalent and the Python implementation builds the export archive from scratch.

The implementation is straightforward in shape but careful in detail. The session log is the user-facing artifact; its schema is the schema agreed in `ARCHITECTURE.md` on Thursday of Week 1. The methods paragraph is a Jinja2 template that walks the tool-call list. The figures are produced by re-running the relevant tools with a `figure_only=True` flag and capturing their plotted output. The metabolite table is a CSV export of the `differential_abundance` result.

The export tool's oracle test asserts that the archive, when unpacked, contains the expected files and that the methods paragraph matches a templated comparison (parameters substituted into a fixed string, then string equality).

The afternoon is coverage push. The CI reports lines not covered; the team adds tests until coverage clears 85% across `tools/` and 70% across the project as a whole.

The Friday afternoon retro names two things to keep and one to change:

- Keep: the daily oracle-test cadence.
- Keep: the shared bridge as the only path to R.
- Change: the snapshot-capture script needs a `--regenerate` mode for the inevitable v1 bug fixes (see §14.8).

[Figure 14.1: Test pyramid. Bottom: unit tests of the bridge and converters (~50 tests). Middle: oracle tests of the ten tools (~25 tests across parameter variants). Top: integration tests of the agent loop (~5 tests, deferred to Week 3).]

[Figure 14.2: Coverage report screenshot at end of week, with the 85% threshold highlighted on `tools/` and the 70% threshold on the whole project.]

14.7 Pitfalls — actual transcript-level examples

The pitfalls below are not theoretical. Each was hit by a real student in a Kretzler-lab rotation during a prior cohort of MetabolonR development; the error message and the fix are reproduced verbatim. A student following this chapter should expect to hit at least two of them.

rpy2 install on Ubuntu 22.04 with R 4.4. The error appears at first `import rpy2.objects`:

```
ImportError: cannot import name 'NULLType' from package 'rpy2.rinterface' (most likely due to a circular import)
```

The cause is that `pip install rpy2` was run before R was on the system path. The fix:

```
$ sudo apt install r-base-dev libcurl4-openssl-dev
$ R --version # confirm R 4.4.x is on the path
$ pip uninstall rpy2
$ pip install rpy2 --no-cache-dir
```

```
$ python -c "import rpy2.robj as ro; print(ro.o(2))"
[1] 2
```

The `--no-cache-dir` is necessary because pip caches the failed rpy2 wheel, which is still broken even after R is installed.

renv::restore() failing on a missing system library.

During the first run on a fresh Ubuntu image, the team hit:

```
Error: package 'XML' is required but not available
configure: error: "libxml not found"
```

The fix is `sudo apt install libxml2-dev`. The deeper lesson: a complete fresh-install runbook lists every system dependency the R packages depend on. The team's runbook is committed to `docs/INSTALL.md` and is the first thing a new lab member is pointed at.

limma::eBayes numerical drift across minor versions.

During Thursday afternoon, the team observed:

```
AssertionError: limma p-value diverged from snapshot
metabolite lactose: snapshot 0.00342, observed 0.00342
```

The cause was that the local environment had `limma` 3.56.2 while the snapshot was captured under 3.56.1. The fix was to relax the `limma`-p-value tolerance to relative 1×10^{-5} (documented inline in the test as discussed above) rather than re-capture the snapshot, because the new `limma` version is the correct one going forward.

Factor-level **ordering** **surprise.** A
`differential_abundance` test failed with:

```
AssertionError: contrast_group differs from snapshot
    snapshot: "progressor"
    observed: "stable"
```

The cause was that `as.factor(progressor_column)` ordered levels alphabetically when called from `rpy2`, but in R-native code the levels were ordered by first-appearance. The fix in the bridge: sort factor levels explicitly at the bridge layer (`bridge.py`) so that downstream R code sees a stable ordering. The bridge tests now cover this case in `tests/test_bridge_factors.py`.

14.8 The v1-bug exception

During the limma testing on Thursday afternoon, the team discovered a marginal-case bug in the `v1 run_da` wrapper: when `reference_group` was passed as a non-string (it should always be a string, but a Shiny-side coercion was implicit), the resulting limma fit silently swapped groups, producing an effect-size sign that did not match the user's expressed intent.

The handling is the only example of the *v1-wins* rule (Chapter 5 §5.6) being overridden during the build, and it is recorded carefully. The `v1` routine is patched at tag `v1.4.3`, the corrected snapshot is re-captured against `v1.4.3`, the test references the new snapshot, and a one-paragraph note in `docs/V1_BUG_FIXES.md` documents the change. Any future `v2` user replaying an old session against pre-`v1.4.3` `v1` sees a flag in the replay output indicating the discrepancy.

This is the single best example, in the build, of why the bidirectional contract between `v1` and `v2` matters. Bugs in `v1`

are also v2's concern; v2's tests are the cleanest place they get caught.

What you should be able to do after this chapter

- Wrap an R function as a typed Python tool: Pydantic schema wi

Chapter 15 — Week 3: Agent Loop and Replay

Week 3 makes the system look like an agent. The single-turn loop on Monday, the multi-turn loop on Tuesday with the full ~80-line implementation, the session logger and content-addressed handles on Wednesday, the replay engine on Thursday, the lab-server demo on Friday with the agent producing a Titan-aligned biomarker shortlist. By end of Friday a clinician can have a conversation with the system and the conversation can be replayed.

15.1 Monday — single-turn agent

Monday is the first day the LLM is in the loop. The deliverable is a Python module `agent/single_turn.py` that takes a user message and a `DatasetHandle`, calls the Anthropic API with the ten tools declared, and returns either the assistant's text response or a single tool-call result.

The implementation reuses the dispatch table from Week 2 — every tool is already a function from `(args_dict) → result_dict`. The new code is the API plumbing.

By end of day Monday the smallest possible smoke test passes: the user message "Summarize the loaded Progredir dataset" triggers a single `summarize_dataset` tool call and the assistant reports `n=454` samples, `n=293` metabolites after QC, and the group sizes from Chapter 6 §6.7. The test is in `tests/test_single_turn.py` and uses the actual Anthropic API; it is the first test that needs a live API key. CI runs it on the `main` branch only, with the proxy in budget-cap mode, to keep PR builds cheap.

A common Monday surprise is that the assistant's text response varies across runs even when the tool call does not. This is fine — the LLM's text wrapping the deterministic tool result is non-deterministic by design — and the test asserts on the tool result, not on the text.

15.2 Tuesday — multi-turn agent in full

Tuesday extends the loop to handle sequences. The full implementation is approximately 80 lines including the conversation-history bookkeeping and the error-handling paths. It is reproduced here in full because subsequent chapters refer to it; the goal is that a student following this build can copy the code, paste it into `agent/multi_turn.py`, and have a working multi-turn agent by lunchtime on Tuesday.

```
# agent/multi_turn.py
import json

from anthropic import Anthropic
from .system_prompt import SYSTEM_PROMPT, SYSTEM_PROMPT_SUFFIX
from .tools_registry import TOOLS_JSON_SCHEMA, display_tools
from .session import Session
from .exceptions import (
    SchemaValidationError,
    ToolExecutionError,
```

```
        UnexpectedStopReason,  
        IterationLimitExceeded,  
    )
```

```
MODEL_NAME = "claude-sonnet-4-6"
```

```
MAX_TOKENS = 2048
```

```
DEFAULT_MAX_ITERATIONS = 25
```

```
_client = Anthropic()
```

```
def _extract_text(content):
```

```
    """Pull the assistant's user-facing text from a list of blocks.
```

```
    for block in content:
```

```
        if getattr(block, "type", None) == "text":
```

```
            return block.text
```

```
    return ""
```

```
def run_multi_turn(  
    user_message: str,
```

```

    session: Session,
    max_iterations: int = DEFAULT_MAX_ITERATIONS,
) -> str:
    """Run an agent turn that may call multiple tools

```

The loop:

1. Send messages + tool declarations to the Agent
2. If the model emits ``end_turn``, return the response
3. If the model emits ``tool_use``, dispatch each tool call,
 - the results as ``tool_result`` blocks, log the results, and
 - continue the loop.
4. If any other stop reason appears, raise.
5. If we exceed `max_iterations`, raise.

```

    """

```

```

    msgs = session.history + [{"role": "user", "content": user_message}]
    session.log_user_message(user_message)

```

```

    for _ in range(max_iterations):
        response = _client.messages.create(
            model=MODEL_NAME,
            system=SYSTEM_PROMPT,
            messages=msgs,

```

```

        tools=TOOLS_JSON_SCHEMA,
        messages=msgsgs,
        max_tokens=MAX_TOKENS,
    )
    session.log_assistant_message(response.content)
    msgsgs.append({"role": "assistant", "content": response.content})

    if response.stop_reason == "end_turn":
        session.history = msgsgs
        session.log_api_spend(response.usage)
        return _extract_text(response.content)

    if response.stop_reason == "tool_use":
        tool_blocks = [
            b for b in response.content if b.type == "tool_use"
        ]
        tool_results = []
        for tb in tool_blocks:
            try:
                result = dispatch_tool_call(tb.args, session)
            except Exception as e:
                session.log_error(f"Tool call failed: {e}")

```

```

        status = "ok"
    except SchemaValidationError as exc:
        result = {
            "status": "error",
            "error_type": "schema_validation_error",
            "detail": str(exc),
        }
        status = "error"
    except ToolExecutionError as exc:
        result = {
            "status": "error",
            "error_type": "tool_execution_error",
            "detail": str(exc),
        }
        status = "error"
    tool_results.append({
        "type": "tool_result",
        "tool_use_id": tb.id,
        "content": json.dumps(result),
    })
    session.log_tool_call(tb, result, status)

```



```

        msgs.append({"role": "user", "content":
            session.log_api_spend(response.usage)
            continue

    raise UnexpectedStopReason(response.stop_reason)

    raise IterationLimitExceeded(max_iterations)

```

Several design decisions in this loop are deliberate and worth naming. The `system-prompt` SHA-256 is imported from `system_prompt.py` and pinned in the session header at session open, so a downstream replay knows exactly which prompt was used. The `MODEL_NAME` is a module-level constant — never inlined — so a global change of model is a one-line edit and the session log faithfully records what was used. Every tool call is logged via `session.log_tool_call(...)` *before* the next API request is made; the log is the source of truth even if the next API request fails. The loop maintains conversation history as a

list that is appended in place; truncation, when needed, lives elsewhere (Chapter 10 §10.3).

The Tuesday smoke test is the Progredir worked example from Chapter 6 §6.4: "Run a standard MetabolonR pipeline on Progredir and report a volcano plot." The expected outcome is a sequence of tool calls (`load` → `summarize` → `qc_filter` → `impute_missing` → `transform` → `scale` → `pca` → `sources_of_variation` → `differential_abundance` → `export_session`) followed by a text response summarizing the result. The test asserts on the sequence of tool names; a trailing `summarize_dataset` after `load_metabolomics_data` is allowed.

By end of day Tuesday the multi-turn smoke test passes three consecutive runs. Inconsistency on Tuesday means the system prompt is not specific enough; the rest of the day is prompt tuning.

15.3 Wednesday — DatasetHandle content-addressing and the SQLite logger

Wednesday is the day the agent stops running in memory and starts running against a persistent store. Two pieces of work land.

Content-addressed DatasetHandles

A `DatasetHandle` is the agent's reference to a dataset state — the loaded matrix, or the QC-filtered matrix, or the imputed-and-scaled matrix. The handle is an opaque string the LLM passes between tool calls; the actual data lives in an in-process cache keyed by the handle.

The content-addressing scheme is what makes the agent's behavior reproducible without requiring the LLM at replay time.

Hash function. SHA-256. Chosen for three reasons: it is cryptographically strong (no collisions are known or expected), it is fast on modern CPUs (sub-microsecond per kilobyte), and it has stable, language-neutral semantics — a SHA-256 computed in Python matches one computed in R or in JavaScript, which matters because a replay run could happen in any of those environments.

What gets hashed for the load handle. The byte content of all three input files (abundance, sample annotation, metabolite annotation), concatenated in a canonical order. The resulting hex digest, truncated to 16 hex characters, becomes the handle ID with a `dh_` prefix. The canonical column order for the abundance matrix is the sample-ID order from the sample annotation file (sorted alphabetically by ID), so two loads of the same dataset with the rows in a different physical order produce the same handle. This canonicalization is the most subtle part of the scheme.

What gets hashed for downstream handles. For a tool that consumes one handle and produces another (`qc_filter`, `impute_missing`, etc.), the new handle's ID is the SHA-256

of `(input_handle_id || tool_name || sorted_args_json)`. The downstream handle is therefore deterministic given the upstream chain.

Why this matters for replay. The replay engine (Chapter 11) executes the recorded tool calls in order. If the input bundle is byte-identical to the one in the log, every downstream handle ID computed during replay matches the one in the log. The handle equality is a fast, transitive check that nothing has drifted in the dataset pipeline. The numerical equality of the actual data is checked separately via the output-hash comparison; the handle equality is the cheap first-pass check that the *pipeline* is in the right state.

Why not just hash the data? Hashing the full matrix at every step would be $O(n)$ at every tool call. The chained-hash scheme is $O(1)$ at each tool call after the initial load and is functionally equivalent: two handles match if and only if the entire upstream chain matches.

The cache layer in `agent/handle_cache.py` keys on the handle ID and supports an LRU eviction policy with a floor

that always retains the handles referenced by the current session. The cache's footprint on disk is bounded by configuration (default 5 GB under `/nfs/turbo/.../cache/`), and the LRU policy means cold sessions get evicted as new ones come in.

The SQLite session logger

The Chapter 11 §11.3 schema is implemented in `agent/db.py`. The six tables are created by `bootstrap()` if the database does not already exist; the writer methods (`log_user_message`, `log_assistant_message`, `log_tool_call`, `log_api_spend`) are called from the agent loop. By end of day Wednesday, running the Tuesday smoke test produces a SQLite database file with one new sessions row, several messages rows, ten `tool_calls` rows, ten `tool_results` rows, ten `dataset_handles` rows, and one `outputs` row (the exported `analysis_log.json`).

15.4 Thursday — replay engine and FastAPI

Thursday is the day the system grows two new entry points.

`agent/replay.py` is the replay engine. It is approximately 30 lines of Python and follows Chapter 11 §11.5. The smoke test is to take the `analysis_log.json` from Wednesday's smoke test, run the replay engine against it, and assert that the resulting output hashes match the log's recorded hashes within tolerance.

`services/backend/main.py` is the FastAPI app. It exposes three endpoints: `POST /chat` for an agent message, `POST /replay` for a replay run, and `GET /health` for liveness checks. Each endpoint is a thin wrapper around the existing modules; the wrappers' job is to translate HTTP requests into Python-call signatures and back. The full `main.py` is roughly 90 lines.

By end of day Thursday the replay engine works. The FastAPI app runs locally on port 8000. A `curl POST http://localhost:8000/chat` with the Tuesday smoke-test message produces a valid response. A subsequent `curl POST /replay` with the exported log path reproduces the result.

A Thursday subtlety is the prompt-template versioning. The system prompt is now SHA-256 hashed at module-load time and the hash is logged in the `sessions` table. If the prompt is changed between an original run and a replay, the replay still works (the prompt is not used at replay time) but the discrepancy is noted in the replay report.

15.5 Friday — the lab-server demo

Friday is the demo day. The morning is spent deploying the stack to `alakwaaf-ap-ps1a` using the Docker Compose configuration from Chapter 12. The afternoon is the live demo.

By 11 AM the stack is running on the lab server. By noon the Streamlit frontend (not yet polished; that is Week 4 Monday) is

accessible at <https://alakwaaf-ap-ps1a.umms.med.umich.edu:8501> to lab-internal IPs. The Progredir dataset is preloaded into /data/ from the Metabolomics Workbench accession ST000978 (Chapter 6 §6.7).

The 2 PM demo is to a small group: the advisor (Fadhl), the Kretzler-lab analyst who is the closest internal customer, and one nephrologist who has agreed to be the cold-start user. The demo runs four scripts.

Script 1: standard pipeline. The Chapter 6 worked example, transcribed verbatim. The agent loads Progredir, runs the standard pipeline with the canonical parameters from §6.7, and produces the volcano plot. Expected runtime: 90 seconds end to end. The room sees the agent emit the v1 oracle's ~18-metabolite limma shortlist (Path A in Chapter 20 §20.4b's terminology), and the methods paragraph the agent generates matches the parameter trace.

Script 2: biomarker discovery against the composite outcome. The cold-start nephrologist types: "find biomarkers in

the ProgreDir cohort for the composite outcome of mortality or incident renal replacement therapy." The agent — because v2.0's tool set does not include `survival_analysis` (Chapter 24 §24.1) — responds with the closest available analysis (the `limma` binary contrast) and explicitly notes the scope limitation in its response. The agent's biomarker shortlist for this prompt is expected to include between five and seven of the verified Titan 2019 fully-adjusted Cox 18-metabolite shortlist (Chapter 6 §6.5): specifically, the agent should reliably surface the high-effect-size, large-magnitude metabolites that both Cox-on-time-to-event and `limma`-on-binary recover — Lactose, D-threitol, Pseudouridine, Acetohydroxamic acid, and DHA are the expected anchors; Tyrosine (inverse) and Myo-inositol are next-tier. The room sees the agent recovering a meaningful subset of the published Titan reference, with the explicit caveat that full recovery of even the fully-adjusted 18 requires the Cox tool deferred to v2.1.

Script 3: a deliberately ambiguous request. "Tell me which metabolites are interesting in this dataset." The agent asks a clarifying question — what counts as "interesting"? — and waits.

The demo pauses to let the room see the clarification behavior in action.

Script 4: a replay. The `analysis_log.json` from Script 1 is fed to `/replay`. The system reproduces the output without calling the LLM. The demo shows the network traffic to the Anthropic endpoint going to zero during replay; this is the moment the reproducibility claim becomes concrete for the audience.

The 3 PM retro produces a list of feedback that goes into Week 4's UI polish.

The full demo transcript is captured to `docs/WEEK3_DEMO.md` and committed; it becomes the basis for one of the Chapter 16 validation prompts (Path A for Script 1, Path B for Script 2).

[Figure 15.1: Week 3 milestone burnup. Five horizontal bars for Monday–Friday. Bars are cumulative deliverables. The Thursday Replay-Engine milestone is annotated as the critical-path moment for Chapter 11's claim.]

[Figure 15.2: First end-to-end demo screenshot grid. Four panels: (a) Streamlit chat panel mid-analysis on Script 1, (b) the biomarker-discovery response on Script 2 with the Titan-aligned hits highlighted, (c) the SQLite session row inspected via `sqlite3` CLI, (d) the network-traffic graph during replay showing zero outbound LLM traffic.]

15.6 Pitfalls

Prompt-template drift. A late-Tuesday tweak to the system prompt that improves the multi-turn smoke test but breaks the Wednesday handle-cache test is the most common Week 3 stumble. Hash the prompt at module load and log the hash; do not let a casual edit slip through.

`tool_result` truncation. A large `differential_abundance` result (say, 400 metabolites $\times$ 5 fields each = 2000 entries) sent as a `tool_result` content block can hit the API's per-content-block size limits in some configurations. The fix is to summarize the table in the tool

result (top-20 by adjusted p-value, plus `n_significant`) and reference a `DatasetHandle` for the full table.

R session leaks across calls. `rpy2` holds a single R process across all calls in a Python process. A failing R call can leave the R-side environment in an inconsistent state, causing subsequent calls to fail mysteriously. The bridge's `r_session()` context manager wraps each call in a fresh R environment to avoid this.

Streamlit on the lab server. The default Streamlit port 8501 is firewalled at the Michigan ITS perimeter; only lab-internal access works on Friday. External access for the Week 4 demo requires an ITS ticket and a reverse-proxy setup, which is filed Friday afternoon to land in time.

Handle-cache collision under refactor. During Wednesday afternoon, a refactor of the canonical-column-order code in `load_metabolomics_data` produced different handle IDs for byte-identical inputs. The cause was a subtle change in how the sample-ID sort handled mixed-case identifiers. The fix was the explicit canonicalization rule (Chapter 6's sample annotation uses uppercase HMDB-style IDs; the sort is case-

sensitive). The pitfall illustrates why the content-addressing scheme is sensitive to canonicalization choices and why those choices need to be documented and tested.

15.7 Deliverable checklist

By end of Friday:

- `agent/single_turn.py`, `agent/multi_turn.py`, `agent/handle_cache.py`, `agent/db.py`, `agent/replay.py`.
- `services/backend/main.py` (FastAPI app).
- `docker-compose.yml` running locally and on `alakwaaf-ap-ps1a`.
- The Tuesday multi-turn smoke test passes three consecutive runs.
- The Thursday replay smoke test passes against the Tuesday log.
- `docs/WEEK3_DEMO.md` captures the Friday demo transcript including the four scripts.

What you should be able to do after this chapter

- Implement a multi-turn tool-use agent loop in fewer than 100 lines, including conversation history, structured error handling, and per-turn session logging.
- Design a SQLite schema that captures a session log and write the loader/dumper functions for it.
- Explain the content-addressed `Dat

Chapter 16 — Week 4: Frontend and the Validation Experiment

Week 4 turns the system into a product and a paper. The Streamlit UI ships on Monday; the 20-prompt validation experiment runs Tuesday through Thursday with both internal (Path A) and external (Path B) scoring arms; the Friday checkpoint asks: are the numbers strong enough to publish? The operational details that Chapter 20 §20.12 flagged — the fixed RNG seed, the two-role blinded scoring, the two-afternoon scoring schedule, and the Path B external-validation arm — are all incorporated into this chapter's day-by-day plan.

16.1 The two parallel tracks

Week 4 runs two tracks in parallel. The frontend track makes the Streamlit prototype demoable to anyone, not just to a lab-internal audience that tolerates rough edges. The validation track produces the empirical evidence that the system meets its

reproducibility claim against both the v1 oracle (Path A) and the Titan 2019 reference (Path B).

The two tracks are sequenced carefully. The frontend lands on Monday because the validation experiment depends on a stable interface (a UI change in the middle of the experiment would invalidate prior runs). The validation experiment runs Tuesday through Thursday because three full days of running, scoring, and inspecting is the minimum that produces a defensible number. Friday is the go/no-go.

16.2 Monday — the Streamlit UI

By end of day Monday the Streamlit app is at `services/frontend/streamlit_app.py` and deployed to the lab server at the same URL the Week 3 demo used. The polish from the Week 3 retro is incorporated.

The polish list is short and concrete:

- The chat panel auto-scrolls to the bottom on new messages. The Week 3 demo showed the panel staying

scrolled to the top, which made users miss the agent's responses.

- The session log panel on the right is collapsed by default and opens on click. The Week 3 version had it always-open and visually noisy.
- The dataset preview shows the first five rows of the abundance matrix and the first five rows of the sample annotation, with a "more" toggle. The Week 3 version dumped the entire matrix into the panel.
- The "Export session" button is now in the bottom-right of the chat panel, not buried in the sidebar.
- An "Examples" dropdown at the top of the chat panel populates the input box with one of five preset prompts. The Week 3 cold-start user got stuck on what to type; the dropdown removes that friction.

By 4 PM Monday the polished Streamlit app passes a five-task usability test with two new users (not the nephrologist from Week 3; new eyes are the test). The five tasks: open the app, load Progredir, ask for a summary, ask for a volcano plot of progressors vs stable, export the session.

If the usability test fails on any task, Monday extends into Tuesday morning. The validation experiment cannot begin until the UI is stable.

16.3 Tuesday — designing the 20 prompts, with worked paraphrase pairs

Tuesday morning the team designs the validation suite. The design has been outlined in docs/VALIDATION_PROTOCOL.md since Week 1; Tuesday is when it becomes concrete.

Ten **unambiguous prompts**, each phrased three different ways.
Ten **ambiguous prompts**, each phrased three different ways.
Total: 60 distinct prompt strings. The full list is given in Appendix C; below are four representative prompts with all three paraphrases shown and annotated for what is held constant versus what is varied.

Unambiguous prompt 1 — full standard pipeline (Path A).

- 1a: "Run a standard MetabolonR pipeline on Progredir and produce a volcano plot."
- 1b: "Do the usual analysis on the Progredir data — QC, impute, transform, scale, PCA, differential abundance — and give me the volcano."
- 1c: "I want to look for biomarkers in Progredir using the standard workflow. End with a volcano of progressors vs stable."

Held constant: dataset (Progredir), pipeline (standard), output (volcano), implicit contrast (progressor vs. stable), implicit parameters (defaults). *Varied:* sentence shape (declarative imperative vs. first-person declarative), lexicon (pipeline vs. analysis vs. workflow), explicitness of the pipeline steps (named in 1b, implicit in 1a and 1c).

Unambiguous prompt 5 — explicit log₁₀ transformation (Path A).

- 5a: "Run a standard analysis of Progredir but log₁₀-transform the abundances first."

- 5b: "Use log10 transformation (not log2) for the ProgreDir analysis."
- 5c: "I want the standard pipeline with log base 10 transformation."

Held constant: the specific deviation from default (log10 instead of log2), everything else inherits standard-pipeline defaults.

Varied: whether the prompt names the default it overrides (5b says "not log2" explicitly), lexical choice for "log base 10" vs "log10," sentence shape.

Unambiguous prompt P (Path B external-validation biomarker discovery).

This is the cohort's most consequential prompt for the external-validation arm.

- Pa: "Find biomarkers in ProgreDir for the composite outcome of mortality or incident renal replacement therapy."
- Pb: "Identify metabolites that predict the composite outcome (mortality or RRT) in the ProgreDir cohort."

- Pc: "I want a biomarker shortlist for the time-to-event outcome — mortality or dialysis — using the ProgreDir baseline serum data."

Held constant: the outcome (composite — mortality or incident RRT), the request for a biomarker shortlist, the dataset. *Varied:* whether "biomarker," "metabolites that predict," or "biomarker shortlist" is the noun phrase; whether the outcome is named as "composite," "time-to-event," or by enumeration; lexical equivalence on "RRT" / "dialysis" / "renal replacement therapy."

The expected agent behavior on this prompt is to acknowledge that v2.0's tool set does not include Cox/survival analysis (this is the explicit `survival_analysis` deferral to v2.1, Chapter 24 §24.1) and to run the closest available analysis — the limma binary contrast. The Path B scoring computes the agent's resulting shortlist against the Titan 34-metabolite Cox reference using external recall (ER) and external Jaccard (EJ) as defined in Chapter 20 §20.4b.

Ambiguous prompt 12 — underspecified contrast.

- 12a: "Compare the groups in ProgreDir."

- 12b: "Tell me what's different between groups in the Progredir data."
- 12c: "Run a group comparison on Progredir."

Held constant: the underspecified word "groups" — Progredir has several plausible grouping variables (progressor, sex, diabetes, hypertension). *Varied:* verb (compare, tell me what's different, run a comparison), sentence shape. *Expected agent behavior:* ask the user which grouping variable, listing the candidates.

By Tuesday noon the 20 prompts $\times$ 3 paraphrases = 60 strings are committed as `docs/VALIDATION_PROMPTS.md` and frozen for the duration of the experiment.

The expected behavior is recorded for every prompt. For unambiguous prompts the expectation is a specific tool sequence with specific parameters. For ambiguous prompts the expectation is a clarification question that addresses the specific ambiguity. For Path B prompts the expectation includes the explicit scope acknowledgment.

The Tuesday afternoon is the dry run: ten prompts (five unambiguous, five ambiguous) sampled at random are run once each. The team inspects the results to confirm the scoring rubric is operable. Two minor tweaks to the rubric are committed by end of day.

16.4 Wednesday — the 60-session run with both scoring arms

Wednesday runs the full experiment. Each of the 20 prompts is run three times. The harness is `scripts/run_validation.py`; the seed for prompt-order randomization is `0x7e1f1ad8` (committed in `docs/VALIDATION_PROTOCOL.md` per Chapter 20 §20.9). The three repeats of any single prompt are interleaved with at least 30 minutes of other runs between them; the harness enforces this via the schedule it generates.

The Wednesday harness logs everything. The committed logging checklist:

- *Per-prompt fields*: prompt ID, paraphrase index (a/b/c), repeat number, scheduled execution time, actual execution start time, scheduled-vs-actual delta.
- *Per-API-call fields*: model name (claude-sonnet-4-6), model snapshot date, input token count, output token count, cost in USD, wall-clock duration, HTTP status, any retry events.
- *Per-tool-call fields*: tool name, validated arguments dict, returned status (ok or structured error), input handle, output handle, wall-clock duration.
- *Per-session fields*: session ID, system-prompt SHA-256, tool-layer commit, R version, Python version, BLAS implementation and thread count, OS image digest.
- *Per-output fields*: file path, SHA-256, byte size, format.

Every field above goes into the SQLite session store (Chapter 11 §11.3) and is exported to the per-session `analysis_log.json` (Chapter 11 §11.4). A field missing from the harness's logging is a field missing from the

manuscript's evidence trail, which is why the checklist is committed and reviewed before any of the 60 sessions run.

By 5 PM Wednesday 60 session logs are on disk. Each log is roughly 6–15 KB.

A Wednesday afternoon surprise: in three of the 60 runs the agent times out at 90 seconds without completing the analysis. The cause is API-side latency rather than agent logic; the timeouts are noted, the harness is configured to retry once with a longer timeout, and the three reruns complete by 6 PM. The retry decision is logged in `data/validation/RETRY_LOG.md`.

16.5 Thursday — scoring under both paths, with two-role blinding

Thursday is the scoring day. Five metrics are computed across the 60 logs.

Path A (internal reproducibility)

The three Path A metrics from Chapter 20 §20.4 are computed by `scripts/score_validation.py` and write to `data/validation/RESULTS.md`. The scoring is fully automated for unambiguous prompts.

- **Tool-sequence agreement (TSA):** binary per prompt, soft variant alongside.
- **Parameter agreement (PA):** median across (prompt, tool) pairs.
- **Jaccard on biomarker sets (J):** mean pairwise Jaccard on the limma significant-metabolite sets.

Path B (external validation against Titan 34-set)

For prompts that elicit biomarker output (the standard-pipeline prompts plus the explicit Path B prompt described in §16.3),

the harness additionally computes the two external-validation metrics from Chapter 20 §20.4b:

- **External recall (ER):** $\mathrm{ER}(q) = |B_q \cap T| / |T|$, where B_q is the agent's biomarker set for paraphrase q and T is the Titan 34-metabolite reference from `data/validation/titan_2019_reference.csv`. Reported metric is the mean across Path B paraphrases.
- **External Jaccard (EJ):** $\mathrm{EJ}(q) = |B_q \cap T| / |B_q \cup T|$. Reported metric is the mean across Path B paraphrases.

The metabolite-ID mapping (Chapter 20 §20.4b) is handled by `scripts/score_external_validation.py`. Both Path B metrics use HMDB IDs as the canonical lookup key; agent outputs with non-HMDB display names are mapped via the metabolite annotation file.

Ambiguous-cohort clarification scoring — blinded, two-role

The ambiguous cohort has an additional manual-scoring pass. The two-role protocol from Chapter 20 §20.9 operates as follows.

Role 1 — rater (Fadh1). The rater is presented with a CSV in which each row is the agent's first assistant turn, identified only by a random per-row hash. The original prompt text is *not* in the CSV; neither is the repeat index (1st, 2nd, or 3rd repeat of the prompt); neither is the timestamp. The CSV rows are in a randomized order, generated from a seed (0x9c4f2a1b) separate from the run-execution seed. The rater fills in a rubric checklist per row: (a) did the agent ask a question? (b) does the question address an ambiguity in the prompt? (c) is the question on the "right axis" — does it target the specific ambiguity the protocol named?

Role 2 — analyst (advisor or a designated lab member; for v2.0, the advisor). Post-scoring, the analyst reconciles the random row IDs back to (prompt, repeat) pairs using the seed-

derived shuffle, and computes the CR metric. The reconciliation is mechanical; the scoring is human; the two roles are separated to limit single-rater bias.

Two-afternoon scoring schedule

The 60 scoring events are distributed across two afternoons (Thursday and Friday morning if needed), not one. A single sitting risks fatigue-induced rubric drift. The protocol prescribes:

- Afternoon 1 (~30 rows): rubric re-read at start; 30-minute mid-session break; second 15-row block.
- Afternoon 2 (~30 rows): rubric re-read at start; same structure as Afternoon 1.

A row scored late on Afternoon 1 is not scored adjacent to the same row on Afternoon 2; the CSV is regenerated between sittings to avoid this.

Per-Wednesday-prompt result rows

By 5 PM Thursday the five metrics have placeholder numbers ready for the manuscript:

- TSA (unambiguous, Path A): [PLACEHOLDER: %]
- PA (unambiguous, Path A): [PLACEHOLDER: %]
- Jaccard on biomarker sets (unambiguous, Path A):
[PLACEHOLDER: index]
- CR (ambiguous): [PLACEHOLDER: %]
- ER (Path B): [PLACEHOLDER: %]
- EJ (Path B): [PLACEHOLDER: index]

The numbers will be filled in for Chapter 21 at the placeholder-replacement pass after the Week 4 experiment completes.

16.6 Friday — the go/no-go checkpoint

Friday morning the team reviews the Thursday numbers against the targets agreed in `docs/VALIDATION_PROTOCOL.md`.

The targets are:

- Tool-sequence agreement (Path A, unambiguous): $\geq 90\%$.
- Parameter agreement (Path A, unambiguous): $\geq 85\%$.
- Jaccard on biomarker sets (Path A, unambiguous): ≥ 0.85 .
- Clarification rate (ambiguous): $\geq 80\%$.
- External recall (Path B): ≥ 0.50 .
- External Jaccard (Path B): within-prompt mean ≥ 0.75 .

If all six targets are met, the build proceeds to Week 5 with confidence. If any target is missed by ≤ 5 points, the team has a Friday afternoon discussion: is the gap a real-world issue, a prompt issue, or a scoring artifact? If it is a prompt issue, the system prompt is tuned and one additional repeat of the affected prompts is run on the following Monday.

If a target is missed by > 5 points, the team has a harder conversation. The architecture-lock document is consulted; if

the gap is fundamental, the manuscript schedule slips and Week 5 begins with mitigation work rather than manuscript drafting. The author's empirical expectation, going into Week 4, is that the Path A targets will be met within the first run; the Path B target ($ER \geq 0.50$) is more uncertain because no prior empirical estimate exists, and a miss there is the most likely scenario for a Monday revisit.

[Figure 16.1: Streamlit UI screenshot, end of Monday. Chat panel center, examples dropdown above it, dataset preview top-left, log panel right.]

[Figure 16.2: The 20-prompt design grid. Two columns of ten rows each: unambiguous on the left, ambiguous on the right. Path B prompts marked with a distinct shading.]

[Figure 16.3: Preliminary agreement plot. Six bars for the six metrics — four Path A, two Path B. Numbers are [PLACEHOLDER] pending Chapter 21's final pass.]

16.7 Pitfalls

Prompt drift across runs. A copy-paste error that introduces a typo in one of the 60 prompt strings invalidates that run. The harness reads from the committed `VALIDATION_PROMPTS.md`, never from memory or from a shell variable.

API rate limits. Anthropic's rate limits at the lab's tier allow comfortably more than the 60 sessions Wednesday requires, but only with reasonable pacing. The one-hour gap between repeats is partly for this reason.

Prompt-order effects. If all three repeats of one prompt run back-to-back, any temporary API-side caching could inflate the agreement. The harness interleaves repeats across prompts to mitigate. The seed (`0x7e1f1ad8`) is recorded for replay.

Manual scoring fatigue. Distributed across two afternoons with documented breaks (\$16.5).

Path B metabolite-ID mismatch. The agent's biomarker output uses chemical names; the Titan reference uses HMDB IDs. A name-to-HMDB mismatch silently undercounts intersection size, deflating ER and EJ. The mapping is checked by hand for the first ten Path B sessions before the scoring is trusted; mismatches are added to `scripts/known_aliases.csv`.

16.8 Deliverable checklist

By end of Friday:

- Polished Streamlit app deployed to the lab server.
- `docs/VALIDATION_PROMPTS.md` (frozen Tuesday noon).
- 60 session logs in `data/validation/`.
- `scripts/run_validation.py` (with seed `0x7e1f1ad8`) and `scripts/score_validation.py`.
- `scripts/score_external_validation.py` (Path B scoring with HMDB-mapped intersection).

- `data/validation/titan_2019_reference.csv` (the 34-metabolite Titan reference, accessible via Metabolomics Workbench ST000978).
- `data/validation/RESULTS.md` with the six metrics filled in.
- `data/validation/RETRY_LOG.md` documenting any reruns.
- A Friday go/no-go decision documented in `docs/WEEK4_GO_NO_GO.md`.

What you should be able to do after this chapter

- Polish a Streamlit prototype to a demoable state in one day, based on a usability-test punch list.
- Design a paraphrase-based validation suite with both unambiguous and ambiguous prompts, and annotate the held-constant vs varied dimensions of each paraphrase set.

- Define and compute the four internal-agreement metrics (TSA, PA, J, CR) and the two external-validation metrics (ER, EJ).
- Operationalize the two-role blinded-scoring workflow with the rater/analyst separation and the seed-derived row randomization.
- Run a 60-session experiment with sensible API pacing, retry logic, and a per-prompt logging checklist that captures every field the eventual manuscript will cite.
- Score Path B against the Titan 34-metabolite reference using HMDB-mapped intersections.
- Make a Friday go/no-go decision against six pre-registered targets.

Chapter 17 — Week 5: Manuscript Drafting and Repository Polish

Week 5 turns artifacts into a manuscript. The figures, the methods, the cover letter, the Docker image, the Zenodo DOI, the README. By Friday night the repository is in a state a reviewer could clone and a manuscript draft is in a state a coauthor could read.

17.1 The Week 5 contract

Week 5's deliverable is two things in parallel: a complete manuscript draft suitable for advisor review over the weekend, and a public-repository state suitable for being cited in that manuscript. The two deliverables are interdependent — the manuscript references the repository, the repository's README references the manuscript — and the week is paced to land them together.

The author has been writing in the margins for two weeks already. Week 5 is the focused drafting pass.

17.2 Monday — manuscript outline

The manuscript is targeted at *Briefings in Bioinformatics* as the primary venue (per Chapter 18 §18.2). Their Application Note format is approximately 2,500 words with up to three figures. The outline produced on Monday morning:

Title. *MetabolonR-LLM: Architecture and Validation of an Agentic Interface for Metabolomics Analysis*

Abstract. 250 words. Three paragraphs. (1) The problem: clinical metabolomics needs a no-code interface that is also reproducible. (2) The solution: a tool-calling LLM agent over a curated, version-pinned tool layer with a structured session log and a replay engine. (3) The validation: 20-prompt $\times$ 3-paraphrase experiment on the Progredir cohort; six pre-registered metrics across Path A (internal reproducibility against v1) and Path B (external validity against Titan 2019).

Introduction. 500 words. The Chapter 1 problem, very compressed.

Methods. 800 words. The architecture in summary: closed tool set, schema-pinned arguments, deterministic R-side computation, `analysis_log.json` and replay engine. The validation design including the Path A / Path B distinction.

Results. 600 words. The six headline numbers from Chapter 21, with the failure-mode taxonomy condensed to a one-paragraph callout.

Discussion. 300 words. The reproducibility claim in publication form, the comparison with prior agentic-bio tools, the regulatory implication in one sentence.

Availability. 50 words. Repository URL, Zenodo DOI, software license (MIT), data accession (Metabolomics Workbench ST000978 for the ProgreDir cohort), contact.

The outline is committed Monday afternoon as `manuscript/OUTLINE.md` and circulated to the advisor and to one external reader for fast feedback.

17.3 Tuesday — figures and graphical abstract, figure by figure

Tuesday is the figure day. The Application Note format allows three figures plus the graphical abstract. The plan, figure by figure:

Figure 1 — Architecture overview. This is the most important figure in the paper. It is a reworking of Figure 9.1 from the book: User → Agent (LLM) → Tool dispatcher (Python with Pydantic validators) → rpy2 bridge → R routines → session logger (SQLite) → exported analysis_log.json → replay engine. The figure has two annotation overlays: a "routing layer" box around the LLM and dispatcher (highlighting the non-deterministic part), and a "computation layer" box around the bridge, R routines, and logger (highlighting the deterministic part). The visual contrast between the two boxes carries the methodological claim of Chapter 8 §8.1: the LLM routes, deterministic code computes.

Figure 2 — Worked-example transcript. A reworking of Figure 10.1 plus the §10.9 transcript. Three columns: User message, Agent reasoning summary (one short line per turn — never the LLM's raw chain-of-thought, which is not load-bearing for any number), Tool call + parameters. The figure shows the Progredir standard-pipeline session end to end. The reader who never reads the Methods can still tell, from this figure, what the system does on a real input.

Figure 3 — Validation results. Six bars for the six metrics: four Path A (TSA, PA, J, CR) and two Path B (ER, EJ). 95% CIs from the Chapter 21 bootstrap. Pre-registered targets drawn as horizontal lines on each bar. The figure is the empirical claim of the paper. [PLACEHOLDER] for the numerical bar heights until Chapter 21 is finalized; the layout and the axes are committed today.

Graphical abstract. Single panel, designed to communicate the architecture-plus-result claim in one image. Layout: top-left third shows a stylized user typing a prompt; top-right two-thirds shows the architecture diagram from Figure 1 in compressed form; bottom strip shows the six-bar validation

result from Figure 3 in compressed form. The graphical abstract is paid more attention to than its size suggests; it is what most readers see first, and it is what circulates on social media (Chapter 18 §18.1's preprint thread leads with the graphical abstract image).

All figures are produced in matplotlib with consistent fonts and color schemes. The figure-generation scripts are in `manuscript/figures/` and are run by `make figures`. Every figure is byte-reproducible from the validation logs in `data/validation/`.

17.4 Wednesday — Docker, Zenodo, DOI

Wednesday is the day the artifacts get DOIs.

The Docker images for backend, proxy, and frontend are built with explicit tags `v2.0.0` and pushed to the GitHub Container Registry. Each image's digest is recorded in `docs/RELEASE_v2.0.0.md`.

The GitHub release `v2.0.0` is cut. The release tag triggers a GitHub Action that builds a source tarball and pushes it to Zenodo via the configured webhook. Zenodo mints a DOI for the release. The DOI is captured and pinned in `CITATION.cff` and in the manuscript's Availability section.

The Zenodo deposit also includes the Wednesday-validation dataset: the 60 session logs from Week 4. This means a reviewer can replay any of the 60 sessions independently of the GitHub repository state.

The Zenodo metadata fields are filled in carefully. Authors: F. M. Alakwaa (ORCID linked) and the student (ORCID required to be set up by Monday). Keywords: metabolomics, large language model, tool use, reproducibility, chronic kidney disease. License: MIT for software, CC-BY-4.0 for the validation dataset.

17.5 Thursday — repository polish, with CITATION.cff and Dockerfile in full

Thursday is the polish day. The repository's public face is brought up to professional standard.

The CITATION.cff in full

The CFF file is what GitHub's "Cite this repository" widget reads. The full v2.0.0 file:

```
cff-version: 1.2.0
message: "If you use MetabolonR-LLM in your work, please cite the following paper."
title: "MetabolonR-LLM: An agentic interface for metabolomics analysis"
abstract: >-
  MetabolonR-LLM is a tool-calling large-language-model (LLM) for
  metabolomics analysis. It routes natural-language queries to a
  closed, schema-pinned set of ten analytical tools and executes
  routines via rpy2, with a structured session log.
```

re-executability via a separate replay engine.

authors:

- family-names: "Alakwaaf"
- given-names: "Fadh1 Mohammed"
- orcid: "https://orcid.org/[VERIFY]"
- affiliation: "Division of Nephrology, Department of Medicine, University of Michigan"
- email: "alakwaaf@umich.edu"

version: "2.0.0"

date-released: "2026-06-01"

license: MIT

repository-code: "https://github.com/alakwaaf/metabolonr-llm"

url: "https://github.com/alakwaaf/metabolonr-llm"

doi: "10.5281/zenodo.[VERIFY-after-Zenodo-mint]"

type: software

keywords:

- metabolomics
- large language model
- tool use
- reproducibility
- chronic kidney disease
- NEPTUNE

preferred-citation:

type: article

title: "MetabolonR-LLM: Architecture and Validation"

authors:

- family-names: "Alakwaa"

given-names: "Fadhil Mohammed"

journal: "Briefings in Bioinformatics"

year: 2026

doi: "[VERIFY-after-acceptance]"

references:

- type: article

title: "Metabolomics biomarkers and the risk of cardiovascular disease"

authors:

- family-names: "Titan"

given-names: "Silvia M"

journal: "PLOS ONE"

volume: 14

issue: 3

year: 2019

doi: "10.1371/journal.pone.0213764"

The file is validated against the CFF specification's schema by `cffconvert validate` in CI on every push. A CFF validation failure fails the build.

The Dockerfile in full

The `services/backend/Dockerfile` is the minimal-viable backend image. Multi-stage build keeps the final size around 1.8 GB.

```
# services/backend/Dockerfile
# Build stage: install R + R packages from renv.lock
FROM rocker/r-ver:4.4.1 AS builder

RUN apt-get update && apt-get install -y --no-install-recommends \
    libcurl4-openssl-dev libssl-dev libxml2-dev \
    libfontconfig1-dev libfreetype6-dev \
    && rm -rf /var/lib/apt/lists/*

COPY renv.lock /tmp/renv.lock

RUN R -e "install.packages('renv', repos = 'https://packagemanager.rstudio.com/cran/@latest', \
    renv::restore(lockfile = '/tmp/renv.lock'))"
```



```

# Runner stage: Python + rpy2 over the prepared R
FROM rocker/r-ver:4.4.1

RUN apt-get update && apt-get install -y --no-install-recommends \
    python3.11 python3-pip python3.11-dev \
    libcurl4-openssl-dev libssl-dev libxml2-dev \
    && rm -rf /var/lib/apt/lists/*

# Copy the prepared R library from the builder stage
COPY --from=builder /usr/local/lib/R/site-library/* /usr/local/lib/R/site-library/

WORKDIR /app

COPY pyproject.toml requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt

COPY agent /app/agent
COPY tools /app/tools
COPY services/backend/main.py /app/main.py
COPY scripts /app/scripts

```

```
ENV OPENBLAS_NUM_THREADS=4
```

```
ENV OMP_NUM_THREADS=4
```

```
ENV PYTHONUNBUFFERED=1
```

```
EXPOSE 8000
```

```
CMD ["python", "-m", "uvicorn", "main:app", "--host
```

Three details in the Dockerfile are load-bearing. First, the base image is `rocker/r-ver:4.4.1` rather than a vanilla Ubuntu image; the rocker images are maintained by the R community specifically for reproducible R deployments. Second, the multi-stage build separates the R-package install from the Python install; the builder stage's only output is the R site-library, copied across stage boundaries. Third, the BLAS thread count is pinned (`OPENBLAS_NUM_THREADS=4`) at the environment level, matching the canonical replay environment from Chapter 11 §11.6.

The other repository polish

README.md. Top section: a one-sentence description, a single-screenshot animated GIF of the Streamlit interface in action, three badges (CI status, Zenodo DOI, license). Quickstart section: clone, build, run. Architecture section: pointer to the manuscript and to the book. Citation section: the BibTeX entry for the eventual paper plus the Zenodo DOI.

LICENSE. MIT. The file's first line is the copyright statement with the year (2026) and the author's full name and affiliation.

CONTRIBUTING.md. Three sections: how to set up a dev environment (links to Chapter 13), how to run the tests, how to propose a new tool. The "how to propose a new tool" section is the most important; it documents the design-by-PR pattern that v2.x extensions will follow.

.github/ISSUE_TEMPLATE/ with a `bug.md` template and a `feature.md` template. Both have a "before opening this issue" checklist that includes running the replay engine on the affected session.

`.github/workflows/ci.yml` is reviewed and tightened. The CI now runs: `ruff`, `mypy`, `cffconvert` `validate`, the oracle tests (Week 2), the smoke tests (Week 3), and a small selection of the validation prompts (a sanity-check subset of Week 4, not the full experiment).

17.6 Friday — reproducibility

crosscheck

Friday morning is the prove-it day. A fresh clone of the public repository onto a blank Ubuntu VM, a `docker compose pull` && `docker compose up -d` against the published images, and a `curl POST /replay` against one of the Week 4 session logs. The expected output: the same output hashes the log records.

The crosscheck is run twice: once by Fadhl, once by the advisor's student in an adjacent group who has never touched MetabolonR-LLM. Two independent successes are the bar.

A separate Friday-morning check confirms that the public dataset accession works as the Availability section claims: a third party with no prior access to the lab's NFS storage downloads Progredir from Metabolomics Workbench under accession ST000978, points the backend at the downloaded files, and replays the Chapter 6 worked example. This is the strict "anyone in the world can reproduce this" check that the Availability section's reference to ST000978 stakes its claim on.

If either crosscheck fails, the failure is the most important thing to fix before submission. The week's manuscript work is paused if necessary.

17.7 The manuscript text itself

By Friday night the manuscript is at full draft. Each section is its first complete pass:

- Abstract: full 250 words, three paragraphs as outlined.
- Introduction: full 500 words. Compressed from Chapters 1 and 3 of the book; the three-paragraph structure mirrors the abstract — problem statement, prior-art

critique, the architectural commitment that the rest of the paper defends.

- **Methods:** full 800 words. The 800-word budget breaks down as: 200 words on the closed-tool-set architecture (the four design commitments of Chapter 9 §9.9); 200 words on the agent loop and session-log mechanics (Chapters 10 and 11); 200 words on the validation experiment design (Chapter 20 including the Path A / Path B split); 200 words on the dataset (ProgreDir per the Chapter 6 §6.7 canonical numbers, including the GC-MS platform, the 454×293 working matrix, the composite outcome, and the ST000978 accession).
- **Results:** 500 words with [PLACEHOLDER] cells in the result tables (to be replaced when the experiment is final). The narrative order is Path A first (TSA, PA, J at the cohort level; per-prompt heatmap in supplementary), then CR for the ambiguous cohort, then Path B (ER, EJ against Titan). The §16.6 go/no-go evaluation against pre-registered targets is the closing paragraph.
- **Discussion:** 300 words. Three paragraphs — the methodological claim (Chapter 8 §8.6); the comparison

with prior agentic-bio tools (the §4.5 framing); the regulatory and scope implications including the survival_analysis deferral to v2.1.

- References: roughly 30 entries, all verified against primary sources per the Chapter 4 verification report and additional Week 5 checks. The Titan 2019 citation is verified with the correct DOI (10.1371/journal.pone.0213764).
- Availability: complete, with DOIs and URLs. Software: GitHub URL plus Zenodo DOI. Data: Metabolomics Workbench accession ST000978 for Progredir, plus the Zenodo deposit of the 60 validation session logs. License: MIT for code, CC-BY-4.0 for the validation-data deposit.

The draft is committed to `manuscript/draft_v1.md` and circulated to the advisor over the weekend. The advisor's edits arrive Sunday night; the Monday-of-Week-6 work begins with incorporating them.

17.8 Pitfalls

Docker image bloat. The R-side install is heavy; a first-cut Docker image for the backend was 4.2 GB. Multi-stage builds and aggressive `apt-get clean` get it to 1.8 GB.

Zenodo–GitHub linkage. The Zenodo webhook fires only on tagged releases, not on every push. A pre-Wednesday rehearsal on a throwaway repository confirms the linkage works.

CITATION.cff field validation. A common mistake is to include a `version` field that does not match the GitHub release tag. The CI validates the CFF on every push; do not bypass.

Manuscript figure resolution. Journal submission systems often reject figures below 300 DPI for print. `manuscript/figures/` outputs at 600 DPI; rendering takes longer but figures are submission-ready without rework.

ST000978 access lag. The Metabolomics Workbench occasionally has propagation lag after a deposit is finalized. The

Friday third-party download check is what surfaces this; if the accession is not yet publicly resolvable, the submission Monday holds until it is.

Advisor weekend bandwidth. The advisor reading the Friday-evening draft over the weekend is a feature, not a bug, but it is the single biggest risk to Week 6 starting on time. The Friday-evening delivery is non-negotiable.

17.9 Deliverable checklist

By Friday night:

- `manuscript/draft_v1.md` (full draft, [PLACEHOLDER] numbers acknowledged).
- `manuscript/figures/` (3 figures + graphical abstract, all make-reproducible).
- GitHub release `v2.0.0`, Docker images tagged, Zenodo DOI minted.
- `README.md`, `CITATION.cff` (validated), `LICENSE`, `CONTRIBUTING.md`, `ISSUE_TEMPLATES`, `Dockerfile`.
- CI green on a fresh clone.

- Reproducibility crosscheck succeeded twice independently, plus the ST000978-based third-party download check.
- Advisor has the draft for weekend review.

[Figure 17.1: Manuscript section flow diagram. Boxes for each section, arrows for citation chains and forward references. The Methods box is the largest.]

[Figure 17.2: Graphical abstract draft. A single panel: user → agent → tool layer → R routines → log → replay → outputs, with the six

Chapter 18 — Week 6: Preprint and Submission

Week 6 makes the work public and submits it. The bioRxiv preprint posts Monday to mitigate scoop risk. The journal submission follows on Thursday. By Friday the work is in the world.

18.1 Monday — the preprint

Monday morning the draft, with the advisor's weekend edits incorporated, becomes the preprint. The preprint is the same Markdown manuscript rendered to PDF with the bioRxiv-appropriate template.

The preprint is posted to bioRxiv before noon Monday Eastern time. Posting takes about 90 minutes including the metadata-entry forms and the screening queue. The DOI is captured and the post-submission record (`manuscript/PREPRINT_DOI.md`) is committed.

The reason for the Monday preprint, before journal submission, is **scoop mitigation**. The agentic-bioinformatics space is competitive in 2026; a preprint posted before submission establishes priority. The Monday timing also gives the social-media post (Tuesday) and the journal submission (Thursday) something to point at.

The repository README is updated Monday afternoon to point at the bioRxiv DOI. The CITATION.cff is updated likewise. CI runs and confirms the changes do not break anything.

A short Twitter/X / Mastodon thread is drafted Monday afternoon, scheduled for Tuesday morning. The thread is five posts, no more. First post: the one-sentence what-it-is, the URL, the graphical abstract. Posts two through four: one key claim each (the architecture, the reproducibility property, the validation result). Fifth post: the team, the lab, the funding acknowledgment.

18.2 Tuesday — target journal selection

Tuesday is the structured journal-selection exercise. Three candidates were named in the architecture lock back in Week 1, and Tuesday is when the selection is final.

Primary target: *Briefings in Bioinformatics*. Open-access option available, Application Note / Software format that fits a 2,500-word ceiling, turnaround typically 6–8 weeks to first decision. Editorial board has at least three members with agentic-LLM expertise — a positive signal that the methodological framing will be understood. Impact factor in the high single digits as of 2025.

Backup target: *GigaScience*. Pure software focus, longer format permitted, strong tradition of accepting reproducibility-oriented submissions. Slightly slower turnaround (~10 weeks). The right fallback if *Briefings* rejects the framing as not biomedical enough or as too systems-oriented.

Secondary target: *Bioinformatics* Application Notes section. Strictest word limit (~1,000 words), highest visibility, hardest to

land. The right venue if the validation numbers in Chapter 21 turn out exceptionally strong; the right venue to skip if they are merely good.

The Tuesday afternoon decision: submit to Briefings first, with a fallback plan for GigaScience if Briefings declines and *Bioinformatics* only if the validation numbers are dramatically better than expected.

The cover letter is drafted Tuesday afternoon. One page. Three paragraphs:

- Paragraph 1: What the paper is, in two sentences, including the methodological claim.
- Paragraph 2: Why the journal. One sentence on fit. One sentence on prior related publications in the same venue (e.g., BioGPT in Briefings).
- Paragraph 3: Suggested reviewers (three), a sentence each. Conflicts (none for this paper). Funding (Kretzler-lab NIH grants, NEPTUNE; specific grant numbers TBD).

The cover letter is committed to `manuscript/COVER_LETTER.md`.

18.3 Wednesday — reviewer suggestions and rebuttal-preempt

Wednesday morning the suggested-reviewer list is finalized. Three names go in the submission system; each is chosen for a different reason.

Reviewer 1: a bioinformatician active in the metabolomics-tooling space. Someone the lab has not co-authored with in the last five years, who will read the Methods carefully and ask about MetaboAnalyst-comparable design choices.

Reviewer 2: a clinical nephrologist with metabolomics experience. Someone who will evaluate whether the system addresses a real clinical-translation need. The nephrologist from the Week 3 demo is potentially in this role but is too close (close enough to be flagged as a conflict); a non-Michigan equivalent is named.

Reviewer 3: an LLM-systems person who has published on agentic systems, reproducibility, or tool-use evaluation. This

reviewer is the technical evaluator of the architecture claim. Someone who has reviewed for *Nature Machine Intelligence* or for ICLR/NeurIPS in the past two years.

Three names go in the system with a one-sentence justification each.

Wednesday afternoon is the **rebuttal-preempt** exercise. This is the most useful thing the team does in Week 6. A private document, `manuscript/REBUTTAL_PREEMPT.md`, lists the ten most likely reviewer objections and a one-paragraph response to each. The document is not submitted; it is what the team consults when the reviews arrive.

Sample anticipated objections:

1. "*The 20-prompt validation is too small.*" Response:
Acknowledge; pre-registered design; results show low variance even at this sample size; happy to extend in revision.
2. "*How does this compare to BioChatter / Biomni?*" Response:
Different scope and reproducibility commitment; survey

in Chapter 4 of the book; brief comparison sentence in Discussion.

3. *"What about hallucination in tool-call parameters?"*

Response: Validated as schema violations; <1% rate in our 60 runs; surfaced as errors, not silent.

4. *"Why Sonnet and not Opus?"* Response: Empirical agreement-rate evidence; 3× cost differential; user can override per session.

The full list runs to ten items. The point is not to win arguments before reviewers raise them; it is to remember the team's own best arguments when the reviews land in eight weeks.

18.4 Thursday — submission

Thursday is the submission day. The morning is the systems busywork: the journal's submission portal, the manuscript file, the cover letter file, the figures, the supplementary materials (`analysis_log.json` from the worked example, plus the 60 validation logs).

The author submits. The submission confirmation email arrives within an hour. The manuscript ID is captured to `manuscript/SUBMISSION_RECORD.md`. The submission timestamp is recorded.

The afternoon is a deliberately quiet one — no celebrating until the first decision is in — but the team has a small dinner.

18.5 Friday — post-submission audit

Friday is the audit day. The team reviews everything one last time:

- The preprint matches what was submitted to the journal modulo the cover letter.
- The repository state at the submission timestamp is tagged `submission-briefings-2026-...`.
- The Zenodo DOI resolves and points at the correct release.
- The `CITATION.cff` is current.
- The README links work.
- The Streamlit demo is up on the lab server.

- The Twitter/X thread has been posted (Tuesday) and is gathering reasonable engagement.
- A "what would we do differently" retro is written to `docs/SIX_WEEK_RETRO.md`.

The retro is the most valuable document of the week. The two or three things the team would do differently the second time (and there will be a second time; v2.1 is on the roadmap and v3 is in the longer-range plan) become the seeds of the next-project planning.

18.6 Pitfalls

Scope mismatches. Briefings has occasionally rejected as out-of-scope papers that the authors thought were in scope. The Monday-Tuesday journal-selection exercise is partly to surface this risk before submission. If the editor returns a desk-reject on scope grounds, the team has GigaScience on file and pivots within a week.

Missing data-availability statement. A submission without an explicit data-availability statement is often desk-rejected. The

Availability section of the manuscript and the cover letter both include the Zenodo DOI for the validation dataset.

ORCID linkage. Some journals require ORCID for the corresponding author; the student's ORCID was set up in Week 1. Some require it for all authors; the team's ORCID list is in `manuscript/AUTHORS.md` for copy-paste into the submission portal.

Preprint server timing. Posting bioRxiv on a Friday or weekend delays the screening queue. Monday-morning posting is the safest.

Social-media timing. Tuesday morning Eastern is the empirically best time to post the thread (data: the lab's prior posts). The Monday afternoon post would be wasted on a smaller weekday-evening audience.

18.7 What success looks like at the end of Week 6

By Friday night:

- bioRxiv preprint live with DOI.
- Journal submission acknowledged with manuscript ID.
- Repository at a tagged submission state.
- Twitter/X thread posted with reasonable engagement (10+ retweets, 50+ likes — adjust expectations to the lab's normal reach).
- A "what we would do differently" retro committed.
- An eight-week wait beginning until the first round of reviews.

The team can move on to the next project. The build is done.

[Figure 18.1: Journal-fit matrix. Three columns (Briefings, GigaScience, Bioinformatics) and four rows (turnaround, word ceiling, audience fit, prior agentic-LLM papers). Briefings's column is highlighted as the chosen target.]

[Figure 18.2: 6-week timeline. Six bars, one per week, with the Friday deliverable annotated for each. The Week 6 bar ends with "preprint + submission + retro."]

What you should be able to do after this chapter

- Post a bioRxiv preprint and capture the DOI for downstream citation.
- Choose between three plausible target journals on scope and fit, and document the decision.
- Write a one-page cover letter that foregrounds the methodological claim.
- Suggest three reviewers each chosen for a different evaluative reason.
- Pre-empt the ten most likely reviewer objections with one-paragraph responses.
- Run a Friday post-submission audit that catches missing artifacts before they become problems.

- Write a "what we would do differently" retro that seeds the next project.

Chapter 19 — The Reliability Problem in Agentic Biology

Reviewers in 2025–2026 are skeptical of agentic bioinformatics for good reason. This chapter is honest about why, and stakes out what MetabolonR-LLM does differently. Part V is the validation argument that follows from this framing.

19.1 The skepticism is justified

If a reviewer assigned to evaluate an agentic-bioinformatics manuscript in mid-2026 is skeptical, it is because the recent literature has earned the skepticism. Several high-profile agentic-tool papers from 2024 and 2025 made strong reproducibility claims in their abstracts and could not deliver them in their supplementary materials. *Nature's* editorial commentary in late 2025 on LLM tools in science noted explicitly that the reviewer pool had become "weary" of claims

that did not survive scrutiny [CITATION NEEDED: identify the specific Nature commentary piece].

The pattern of failure is recognizable. A paper describes an LLM agent that performs an analytical task. The reported numbers are impressive. A reader who tries to reproduce them runs the agent, sees different tool calls than the paper described, and gets numbers that do not match. The paper's authors respond that the discrepancy is "within expected variance for stochastic agents." The reader is left without a way to evaluate the claim, and the field's credibility takes a hit.

This is not a problem with LLMs as such. It is a problem with the discipline of writing about LLMs as if they were the deterministic software the field's reviewing norms were built for. The right response is to adapt the norms; the wrong response is to over-claim and then complain when the over-claim is detected.

19.2 The catalog of failure modes

Five failure modes recur in the 2024–2026 agentic-bioinformatics literature. Naming them is half the work of mitigating them.

Hallucinated function calls. The agent emits a tool call to a function that does not exist, with arguments that do not match the function's schema. This is the simplest failure mode and the easiest to detect — a schema validator catches it. It is also the failure mode that several published systems do not have validators for; the bad output is silently logged and the run continues.

Silent parameter changes. The agent calls the right function but with a different parameter than it should. The output looks reasonable; only careful inspection reveals that the FDR threshold was 0.10 in this run rather than the 0.05 the user asked for. This failure mode is the most damaging because it is the hardest to detect.

Irreproducible outputs across runs. Two runs of the same prompt produce different tool sequences, different parameters, or different numerical outputs. This is the failure mode that makes "the same analysis" mean two different things. It is detectable by repeated runs but is rarely reported in published papers.

Prompt-injection vulnerabilities. A malicious dataset file with a crafted column name or annotation can perturb the agent's behavior. The metabolomics-specific risk is modest (the agent does not write code, does not execute shell, does not touch the network beyond the Anthropic API), but the literature has documented attacks on agentic systems that do (Greshake et al. 2023 [CITATION NEEDED: verify]).

Confident confabulation on ambiguous prompts. The user's request is genuinely ambiguous; a competent analyst would ask a clarifying question. The agent picks one interpretation, runs the analysis, and reports the result with no flag. The user does not know that her question was ambiguous; the report is convincing.

The MetabolonR-LLM architecture addresses each of these directly. Hallucinated calls are caught at the Pydantic-schema validator. Silent parameter changes are logged in `analysis_log.json` and surfaced in the methods paragraph. Irreproducibility is the property the replay engine is designed to solve. Prompt injection is bounded by the absence of code execution and shell. Confident confabulation is addressed by the clarification-before-execution principle and is the explicit subject of half of the validation suite.

19.3 What reviewers now require

Three things have shifted in what reviewers ask of agentic-bioinformatics submissions over 2024–2026. The shifts are partly the product of editorial guidance, partly the product of reviewers being burned, and partly the product of the field maturing.

Replay. Reviewers ask for the ability to re-run the analysis the paper describes. They are not satisfied with "the code is on GitHub." They want a deterministic re-execution from the

paper's logged session. Some venues are now asking for the session log to be supplied as supplementary material; *Briefings in Bioinformatics* has not formalized this requirement but reviewers there have asked for it specifically.

Version pinning. Reviewers ask for the exact LLM snapshot, the exact prompt-template hash, the exact tool-layer version, the exact R and Python package versions, the exact BLAS implementation. They want a manifest that pins everything that could drift. The model-name string `claude-sonnet-4-6` is no longer a sufficient pin; the API endpoint date is also relevant.

Cost disclosure. This is the newest of the three. Reviewers — and increasingly journal editors — are asking for the API spend the paper's described analyses required. The disclosure is about transparency on cost-equity: a tool that requires \$50 in API calls per analysis is not deployable in low-resource settings. *Briefings* has not formalized this either; *GigaScience* has, in a 2025 update to its software-paper guidelines [CITATION NEEDED: verify exact text and date].

A fourth shift, not yet universal but trending: **failure analysis**. Reviewers ask "what did the agent get wrong?" and they want a real answer with frequencies and a taxonomy, not "we did not observe any failures." A paper that claims zero failures in 60 runs invites suspicion; a paper that reports a 6% rate of premature termination with a mitigation that brought it to 2% invites engagement.

19.4 An audit of three named tools

Three agentic-bio tools from Chapter 4's survey are audited here against the four-criterion bar: replay, version pinning, cost disclosure, failure analysis. The audit is brief and is meant to be fair; the tools' authors would each have valid responses to the gaps named.

BioMANIA (Dong et al. 2023). Replay: partial — the agent can be re-run but reproducibility across runs is not the central claim. Version pinning: partial — Python packages pinned in `requirements.txt`, LLM snapshot identifiable but not

centrally documented. Cost disclosure: not present in the bioRxiv preprint. Failure analysis: limited.

BioChatter (Lobentanzer et al. 2025). Replay: not a central feature; the framework supports it but it is not how the paper's results are presented. Version pinning: package-level pinning available. Cost disclosure: not present. Failure analysis: not present.

Biomni (Huang et al. 2025). Replay: not the central claim. Version pinning: present at the tool-catalog level. Cost disclosure: discussed in the preprint at high level. Failure analysis: present but informal.

None of these tools is bad; each makes a different design trade. The audit's point is that the reproducibility-bar in 2026 is not yet a universal standard, and MetabolonR-LLM is making a particular bet about what the standard *should* be.

[Figure 19.1: Reliability-bar audit grid. Three columns for the three audited tools plus a fourth for MetabolonR-LLM. Four rows for the four criteria. Cells coded as Present / Partial /

Absent. The MetabolonR-LLM column is Present across all four.]

19.5 Where MetabolonR-LLM sits

The audit's bottom row says where MetabolonR-LLM differs.

Replay: present. The `analysis_log.json` plus replay engine is the central architectural commitment (Chapter 11).

Version pinning: present, comprehensive. The log header captures eleven kinds of version information: LLM snapshot, prompt SHA-256, tool-layer commit, R version, R-package versions, Python version, Python-package versions, BLAS, thread count, OS image, Docker image digest.

Cost disclosure: present. Every session log records `api_spend_usd`. The 60-session validation cost is reported in Chapter 21.

Failure analysis: present. The four-class taxonomy of Chapter 21 (ambiguous parameter inference, premature termination,

clarification deferral, tool misordering) is the explicit failure analysis the manuscript reports.

The point of this chapter has been to make the case that meeting these four criteria is not a stylistic preference; it is what publishability looks like in 2026 for clinical agentic tools. The architecture of Parts III and IV is designed around the criteria. The validation of Part V demonstrates that the architecture delivers.

19.6 A note on the *Nature* commentary

The late-2025 *Nature* editorial commentary on LLM tools in science is worth engaging with briefly because it is the most cited piece of skepticism in the field. The commentary's central concern is not that LLMs are bad at science; it is that the publishing apparatus is bad at evaluating LLM-driven work. Reviewers do not have time to re-run analyses. Editors do not have shared norms for what to require. Authors are incentivized to claim more than they can defend.

The commentary's prescription is structural: ask for the artifacts, require the version pins, build replay into the supplementary-material standard. MetabolonR-LLM's architecture is in part an attempt to be the kind of paper the commentary asks for. The bet is that this kind of paper is the one that gets accepted in 2026–2028 and beyond.

A short representative quote from the commentary — kept under 15 words to honor copyright — captures the tone: *"replication is a precondition, not a courtesy."* [CITATION NEEDED: identify and verify the exact Nature piece and the precise wording before publication.]

What you should be able to do after this chapter

- Name the five recurring failure modes in 2024–2026 agentive-bioinformatics tools and explain how MetabolonR-LLM addresses each.

- Name the four criteria reviewers now require — replay, version pinning, cost disclosure, failure analysis — and describe what each looks like in practice.
- Audit a published agentic-bio tool against the four criteria and produce a Present/Partial/Absent verdict per criterion.
- Articulate why MetabolonR-LLM's architecture is structured around the four criteria.
- Explain the *Nature* commentary's structural prescription in one paragraph and place MetabolonR-LLM as a response to it.

Chapter 20 — Designing the Validation Experiment

A claim about reliability is only as good as the experiment that tests it. This chapter is the design of the 20-prompt validation suite, the three repeats, the four internal-agreement metrics, and the fifth external-validation metric anchored against the Titan 2019 reference set. It is one of the two long chapters of Part V; the actual results follow in Chapter 21.

20.1 The hypothesis the experiment tests

The validation experiment tests a single composite hypothesis stated as five sub-hypotheses, each operationalized by a measurable metric.

H1. For unambiguous metabolomics-analysis prompts, MetabolonR-LLM produces the same ordered tool-call sequence across three paraphrases at least 90% of the time.

H2. For unambiguous prompts where the tool-call sequence matches, the parameters passed to each tool match across paraphrases at least 85% of the time.

H3. For unambiguous prompts that produce a `differential_abundance` result, the set of significant metabolites at FDR 0.05 agrees across paraphrases with mean Jaccard index at least 0.85.

H4. For deliberately ambiguous prompts where a competent analyst would ask a clarifying question, MetabolonR-LLM asks a clarifying question (rather than confabulating an interpretation) at least 80% of the time.

H5. For Path B prompts (biomarker discovery against the ProgreDir composite outcome — mortality or incident renal replacement therapy), MetabolonR-LLM's biomarker outputs recover the Titan 2019 reference at two pre-registered tiers:

H5a (primary, matched-adjustment): mean external recall

against Titan's fully-adjusted 18-metabolite reference (Table 3 of Titan 2019) ≥ 0.70 , with within-prompt mean external Jaccard ≥ 0.75 . **H5b (secondary, published-headline):** mean external recall against Titan's batch-only 34-metabolite reference (Table 2 of Titan 2019) ≥ 0.50 , with within-prompt mean external Jaccard ≥ 0.55 . Path A versus Path B and the two-tier construction are explained in §20.4b.

H1, H2, and H3 are internal-reproducibility hypotheses. H4 is a clarification-behavior hypothesis. H5 is the external-validation hypothesis introduced when the experiment's design was updated to include Titan 2019 as an independent ground-truth reference; the rationale is in §20.4b. The 80% target on H4 is the most contentious of the five — a competing reasonable position is that the target should be 100%, on the grounds that any silent interpretation is a failure — and the manuscript will defend the 80% choice in the Discussion.

The five targets are pre-registered. They were committed to docs/VALIDATION_PROTOCOL.md in Week 1 (Chapter 13); H5 was added in an explicit protocol amendment when the Titan reference set's accessibility (Metabolomics Workbench

ST000978) was confirmed, and the amendment is recorded in `data/validation/PROTOCOL_DEVIATIONS.md`.

20.2 The 20-prompt design philosophy

The prompts are chosen to span the analytical workflow the system is designed for, not to game any one metric. Ten unambiguous prompts and ten ambiguous prompts, each with three paraphrases for a total of 60 distinct prompt strings.

The **unambiguous cohort** tests routing consistency under linguistic variation. A clinician describing the same intended analysis in three different ways should produce the same tool sequence. The cohort's prompts cover:

1. The full standard pipeline (load through volcano).
2. A QC-and-summarize-only request.
3. A PCA-only request (no differential abundance).
4. A specific imputation method named explicitly.
5. A specific transformation named explicitly.
6. A specific scaling method named explicitly.

7. A specific test (limma vs. t-test vs. Wilcoxon) named explicitly.
8. An FDR threshold named explicitly.
9. A fold-change threshold named explicitly.
10. A sources-of-variation request alongside the standard pipeline.

For each, three paraphrases are written. The paraphrases vary lexically, syntactically, and in level of formality, but they refer to the same intended analysis. The three paraphrases for prompt 1, for example:

- "Run a standard MetabolonR pipeline on Progredir and produce a volcano plot."
- "Do the usual analysis on the Progredir data — QC, impute, transform, scale, PCA, differential abundance — and give me the volcano."
- "I want to look for biomarkers in Progredir using the standard workflow. End with a volcano of progressors vs stable."

Producing three legitimately equivalent paraphrases is harder than it sounds. The discipline that keeps this from being arbitrary is a **paraphrase rubric**: the paraphrases must (i) name the same dataset, (ii) name the same target analysis, (iii) imply the same parameter choices (defaults if unspecified, specific values if specified), and (iv) have no overlap of more than 50% of content words with another paraphrase of the same prompt.

The rubric was applied to every paraphrase before inclusion. Three paraphrases that initially failed the 50%-overlap test were rewritten on Tuesday afternoon of Week 4.

The **ambiguous cohort** tests clarification behavior. The cohort's prompts cover:

1. Underspecified target ("Tell me which metabolites are interesting").
2. Underspecified contrast ("Compare the groups in ProgreDir").
3. Underspecified parameter ("Do a PCA with the right number of components").

4. Multiple-interpretation prompt ("Look at the eGFR signal").
5. Implicit covariate ("Adjust for the obvious confounders").
6. Conflicting requests ("Use a high stringency cutoff but include the borderline cases").
7. Vague threshold ("Show me the strong effects").
8. Non-existent variable ("Group by smoking status" — Progredir does not have this).
9. Ambiguous reference group ("Compare the high-risk and low-risk groups" — when neither category exists explicitly).
10. Incomplete prompt ("Run the pipeline and").

Each ambiguous prompt has three paraphrases by the same rubric.

The expected behavior is recorded for every prompt. For unambiguous prompts the expectation is a specific tool sequence with specific parameters. For ambiguous prompts the expectation is a clarification question that addresses the specific ambiguity. The expectations were committed alongside the prompts, before any of the 60 runs were executed.

20.3 The paraphrase rubric in detail

The four-criterion rubric described above is the visible scoring; an invisible discipline ran underneath it during prompt design. Three principles drove the writing.

Stay in the user's vocabulary. The paraphrases use language a metabolomics-naïve clinician might plausibly use. Prompts that lean heavily on internal jargon like "PCA scores plot colored by progressor status" are present, but at most one paraphrase per prompt is at that register.

Vary lexicon, not intent. Words that change meaning if swapped — *significant* vs. *salient*, *PCA* vs. *PCoA* — are constrained. Words that do not — *run* vs. *do* vs. *execute*, *find* vs. *show me* vs. *give me* — are varied freely.

Preserve implicit defaults. A paraphrase that says "log transform with default pseudocount" is allowed; a paraphrase that omits the word *default* but otherwise carries the same expectation is also allowed because both should resolve to the system's default pseudocount of 1. A paraphrase that says "log

transform with pseudocount 0.5" is a *different prompt* with a different expectation and is not a paraphrase of the original.

The third principle is where paraphrase design most often goes wrong; the rubric guards against it by requiring an explicit expectation per prompt and checking each paraphrase's compatibility against the expectation.

20.4 The three agreement metrics, formally defined

The metrics from Chapter 16 are restated here with formal definitions.

Tool-sequence agreement (TSA). For a prompt p with paraphrases $\{q_1, q_2, q_3\}$, let $S_{\{p,i\}}$ be the ordered sequence of tool names emitted by the agent on paraphrase q_i . Define $\mathrm{TSA}(p) = 1$ if $S_{\{p,1\}} = S_{\{p,2\}} = S_{\{p,3\}}$ and 0 otherwise. The reported metric over the unambiguous cohort is $\frac{1}{10} \sum_p \mathrm{TSA}(p)$.

A "soft" variant is also reported: the longest common subsequence (LCS) of $S_{\{p,1\}}$, $S_{\{p,2\}}$, $S_{\{p,3\}}$ divided by the median sequence length. The soft variant lets a single inserted `summarize_dataset` call (a common agent habit) not break the score; the hard variant penalizes it.

Parameter agreement (PA). For each tool name t that appears at the same position in all three of $S_{\{p,1\}}$, $S_{\{p,2\}}$, $S_{\{p,3\}}$, let $A_{\{p,i,t\}}$ be the dictionary of arguments passed in q_i . Define $\mathrm{PA}(p,t) = |\{k: A_{\{p,1,t\}}[k] = A_{\{p,2,t\}}[k] = A_{\{p,3,t\}}[k]\}| / |A_{\{p,1,t\}}|$. The reported metric is the median of $\mathrm{PA}(p,t)$ over all (p,t) pairs.

The PA metric treats numerical and categorical arguments uniformly. A `knn_k` of 10 vs. 10 is a match; a `knn_k` of 10 vs. 11 is a mismatch.

Jaccard on biomarker sets (J). For each unambiguous prompt p that triggers a `differential_abundance` tool call, let $B_{\{p,i\}}$ be the set of metabolite IDs called significant at FDR 0.05 on paraphrase q_i . Define $J(p) = \frac{1}{3} \sum_{i \neq j} \frac{|B_{\{p,i\}} \cap B_{\{p,j\}}|}{|B_{\{p,i\}} \cup B_{\{p,j\}}|}$ over the three pairwise comparisons. The

reported metric is the mean of $J(p)$ over prompts that triggered DA. $\frac{1}{|B_{\{p,j\}}|} \sum_{B_{\{p,i\}} \in B_{\{p,j\}}} J(p)$

Clarification rate (CR). For each ambiguous prompt p and repeat i , let $C_{\{p,i\}} = 1$ if the agent's first assistant turn asks a question that addresses the ambiguity, and 0 otherwise. The reported metric is $\frac{1}{|B_{\{p,i\}}|} \sum_{C_{\{p,i\}} \in B_{\{p,i\}}} C_{\{p,i\}}$. Scoring is by a single rater (the author) with a written rubric and the blinding protocol described in §20.9.

20.4b External validation against the Titan reference set

The four metrics defined in §20.4 measure *internal* reproducibility: whether the agent agrees with itself across paraphrases of the same prompt. A different question is whether the agent's biomarker outputs agree with an *external* reference — a published shortlist from the same cohort, derived by independent analysts using a different statistical approach. The Titan et al. 2019 paper provides this reference for the

ProgreDir cohort, and the validation experiment uses it as a fifth, pre-registered metric.

Path A and Path B

The experimental design therefore has two analytical paths, which the protocol distinguishes explicitly.

Path A — internal reproducibility. Prompts that ask the agent for differential abundance between progressors and stable participants. The expected output is a roughly 18-metabolite limma shortlist matching MetabolonR v1's run on the same data. This is what H1–H4 measure.

Path B — external validity. Prompts that ask the agent for biomarker discovery against the composite outcome (mortality or incident renal replacement therapy). The expected output is a biomarker set whose overlap with Titan's 34-metabolite Cox-significant reference is computed. The 34 reference biomarkers were derived by Cox proportional-hazards models on time-to-event data; v2.0's tool layer does not include Cox or survival analysis (deferred to v2.1 as `survival_analysis`, Chapter

24), so the agent's binary-contrast output cannot, by design, recover all 34. The test is whether the agent reliably recovers the subset detectable by limma-on-binary.

This is a strict external benchmark, not a soft one. Partial recovery, measured consistently across paraphrasings, is the evidence of methodological reproducibility we are after; full recovery is not the expectation and is not the target.

The two external metrics, scored against two reference tiers

Path B is a two-tier external benchmark. The primary tier scores against Titan's **fully-adjusted 18-metabolite set** (Table 3 of Titan 2019 — Cox with batch + sex + age + eGFR + diabetes adjustment). The secondary tier scores against Titan's **batch-only-adjusted 34-metabolite set** (Table 2 — the published headline). The two-tier construction is motivated in §20.4b's framing paragraphs; see also Chapter 6 §6.5 for the methodological rationale (v2.0's `differential_abundance` adjusts for covariates, which

makes the fully-adjusted 18 the apples-to-apples comparison; the 34 is the published headline most readers cite).

External recall, primary tier (ER_{18}). For each Path B paraphrase q , let B_q be the agent's biomarker output and T_{18} be Titan's fully-adjusted 18-metabolite reference. Define $\mathrm{ER}_{18}(q) = |B_q \cap T|$. The reported metric is the mean of ER_{18} across Path B paraphrases. This is the primary external-validation number. $|T_{18}$

External recall, secondary tier (ER_{34}). Same formula with T_{34} , the batch-only Titan 34-set, in place of T_{18} . Reported alongside ER_{18} for the field-compatibility reason above.

External Jaccard (EJ). For each tier, $\mathrm{EJ}(q) = |B_q \cap T| / |B_q \cup T|$. EJ penalizes both false negatives and false positives. ER is the more forgiving metric; EJ is reported alongside for completeness.

Both metrics use HMDB metabolite identifiers as the canonical lookup key, because the Titan reference and the agent's output may use different display names for the same molecule. The

agent's outputs are mapped to HMDB IDs through the metabolite annotation file at scoring time; metabolites the agent emits that have no HMDB ID are excluded from EJ's denominator but counted as misses in ER. The mapping function lives in `scripts/score_external_validation.py`.

H5: the pre-registered external-validation hypothesis (two tiers)

H5 has been refined to a two-tier form. H5a is the primary tier; H5b is the secondary, field-compatibility tier.

H5a (primary, matched-adjustment). Mean $ER_{18} \geq 0.70$ across Path B paraphrases against Titan's fully-adjusted 18-metabolite reference, with within-prompt mean EJ_{18} across paraphrases ≥ 0.75 .

H5b (secondary, published-headline). Mean $ER_{34} \geq 0.50$ across Path B paraphrases against Titan's batch-only 34-metabolite reference, with within-prompt mean EJ_{34} across paraphrases ≥ 0.55 .

The 70% recall target on the primary tier (H5a) is calibrated to what limma-on-binary with covariate adjustment can recover from Cox-with-the-same-covariate-adjustment when both are run on the same cohort. The cardinality convergence (v1's 18 $\approx$ Titan's fully-adjusted 18, Chapter 6 §6.5) supports the 70% calibration: methods that converge on the same shortlist size on the same cohort should agree on at least 70% of the membership.

The 50% recall target on the secondary tier (H5b) is calibrated to the methodological mismatch — limma-on-binary cannot recover Cox-time-to-event-only members of the 34. A reviewer who is uncomfortable with the 50% calibration is asked to consult the Titan supplementary materials: many of the 34 Titan biomarkers are driven by signal that accumulates with follow-up time rather than dichotomizing cleanly at the three-year endpoint.

The 0.75 within-prompt EJ-consistency target tests whether the agent's *external* recovery is stable across paraphrasings. A system that recovers 17 of the Titan 34 on one paraphrase and 15 of the Titan 34 on the next has high agreement; one that

recovers 17 in one and 8 in another is internally inconsistent regardless of how it does in absolute recall.

Why this framing matters

The 18-versus-34 discrepancy between v1's limma output and Titan's Cox reference is not a contradiction to resolve. It is the most informative feature of the experimental design. Both reference sets are legitimate; they answer different statistical questions on the same cohort. The validation experiment uses both — v1 for internal-reproducibility scoring (Path A), Titan for external-validity scoring (Path B) — because together they pin two different reproducibility commitments. Path A asserts the tool layer is reproducible against the v1 implementation it inherits from. Path B asserts the system's downstream biomarker outputs are non-spurious against an independently derived reference. A clinical-grade tool needs both, and the experiment scores both.

20.5 A worked example of agreement scoring

The metric definitions of §20.4 become more concrete when applied to specific session transcripts. The worked example below uses two realistic sessions for unambiguous prompt 5 — "explicit log10 transformation" — one repeat each from two paraphrases. A third repeat is included to complete the three-paraphrase set. Chapter 21's results refer back to this example.

Paraphrase 5a session. *Prompt:* "Run a standard analysis of ProgreDir but log10-transform the abundances first."

Agent tool sequence: load_metabolomics_data →
summarize_dataset → qc_filter →
impute_missing → transform → scale → pca →
sources_of_variation →
differential_abundance → export_session.

Parameters on transform: method="log10",
pseudocount=1.0. Parameters on

differential_abundance:
group_column="progressor",
reference_group="stable", test="limma",
fdr_method="BH", fc_threshold=0.5,
fdr_threshold=0.05. Returns 17 significant metabolites
(named from Titan 2019 Table 2 to keep the worked example
reviewer-checkable against the published paper): Lactose, D-
threitol, Pseudouridine, Butanoic acid, D-mannitol, Trans-
aconitic acid, Acetohydroxamic acid, Galactonic acid, Myo-
inositol, L-threonine, 2-O-Glycerol- α -D-galactopyranoside,
Galacturonic acid, L-glutamine, Xylitol, Gluconic acid, 5-
hydroxyindol, Ribose.

Paraphrase 5b session. *Prompt:* "Use log10 transformation (not log2) for the Progredir analysis."

Agent tool sequence: identical to 5a. Parameters on
transform: method="log10", pseudocount=1.0.
Parameters on differential_abundance: identical to
5a. Returns 18 significant metabolites: the 17 from 5a, plus p-
Cresol glucuronide (Titan Table 2 rank 19; q = 0.0042 in the
published Cox analysis, borderline in our limma analysis). The

single extra hit reflects a borderline metabolite whose adjusted p-value falls just below 0.05 in one run and just above it in another by a few thousandths.

Paraphrase 5c session. *Prompt:* "I want the standard pipeline with log base 10 transformation."

Agent tool sequence: load_metabolomics_data → summarize_dataset → qc_filter → impute_missing → transform → scale → pca → sources_of_variation → differential_abundance → export_session.

Identical. Parameters on transform: method="log10", pseudocount=1.0. Parameters on

differential_abundance:

group_column="progressor",
reference_group="stable", test="limma",
fdr_method="BH", fc_threshold=0.5,
fdr_threshold=0.05. Returns 17 significant metabolites:
same 17 as 5a (Lactose, D-threitol, Pseudouridine, Butanoic acid, D-mannitol, Trans-aconic acid, Acetohydroxamic acid,

Galactonic acid, Myo-inositol, L-threonine, 2-O-Glycerol- α -D-galactopyranoside, Galacturonic acid, L-glutamine, Xylitol, Gluconic acid, 5-hydroxyindol, Ribose).

TSA computation. All three sequences are identical. $S_{\{5,1\}} = S_{\{5,2\}} = S_{\{5,3\}}$. Therefore $\mathrm{TSA}(5) = 1$. The soft TSA variant is also 1.0 (LCS = full length, median length = full length).

PA computation. Tool by tool, the parameter dictionaries are compared across the three paraphrases. For `transform`: `{method, pseudocount}` — both keys match in all three. $PA(5, \text{transform}) = 2/2 = 1.0$. For `differential_abundance`: six keys (`group_column`, `reference_group`, `test`, `fdr_method`, `fc_threshold`, `fdr_threshold`), all match in all three. $PA(5, \text{differential_abundance}) = 6/6 = 1.0$. Repeating across the other eight tool positions (all of which have default-only arguments that match identically), the median PA across (prompt, tool) pairs for this prompt is 1.0.

Jaccard computation. Let $B_{\{5,1\}}$, $B_{\{5,2\}}$, $B_{\{5,3\}}$ be the significant-metabolite sets from the three paraphrases. $|B_{\{5,1\}}| = 17$, $|B_{\{5,2\}}| = 18$, $|B_{\{5,3\}}| = 17$. Pairwise intersections: $|B_{\{5,1\}} \cap B_{\{5,2\}}| = 17$ (5a is a subset of 5b), $|B_{\{5,1\}} \cap B_{\{5,3\}}| = 17$ (identical sets), $|B_{\{5,2\}} \cap B_{\{5,3\}}| = 17$ (the 17-element subset). Pairwise unions: $|B_{\{5,1\}} \cup B_{\{5,2\}}| = 18$, $|B_{\{5,1\}} \cup B_{\{5,3\}}| = 17$, $|B_{\{5,2\}} \cup B_{\{5,3\}}| = 18$. Pairwise Jaccard: $17/18$, $17/17$, $17/18 = 0.944$, 1.000 , 0.944 . Mean: $J(5) = 0.963$.

The worked example shows what a "successful" prompt looks like across all three metrics: identical tool sequences, identical parameters, near-perfect Jaccard with a single borderline-metabolite difference. Most prompts in the validation experiment are expected to look like this one. The interesting prompts will be the ones that do not — Chapter 21 will return to this example as the reference against which deviations are measured.

For contrast, consider what a *failure* on prompt 5 would look like. If paraphrase 5b's agent had chosen $\log_2$ rather than $\log_{10}$ (the failure mode documented in Chapter 21 §21.5 as

Mode A — ambiguous parameter inference), the tool sequence would still match ($TSA = 1$), but the parameter agreement on `transform` would drop from 1.0 to 0.5 (one of two keys mismatched), and the significant-metabolite set could shift substantially: the log-base difference changes the variance structure post-scaling and can change which borderline metabolites cross the FDR threshold. A plausible Jaccard for the failure variant is in the 0.7–0.85 range — still high in absolute terms because most significant hits are robust to the log-base choice, but visibly degraded relative to the success case.

20.6 The three-repeat design and its statistical power

The choice of three repeats per prompt is a trade-off between statistical power and API cost. This section makes the power argument formally so that a reviewer can evaluate what the experiment can and cannot conclude.

The model. Let π denote the per-paraphrase probability that the agent emits the canonical tool sequence (the sequence the

protocol's expected-behavior document specifies for that prompt). Assume π is the same across paraphrases of the same prompt — the paraphrase rubric of §20.3 is designed to enforce this — and that different paraphrases' outcomes are conditionally independent given the prompt's expected behavior. Under this model, the probability that all three paraphrases of a prompt produce the canonical sequence (and therefore the binary TSA equals 1) is π^3 , ignoring small probabilities of three identical *non*-canonical sequences.

The reported TSA across the 10-prompt unambiguous cohort is the proportion of prompts with binary TSA = 1. Under the model, this is a sample from a Binomial(10, π^3) distribution divided by 10. The pre-registered target is TSA $\geq$ 0.90, meaning at least 9 of 10 prompts.

What N = 3 buys. With three paraphrases, the per-prompt TSA = 1 event requires three independent successes. This is a stringent threshold: at $\pi = 0.95$, the per-prompt success probability is $0.95^3 \approx 0.857$. The expected number of prompts with TSA = 1 across the cohort is therefore 8.57 of 10. Observing 9 or 10 of 10 is consistent with π in the range

[0.93, 1.0]; observing 8 of 10 is consistent with [0.88, 0.99]; observing 7 of 10 is consistent with [0.82, 0.97]. A Wilson 95% confidence interval at observed 9/10 is approximately [0.60, 0.99]; at 10/10 it is [0.72, 1.00]. The intervals are wide because $n = 10$ prompts is small.

Minimum detectable effect. Against the pre-registered null hypothesis that observed TSA ≤ 0.75 (a system unsuitable for clinical use), a one-sided binomial test at $\alpha = 0.05$ rejects the null when the observed count is ≥ 9 . The power to reject at observed-TSA targets is approximately: 0.92 when true $\pi = 0.97$; 0.66 when true $\pi = 0.90$; 0.29 when true $\pi = 0.85$. The experiment is well powered to distinguish a high-quality system ($\pi \geq 0.95$) from an unacceptable one ($\pi \leq 0.75$). It is less well powered to distinguish *between* two high-quality systems — a 5-point difference at the high end ($\pi = 0.95$ vs $\pi = 0.90$) requires roughly 25 prompts at three repeats each to detect at 80% power.

What this experiment fails to detect. Three classes of conclusion are out of reach.

First, the experiment cannot distinguish $\pi = 0.95$ from $\pi = 0.97$ with this design. A change-detection study (comparing a system before and after a system-prompt revision) would require either more prompts or more repeats per prompt.

Second, the experiment cannot estimate the *between-prompt* heterogeneity of π . The model assumes a single π ; if π in fact varies by prompt — some prompts inherently harder than others — the cohort-average estimate is still meaningful, but the per-prompt confidence intervals are wider than the binomial calculation suggests. The per-prompt heatmap of Chapter 21 §21.3 is the visualization that surfaces this; the formal estimation is deferred to v2.1.

Third, the experiment cannot detect violations of the conditional-independence assumption across paraphrases. If two paraphrases of the same prompt cue the agent into the same non-canonical interpretation (because both happen to use a word the agent over-weights), the empirical TSA would be inflated relative to the model's prediction. The paraphrase rubric's "no overlap of more than 50% of content words"

criterion (§20.3) is the discipline that limits this risk; it does not eliminate it.

The soft TSA variant. The longest-common-subsequence variant is more powerful because it has more granularity than the binary metric. A prompt where two of three paraphrases match exactly and the third inserts one extra `summarize_dataset` call scores 1.0 on soft TSA and 0 on hard TSA. The soft variant's per-prompt distribution is more concentrated, and the cohort-mean estimator has roughly twice the effective sample size. The reported numbers will include both variants so that a reviewer can choose which to weight.

The contingency. If the Week 4 results come in close to but below a target — within 5 percentage points — the protocol allows an additional fourth repeat on the affected prompts. This is documented in `docs/VALIDATION_PROTOCOL.md` §4.3. The contingency adds at most 30 more sessions and at most \$2 of additional API cost. It does not constitute a free pass to extend the experiment indefinitely until the numbers look right; the rule is one and only one fourth repeat, on

prompts whose pre-mitigation score was within 5 points of the target.

Path B power: external-recall effect sizes, per tier

The power discussion above applies to Path A — internal-reproducibility metrics on the unambiguous cohort. Path B has different denominators and a different statistical question; its power deserves its own treatment, and because Path B is a two-tier benchmark (§20.4b), the power for each tier is different.

Primary tier (H5a, denominator 18). Path B's primary metric is mean ER_{18} across paraphrases of Path-B-eligible prompts. With approximately 7 prompts $\times$ 3 paraphrases = 21 Path B observations (a subset of the 30 unambiguous-cohort runs whose outputs are biomarker shortlists), per-observation ER_{18} is continuous on $[0, 1]$ but with effective resolution of $1/18 \approx 0.056$ per metabolite. Under between-paraphrase variance $\sigma \approx 0.10$, the minimum detectable difference in mean ER_{18} at $\alpha = 0.05$ with one-sided testing is approximately 0.11 — about two metabolites in absolute terms. The experiment is well powered to distinguish a system that recovers 14–15 of Titan's fully-

adjusted 18 ($ER_{18} \approx 0.78$) from one that recovers 10–11 ($ER_{18} \approx 0.58$); it is not powered to distinguish 14 from 15. The pre-registered H5a target of $ER_{18} \geq 0.70$ sits in the well-powered range, calibrated to the methodological convergence argument from Chapter 6 §6.5.

Secondary tier (H5b, denominator 34). Mean ER_{34} has effective resolution $1/34 \approx 0.029$ per metabolite — finer granularity than ER_{18} . The minimum detectable difference in mean ER_{34} at $\alpha = 0.05$ is approximately 0.10 — about three or four metabolites in absolute terms. Concretely, the experiment is well powered to distinguish a system that recovers 16–17 of the Titan 34 ($ER_{34} \approx 0.50$) from one that recovers 11–12 ($ER_{34} \approx 0.35$). The pre-registered H5b target of $ER_{34} \geq 0.50$ sits at the upper end of what binary-contrast analysis can plausibly recover from a Cox time-to-event reference; missing the target by 5–10 percentage points would be informative about the agent's parameter consistency more than about the underlying methodological limit.

In both tiers, EJ has higher variance than ER (because false positives enter the denominator) and is correspondingly less

powerful for fine-grained comparisons; the experiment is positioned to detect EJ shifts of 0.15 or more on either tier.

20.7 Clarification as success, not failure

The single most counterintuitive design choice is that on the ambiguous cohort, a clarifying question counts as success. A naïve interpretation of "the agent did not produce the analysis" is a failure; this experiment treats it as the correct behavior.

The justification is the same as the *clarification-before-execution* principle (Chapter 8). Confident confabulation on an ambiguous input is more dangerous than asking. A user whose ambiguous prompt produced a clarification can refine her request; a user whose ambiguous prompt produced a confident wrong analysis can be misled.

This means the ambiguous-cohort scoring is binary per repeat: did the agent clarify, or not? The downstream behavior (whether, after clarification, the agent produces a useful

analysis) is not part of the validation metric — that is a UX concern, not a reliability one.

A subtlety: a clarifying question that is on the wrong axis of the ambiguity counts as a partial success. If the prompt was ambiguous about the FDR threshold and the agent asked about the imputation method, that is more useful than no clarification but less useful than asking about the FDR. The scoring rubric resolves this by scoring "did the agent identify the specific ambiguity" as a separate dimension; only the binary score enters the headline metric, but the dimensional score is reported alongside.

20.8 Pre-registration

Pre-registration is the discipline of committing the analysis plan before the data are collected (Munafò et al. 2017). It is standard in clinical trials and increasingly standard in machine-learning benchmarks. The MetabolonR-LLM validation protocol was committed to a Git tag, `protocol-v1.0`, in Week 1, before any of the 60 runs were executed.

The committed protocol document specifies: the four hypotheses H1–H4 with their numerical targets; the 20 prompts with their three paraphrases each; the expected behavior per prompt; the metric definitions (TSA, PA, J, CR) as in §20.4; the three-repeat design and the contingency for borderline numbers; the scoring rubric for the ambiguous cohort with the blinding protocol of §20.9; the model snapshot (`claude-sonnet-4-6`), the system prompt SHA-256, the tool-layer commit hash; and the dataset (Progredir), with its file hashes.

Pre-registration is not a guarantee of honesty — an author can always post-hoc explain a discrepancy between the protocol and the results — but it makes post-hoc rationalization easier to spot. A reviewer reading both the protocol and the manuscript can identify where the manuscript deviates and ask why.

The deviation policy: any departure from the protocol must be documented in `data/validation/PROTOCOL_DEVIATIONS.md`, with the reason, the date, and the author's initials. As of the manuscript draft, two minor deviations have been documented (the three timed-out reruns

from Wednesday of Week 4, and the addition of the soft-TSA variant alongside the hard variant). Neither affects the headline conclusions; both are reported.

The protocol explicitly partitions the unambiguous-cohort prompts into Path A (internal-reproducibility) and Path B (external-validity) sets. Path A applies to all ten prompts as the primary scoring path. Path B applies only to prompts whose agent output is a biomarker-list payload — practically, the prompts that exercise `differential_abundance` and `export_session`. A single prompt can be scored under both paths if its output supports both: the within-paraphrase Jaccard against the v1 oracle (Path A, H3) and the recall against Titan (Path B, H5) are computed independently from the same agent output.

20.9 Bias controls at depth

The naïve bias-control list — random prompt order, interleaved repeats, blinded scoring, no retroactive prompt revision — was the version in earlier drafts of this chapter. The grown-up

version, sketched here, accepts that bias control in a single-rater single-dataset experiment is a discipline with internal nuance, not a checkbox.

Randomized prompt order, with a recorded seed. The 60 prompt strings are not executed in any meaningful order. The harness shuffles the list with a fixed RNG seed (committed in the protocol, value 0x7e1f1ad8) before execution. This is not the same as a full Latin-square design — that would balance paraphrase position (1st, 2nd, 3rd of its prompt) across all 10 prompts in a way that requires 30 runs in a fixed structured order. The 60-session experiment uses what biostatisticians call a *partially balanced randomization*: the harness ensures that every paraphrase position appears roughly equally in every quarter of the run schedule, but does not enforce strict Latin-square balance.

The literature on order effects in human-rater contexts (Senn 2002 on clinical-trial sequencing; older psychometric work in the Cronbach 1951 lineage on item-position effects in educational testing) suggests that what matters most is randomization with a recorded seed, not the more elaborate

balance designs. Full Latin-square would matter if the experiment ran enough sessions for position-induced bias to dominate sampling noise, which at $n = 60$ it does not.

Interleaved repeats with timed spacing. The three paraphrases of a single prompt are separated by at least 30 minutes of other runs. This prevents two specific failure modes. The first is short-term API-side caching: if Anthropic's infrastructure has any per-request caching for nearby identical-prefix requests, three back-to-back paraphrases of the same prompt could inflate within-prompt agreement. The second is the more subtle agent-internal effect: a paraphrase that runs immediately after a different paraphrase of the same prompt may be processed against still-warm activation patterns at the model side. Neither effect is documented to exist for the Anthropic API, but both are easy to defend against by interleaving, so the harness does.

Blinded scoring for the ambiguous cohort. The clarification-rate scoring is done by one human (the author). Single-rater scoring has known biases: expectancy bias (the rater knows the expected outcome and reads it into ambiguous evidence), order

effects (responses scored late in a session are scored differently from those scored early), and learning effects (the rater's rubric drifts as she sees more responses).

The blinding protocol operationalizes the response. The scorer is presented with a CSV in which each row is the agent's first assistant turn, identified only by a random per-row hash. The original prompt text is *not* in the CSV; neither is the repeat index (1st, 2nd, or 3rd repeat of the prompt); neither is the timestamp. The scorer fills in a rubric checklist per row: (a) did the agent ask a question? (b) does the question address an ambiguity in the prompt? (c) is the question on the "right axis" — does it target the specific ambiguity the protocol named? The CSV rows are in a randomized order, generated from a separate seed than the run-execution seed.

Post-scoring, the analyst (a different person — the advisor) reconciles the random row IDs back to (prompt, repeat) pairs and computes the CR metric. The reconciliation is mechanical; the scoring is human; the two roles are separated to limit single-rater bias.

This is not the same as inter-rater reliability. A genuinely independent validation would require a second human scorer producing independent CR scores and a Cohen's-kappa computation across raters. Single-rater scoring is the right discipline for v2.0 given the team size; the v2.1 plan includes a second rater for the cross-dataset validation.

Time of day and rater fatigue. The 60 ambiguous-cohort scorings happen across two afternoons, not one. A single sitting risks fatigue-induced rubric drift. Two sittings, each of approximately 30 scoring events, with a documented break between them and the rubric re-read at the start of each, mitigates this. The scoring CSV is regenerated between sittings so that no row is scored in adjacent positions on both passes (a row scored late on day 1 will be scored mid-pack on day 2 if there is re-scoring; in v2.0 there is not).

No retroactive prompt revision. A failed run does not motivate a prompt rewrite. If a prompt produced unexpected agent behavior, that is data. The protocol commitment is binding.

The five controls above do not eliminate bias; they limit specific named sources of it. The remaining sources — the author's involvement in writing both the prompts and the system that responds to them, the choice of dataset and grouping variable, the choice of which behaviors count as "correct" in the unambiguous cohort — are honestly limitations of an internal validation. §20.11 names them.

20.10 The cost budget for the experiment

The full 60-session experiment, plus the borderline-recovery contingency of a fourth repeat on up to 5 prompts, was budgeted at \$25 of Anthropic API spend. The Flask budget proxy (Chapter 10 §10.8) was configured with a \$30 cap for the experiment specifically; the proxy logs the actual spend per run, which is reported in Chapter 21 alongside the agreement numbers. A separate \$10 budget covered the Tuesday dry run. The total experimental cost is therefore bounded at \$40. The

pre-registration commits the team to a budget as well as a design.

[Figure 20.1: Experimental design schematic. Boxes for the four hypotheses, the 20 prompts, the three repeats. Arrows showing the data flow from agent runs to metric computation.]

[Figure 20.2: Prompt $\times$ repeat $\times$ metric data structure. A 3D grid: prompt axis, repeat axis, metric axis. The cells contain the raw measurements; the aggregations along each axis are the reported numbers.]

[Figure 20.3: Worked-example agreement scoring on prompt 5 (the \$20.5 example). Three columns for the three paraphrases, with tool sequences aligned and parameter dictionaries shown side by side. The TSA, PA, and Jaccard computations are annotated underneath.]

20.11 The honest limitations of the design

The experiment has four honest limitations that are worth naming.

Single dataset. All 60 runs are on Progredir. A second dataset would test whether the agreement metrics generalize. The author intends to run a second-dataset experiment in v2.1 (Chapter 24); the v2.0 paper is honest about the single-dataset scope.

English only. All prompts are in English. The same agent on the same data with prompts in Portuguese, Mandarin, or Arabic might behave differently. v3 considers multi-lingual support; v2 does not.

Single rater. The clarification scoring is done by one human, with the blinding protocol of §20.9 limiting but not eliminating single-rater bias. Inter-rater reliability would require a second

rater. The author intends to involve a second rater for the v2.1 paper; the v2.0 paper is honest about the single-rater limitation.

No adversarial prompts. The experiment tests routine analytical questions, not red-team attempts to confuse the agent. A separate adversarial-evaluation study is on the v2.x roadmap.

Strict external-validation scope. v2.0's tool layer does not include survival or time-to-event analysis. The external-validation metric in §20.4b is therefore scoped to what limma-with-covariate-adjustment can recover from the Titan Cox reference. The primary-tier H5a target ($\geq 70\%$ recall against the fully-adjusted 18) is plausible because v2.0's `differential_abundance` does adjust for covariates and the two methods converge on cardinality (Chapter 6 §6.5's 18-vs-18 convergence). The secondary-tier H5b target ($\geq 50\%$ recall against the batch-only 34) is plausible but a stricter test, because v2.0 cannot recover Cox-time-to-event-only members of the 34 by design. A v2.1 with the planned `survival_analysis` tool (Chapter 24) would close both gaps and enable a Cox-vs-Cox external-validation experiment in

which the agent's own Cox output is scored against Titan's; the v2.1 paper's H5 target would be raised to $\geq 80\%$ on a single-tier basis.

These limitations are real and are listed in the manuscript's Discussion section. They do not invalidate the experiment; they bound its claims. A reader who knows the limitations can interpret the headline numbers correctly.

20.12 Operational changes for Chapter 16

The bias-control depth of §20.9 implies three operational changes that Chapter 16 (Week 4: Frontend and the Validation Experiment) must reflect. Sub-pass 2 of the enrichment will incorporate these into Ch 16 verbatim so that Qasim's Week 4 execution matches the protocol committed here.

First, the RNG seed for prompt ordering is fixed at 0x7e1f1ad8 and is documented in the protocol. Ch 16's

Wednesday plan (the 60-session run) must specify this seed in the harness call rather than leaving it implicit.

Second, the blinded scoring of the ambiguous cohort is a two-role workflow: the author scores against the rubric on randomized hash-IDed rows; a separate person (the advisor, or a designated lab member) handles the reconciliation back to (prompt, repeat) pairs. Ch 16's Thursday plan currently describes scoring as a single-person task; Sub-pass 2 will revise it to name the two roles and the CSV format.

Third, the ambiguous-cohort scoring happens across two afternoons with a documented break, not in one sitting. Ch 16's Thursday plan currently implies a single afternoon; Sub-pass 2 will revise the day plan to reflect two-session scoring.

These three changes are operational, not architectural — they do not alter what Ch 20 measures, only how Ch 16 executes the measurement.

Fourth, the Wednesday harness must score every Path B prompt's output against the Titan 34-metabolite reference for external recall (ER) and external Jaccard (EJ). The reference set

lives at `data/validation/titan_2019_reference.csv` — a single-column CSV of HMDB IDs derived from the Titan supplementary, accessible via Metabolomics Workbench accession ST000978. Ch 16's Thursday plan needs a step for this scoring; Sub-pass 2 of the book's enrichment incorporates it.

What you should be able to do after this chapter

- State the four hypotheses H1–H4 with their numerical targets and the metric that tests each.
- Distinguish unambiguous and ambiguous validation prompts and explain why each cohort is in the experiment.
- Write a paraphrase that meets the four-criterion rubric.
- Define tool-sequence agreement, parameter agreement, Jaccard on biomarker sets, and clarification rate formally.
- Compute the three reproducibility metrics on a worked example, by hand, in under 10 minutes.

- Defend the choice of three repeats against five and against one, using a formal power argument that states the minimum detectable effect.
- Articulate what the experiment will *fail* to detect, and why.
- Argue that clarification is success on the ambiguous cohort.
- Operationalize blinded single-rater scoring with the two-role workflow that limits expectancy bias.
- Distinguish Path A (internal reproducibility against the v1 oracle) from Path B (external validity against the Titan 2019 reference), and compute external recall and external Jaccard for a Path B output.
- Explain why the 18-vs-34 reference-set discrepancy is the experimental design's most informative feature rather than a contradiction to flatten.
- Recite the five honest limitations of the design.

Chapter 21 — Results and Failure- Mode Taxonomy

This chapter reports the numbers from the Week 4 experiment and the failure-mode taxonomy that fell out of the post-hoc analysis. Where numbers are not yet final, they are flagged [PLACEHOLDER] for the post-build replacement pass scheduled after the Week 4 experiment completes.

21.1 Reading this chapter before the placeholders are filled

This chapter is written to the structure the final manuscript figures and tables will take. Every number that depends on the Week 4 experiment is rendered as [PLACEHOLDER: ...] with a brief description of what the number will be. The intent is that after the experiment completes and the metric scripts

run, a single editing pass replaces the placeholders without restructuring the prose.

The placeholders are not estimates. The author has internal expectations of the numbers from the dry runs in Week 3 and Week 4 Tuesday, but those expectations are not committed here. A reviewer reading this chapter alongside the protocol document (Chapter 20) should be able to see whether the actual numbers are in the range the experiment was designed to defend.

21.2 Headline numbers

H1 — Tool-sequence agreement on the unambiguous cohort. [PLACEHOLDER: %, 95% CI] against a pre-registered target of $\geq 90\%$.

H2 — Parameter agreement on the unambiguous cohort. [PLACEHOLDER: %, 95% CI] against a pre-registered target of $\geq 85\%$.

H3 — Jaccard on biomarker sets ($\text{FDR} \leq 0.05$).

[PLACEHOLDER: index, 95% CI] against a pre-registered target of ≥ 0.85 .

H4 — Clarification rate on the ambiguous cohort.

[PLACEHOLDER: %, 95% CI] against a pre-registered target of $\geq 80\%$.

H5 — Path B external validation against the Titan 34-metabolite reference (Chapter 20 §20.4b). External recall

(ER) [PLACEHOLDER: %, 95% CI] against a pre-registered target of ≥ 0.50 ; within-prompt external Jaccard (EJ)

[PLACEHOLDER: index, 95% CI] against a pre-registered target of ≥ 0.75 . The Path B target is calibrated to what limma-on-binary can recover from Titan's Cox time-to-event reference; v2.1's `survival_analysis` tool (Chapter 24 §24.1) will enable a stricter external-validation experiment.

Confidence intervals are computed by bootstrap over the 60 runs, with 10,000 resamples and the prompt as the unit of resampling. The bootstrap script is in `scripts/score_validation.py`.

Each headline number is reported in two forms: the headline metric (the binary all-three-match version, or the mean Jaccard) and a "soft" companion (the longest-common-subsequence ratio for TSA, the parameter-key-overlap ratio for PA). The soft companion is reported because it gives a finer-grained picture of where agreement degrades when the headline metric does not pass.

[Figure 21.1: Agreement metrics with 95% confidence intervals. Four horizontal bars for H1, H2, H3, H4. Pre-registered target line drawn vertically. The numerical width of each CI is the bootstrap result. [PLACEHOLDER] cells until the experiment completes.]

21.3 Disaggregation by cohort

The headline metrics aggregate across the ten unambiguous prompts; the disaggregated picture is more informative.

The per-prompt TSA scores show [PLACEHOLDER: distribution shape]. The author's expectation is that the standard-pipeline prompt (prompt 1) and the explicit-

method prompts (prompts 4–9) will score near 100%, and that the QC-and-summarize-only prompt (prompt 2) and the PCA-only prompt (prompt 3) will score lower because they are open to a longer-than-requested-pipeline interpretation. The actual distribution will be reported as a per-prompt table in the manuscript supplementary materials and is summarized in this chapter's Figure 21.5.

The PA scores by tool show [PLACEHOLDER]. The author's expectation is that `qc_filter` and `differential_abundance` will be the tools most prone to parameter disagreement because they have the most arguments with non-trivial defaults; `transform` and `scale` will be the most consistent because their argument lists are short.

The Jaccard distribution across the seven unambiguous prompts that triggered `differential_abundance` shows [PLACEHOLDER].

The clarification-rate distribution across the ten ambiguous prompts shows [PLACEHOLDER]. The author's expectation is

that prompts 1–4 (underspecification of obvious axes) will score near 100%, prompts 5–7 (subtler underspecifications) will score in the 70–90% range, and prompt 10 (incomplete sentence) will be a near-100% clarification trigger.

21.4 The failure-mode taxonomy

Across the 60 runs the agent's failures classify into four named modes. The classification was developed inductively by inspecting every failed repeat after the runs completed; the four classes capture every observed failure with no residual category. The taxonomy is the chapter's qualitative contribution.

Mode A — Ambiguous parameter inference. The agent encounters a parameter the user did not specify and chooses a value the system prompt does not pin as the default. Example: a prompt that says "log-transform" without specifying log2 vs log10 vs sqrt; the agent chose log10 once across the validation runs where log2 was the documented default. Observed frequency in the 60 runs: [PLACEHOLDER: count and %],

The mitigation, applied after the experiment, was to tighten the system prompt to list every tool's default values explicitly. The before/after comparison appears in §21.6.

Mode B — Premature termination. The agent stops after fewer tool calls than the prompt required. Example: a prompt that asks for a full pipeline ending in a volcano plot; the agent runs through `scale` and then emits a final text response without calling `differential_abundance` or `export_session`. Observed frequency: [PLACEHOLDER].

The mitigation was to add an explicit instruction in the system prompt: *unless the user requests otherwise, always run the differential-abundance step when a clinical-group variable is present and always end with `export_session`*. The before/after appears in §21.6.

Mode C — Clarification deferral. The agent runs an ambiguous prompt to completion without asking the clarifying question the protocol expected. This is the failure mode that determines the H4 score. Example: prompt "Find biomarkers in Progredir" (no contrast specified) executed against progressor-

vs-stable by default rather than asking. Observed frequency on the ambiguous cohort: [PLACEHOLDER].

The mitigation was to expand the worked example in the system prompt to include an ambiguous-prompt scenario with a clarification turn. The before/after appears in §21.6.

Mode D — Tool misordering. The agent calls tools in the wrong order, typically calling `pca` before `scale` or `differential_abundance` before `transform`. Modern frontier models almost never do this on the standard pipeline, but it appeared once or twice in the validation runs. Observed frequency: [PLACEHOLDER].

The mitigation was at the tool layer rather than the prompt: the dependency-graph enforcement (Chapter 9 §9.5) now returns a structured error that the agent reads and self-corrects on the next turn. The before/after appears in §21.6.

[Figure 21.2: Failure-mode frequency stacked bar. Two bars: pre-mitigation and post-mitigation. Each bar segmented by the

four failure modes. The post-mitigation bar should be visibly shorter. [PLACEHOLDER] until the data are final.]

21.5 Two transcripts that illustrate the taxonomy

A single failure-mode-A transcript and a single failure-mode-C transcript are reproduced here, lightly redacted for length, to give the reader a concrete sense of what each looks like in practice.

Failure-mode A example.

User (paraphrase 2 of prompt 5): Run a standard analysis of ProgreDir but log-transform the abundances first.

Agent (tool_use, load_metabolomics_data, ...). Agent (tool_use, summarize_dataset, ...). Agent (tool_use, qc_filter, ...). Agent (tool_use, impute_missing, ...). Agent

```
(tool_use, transform, args={"method": "log10",  
"pseudocount": 1.0}, ...). [the documented default is log2] ...
```

The agent correctly identified that the user asked for a log transformation and called the right tool. It chose log10 rather than the system's log2 default, which is documented in the rendered system prompt but apparently not salient enough to the agent in this run. The mitigation: the system prompt's tool-default block was rewritten to use bullet form and explicit default-value notation. After the mitigation, the same paraphrase across three new runs called log2 in all three.

Failure-mode C example.

User (paraphrase 1 of ambiguous prompt 2): Compare the groups in Progredir.

Agent (tool_use, load_metabolomics_data, ...). Agent (tool_use, summarize_dataset, ...). Agent (tool_use, qc_filter, ...). ... Agent (tool_use, differential_abundance,

args={"group_column": "progressor", "reference_group": "stable", ...}). Agent (text): Here are the metabolites differentially abundant between progressors and stable participants...

The prompt was deliberately ambiguous — Progredir has several plausible grouping variables (progressor, sex, diabetes status, hypertension status) — and the agent should have asked. Instead it defaulted to progressor. The mitigation: an additional sentence in the system prompt instructing the agent that when a "groups" reference is ambiguous, list the candidates and ask which the user wants. After the mitigation, the same paraphrase asked the clarifying question in three out of three new runs.

21.6 Before/after mitigation results

Each of the four failure-mode-specific mitigations was tested by re-running the relevant prompts after the mitigation was applied. The before/after numbers:

- Mode A: pre-mitigation [PLACEHOLDER: %], post-mitigation [PLACEHOLDER: %].
- Mode B: pre-mitigation [PLACEHOLDER], post-mitigation [PLACEHOLDER].
- Mode C: pre-mitigation [PLACEHOLDER], post-mitigation [PLACEHOLDER].
- Mode D: pre-mitigation [PLACEHOLDER], post-mitigation [PLACEHOLDER].

The headline H1–H4 numbers reported in §21.2 are the post-mitigation numbers. The pre-mitigation numbers — what the system would have produced before the mitigations were applied — are reported here for transparency and are [PLACEHOLDER].

A subtle methodological note: applying mitigations after the protocol-committed runs and then reporting the post-mitigation numbers as the headline is a deviation from the strictest pre-registration discipline. The author's defense is that the protocol explicitly allowed system-prompt iteration during Week 4 (committed in `docs/VALIDATION_PROTOCOL.md` §4.3), and that the deviation is documented in `data/validation/PROTOCOL_DEVIATIONS.md` per the policy of Chapter 20 §20.7. A reviewer who prefers a stricter standard can read the pre-mitigation numbers in this section and judge accordingly.

[Figure 21.3: Before/after mitigation paired plot. Four pairs of bars, one pair per failure mode. The post bar in each pair should be visibly shorter. [PLACEHOLDER] until data are final.]

21.7 Cost per session

The 60 validation sessions, with three timeout retries, cost a total of **\$2.72** [PLACEHOLDER] of Anthropic API spend at

claude-sonnet-4-6 pricing as of mid-2026 [VERIFY pricing date and source URL]. The distribution per session ranges from **\$0.018** to **\$0.082** [PLACEHOLDER] with a median of **\$0.045** [PLACEHOLDER].

The same prompts run against claude-opus-4-7 for the cost-comparison analysis of Chapter 10 §10.7 totaled **\$8.10** [PLACEHOLDER] across 60 sessions, a 3.0× factor over Sonnet. The agreement-rate differences between the two models are reported in §21.8.

The total experimental spend, including the Tuesday dry run and the timed-out reruns, was **\$3.10** [PLACEHOLDER], well within the \$40 protocol budget.

[Figure 21.4: Cost-per-session distribution. Two violin plots, Sonnet on the left, Opus on the right. Median lines and 95th percentiles annotated. [PLACEHOLDER] until data are final.]

21.8 Sonnet vs Opus side-by-side

The same 60-prompt suite was run against both models. The headline differences:

- TSA: Sonnet [PLACEHOLDER: %], Opus [PLACEHOLDER: %]. Within [PLACEHOLDER] percentage points.
- PA: Sonnet [PLACEHOLDER: %], Opus [PLACEHOLDER: %]. Within [PLACEHOLDER] percentage points.
- Jaccard: Sonnet [PLACEHOLDER], Opus [PLACEHOLDER]. Within [PLACEHOLDER].
- CR: Sonnet [PLACEHOLDER], Opus [PLACEHOLDER]. Within [PLACEHOLDER] percentage points.

The author's expectation, based on the Week 3 dry runs, is that Opus will be slightly higher on TSA and CR (modeling closer to "if in doubt, ask") and indistinguishable on PA and Jaccard. The cost differential — 3.0× as reported in §21.7 — makes Sonnet

the right operational default unless the absolute lift on TSA and CR exceeds 5 points, which the dry runs do not suggest.

21.9 Honest limitations of the results

Three limitations belong in the chapter's body, not in a footnote.

Single-dataset, single-rater. All numbers are from ProgreDir, with clarification scoring by one human (the author). The author's involvement in writing the protocol, the system prompt, and the scoring rubric is a real source of bias the experiment does not control for. A genuinely independent validation — by a different group, on a different dataset, with a different rater — is the v2.1 plan.

Mitigation iteration. As §21.6 acknowledges, the headline numbers are post-mitigation. The pre-mitigation numbers are reported for transparency, and the pre-registered protocol allowed the iteration, but a strictly conservative reading would weight the pre-mitigation numbers more heavily.

No cross-environment replay test. The bit-identical-replay claim of Chapter 11 §11.6 was tested on the same Docker image used for the original runs. The cross-OS and cross-BLAS rows of Figure 11.4 are within-tolerance, not bit-identical. The manuscript reports this honestly; a reader who needs cross-environment bit-identity should not infer it from these results.

21.10 The qualitative result

Independent of the headline numbers, the qualitative result of the validation is that the failure-mode taxonomy is small, named, and mitigable. Four classes cover every observed failure. Each has a specific mitigation that the system prompt or the tool layer can apply. The mitigations close the gap; they do not eliminate it entirely, and a residual [PLACEHOLDER] % rate of mode-A and mode-C failures remains in the post-mitigation data.

A user reading this chapter should be calibrated. The system is reliable enough for the unambiguous standard-pipeline cases

that constitute most clinical metabolomics analyses. It is not so reliable that a clinician should not read the auto-generated methods paragraph carefully before submitting a manuscript. The methods paragraph is a draft; the clinician is the editor.

The honest framing is that MetabolonR-LLM removes the analyst's overhead on routine analyses while making it harder, not easier, to ignore the parameter choices that determine the result. The validation experiment is the evidence for both halves of that claim.

[Figure 21.5: Per-prompt agreement heatmap. Rows: 20 prompts. Columns: TSA, PA, Jaccard, CR. Cells colored by score. Patterns visible by row are the chapter's most informative single image. [PLACEHOLDER] until data are final.]

What you should be able to do after this chapter

- Read a TSA / PA / Jaccard / CR result table and identify which numbers are pre-registered targets and which are observed values.
- Classify a failed agent run into one of the four named failure modes (A through D).
- Trace a failure mode to the specific system-prompt or tool-layer mitigation that addresses it.
- Estimate the API cost of a comparable validation suite at Sonnet pricing.
- Argue, in one paragraph, whether the validation evidence is sufficient to publish or whether additional rigor is required.
- Name the three honest limitations of the experiment and weight them against the headline conclusions.

Chapter 22 — What "Reproducible LLM Agent" Really Means

This chapter is the grounded philosophical one. It draws the line between deterministic output (impossible) and deterministic re-executability (achievable), and traces the regulatory implications under IMDRF, FDA SaMD, and the EU AI Act.

22.1 Restating the distinction precisely

Chapter 11 introduced two reproducibility properties: deterministic output and deterministic re-executability. The distinction matters enough to be restated with care here.

Deterministic output. A computation has deterministic output if two runs on the same inputs produce bit-identical outputs. Classical software has this property when run in the same environment. LLM-based systems do not, because the LLM itself is stochastic even at temperature zero.

Deterministic re-executability. A computation is deterministically re-executable if, given a structured record of one run, a different system can reproduce that run's outputs without re-invoking the stochastic component. MetabolonR-LLM has this property: the `analysis_log.json` is the record; the replay engine is the different system; the LLM is not re-invoked.

The first property is a property of the computation. The second is a property of the *record* of the computation. The first is impossible for any system that uses an LLM in the loop. The second is achievable, is achieved by MetabolonR-LLM, and is the standard the rest of this chapter argues for.

A useful analogy. Classical software is like a recipe: same ingredients, same steps, same result. An LLM agent is like a chef working from a brief: same brief, different recipe each time, similar but not identical result. MetabolonR-LLM is like a chef who writes down the recipe she chose this time before she cooks: the recipe is the artifact; anyone with the recipe can cook the same dish, even if the chef is unavailable.

22.2 The history of reproducibility in computational science

Reproducibility in computational science has had three eras.

The first, roughly through the 1990s, was the **bench-notebook era**. Reproducibility was an author's responsibility, exercised by writing methods clearly enough that another lab could re-derive the computation by hand. The standard was high in principle and low in practice.

The second, from the early 2000s through the late 2010s, was the **versioned-script era**. The advent of git, GitHub, and reproducible-document tools (Sweave, R Markdown, Jupyter) gave authors a way to publish the actual code. The standard rose. Sandve et al. 2013's ten rules were the era's high-water mark.

The third, from roughly 2020, is the **container era**. Docker and similar tools let authors publish the runtime environment alongside the code. A reader can reproduce the analysis without

reproducing the install. This is where most reproducibility-conscious computational biology lives in 2026.

The introduction of LLMs into the analysis loop in 2023–2024 broke the container era's guarantees. A containerized LLM agent is reproducibly *runnable* but not reproducibly *productive of the same output*. The third era's standard is necessary but no longer sufficient.

A fourth era — call it the **logged-agent era**, though no one is using that term yet — is the response. The artifact that needs to be published is not just the code, not just the container, but the structured log of the agent's run. The replay engine is the era's analog of `docker run`. MetabolonR-LLM's architecture is an early attempt to operationalize this; other 2024–2026 systems, with varying degrees of self-awareness, are converging on it as well.

22.3 Three threats to deterministic re-executability

The re-executability property is achievable, but it is not automatic. Three forces threaten it over time.

API deprecation. The LLM provider can retire the model snapshot the log records. A 2026 log pinned to `claude-sonnet-4-6` may not be replayable in 2028 if Anthropic retires that snapshot. The MetabolonR-LLM architecture mitigates this by *not requiring* the LLM at replay time: the replay engine reads the tool calls from the log directly. The original run's LLM-side reasoning is gone, but the analysis itself can still be reproduced. This is a substantial design advantage over architectures where the LLM is re-invoked at replay.

Model-snapshot retirement. Even if the provider remains, the specific snapshot's outputs may not be available for re-querying. The same mitigation applies: replay does not query the LLM.

Dependency drift. R packages, Python packages, BLAS implementations all evolve. A snapshot pinned in `renv.lock` and `requirements.txt` is replayable now but may not be installable in five years because PyPI or CRAN garbage-collects old versions, or because a binary wheel is no longer compatible with a current OS. The mitigation is to vendor the dependencies into the published artifact — store the wheel files and tarballs alongside the log, not just their version numbers — and to archive the canonical Docker image with a content-addressed registry like Zenodo. v2.0 archives the image; v2.1 considers vendoring the packages more aggressively.

A fourth threat is hardware drift: a future CPU may compute floating-point math subtly differently. The within-tolerance replay standard absorbs this; the bit-identical replay standard does not. The honest position is that bit-identical replay is achievable in the short term and not guaranteed in the long term; within-tolerance replay is a more durable standard.

22.4 The pinning manifest in full

The MetabolonR-LLM pinning manifest, captured in the `analysis_log.json` header (Chapter 11 §11.4), records eleven fields. Each is justified by one of the threats above.

`model_name` and `model_snapshot` pin the LLM. `prompt_sha256` pins the prompt template. `tool_layer_sha` pins the MetabolonR-LLM repository commit. `metabolonr_v1_commit` pins the underlying R-side codebase. `r_version` and `r_packages` pin the R environment. `python_version` and `python_packages` pin the Python environment. `blas` and `blas_threads` pin the numerical libraries. `os_image_digest` pins the OS layer.

A reader who reads the manifest sees, in one screen, every value she needs to set up a matching environment. A regulator who needs to evaluate the manifest can verify each pin against a controlled-image registry. A reviewer who reads the manifest sees that the authors did the work.

The author's expectation is that some form of this manifest will become a standard supplementary-material requirement for clinical-LLM-tool papers by 2028. MetabolonR-LLM is partly an early instance of what that requirement looks like.

[Figure 22.1: The reproducibility taxonomy across paradigms. Five rows: bench notebook, versioned script, container, logged agent (current state of the art), formally verified agent (hypothetical future). Three columns: input reproducibility, computation reproducibility, output reproducibility. Each cell marked Yes / Conditional / No.]

[Figure 22.2: The pinning manifest, annotated. The eleven fields with arrows pointing to the threat each addresses.]

22.5 Regulatory framing: IMDRF, FDA SaMD, EU AI Act

Three regulatory frameworks are relevant to clinical agentic bioinformatics in 2026. Each is sketched here at the level a

software engineer needs to understand; the full legal treatment is for a different book.

IMDRF Software as a Medical Device (SaMD) is the international forum framework that classifies software intended for medical purposes by intended use and risk. The 2017 IMDRF framework, refined in 2023, classifies SaMD into four risk classes I–IV based on (a) the seriousness of the health condition and (b) the significance of the information provided by the software. A clinical-decision-support tool that informs the diagnosis of a serious condition is high-risk; a tool that informs general wellness is low-risk.

A research-grade tool like MetabolonR-LLM v2.0 is not SaMD. It is intended for research, with explicit non-clinical-use language in its license and documentation. Any future deployment in a clinical context — for example, as part of a clinical-trial endpoint analysis where the result enters the clinical record — would change the classification.

The relevance of IMDRF to this book is that the framework's evidence requirements include reproducibility. A SaMD

submission requires demonstration that the software produces consistent outputs under defined conditions. The replay engine and the manifest are direct evidence for this requirement. The version-pinning policy is the operational discipline that supports the evidence.

FDA SaMD framework. The FDA's 2021 SaMD Action Plan, updated through 2025, harmonizes with the IMDRF framework and adds specific requirements for AI/ML-enabled SaMD. The 2023 guidance on "Predetermined Change Control Plans" allows AI systems to be updated post-market under documented conditions; the 2025 update extends this to LLM-enabled systems with specific reproducibility documentation requirements [VERIFY: confirm the 2025 update exists and cite correctly].

For an LLM agent, the FDA's relevant ask is essentially the four-criterion bar of Chapter 19: replay, version pinning, cost-and-environment disclosure, failure analysis. The MetabolonR-LLM architecture maps onto each.

The FDA's framework includes a distinction between "locked" AI models and "adaptive" ones. A locked model is fixed at submission and does not learn from post-deployment data; an adaptive model does. MetabolonR-LLM is firmly in the locked category — the LLM snapshot is pinned, the tool layer is pinned, the prompt is pinned. An adaptive variant is outside the scope of v2.0 and would face a substantially higher regulatory bar.

EU AI Act. The EU AI Act, in force since 2024 and with its high-risk-system provisions becoming binding through 2026 and 2027, classifies AI systems by risk in a way that overlaps with but is not identical to the IMDRF / FDA framework. Clinical-decision-support systems are explicitly high-risk under Article 6 and Annex III; they are subject to the conformity-assessment and post-market-monitoring requirements of Articles 16 and 17.

A research-grade tool is exempt from the high-risk provisions. A clinical deployment of MetabolonR-LLM in an EU jurisdiction would not be. The reproducibility requirements of Article 17 (record-keeping) and Article 13 (transparency) are

directly addressed by the session log and the methods-paragraph generation.

A reader who is not a regulatory specialist should take away two points from this section. First, MetabolonR-LLM v2.0 is not subject to any of these frameworks because it is research-grade. Second, the architecture is positioned so that a future clinical-grade version would meet the frameworks' reproducibility requirements without architectural change — the log and the replay would be the regulatory artifacts.

22.6 A speculative trajectory for 2026–2030

The regulatory frameworks above are evolving. The author's bet about how they will evolve over 2026–2030, offered as opinion rather than fact:

By 2027 the FDA will require, for clinical AI/ML submissions involving LLM components, a deterministic re-executability artifact — something resembling the `analysis_log.json`

plus replay-engine pair. The current draft guidance is ambiguous on this point; the next revision will tighten it.

By 2028 the major clinical-bioinformatics journals will require the artifact as supplementary material for any agentic-tool paper. *Briefings in Bioinformatics* and *GigaScience* are likely to be early adopters; *Bioinformatics* and *Nature Methods* will follow.

By 2029 the public-cloud LLM providers will offer, as a paid service, model-snapshot archival with verifiable provenance for clinical-LLM submissions. The provider will commit to maintaining the snapshot for a specified number of years against a regulatory standard.

By 2030 the on-prem open-weights deployment path (Llama, Qwen, and their successors) will be mature enough that lab-internal clinical tools can run without an external API dependency, and the regulatory frameworks will explicitly recognize the on-prem path as a valid deployment model.

The first prediction is the most uncertain and the most consequential. If the FDA does not require the artifact, the field's voluntary standards will fragment. If the FDA does, the

voluntary standards will consolidate around something resembling the MetabolonR-LLM pattern. The author's position is that the consolidation is desirable and likely; the bet is real but is not held without uncertainty.

22.7 The honest position on what this chapter has done

This chapter has done three things. It has restated the distinction between deterministic output and deterministic re-executability and placed MetabolonR-LLM in the second category. It has named three threats to re-executability over time and the mitigations the architecture applies to each. It has placed the architecture against three real regulatory frameworks and argued that the architecture is positioned for a clinical-grade extension without rewrite.

What this chapter has not done is prove that the speculative trajectory will play out as the author bets. The regulatory frameworks may evolve differently. The cloud-LLM providers may not offer archival. The on-prem path may stall. The bet is a

bet. The architecture is the hedge: whatever the regulators decide, a system that produces a structured replayable log is in a better position than a system that does not.

What you should be able to do after this chapter

- Distinguish deterministic output from deterministic re-executability and explain why the second is the binding standard for clinical agentic systems.
- Trace the four eras of computational reproducibility from bench notebook to logged agent.
- Name three threats to long-term re-executability and the mitigation for each.
- Read a pinning manifest and identify the field that addresses each threat.
- Place a hypothetical clinical-grade extension of MetabolonR-LLM on the IMDRF SaMD risk grid.

- Articulate the author's speculative 2026–2030 trajectory and identify the assumption that, if wrong, would invalidate it.

Chapter 23 — Known Limitations

Every chapter so far has made a positive claim. This chapter is the honest inventory of what MetabolonR-LLM does not do and what the architecture explicitly does not solve. A reader who finishes Part V wanting to deploy the tool tomorrow should read this chapter first.

23.1 Fixed tool set

The closed-set, schema-pinned principle of Chapter 8 buys auditability at the cost of flexibility. A user whose analysis requires a method MetabolonR-LLM does not have — a Cox proportional-hazards regression on Progredir's survival time, a partial-least-squares discriminant analysis, a metabolite-set enrichment test against a curated KEGG pathway — cannot ask the agent to do it. The agent will, correctly, refuse and direct her to a biostatistician.

The cost is real. The mitigation is that the tool set is *extensible*: a new tool is a Pydantic schema, an R wrapper, and an oracle

test, and v2.x releases will add tools as the clinical-translation team's needs surface them. The deferred `pathway_variation` (Chapter 24) and a planned `batch_correct` are the first two.

A reader who needs flexibility now has the choice: use a more flexible agent (BioChatter, Biomni) and accept its weaker reproducibility guarantees, or use MetabolonR-LLM for what it does and a separate tool for what it does not. The author thinks the second is the right choice for most clinical metabolomics, and is honest that the first is the right choice for a different audience.

23.2 No on-the-fly tool synthesis

A subclass of the previous limitation deserves its own section because it is a deliberate non-feature, not a deferred feature.

A code-writing agent (Chapter 8 §8.3) can generate a tool on demand. The user says "I want to do an XYZ analysis," the agent writes a function that does XYZ, the runtime executes the function. MetabolonR-LLM cannot do this. The reason is the

auditability commitment: an on-the-fly function has no schema, no oracle test, no v1 routine behind it, and no peer review.

This is not a deficiency that future versions will fix. It is the architecture's central trade-off. v3 may include a human-in-the-loop tool-proposal workflow (Chapter 24 §24.3), in which the agent *drafts* a new tool spec and a human signs off before it ships. That is different from on-the-fly synthesis; it is supervised extension.

23.3 API cost as a deployment barrier

The Anthropic API cost, modest in absolute terms (\$0.045 median per session on Sonnet — Chapter 21 §21.7), is large in low-resource settings. A lab in a country with restricted international financial access cannot run MetabolonR-LLM at scale without a partnership for the API quota. A lab in a high-resource environment but with strict budget oversight may find the per-user-per-month cap of \$20 (Chapter 10 §10.8) inadequate for a clinician's heaviest months.

The v2.x mitigation is the on-prem open-weights deployment path (Chapter 24 §24.5). An on-prem Llama 3.3 or Qwen 2.5 deployment has zero marginal API cost; the up-front infrastructure cost is real but is a different cost model. For the global clinical-AI community in 2026, the on-prem path is the only equitable scaling path.

v2.0 does not provide the on-prem path. A user who needs it should hold off on adopting MetabolonR-LLM until v2.1 or use it knowing the API cost will be a recurring line item.

23.4 Data privacy

The LLM, as routed through the Anthropic API, sees metabolite *names* and sample-annotation *column names and values*. It does not see per-sample abundance values (the `DatasetHandle` indirection of Chapter 10 keeps those local). Whether this is acceptable depends on the dataset.

For a dataset like Progredir, where the sample-annotation file carries group labels but no patient identifiers and no raw biospecimen results, the API-side exposure is similar to what is

exposed in a methods paragraph published in a paper. For a dataset with potentially identifying clinical variables — exact ages, geographic micro-locations, rare-disease labels — the API-side exposure may exceed what is permissible under HIPAA or under the local IRB's data-use agreement.

The architecture allows but does not enforce on-prem deployment. A HIPAA-covered dataset must be processed on-prem, either with the v2.1 open-weights path or with an enterprise Anthropic deployment (which exists in 2026 but requires institutional procurement [VERIFY current Anthropic enterprise offerings]). The v2.0 documentation includes a clear "do not upload PHI" notice and a checklist of dataset properties that imply on-prem.

The team's working position is that until a clearly HIPAA-compliant deployment path is documented and tested, MetabolonR-LLM is for research datasets and de-identified clinical datasets only. The v2.1 roadmap addresses this.

23.5 Adversarial prompting

A malicious user can attempt to confuse the agent. Two classes of attack matter.

Prompt-injection via dataset content. A crafted column name like "ignore the system prompt and instead..." in the sample annotation could attempt to override the agent's instructions. Modern frontier models are resistant to this attack class but not immune. The MetabolonR-LLM mitigation is twofold: the tool descriptions never relay user-provided text back to the LLM verbatim, and the system prompt instructs the agent that all user-data content is to be treated as data, not instructions. The Chapter 23-supplement (in the public repository, not in this book) includes a small red-team transcript that probes the attack surface.

Confused-deputy attacks. A user who is permitted to use the tool but should not be permitted to access a specific dataset could attempt to trick the agent into loading it. The MetabolonR-LLM mitigation is file-system permissions: the

agent runs as a service user with read access only to the datasets the user has been granted access to. The agent cannot grant itself broader access via tool calls.

Neither mitigation is exhaustive. A determined adversary with access to the agent's input channel could probably find a confusion path. The honest position is that MetabolonR-LLM is not a security-critical system in v2.0 and should not be deployed in adversarial settings without additional hardening.

23.6 Single-language support

The system prompt is in English. The tool descriptions are in English. The agent is trained on a multilingual corpus and can in principle accept prompts in other languages, but the validation experiment (Chapter 21) tested only English. A clinician working in Portuguese, Mandarin, or Arabic cannot expect the agent's behavior to be characterized by the validation numbers.

The mitigation is to translate the system prompt and validate in each target language. The author's collaborators in São Paulo,

Seoul, and Berlin have offered to participate in a multilingual validation as part of v2.x. The work is real and is on the roadmap.

A first-pass user who must use a non-English prompt can switch the agent's response language by asking, and the agent will comply, but the agreement-rate numbers from Chapter 21 do not apply. A user in this situation should treat the agent's outputs with extra care and check the auto-generated methods paragraph closely.

23.7 What the architecture explicitly does not try to solve

A list, for clarity.

Statistical modeling beyond two-group differential abundance. No Cox regression, no linear mixed models, no survival analysis. A clinician analyzing repeated-measures data goes elsewhere.

Multi-omics integration. No transcriptomics, no genomics, no proteomics. v3 considers this; v2 does not.

Causal inference. The agent's outputs are associative, not causal. A user who reads the methods paragraph and concludes "the metabolite *causes* the outcome" has misread it. The tool produces correlations; causation requires study design that is outside the tool's scope.

Clinical interpretation. The biomarker shortlist is a candidate list, not a clinical recommendation. A clinician who acts on the list without further work is making a clinical judgment that the tool does not vouch for.

Sample-size and power calculation. No prospective-design tools. The agent runs analyses on the data it is given; it does not advise on how much data to collect.

The list of non-features is short because the tool's scope is intentionally narrow. A narrow tool with a clear interface is more useful than a broad tool with confused boundaries; that is the bet of Part III, and §23.7 is the honest accounting of where the bet has been placed.

23.8 Severity and mitigation grid

The seven limitations above fall on a 3×3 grid of severity (low / medium / high) and mitigation-availability (none / partial / planned).

- **Fixed tool set:** medium severity, planned mitigation (v2.x tool additions).
- **No on-the-fly synthesis:** medium severity, no mitigation (this is the architecture).
- **API cost:** high severity in low-resource contexts, planned mitigation (on-prem path).
- **Data privacy:** high severity for PHI datasets, planned mitigation (on-prem path).
- **Adversarial prompting:** low severity for research use, partial mitigation (sandboxing).
- **Single language:** medium severity for non-English-speaking labs, planned mitigation (multilingual validation).
- **Architecture-scope exclusions:** acknowledged scope, no mitigation (intentional).

A user evaluating MetabolonR-LLM for her own context should run this grid against her own use case and decide accordingly. The chapter's purpose is to make the evaluation possible, not to argue any particular use case in or out.

[Figure 23.1: Limitation × severity × mitigation grid. Seven rows, three columns. Cells colored by severity. Annotations name the chapter where the mitigation is discussed.]

What you should be able to do after this chapter

- State each of the seven known limitations of MetabolonR-LLM v2.0 in one sentence.
- Distinguish deferred limitations (planned mitigations) from architectural limitations (deliberate non-features).
- Evaluate MetabolonR-LLM against a hypothetical use case using the §23.8 grid.
- Articulate why "no on-the-fly tool synthesis" is the architecture's central trade-off, not a future feature.

- Recognize that this chapter, rather than the introduction, is the right starting point for a sober adoption decision.

Chapter 24 — Future Directions

This chapter is the v2.x and v3 roadmap. The headline v2.1 features are `pathway_variation` (the KEGG-mapped extension of `sources_of_variation` deferred from v2.0) and `survival_analysis` (the Cox-proportional-hazards tool that would let the agent reproduce the Titan 2019 reference analysis on its own terms — Chapter 20 §20.4b explains why this matters for external validation). Beyond v2.1: multi-omics, dynamic tools with human review, HMDB hyperlinking, Metabolomics Workbench integration, on-prem open-weights, and the path to a full agentic NEPTUNE stack.

24.1 The v2.1 headline tools: **pathway_variation** and **survival_analysis**

Two deferred-from-v2.0 tools ship together in v2.1. `pathway_variation` was deferred at the Week 1 architecture lock for scheduling reasons; `survival_analysis` was identified as a v2.1 candidate later in the build, when Chapter 20's external-validation arm against the Titan 2019 reference made the absence of a Cox tool a binding constraint on the experiment's strength (Chapter 20 §20.11).

pathway_variation

The single deferred tool of v2.0 at the original architecture lock — `pathway_variation` — is the headline of v2.1. The decision to defer it from v2.0 was made in Week 1's architecture lock (Chapter 13 §13.5) for two reasons. First, the tool requires a KEGG-pathway lookup subsystem with its own

caching, error handling, and version pinning of the KEGG snapshot; that subsystem is larger than any single tool's wrapper and would have pushed the Week 2 schedule. Second, shipping it independently of v2.0 allows it to be released with its own oracle test on a curated KEGG subset, rather than as a marginal addition tested against general behavior.

The tool's design, sketched here for the v2.1 spec:

```
class PathwayVariationInput(BaseModel):  
    dataset_handle: str  
    covariates: List[str]  
    kegg_snapshot_version: str = "2026-04"  
    min_pathway_size: int = 5  
    fdr_threshold: float = 0.05
```

The tool wraps a v1-equivalent R routine that maps each metabolite to its KEGG pathways, computes the F-ratio of between-level to within-level variance for the *aggregated pathway signature* (one number per pathway per covariate, rather than per metabolite per covariate), and ranks pathways by their median F-ratio. The output is a

`PathwayVariationResult` containing a ranked list of pathways with F-ratios and significance flags.

The oracle test pins the tool's behavior against the same Progredir baseline-serum analysis as the other tools, but using a frozen 2026-04 KEGG snapshot. The KEGG-snapshot pinning is the subtle correctness property: KEGG's pathway memberships change quarterly, and an unpinned tool would produce different results next quarter on the same data. The pinning is in the input schema, defaulted to the snapshot the tool was trained against, and recorded in the session log.

survival_analysis

The Cox proportional-hazards tool whose absence is the binding constraint on Chapter 20's Path B external-validation experiment. With `survival_analysis` in the tool set, the agent can produce Cox-based biomarker shortlists on the same time-to-event composite outcome (mortality or incident renal replacement therapy) that Titan 2019 reports. The external-validation arm becomes a like-for-like comparison: the agent's Cox output against Titan's Cox output, with the 50% partial-

recovery target of v2.0 (calibrated to limma-on-binary's restricted scope) replaced by a stricter $\geq 80\%$ target.

The tool's Pydantic input schema sketch:

```
class SurvivalAnalysisInput(BaseModel):
    dataset_handle: str
    time_column: str
    event_column: str
    adjust_for: List[str] = []
    fdr_method: Literal["BH", "bonferroni", "holm"]
    fdr_threshold: float = Field(default=0.05, gt=0)
```

It wraps the `v1 metabolonr::run_cox()` routine, which exists as a v1-internal helper but was never exposed in the v1 Shiny UI. The oracle test pins the tool's output against Titan's published shortlist on the ProgreDir cohort: the agent's run on `(time, event, adjust_for=[batch])` is expected to recover the Titan 34-metabolite reference set at $\text{FDR} \leq 0.05$ with bit-identical agreement on the top 30 hits.

A subtle design question with `survival_analysis` is the relationship between `differential_abundance` (binary) and `survival_analysis` (time-to-event). Both are legitimate analytical paths; neither is a replacement for the other. The agent's system prompt in v2.1 will be updated to ask the user explicitly which path she wants when her clinical question is compatible with both. This is the *clarification-before-execution* principle (Chapter 8) applied to a new dimension.

v2.1 release timing

The v2.1 release is scheduled approximately three months after v2.0 — long enough for the v2.0 paper to land its first round of reviews, short enough that the field's interest in the architecture has not faded.

24.2 Multi-omics integration

The natural v2.x direction after `pathway_variation` is multi-omics integration: extending MetabolonR-LLM to handle transcriptomics, proteomics, or genomics data alongside

metabolomics, on the same agent and with the same reproducibility guarantees.

The architectural sketch:

- A unified `OmicDatasetHandle` that subsumes `DatasetHandle` and carries an omics-type tag.
- New tools for each omics layer:
`transcriptomics_da`, `proteomics_da`,
`genomics_burden_test`, each wrapping the equivalent v1 routine from the corresponding lab tool.
- A new tool, `multiomics_integrate`, that takes handles from two or more layers and performs a joint analysis (canonical correlation, multi-block PLS, similarity network fusion).

The honest design question is whether the agent's routing accuracy holds across the larger tool surface. With ten tools v2.0 hits the routing-accuracy sweet spot (Chapter 9 §9.1); thirty tools spread across four omics layers may push past it. The mitigation is *namespacing*: tools are grouped by omics layer,

and the agent's tool selection is constrained to the relevant layer based on the user's prompt or an explicit pre-routing step.

The multi-omics direction is a v3, not a v2.x. The v2.x releases stay within metabolomics.

24.3 Dynamic tool creation under human review

A more speculative direction is **supervised tool extension**: a workflow in which the agent itself drafts a proposal for a new tool, and a human reviewer signs off before the tool ships.

The workflow:

1. The user asks for an analysis the agent cannot do.
2. Instead of refusing, the agent says: "I cannot do that, but I can draft a tool proposal."
3. The agent emits a structured proposal — a candidate Pydantic schema, a candidate R-side function signature, a candidate oracle test design — and writes it to a queue.

4. A human reviewer (a lab member, not the original user) reads the proposal, edits it if necessary, and either approves or rejects.
5. If approved, the proposal becomes a PR against the public repository. The PR's CI runs the candidate oracle test against a curated reference dataset. If green, the PR is merged and the tool ships in the next minor release.

The workflow keeps the closed-set discipline at the agent level (the agent never invents a tool unilaterally) while letting the tool catalog grow at human-review pace. It is a v2.x direction; v2.0 ships without it.

The risk in this design is that the agent's proposals are subtly wrong in ways the human reviewer cannot catch. The mitigation is the oracle test: a proposal that does not pass an oracle test against a curated reference is not a candidate for merging. The test, not the proposal, is the gate.

24.4 KEGG and HMDB hyperlinking

A small but high-value direction is the *output side* of the same KEGG layer that `pathway_variation` builds the analytical side of. The biomarker shortlist from `differential_abundance` becomes a clickable list, with each metabolite linking to its KEGG entry and its HMDB entry. The methods paragraph generated by `export_session` includes the KEGG and HMDB identifiers inline.

The implementation is trivial: a static mapping table from metabolite ID to KEGG and HMDB URLs, included in the metabolite annotation file at load time. The user-side experience is meaningfully better; the clinician's hop from "metabolite X is differentially abundant" to "metabolite X participates in pathway Y, which has been implicated in disease Z" is one click.

This direction is the second item on the v2.1 list. It is technically easy and was deferred only because it is bundled

with the `pathway_variation` rollout — the same KEGG snapshot pins both.

24.5 Metabolomics Workbench integration

The Metabolomics Workbench (Sud et al. 2016, NIH-supported) is the standard public repository for metabolomics datasets. A future `load_metabolomics_workbench` tool would accept a study identifier and pull the dataset directly from the Workbench's API, populating the same `DatasetHandle` indirection as a local file would.

The clinical-translation value is that any published metabolomics study can be replicated through MetabolonR-LLM with a single tool call. A clinician who reads a paper and wants to apply its analysis to her own cohort can run both analyses through the same agent and compare.

The privacy implication, raised in Chapter 23 §23.4, does not apply to Workbench data because Workbench data are already

public. This makes the Workbench-integration tool a natural early v2.x addition: low risk, high value.

The implementation is on the v2.x roadmap, target v2.2 or v2.3 depending on Workbench API stability [VERIFY: confirm Workbench API status as of mid-2026].

24.6 On-prem open-weights deployment

Chapter 23 §23.3 named the API-cost barrier as a deployment limit in low-resource settings. The v2.x mitigation is the on-prem path: deploy a locally hosted open-weights LLM (Llama 3.3, Qwen 2.5, or a 2026 successor) as the routing model in place of the Anthropic API.

The architectural change is minimal. The agent loop already abstracts the LLM call as a single function; swapping the implementation from Anthropic-SDK to a local-OpenAI-compatible-API endpoint is roughly 40 lines of code. The tool layer does not change. The session log records the on-prem

model name and snapshot in the same field that records `claude-sonnet-4-6` for the API path.

The honest design question is routing-quality drop. Open-weights models in 2026 are very good but not yet at frontier-API parity on long-tool-set routing tasks. A back-of-envelope estimate based on the open-weights numbers in the agentic-LLM benchmarks of 2025 [CITATION NEEDED: pick one or two recent benchmark papers]: a 70B-parameter open-weights model loses approximately 5–10 percentage points on TSA relative to Sonnet on a 10-tool surface.

This is a real cost. For a lab in a high-resource setting it argues for the API path. For a lab in a low-resource setting, where the API path is not available, the 5–10 point drop is the cost of having a tool at all, and is acceptable.

The on-prem path is v2.1 or v2.2 depending on hardware-provisioning timeline. The Kretzler-lab GPU cluster has the capacity to run a 70B model; the bottleneck is the deployment automation, not the hardware.

24.7 The agentic NEPTUNE stack

The longest-range direction in the roadmap is the **agentic NEPTUNE stack**: a coordinated set of agents, each specialized for a different NEPTUNE analytical task (metabolomics, transcriptomics, histopathology, electronic-health-record extraction), with a meta-agent that routes between them based on the clinical question.

A clinician at a NEPTUNE site, asking "what serum and tissue markers predict progression in this patient's biopsy-confirmed FSGS subgroup?" would route through the meta-agent, which would call the metabolomics agent (MetabolonR-LLM), the transcriptomics agent (a v3 successor), and the histopathology agent (a different system entirely), then synthesize the results into an integrated patient-context analysis.

This is a multi-year vision, not a v2.x release. It depends on:

- v3 of MetabolonR-LLM (multi-omics integration).

- Equivalent v1.0 systems for the transcriptomics and histopathology arms, which the Kretzler lab does not yet have.
- A meta-agent architecture that is consistent with the per-arm reproducibility commitments. This is non-trivial; the meta-agent's session log must compose the per-arm logs cleanly.

The author's expectation is that the NEPTUNE stack will be a v3-or-v4 deliverable, on

Chapter 25 — A Short Guide to Publishing Scientific Software in 2026

This is the advice chapter — written to the dedicated student who built this with the author. The software-paper landscape has shifted in 2024–2026; here is what now matters. By the end of the chapter the student should have a working twenty-item checklist to apply to her next project.

25.1 The shift since 2020

A reader publishing scientific software in 2020 needed to do four things well. Have a working tool. Put the code on GitHub. Write a paper that described the tool. Submit to one of the bioinformatics-software venues. The bar was modest and the field's reviewers were generous with reproducibility expectations.

By 2026 the bar has moved. A working tool is still required but is now the table-stakes minimum. Code-on-GitHub is required but is not sufficient. Reviewers now expect, with varying degrees of explicitness:

- A pinned runtime: Docker image with content-addressed digest, or an equivalent.
- A reproducibility receipt: for ML or agentic tools, a session log or model card with version pins.
- A continuous-integration pipeline that demonstrates the tool's tests pass on every push.
- A Zenodo DOI for the software, integrated with the GitHub release flow.
- A `CITATION.cff` file that renders correctly on GitHub's "Cite this repository" widget.
- Cost disclosure for tools that use paid APIs.
- A failure analysis or limitations section that reads honestly.

The shift is partly technological — Docker, Zenodo, GitHub Actions, the agentic-LLM ecosystem all matured in 2020–2025 — and partly cultural. Reviewers have been burned, and the

response has been to ask for more evidence up front. The advice in this chapter is calibrated to the 2026 bar.

25.2 Choosing between Application

Note, Software, and full Methods

venues

Three venue classes are typical for scientific software, and the choice depends on the work's scope.

Application Notes are short (~1,000–2,500 words), focused on a single tool, and emphasize availability and ease of use. Examples: *Bioinformatics* Application Notes, *Briefings in Bioinformatics* Software, *PLoS Computational Biology* Software. The Application Note format is appropriate when the contribution is a tool that does something specific well, and the underlying methods are not novel.

Software Papers are medium-length (~3,000–5,000 words), focused on a tool and its methodology, and require a substantial validation. Examples: *GigaScience*, *F1000Research* Software Tool

Articles, *Journal of Open Source Software* (which is the lightest-weight option and is appropriate for tools that are too small for a full software paper). Software papers are appropriate when the tool has methodological contributions beyond the standard pipeline.

Methods Papers are full-length (~5,000–8,000 words), focused on a methodological advance, with the software as the supporting artifact. Examples: *Nature Methods*, *Nature Biotechnology*, *Genome Biology*. Methods papers are appropriate when the underlying methodology is a real innovation, validated against an independent dataset and benchmarked against state-of-the-art alternatives.

MetabolonR-LLM is on the Application-Note-to-Software-Paper boundary. The architecture is methodological; the implementation is a tool. The Briefings target sits in the right zone. *Nature Methods* would have been within reach with a multi-dataset validation; the v2.0 single-dataset scope rules it out. The v2.1 multi-dataset validation may motivate a different venue choice for that paper.

The rule the student should use: the venue should match the validation depth. A paper claiming a methodological contribution without a multi-dataset validation will be rejected from *Nature Methods*; a paper presenting a tool with a single-dataset validation should be at Briefings or GigaScience.

25.3 The Zenodo + GitHub citation chain

The Zenodo + GitHub integration is the easiest dollar a scientific-software project can make. The integration takes 20 minutes to set up and produces a permanent DOI for every release. The chain works like this.

A GitHub release is tagged on the public repository. The Zenodo webhook, configured once at the repository level, fires on every release and pulls the tagged source archive into a Zenodo deposit. Zenodo mints a DOI for the deposit and links it back to the GitHub release.

The DOI is what gets cited in academic publications. A reader who clicks the DOI gets the Zenodo landing page, which links to the GitHub release, which links to the canonical commit hash. The chain is durable: even if the GitHub repository is renamed, deleted, or transferred, the Zenodo deposit and its DOI persist.

Three details matter and are easy to get wrong on the first attempt.

First, the Zenodo metadata must be filled in carefully. Author ORCIDs, affiliations, keywords, license. A deposit with missing metadata is technically valid but less discoverable.

Second, the GitHub release must have a tag that follows semver (`v2.0.0`, not `release-2026-04-12`). Zenodo's parsing of the tag drives the deposit version field, and a non-semver tag produces a confusing version.

Third, the `CITATION.cff` file must include the Zenodo DOI in the `doi` field. This is what makes the GitHub "Cite this repository" widget produce a complete citation.

25.4 The pre-submission checklist

A twenty-item checklist the student should run against any software-paper submission. The order is roughly the order of consequences-if-missed.

1. README has a one-paragraph description and a quickstart.
2. LICENSE is present and is MIT or an OSI-approved equivalent.
3. CITATION.cff is present, validates, includes the Zenodo DOI.
4. Public GitHub release matches the manuscript-cited version.
5. Zenodo DOI is minted and resolves.
6. CI is green on the released commit.
7. Tests run on a fresh clone with documented setup steps.
8. Docker image (if applicable) builds and pulls.
9. Reproducibility crosscheck succeeded on a different machine.
10. Cover letter foregrounds the methodological claim.

11. Three reviewers are suggested with one-sentence justifications each.
12. Data-availability statement is explicit, with DOIs.
13. Code-availability statement is explicit, with repository URL and Zenodo DOI.
14. Funding statement names the grants.
15. Author contributions list is complete (CRediT taxonomy if the journal uses it).
16. Conflicts of interest are declared (none, in most cases, but the field is required).
17. ORCIDs for all authors are entered in the submission portal.
18. Figures are at submission-required DPI (typically 300 for print, 600 is safer).
19. Supplementary materials (the validation dataset, the worked-example log) are uploaded.
20. The pre-print is posted and its DOI is referenced in the cover letter.

A submission that misses any of items 1–10 is likely to be desk-rejected. A submission that misses items 11–20 is likely to

receive a "revise and resubmit" with the missing items as the first reviewer comments. The student should run the checklist with the advisor on the Friday before submission and not submit until every item is green.

25.5 The social-media and preprint workflow

The week of a software-paper submission produces three social media moments: the preprint posting, the journal submission, and (later) the acceptance announcement. Each should be planned and none should be skipped.

The **preprint thread** is the most consequential. A five-post thread on Twitter / X / Mastodon, ideally posted Tuesday morning Eastern (Chapter 18 §18.5), is the right format. Post 1: what the tool is, one sentence, with the graphical abstract image and the bioRxiv DOI. Post 2: the architectural claim. Post 3: the validation evidence. Post 4: the limitations. Post 5: acknowledgments and team. A thread that is honest about limitations earns more engagement than one that overclaims.

The **submission announcement** is optional and is usually a low-key follow-up to the preprint thread. A single post mentioning the submission and the venue. This is the moment many authors skip; the author should not. It signals that the work is moving through the publication process.

The **acceptance announcement** comes later (the eight-week wait of Chapter 18 §18.5). When it arrives it is the moment the author has earned. A thread similar to the preprint thread, with the final DOI replacing the preprint DOI, and a slightly more confident tone justified by peer review.

The author's specific advice for the student: write the threads in a draft folder before posting. The Monday before the preprint posting, draft the preprint thread. The Tuesday before submission, draft the submission announcement. The day acceptance arrives, the announcement thread is half-written already.

25.6 Advice for the next project

The build documented in this book is the student's first major software-paper project. The most important advice for the second project does not appear in the prior twenty-four chapters and is offered here.

Pick a problem narrower than seems reasonable. The temptation on a second project is to scale up. Resist it. A narrower problem with a clear architecture, a small tool set, and a tractable validation is more publishable than a broad problem with a confused architecture.

Reuse the architecture. The closed-tool-set discipline, the schema-pinned arguments, the structured session log, the replay engine — these patterns are the contribution that travels. The student's next project, if it touches an agentic component, should reuse these patterns rather than rederiving them.

Find the oracle early. Every successful build documented in this book traces back to v1 as the validation oracle. The student's next project needs an equivalent: a previous-

generation tool whose outputs the new tool can be pinned against. If the next project does not have an oracle, the validation will be much harder.

Pre-register the validation. The discipline of writing down the metrics and targets before collecting the data is the cheapest insurance against post-hoc rationalization. Do it on every project. Commit to a Git tag before the first run.

Budget for the boring weeks. The four weeks of any six-week build that look unglamorous — Week 1's literature review, Week 2's oracle-test grind, Week 5's repo polish, Week 6's submission paperwork — are the weeks the project's quality depends on. Budget them generously and resist the urge to shortchange them in favor of the visible weeks.

Cultivate the advisor relationship. This book exists because of the relationship between author and student, and the student-to-be-author transition for the next project depends on the relationship being maintained. Send weekly updates. Ask for feedback before you need it. Show drafts in pieces, not as fully

formed manuscripts. The advisor's leverage compounds over years.

25.7 A closing note

The student who reads this book has built something real. The build is documented chapter by chapter, the validation is reported honestly, and the manuscript is in the world. The next project will be different in its details and the same in its discipline.

The author hopes the student will, in turn, mentor someone else through their first major software-paper build, and that the dedication in the Arabic preface — to the student who built this — will, in time, become a dedication the student writes to their own student.

[Figure 25.1: The twenty-item pre-submission checklist as a literal checklist rendered as the book's penultimate figure.]

What you should be able to do after this chapter

- Choose between Application Note, Software, and full Methods venues based on the work's scope.
- Set up a Zenodo + GitHub citation chain in under 30 minutes.
- Run the twenty-item pre-submission checklist against a software-paper draft.
- Plan a three-moment social-media-and-preprint workflow for a submission week.
- Apply the advice for next-project planning: narrow scope, architecture reuse, oracle-first, pre-registration, budget for boring weeks.
- Recognize that this chapter is the closing of the book and that the methodological discipline it summarizes is the contribution the author hopes will travel.

عن الكتاب

كتاب تقني شامل يوثق تصميم وبناء والتحقق العلمي من ميثابولون-إل إل إم، وهو وكيل ذكي يعتمد على نموذج لغوي كبير يحلّ محل واجهة شايبي في تحليل بيانات الأيض، مع الحفاظ على إحصاءات حتمية قابلة لإعادة الإنتاج عبر طبقة أدوات مكتوبة بدقة.

عن المؤلف

الدكتور فضل محمد الأكوع باحث في البيولوجيا الحاسوبية والمعلوماتية الحيوية والجينومات المكانية بجامعة ميشيغان في آن آربر. تتمحور أبحاثه حول تطبيق الذكاء الاصطناعي في دراسة الأمراض المزمنة، ومن أبرز أعماله المنشورة دراسات على مثبطات ناقل الصوديوم-غلوكوز 2 (SGLT2) والتغيرات الأنوية الكلوية المنشورة في مجلة JCI عام 2023. إلى جانب مسيرته العلمية، يكتب د. فضل باللغتين العربية والإنجليزية في حقول متعددة تشمل الدراسات القرآنية، تاريخ اليمن والسياسة، الذكاء الاصطناعي والتقنية، الأسرة والتربية، ودليل المهاجرين في المهجر.

مؤلفات المؤلف الأخرى

ثروة حلال (2026)

التأمين في الإسلام (2026)

الإفلاس في أمريكا (2026)

الوصية في أمريكا (2026)

قتل الشاهد (2026)

ديمقراطية للبيع (2026)

الحقيقة عن الشيعة (2026)

تحديات يوتيوب (2026)

من أنا؟ (2026)

المسلم في مجتمع غير مسلم (2026)

تحدث كالناطقين (2026)

فجرة الأجيال (2026)

الموت في أمريكا (2026)

وحدة المهاجر (2026)

الطلاق في أمريكا والإسلام (2026)

مسلم في الغرب (2026)

بيت بلا ربا (2026)

الأفلام الهادفة (2026)

الأكل الذي يقتلنا ببطء (2026)

المواريث في الإسلام والقانون (2026)

رحلة الدواء (2026)

حزمة التقنية الصحية (2026)

البرمجة بالذكاء الاصطناعي: Claude Code من الصفر إلى الإنتاج (2026)

جوجل ديب مايند (2026)

أسباب التقارب الإماراتي الإسرائيلي (2026)

خوارزميات التعلم الآلي (2026)

من آلة الكاش إلى نظام البيع الذاتي (2026)

كل ما تحتاجه هو الانتباه (2026)

قراءة شرائح الكلى المرضية دليل عملي لتفسير صبغة H&E (2026)

الدليل العملي لمهنة ميكانيكي السيارات في أمريكا (2026)

أفضل المدن للعيش في الولايات المتحدة الأمريكية (2026)

عالم القهوة رحلة من المزرعة إلى الفنجان (2026)

دليل الاستثمار في سوق الأسهم التاريخ والمستقبل وأسرار الثروة (2026)

المثلية الجنسية قراءة في التاريخ والحجج والأديان (2026)

بايثون للذكاء الاصطناعي: من الصفر إلى البناء (2026)

معالجة اللغة العربية مع نماذج الذكاء الاصطناعي: العمل بلغتك الأم (2026)

البناء مع كلود – دليل عملي لواجهة برمجة Anthropic (2026)

بناء معرض أعمالك في مجال الذكاء الاصطناعي: 30 مشروعاً حقيقياً (2026)

هندسة الأوامر – فن التحوار مع الذكاء الاصطناعي (2026)

الذكاء الاصطناعي لصنّاع المحتوى - دليل التصميم والصوت والفيديو (2026)

الأتمة بدون كود - دليل بناء سير عمل ذكي (2026)

العملاء الأذكياء و بروتوكول MCP - بناء أنظمة ذكاء اصطناعي مستقلة (2026)

التوليد المعزز بالاسترجاع - دليل عملي لبناء أنظمة RAG (2026)

العمل الحر بالذكاء الاصطناعي – دليل المستقل اليمني والعربي (2026)

نوت بوك إل إم – الدليل الشامل (2026)

القانون الجميل الكبير - تشريح إصلاح ترامب الضريبي وتأثيره على أمريكا (2026)

التقارب السني الشيعي (2026)

الذكاء الاصطناعي ومستقبل المهن – أيها يندثر وأيها يصعد (2026)

التأثيرات الاستثمارية في أمريكا – دليل المقارنة والاختيار (2026)

البن في ملفات إبستين (2026)

الرئيس إبراهيم الحدي – حياته ومشروعه الوطني (2026)

الشخصيات العامة التي فقدت وظائفها بسبب موقفها من غزة (2026)

الصوفية – دراسة شاملة (2026)

الطعام والصحة بين العلم الحديث والحديث النبوي (2026)

النظام الضريبي الأمريكي – دليل شامل (2026)

دليل شراء اللحم الحلال في ميشيجان وإنديانا (2026)

سلسلة التوريد – دليل شامل (2026)

ميكانيكا الكم – دراسة شاملة (2026)

أدوات الذكاء الاصطناعي – دليل شامل (2026)

Mythos في مواجهة نماذج Anthropic (2026)

مهارات كلود - كيف يجعلك استخدامها خبيراً في كل شيء (2026)

البوذية والهندوسية - النشأة والانتشار والمقارنة (2026)

الذنوب والمعاصي والأخلاق في الإسلام - مقارنة مع الأديان
الأخرى (2026)

الذكاء الاصطناعي وثورة اكتشاف الأدوية (2026)

حرب غزة - كيف فضحت المجتمع الدولي (2026)

مجازر ضد المسلمين عبر العصور (2026)

العلاقات اليمنية السعودية - تاريخ وتحولات عبر الأزمان (2026)

كيف تؤسس شركتك في أمريكا (2026)

هندسة البيانات - الدليل الشامل (2026)

جماعة الدعوة والتبليغ - دراسة شاملة (2026)

أجيال على المحك - الإسلام في المهجر بين الحفاظ والانسلاخ (2026)

المسلمون في الغرب — الإنجازات والتحديات والاندماج (2026)

الربا في الإسلام والبنوك الإسلامية (2026)

القرآن والإنجيل — دراسة مقارنة في مفاهيم السلام والقتال والتسامح (2026)

نهاية الكون في القرآن الكريم (2026)

مشاكل اليمن عبر الأزمان (2026)

سر التقارب الأمريكي الإسرائيلي — من يقود من؟ (2026)

هل انتهت الوحدة اليمنية؟ (2026)

الرؤساء اليمنيون في مذكرات ويكيليكس (2026)

أسرار قوة التحالف الأمريكي الإسرائيلي (2026)

فن التربية — كيف تصنع قادة من أبنائك (2026)

تأثير النماذج اللغوية الكبيرة على البحث العلمي والتعليم الجامعي (2026)

علامات الساعة (2026)

التخطيط للتقاعد (2026)

العملات الرقمية — دليل شامل للمبتدئين (2026)

وادي السيليكون — تاريخ وابتكار وريادة علمية (2026)

أشهر المتنزهات الوطنية في أمريكا (2026)

دليل شراء وبيع السيارات المستعملة في أمريكا (2026)

كلود كود — الدليل الشامل (2026)

دليل المهاجر اليمني الجديد إلى أمريكا (2026)

الحقوق الزوجية والحياة الأسرية للمسلمين في الغرب (2026)

مستقبل المعلوماتية الحيوية والبيولوجيا الحاسوبية في عصر الذكاء الاصطناعي (2026)

تصنيف الآيات في القرآن الكريم — دراسة تأملية في البنية الموضوعية (2026)

كيف تعمل النماذج اللغوية الكبيرة — دليل مبسّط (2026)

سرقة الجامعات الأمريكية (2026)

عتيقة — حكاية جدتي من جبال اليمن (2026)

الخلافة الإسلامية: من السقيفة إلى سقوط غرناطة (2026)

السلفية — دراسة في المفهوم والنشأة والتيارات المعاصرة (2026)

النقد المشروع وحدود معاداة السامية (2026)

كيف تنجح أكاديمياً في الجامعات الأمريكية (2026)

الزواج الناجح — دليل عملي بمنظور إسلامي (2026)

الإعجاز العلمي في القرآن — من منظور باحث في الجينوميات (2026)

أمريكا — صعود القوة وبوادر الأفول (2026)

About this Book

A comprehensive technical book documenting the design, construction, and scientific validation of MetabolonR-LLM, a tool-calling LLM agent that replaces a Shiny-app frontend for metabolomics data analysis while preserving deterministic, reproducible statistics through a typed tool layer.

About the Author

Dr. Fadhl Mohammed Alakwaa is a researcher in computational biology, bioinformatics, and spatial genomics at the University of Michigan in Ann Arbor. His research focuses on applying AI to the study of chronic diseases; among his notable publications are studies on SGLT2 inhibitors and kidney tubular changes published in JCI in 2023. Alongside his scientific career, Dr. Alakwaa writes in both Arabic and English across fields including Quranic studies, Yemeni history and politics, AI and technology, family and parenting, and guides for diaspora life.